Bangladesh University of Engineering and Technology

Department of Computer Science & Engineering

CSE 105 Question Bank With Answers

Data Structures & Algorithms

CSE 105 (L-1/T-2)
CSE 203 (L-2/T-1) & CSE 207 (L-2/T-2)

Topics: Asymptotic Analysis, Data Structures, Graph Traversals, Divide & Conquer, Greedy, Dynamic Programming, Sorting & more

Authors & Contributors

Md Ratul Hasan (2405009) — Question Curation & Organisation
— LaTeX Typesetting
Claude AI — LaTeX Typesetting
Gemini — LaTeX & Diagram Generation

Table of Contents

BUET · CSE 105

DSA

Introduction to Algorithms

Intro, Basic Search, Array Mapping

T1 — Introduction to Algorithms

Q1. Consider the code snippet for a binary search, where you are searching for an item **x** from an array **a** of size **n**. If all primitive operations (addition/subtraction, comparison, assignment, etc.) take one unit of time each to execute, what will be the time complexity of the above algorithm by counting all the primitive operations performed. Can you show the worst-case time complexity of the above algorithm using big-Oh notation? **(5 marks)** [CSE 203 | 2019–20]

Algorithm 1 BinarySearch(a, x)

```
1: left ← 0
2: right ← a.size()
3: while left < right do
4:     middle ← (left + right) / 2
5:     if a[middle] ≤ x then
6:         left ← middle + 1
7:     else
8:         right ← middle
9:     end if
10: end while
11: if left > 0 and a[left - 1] == x then
12:     return true
13: else
14:     return false
15: end if
```

Answer

Counting primitive operations: Each iteration of the `while` loop performs: 1 comparison (`left < right`), 1 addition + 1 division for `middle`, 1 comparison (`a[middle] <= x`), 1 addition or 0 assignments. Approximately 5 operations per iteration. The loop runs $\lfloor \log_2 n \rfloor$ times. The final `if` block adds 2 more operations.
Total $\approx 5\lfloor \log_2 n \rfloor + 2$ primitive operations.
Worst-case Big-O: $T(n) = O(\log n)$, since the number of operations grows logarithmically with n.

Q2. Write down two algorithms for determining whether a year is a leap year. Deduce in detail the average-case complexity of both algorithms. **(5+5 marks)** [CSE 207 | 2006–07, 2013–14]

Answer

Algorithm 1 — Standard 3-condition check:

Algorithm 2 isLeapYear_A(y)

```
1: if y mod 400 = 0 then return true
2: end if
3: if y mod 100 = 0 then return false
4: end if
5: if y mod 4 = 0 then return true
6: end if
7: return false
```

Average-case complexity: Each condition is $O(1)$. In a uniform distribution over all years, 1/400 reach line 1 (return true), 3/400 reach line 2 (return false), 96/400 reach line 3, and 300/400 reach line 4. The expected number of comparisons $= 1 \cdot \frac{1}{400} + 2 \cdot \frac{3}{400} + 3 \cdot \frac{96}{400} + 4 \cdot \frac{300}{400} \approx 3.75$. This is a constant, so the average-case complexity is $\Theta(1)$.

Algorithm 3 isLeapYear_B(y)

```
1: return ((y mod 4 = 0) and (y mod 100 ≠ 0 or y mod 400 = 0))
```

Average-case complexity: This evaluates at most 3 modular conditions, all $O(1)$. With short-circuit evaluation, the average number of conditions checked is constant. Thus average-case complexity is also $\Theta(1)$.

Both algorithms are $\Theta(1)$ in all cases since the input size is fixed (one integer) and the number of operations is bounded by a constant.

Q3. Write down two versions of the binary search algorithm. Deduce the average-case complexity of both. **(5+5 marks)** [CSE 207 | 2013–14]

Answer

Version 1 — Standard (early exit on equality): Checks $A[mid] = x$ at each step; terminates as soon as element is found. Average-case comparisons (successful): $\approx \log_2 n - 1 + \frac{1}{n} = \Theta(\log n)$.
Version 2 — Delayed (equality check at end): Only checks $A[mid] \leq x$ in the loop; performs one equality check after the loop exits. Always runs exactly $\lceil \log_2 n \rceil$ loop iterations then 1 check $= \Theta(\log n)$.

Both versions have average-case complexity $\Theta(\log n)$. Version 2 is more uniform (same cost for all inputs of size n), while Version 1 is faster in the best case ($\Theta(1)$) but has the same asymptotic average.

Q4. Write down traditional algorithms for finding the minimum and maximum of an array, and a recursive algorithm. Compare their complexities. **(10 marks)** [CSE 207 | 2013–14]

Answer

Traditional (linear scan): 2 comparisons per element $\Rightarrow 2(n-1)$ total $= \Theta(n)$.
Recursive divide-and-conquer:

Algorithm 4 MinMaxRec(A, lo, hi)

```
1: if lo = hi then
2:     return (A[lo], A[lo])
3: end if
4: if hi = lo + 1 then
5:     if A[lo] < A[hi] then
6:         return (A[lo], A[hi])
7:     else
8:         return (A[hi], A[lo])
9:     end if
10: end if
11: mid ← ⌊(lo + hi)/2⌋
12: (lmin, lmax) ← MinMaxRec(A, lo, mid)
13: (rmin, rmax) ← MinMaxRec(A, mid + 1, hi)
14: return (min(lmin, rmin), max(lmax, rmax))
```

Recurrence: $T(n) = 2T(n/2) + 2$, with $T(2) = 1$. Solving: $T(n) = \frac{3n}{2} - 2$ comparisons.

Comparison:

Algorithm	Comparisons	Complexity
Traditional	$2(n-1)$	$\Theta(n)$
Recursive	$\frac{3n}{2} - 2$	$\Theta(n)$

Both are $\Theta(n)$, but the recursive version uses $\approx 25\%$ fewer comparisons.

Q5. What is a data structure? When is a data structure called a linear data structure? Write 4 (four) properties of a linear data structure. **(1.5+1.5+4=7 marks)** [CSE 203 | 2013–14]

Answer

Data structure: An organised way of storing and managing data so operations can be performed efficiently.
Linear data structure: A data structure is *linear* if its elements form a sequence, i.e. each element (except the first and last) has exactly one predecessor and one successor, and all elements can be traversed in a single pass.
Four properties:

1. Elements are arranged in a sequential, one-dimensional order.
2. Every element (except the first) has a unique predecessor; every element (except the last) has a unique successor.
3. Can be traversed from beginning to end in one pass.
4. Memory is allocated either contiguously (array) or via pointers (linked list). Examples: array, stack, queue, linked list.

Q6. Let `a : array[`$l_1 \,..\, u_1$`, `$l_2 \,..\, u_2$`]` be a 2D integer array. Assume b is the base address of the array `a`. Write the Array Mapping Function to calculate `address(a[i,j])` in:

(a) Column-major order

(b) Row-major order

(5+5=10 marks) [CSE 203 | 2013–14]

Answer

Let $d_1 = u_1 - l_1 + 1$ and $d_2 = u_2 - l_2 + 1$ be the sizes. Let c = size of one integer element.

(a) **Column-major order** (FORTRAN-style): column index varies fastest.

$$\text{address}(a[i,j]) = b + c \cdot \big[(j - l_2) \cdot d_1 + (i - l_1)\big]$$

(b) **Row-major order** (C-style): row index varies fastest.

$$\text{address}(a[i,j]) = b + c \cdot \big[(i - l_1) \cdot d_2 + (j - l_2)\big]$$

Derivation (row-major): To reach row i, skip $(i - l_1)$ complete rows of d_2 elements. Within row i, element j is at offset $(j - l_2)$. Multiply total offset by c and add base address b.

Q7. What is a data structure? Write the properties of a linear data structure. **(1+4=5 marks)** [CSE 203 | 2012–13]

Answer

Data Structure: A data structure is a systematic way of organising, storing, and managing data in a computer so that it can be accessed and modified efficiently. It defines both the data and the operations that can be performed on that data (e.g. insertion, deletion, search).

Properties of a Linear Data Structure:

1. **Sequential arrangement:** Elements are arranged in a linear (one-dimensional) sequence, where each element has a unique predecessor (except the first) and a unique successor (except the last).
2. **Single level:** All elements reside on a single level; there is no hierarchical or branching structure.
3. **Traversal in one pass:** The entire structure can be traversed in a single run from the first element to the last.
4. **Memory:** Elements are stored either in contiguous memory locations (e.g. arrays) or linked via pointers (e.g. linked lists). Examples include arrays, stacks, queues, and linked lists.

Q8. Let `a : array[`$l_1 \,..\, u_1$`,` $l_2 \,..\, u_2$`,` $l_3 \,..\, u_3$`]` be a 3-dimensional array. Assume that b and c are the base address and component length of the array `a` respectively. Write the Array Mapping Function to calculate addr$(a[i, j, k])$. **(10 marks)** [CSE 203 | 2012–13]

Answer

Let $d_2 = u_2 - l_2 + 1$ and $d_3 = u_3 - l_3 + 1$ be the sizes of the second and third dimensions respectively.
Using **row-major order**, the Array Mapping Function is:

$$\text{addr}(a[i, j, k]) = b + c \cdot \big[(i - l_1) \cdot d_2 \cdot d_3 + (j - l_2) \cdot d_3 + (k - l_3)\big]$$

Derivation: To reach the i-th slab, we skip $(i - l_1)$ complete slabs each containing $d_2 \times d_3$ elements. Within that slab, to reach row j we skip $(j - l_2)$ rows of d_3 elements each. Within that row, k is at offset $(k - l_3)$ from the start. Each element occupies c bytes, so we multiply the total element offset by c and add to the base address b.

Q9. Write down the algorithm for binary search. Analyze its best-case, average-case, and worst-case complexities. Simulate the search of elements 2 and 10 in the array $[1, 2, 4, 7, 9, 11, 12, 13, 14]$ by showing the values of `low`, `high`, `mid`, and `found`. **(5+5+5=15 marks)** [CSE 207 | 2008–09]

Answer

Algorithm 5 BinarySearch(A, n, x)

```
1: low ← 1;   high ← n;   found ← false
2: while low ≤ high and not found do
3:     mid ← ⌊(low + high)/2⌋
4:     if A[mid] = x then
5:         found ← true
6:     else if A[mid] < x then
7:         low ← mid + 1
8:     else
9:         high ← mid − 1
10:    end if
11: end while
12: if found then
13:    return mid
14: else
15:    return −1
16: end if
```

Complexities: Best case: $\Theta(1)$ (element at midpoint). Worst/Average case: $\Theta(\log n)$.

Simulation on $A = [1, 2, 4, 7, 9, 11, 12, 13, 14]$ ($n = 9$, 1-indexed):
Search for 2:

Step	low	high	mid	A[mid]
1	1	9	5	$9 > 2 \Rightarrow$ high=4
2	1	4	2	$2 = 2 \Rightarrow$ found=true

Result: found at index 2.
Search for 10:

Step	low	high	mid	A[mid]
1	1	9	5	$9 < 10 \Rightarrow$ low=6
2	6	9	7	$12 > 10 \Rightarrow$ high=6
3	6	6	6	$11 > 10 \Rightarrow$ high=5
4	low>high			not found

Result: not found.

Q10. Write down an algorithm for finding the minimum and maximum of an array. Deduce its complexity. **(10 marks)** [CSE 207 | 2007–08]

Answer

Algorithm 6 MinMax(A, n)

```
1: min ← A[1]
2: max ← A[1]
3: for i = 2 to n do
4:     if A[i] < min then
5:         min ← A[i]
6:     end if
7:     if A[i] > max then
8:         max ← A[i]
9:     end if
10: end for
11: return (min, max)
```

Complexity: The loop runs $n-1$ iterations. In each iteration, exactly 2 comparisons are made (regardless of values). Total comparisons $= 2(n-1)$. Therefore the time complexity is $\Theta(n)$.

Improved algorithm (Tournament / Pair comparison): Process elements in pairs: compare the pair first, then compare the smaller with current min and the larger with current max. This uses only $\frac{3(n-1)}{2}$ comparisons instead of $2(n-1)$, but remains $\Theta(n)$.

Q11. Deduce the best-case, worst-case, and average-case complexity of sequential search in both successful and unsuccessful cases. **(10 marks)** [CSE 207 | 2007–08]

Answer

Algorithm: Search for element x in array $A[1..n]$ by scanning left to right.

Case	Successful	Unsuccessful	Comparisons
Best case	$x = A[1]$ (found at pos 1)	N/A	1
Worst case	$x = A[n]$ (found at end)	$x \notin A$ (scan all)	n
Average case	element equally likely at any of n positions	full scan	$\frac{n+1}{2}$ (success); n (fail)

Derivation of average-case (successful): If x is at position i with equal probability $\frac{1}{n}$, the expected comparisons are:

$$\sum_{i=1}^{n} \frac{1}{n} \cdot i = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2} = \Theta(n)$$

Unsuccessful: Always scans all n elements $\Rightarrow \Theta(n)$.
All non-trivial cases are $\Theta(n)$; best case is $\Theta(1)$.

Q12. Write down an algorithm for binary search that delays the finding until the end. Deduce its average-case complexity. **(5+10 marks)** [CSE 207 | 2006–07]

Answer

Algorithm 7 DelayedBinarySearch(A, n, x)

```
1: low ← 1
2: high ← n
3: while low < high do
4:     mid ← ⌊(low + high)/2⌋
5:     if A[mid] ≤ x then
6:         low ← mid + 1
7:     else
8:         high ← mid
9:     end if
10: end while
11: if A[low] = x then
12:     return low
13: else
14:     return Not_Found
15: end if
```

Unlike standard binary search which checks equality at each step, this version only checks $A[mid] \leq x$ in the loop, delaying the equality check until $low = high$.

Average-case complexity: The loop always runs $\lceil \log_2 n \rceil$ iterations regardless of where the element is, because equality is never checked inside the loop. In each iteration, exactly one comparison is made. Therefore:

$$T_{\text{avg}}(n) = \lceil \log_2 n \rceil + 1 = \Theta(\log n)$$

This is actually better than standard binary search in the average case (which does $\sim \log n - 1$ comparisons for successful search) because it makes only $\lfloor \log_2 n \rfloor + 1$ total comparisons in every case, making it more uniform and cache-friendly.

BUET · CSE 105

DSA

Asymptotic Notations: $\mathcal{O}$, Ω, Θ, o, ω

Growth of Functions, Notations, Code Complexity

T2 — Asymptotic Notations: $\mathcal{O}$, Ω, Θ, o, ω

Q1. Analyze and compare the growth rate of $n^{\log\log n}$ and $(\log n)^n$. **(10 marks)** [CSE 105 | 2022–23]

Answer

Take logarithms to compare: $\ln(n^{\log\log n}) = \log\log n \cdot \ln n$ and $\ln((\log n)^n) = n\ln(\log n)$.
Ratio: $\frac{n\ln(\log n)}{\log\log n \cdot \ln n}$. As $n \to \infty$, n grows much faster than $\ln n \cdot \log\log n$, so the ratio $\to \infty$.
Therefore $(\log n)^n$ grows **much faster** than $n^{\log\log n}$, i.e. $n^{\log\log n} = o((\log n)^n)$.
Intuitively: the exponent of $(\log n)^n$ is n itself (linear in n), whereas the exponent of $n^{\log\log n}$ is only $\log\log n$ (doubly logarithmic).

Q2. Derive the time complexity of the following code fragments:

(a)
```
for(int i=0; i<n; ++i)
  for(int j=0; j<i; ++j)
    sum += i+j;
```

(b)
```
for(int i=0; i<n; ++i)
  for(int j=0; j<i*i; j++)
    sum += i+j;
```

(c)
```
for(int i=0; i<n*n; i++)
  for(int j=0; j<i; j++)
    sum += i+j;
```

(d)
```
for(int k=1; k<=n; k*=2)
  for(int j=1; j<=k; j++)
    sum++;
```

(4×4=16 marks) [CSE 105 | 2021–22]

Answer

(a) **i from 0 to $n-1$; j from 0 to $i-1$:** Inner loop runs $0+1+2+\cdots+(n-1) = \frac{n(n-1)}{2}$ times. Complexity: $\Theta(n^2)$.

(b) **j from 0 to i^2-1:** Inner loop runs $\sum_{i=0}^{n-1} i^2 = \frac{(n-1)n(2n-1)}{6} = \Theta(n^3)$. Complexity: $\Theta(n^3)$.

(c) **Outer i from 0 to n^2-1; inner j from 0 to $i-1$:** Inner loop total $= \sum_{i=0}^{n^2-1} i = \frac{n^2(n^2-1)}{2} = \Theta(n^4)$. Complexity: $\Theta(n^4)$.

(d) **Outer k doubles: $k = 1, 2, 4, \ldots, n$; inner j from 1 to k:** k takes values $1, 2, 4, \ldots, 2^{\lfloor \log_2 n \rfloor}$. Total inner iterations $= 1+2+4+\cdots+2^{\lfloor \log_2 n \rfloor} = 2^{\lfloor \log_2 n \rfloor + 1} - 1 \leq 2n-1 = \Theta(n)$. Complexity: $\Theta(n)$.

Q3. In an array-based implementation of a stack, when the array space runs out, the array capacity is increased. Analyze and explain the following scenarios through an amortized analysis of the `push()` function:

(a) Increasing the capacity of the array by 1

(b) Doubling the capacity of the array

(2×7=14 marks) [CSE 105 | 2021–22]

Answer

(a) **Increase by 1:** When the array is full at size k, we allocate a new array of size $k+1$ and copy all k elements. A sequence of n pushes starting from size 1 incurs copies: $1+2+3+\cdots+(n-1) = \frac{n(n-1)}{2} = O(n^2)$ total. Amortized cost per push: $\frac{O(n^2)}{n} = O(n)$. This is **inefficient**.

(b) **Double the capacity:** When the array is full at size k, allocate a new array of size $2k$ and copy k elements. Doubling happens at sizes $1, 2, 4, \ldots, 2^{\lfloor \log_2 n \rfloor}$. Total copies $= 1+2+4+\cdots+2^{\lfloor \log_2 n \rfloor} < 2n$. Amortized cost per push: $\frac{O(n)}{n} = O(1)$. This is **efficient** — the amortized cost of each `push()` is $O(1)$.

Q4. Explain what the three figures (Figure 1(i)–(iii)) signify with respect to asymptotic notations. **(– marks)** [CSE 105 | 2021–22]

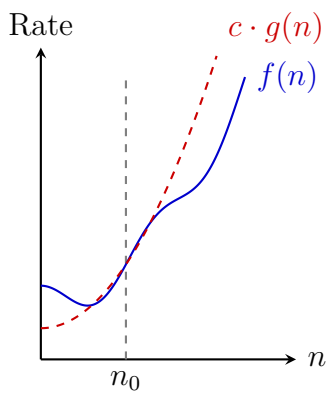


Figure : (i)

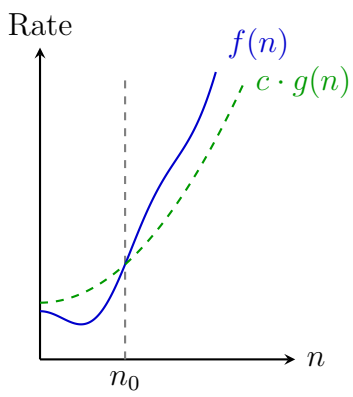


Figure : (ii)

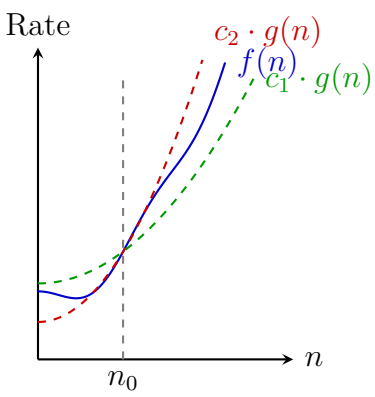


Figure : (iii)

Answer — Significance of Asymptotic Notations

The three figures visually define the primary asymptotic bounds used to analyze and classify the time or space complexity of algorithms. In all figures, n_0 represents a threshold input size, and c, c_1, c_2 are strictly positive constants.

I. Big-O Notation (O) — Asymptotic Upper Bound
Figure 1(i) illustrates $f(n) = O(g(n))$. It signifies that for all inputs larger than n_0, the running time $f(n)$ is bounded from above by a constant multiple of $g(n)$.
Formal Definition: $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.
Significance: It provides the *worst-case scenario*, guaranteeing that the algorithm will not take longer than this bounded rate to execute.

II. Big-Ω Notation (Ω) — Asymptotic Lower Bound
Figure 1(ii) illustrates $f(n) = \Omega(g(n))$. It signifies that for all inputs beyond n_0, the function $f(n)$ will always grow at least as fast as a constant multiple of $g(n)$.
Formal Definition: $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.
Significance: It provides the *best-case scenario* (or a hard minimum bound), guaranteeing that the algorithm will take *at least* this much time or resources.

III. Big-Θ Notation (Θ) — Asymptotic Tight Bound
Figure 1(iii) illustrates $f(n) = \Theta(g(n))$. It signifies that $f(n)$ is completely "sandwiched" between two constant multiples of $g(n)$ for all inputs beyond n_0.
Formal Definition: $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.
Significance: It indicates that $f(n)$ grows at *exactly* the same rate as $g(n)$. It is used when an algorithm's worst-case and best-case complexities scale identically.

Q5. Find a tight upper bound on the complexity of the following code segments:

(a)
```
void function(int n) {
    int count = 0;
    for (int i=n/2; i<=n; i++)
        for (int j=1; j+n/2<=n; j++)
            for (int k=1; k<=n; k = k*2)
                count++;
}
```

(b)
```
void function(int n) {
    int count = 0;
    for (int i=n/2; i<=n; i++)
        for (int j=1; j<=n; j = 2*j)
            for (int k=1; k<=n; k = k*2)
                count++;
}
```

(10 marks) [CSE 203 | 2021–22]

Answer

(a) **Three nested loops:** Outer: `i` from $n/2$ to $n \Rightarrow O(n)$ iterations. Middle: `j` from 1 while $j + n/2 \leq n \Rightarrow j \leq n/2 \Rightarrow O(n)$ iterations. Inner: `k` doubles from 1 to $n \Rightarrow O(\log n)$ iterations. Total: $O(n) \cdot O(n) \cdot O(\log n) = O(n^2 \log n)$.

(b) **Three nested loops (j doubles too):** Outer: $O(n)$. Middle: `j` doubles from 1 to $n \Rightarrow O(\log n)$. Inner: `k` doubles $\Rightarrow O(\log n)$. Total: $O(n \log^2 n)$.

Q6. Deduce the time complexity of the following code fragments:

(a)
```
k = 1;
while (n > 1) {
    n = n/2;
    k++;
}
```

(b)
```
sum = 0;
for (k=1; k<=n; k*=2)
    for (j=1; j<=k; j++)
        sum++;
```

(2×6=12 marks) [CSE 203 | 2020–21]

Answer

(a) **`n = n/2` loop:** Each iteration halves n. Starting from n, after k iterations $n = n_0/2^k$. Loop ends when $n \leq 1$, i.e. $k = \lfloor \log_2 n_0 \rfloor$. Complexity: $\Theta(\log n)$.

(b) **Double-loop (`k*=2`, `j<=k`):** Outer loop: $k = 1, 2, 4, \ldots, n \Rightarrow \log_2 n$ iterations. Inner loop runs k times. Total: $\sum_{i=0}^{\log_2 n} 2^i =$

$2n - 1 = \Theta(n)$.

Q7. Identify whether the following statements are true or false, clearly justifying your answer:

(a) If an algorithm has time complexity $O(n^2)$, then it always makes n^2 steps.

(b) An algorithm with time complexity $O(n)$ is always slower than an algorithm with time complexity $O(\log n)$, for any input.

(c) An algorithm that makes $c_1 \log_2 n$ steps and an algorithm that makes $c_2 \log_{10} n$ steps are in the same complexity class (c_1, c_2 are constants).

(d) An $O(2^n)$ algorithm can never be faster than an $O(n)$ algorithm.

(12 marks) [CSE 203 | 2020–21]

Answer

(a) **False.** $O(n^2)$ is an upper bound — the algorithm makes *at most* cn^2 steps; it may make far fewer. E.g. an algorithm that always runs in n steps is also $O(n^2)$.

(b) **False.** Asymptotic notation describes growth rates for large n. For small n, an $O(n)$ algorithm may be slower if it has a large constant factor, e.g. $T_1(n) = 1000n$ vs $T_2(n) = \log_2 n$: for $n < 2^{1000}$, $T_1 > T_2$.

(c) **True.** $\log_2 n = \frac{\log_{10} n}{\log_{10} 2}$, so $c_1 \log_2 n = \frac{c_1}{\log_{10} 2} \log_{10} n$. Both are constant multiples of each other $\Rightarrow$ same complexity class $\Theta(\log n)$.

(d) **False.** For small inputs, 2^n may be smaller. E.g. at $n = 1$: $2^1 = 2 < 1$? No, but at $n = 1$: $O(2^n) = 2$ and $O(n) = 1$. However for very large constants in $O(n)$, e.g. $T_1(n) = 10^{10} \cdot n$ and $T_2(n) = 2^n$: for $n < 33$, $T_2 < T_1$.

Q8. What does the term *asymptotic* mean in complexity analysis? Can we show the worst-case and average-case time complexity using Big-Θ notation? If yes, how and when? **(5+5+5 marks)** [CSE 203 | 2019–20]

Answer

Asymptotic: In complexity analysis, *asymptotic* means the behaviour of a function as the input size n grows without bound ($n \to \infty$). We ignore constant factors and lower-order terms and focus on the *dominant* term, which characterises long-run growth.
Can we use Θ for worst-case / average-case? Yes — Θ is a *tight* bound, not a specific-case bound. We can use Θ for any case as long as the bound is tight for that case:

- **Worst-case Θ:** Applicable when the worst-case cost has both a matching upper and lower bound. E.g. Insertion Sort worst-case is $\Theta(n^2)$.
- **Average-case Θ:** Applicable when the expected cost over all inputs of size n is tightly bounded. E.g. Quicksort average-case is $\Theta(n \log n)$.

We *cannot* use a single Θ expression to simultaneously describe all cases if they differ (e.g. Binary Search is $\Theta(1)$ best-case but $\Theta(\log n)$ worst-case).

Q9. Derive the time complexity of the following code snippet and write down the asymptotic complexity using Big-O notation. Can we show the complexity using Big-Θ notation as well? What is the difference in meaning?

```
int i, j, k = 0;
for (i = n/2; i <= n; i++) {
    for (j = 2; j <= n; j = j*2) {
        k = k + n/2;
    }
}
```

(10 marks) [CSE 203 | 2019–20]

Answer

Analysis of the snippet: Outer loop: `i` runs from $n/2$ to n, giving $\approx n/2$ iterations. Inner loop: `j` starts at 2 and doubles until n, giving $\lfloor \log_2 n \rfloor$ iterations. Total: $\frac{n}{2} \cdot \log_2 n$ iterations.
Big-O: $O(n \log n)$.
Can we use Big-Θ? Yes — the loop bounds are deterministic (not data-dependent), so the exact count is $\Theta(n \log n)$ in all cases. $\Theta(n \log n)$ is *tighter*: it says the running time grows *exactly* like $n \log n$ (up to constants), whereas $O(n \log n)$ only says it grows *at most* that fast.

Q10. Express the following function in Θ notation with proper explanation:

$$f(n) = 5n - 2n^2$$

(8 marks) [CSE 203 | 2019–20]

Answer

For large n, the $-2n^2$ term dominates. We drop lower-order terms and constants: $f(n) = 5n - 2n^2 = n^2(5/n - 2)$. For $n \geq 10$, $5/n \leq 0.5$, so $f(n) \leq -1.5n^2 < 0$.
Since $f(n) < 0$ for large n and complexity analysis applies to non-negative functions (e.g. counts of operations), we consider $|f(n)| = 2n^2 - 5n$.
For $n \geq 5$: $n^2 \leq 2n^2 - 5n \leq 2n^2$. So $|f(n)| = \Theta(n^2)$.
Answer: $f(n) = \Theta(n^2)$ (the dominant term is n^2).

Q11. Suppose that you observe that a program takes 30 seconds to complete on inputs of size 600, 4.5 minutes on inputs of size 1800, and 20 minutes on inputs of size 2000. Develop a reasonable assumption for the order of growth of the running time. **(10 marks)** [CSE 203 | 2018–19]

Answer

Convert times: 600 s $\to$ 30 s; 1800 s $\to$ 270 s; 2000 s $\to$ 1200 s.
Check $O(n^2)$: If $T(n) = cn^2$, then $\frac{T(1800)}{T(600)} = \frac{1800^2}{600^2} = 9$. Observed ratio: $\frac{270}{30} = 9$. ✓
Check $\frac{T(2000)}{T(600)} = \frac{2000^2}{600^2} \approx 11.1$. Observed: $\frac{1200}{30} = 40$. ×
Check $O(n^3)$: $\frac{T(1800)}{T(600)} = 3^3 = 27 \neq 9$.
Try $O(n^2 \log n)$: $\frac{1800^2 \log 1800}{600^2 \log 600} \approx 9 \cdot \frac{10.74}{9.23} \approx 10.5$. Still not 9.
The ratio $T(600) : T(1800) = 1 : 9$ matches $O(n^2)$ exactly, while the ratio at 2000 is anomalous (possibly different cache/OS effects).
A **reasonable assumption** is $T(n) = \Theta(n^2)$.

Q12. What is the difference between Big-O (O) and Little-o (o) asymptotic notations? Find appropriate $f(n)$ and $g(n)$ (if any) for each of the following cases:

(a) $f(n) = O(g(n))$, but $g(n) \neq O(f(n))$
(b) $f(n) = O(g(n))$, and $f(n) = \Omega(g(n))$
(c) $f(n) = \Omega(g(n))$, and $f(n) = \omega(g(n))$
(d) $f(n) = \Theta(g(n))$, and $f(n) = o(g(n))$

(5+10 marks) [CSE 203 | 2017–18]

Answer

Big-O vs Little-o: $f = O(g)$ means f grows *at most as fast* as g (non-strict upper bound; $f = g$ is allowed). $f = o(g)$ means f grows *strictly slower* than g: $\lim_{n\to\infty} f(n)/g(n) = 0$.

(a) $f = O(g)$ but $g \neq O(f)$: e.g. $f(n) = n$, $g(n) = n^2$. $n = O(n^2)$ but $n^2 \neq O(n)$.

(b) $f = O(g)$ and $f = \Omega(g)$ (i.e. $f = \Theta(g)$): e.g. $f(n) = 2n$, $g(n) = n$.

(c) $f = \Omega(g)$ and $f = \omega(g)$: **Impossible.** $f = \Omega(g)$ means f grows at least as fast as g; $f = \omega(g)$ (strict) also requires f to grow strictly faster. Together they mean $\lim f/g = \infty$ which is consistent — e.g. $f = n^2$, $g = n$: $n^2 = \Omega(n)$ and $n^2 = \omega(n)$.

(d) $f = \Theta(g)$ and $f = o(g)$: **Impossible.** $f = \Theta(g)$ implies $\lim f/g = c > 0$; $f = o(g)$ implies $\lim f/g = 0$. Contradiction.

Q13. Define Big-O, Ω, and Θ notations. Arrange the following functions in ascending order of growth rate (i.e., if $f(n)$ precedes $g(n)$, then $f(n) = O(g(n))$). Provide justifications.

$$f_1 = n, \quad f_2 = \log_2 n, \quad f_3 = 2^{\sqrt{\log_2 n}}, \quad f_4 = n \ln n, \quad f_5 = 2^n$$

(6+9 marks) [CSE 203 | 2016–17]

Answer

Definitions:

- $f(n) = O(g(n))$: $\exists c > 0, n_0$ s.t. $0 \leq f(n) \leq cg(n)\ \forall n \geq n_0$. (upper bound)
- $f(n) = \Omega(g(n))$: $\exists c > 0, n_0$ s.t. $0 \leq cg(n) \leq f(n)\ \forall n \geq n_0$. (lower bound)
- $f(n) = \Theta(g(n))$: $f = O(g)$ and $f = \Omega(g)$. (tight bound)

Ascending order of growth:

$$f_2 = \log_2 n \ < \ f_3 = 2^{\sqrt{\log_2 n}} \ < \ f_1 = n \ < \ f_4 = n \ln n \ < \ f_5 = 2^n$$

Justification: $\log n \ll 2^{\sqrt{\log n}}$ (since $\sqrt{\log n} \to \infty$ but slower than $\log n$); $2^{\sqrt{\log n}} \ll n$ (take logs: $\sqrt{\log n} \ln 2 \ll \ln n$); $n \ll n \ln n \ll 2^n$ (standard hierarchy).

Q14. Prove that $15n^4 + 13n^2 + 11n = O(n^4)$, but $15n^4 + 13n^2 + 11n \neq O(n^2)$. **(10 marks)** [CSE 203 | 2015–16]

Answer

$15n^4 + 13n^2 + 11n = O(n^4)$**:** For $n \geq 1$: $13n^2 \leq 13n^4$ and $11n \leq 11n^4$, so $15n^4 + 13n^2 + 11n \leq 39n^4$. Choose $c = 39$, $n_0 = 1$. □
$15n^4 + 13n^2 + 11n \neq O(n^2)$**:** If it were $O(n^2)$, then $\exists c$ with $15n^4 + 13n^2 + 11n \leq cn^2$ for large n, giving $15n^2 \leq c - 13 - 11/n$. But $15n^2 \to \infty$, contradiction. □

Q15. For each of the following pairs indicate whether $f(n)$ is $O(g(n))$, $\Omega(g(n))$ or both (i.e. $\Theta(g(n))$). Provide brief justifications

$f(n)$	$g(n)$
$\sqrt{n}$	$\log_2 n$
$n^{1.01}$	$n \log_2^2 n$
2^n	2^{n+1}
$n!$	2^n
n^{100}	1.01^n
$\sum_{i=1}^{n} i^k$	n^{k+1}

(6+9=15 marks) [CSE 207 | 2015–16]

Answer

$f(n)$ vs $g(n)$	Relation	Justification
$\sqrt{n}$ vs $\log_2 n$	Ω	$\sqrt{n}/\log n \to \infty$; $\sqrt{n}$ grows faster
$n^{1.01}$ vs $n \log_2^2 n$	Ω	$n^{1.01}/(n \log^2 n) = n^{0.01}/\log^2 n \to \infty$
2^n vs 2^{n+1}	Θ	$2^{n+1} = 2 \cdot 2^n$; constant factor difference
$n!$ vs 2^n	Ω	Stirling: $n! \sim \sqrt{2\pi n}(n/e)^n \gg 2^n$
n^{100} vs 1.01^n	O	Exponential always dominates polynomial
$\sum_{i=1}^{n} i^k$ vs n^{k+1}	Θ	$\sum_{i=1}^{n} i^k \sim \frac{n^{k+1}}{k+1}$

Q16. What is asymptotic analysis of algorithms? Define Big-O, Ω, and Θ notations. For the functions $f(n) = n \log n$ and $g(n) = 10n \log_{10} n$, show whether f is $O(g)$, or f is $\Omega(g)$, or f is $\Theta(g)$. **(3+6+8 marks)** [CSE 207 | 2014–15]

Answer

Asymptotic analysis studies the behaviour of algorithms as input size $n \to \infty$, ignoring constants and lower-order terms to focus on the dominant growth factor.
Definitions: (same as above — O, Ω, Θ)
Comparing $f(n) = n \log_2 n$ and $g(n) = 10n \log_{10} n$:
Using change of base: $\log_{10} n = \frac{\log_2 n}{\log_2 10} = \frac{\log_2 n}{\log_2 10}$.
So $g(n) = 10n \cdot \frac{\log_2 n}{\log_2 10} = \frac{10}{\log_2 10} n \log_2 n \approx \frac{10}{3.32} n \log_2 n \approx 3.01\, f(n)$.
Since $g(n) = c \cdot f(n)$ for constant $c \approx 3.01$, we have:

$$f(n) = \Theta(g(n))$$

(and hence also $f = O(g)$ and $f = \Omega(g)$).

Q17. Prove that $5n^3 + 3n = O(n^4)$, but $5n^3 + 3n \neq O(n^2)$. **(5+5=10 marks)** [CSE 203 | 2012–13]

Answer

Part 1 — $5n^3 + 3n = O(n^4)$**:** For $n \geq 1$: $5n^3 + 3n \leq 5n^4 + 3n^4 = 8n^4$. Choose $c = 8$, $n_0 = 1$. Then $5n^3 + 3n \leq 8n^4$ for all $n \geq 1$. By definition, $5n^3 + 3n = O(n^4)$. □
Part 2 — $5n^3 + 3n \neq O(n^2)$**:** Suppose for contradiction that $5n^3 + 3n = O(n^2)$. Then $\exists c, n_0$ such that $5n^3 + 3n \leq cn^2$ for all $n \geq n_0$. Dividing by n^2: $5n + 3/n \leq c$. But $5n \to \infty$ as $n \to \infty$, so no constant c can bound $5n$ for all large n. Contradiction. Hence $5n^3 + 3n \neq O(n^2)$. □

Q18. Prove that $O(n^3) \cdot O(n) = O(n^4)$. **(5 marks)** [CSE 203 | 2011–12]

Answer

Let $f(n) \in O(n^3)$ and $g(n) \in O(n)$. Then there exist constants $c_1, c_2 > 0$ and $n_1, n_2 \geq 0$ such that $f(n) \leq c_1 n^3$ for all $n \geq n_1$ and $g(n) \leq c_2 n$ for all $n \geq n_2$.
For $n \geq \max(n_1, n_2)$:

$$f(n) \cdot g(n) \leq c_1 n^3 \cdot c_2 n = (c_1 c_2)\, n^4$$

Setting $c = c_1 c_2$ and $n_0 = \max(n_1, n_2)$, we have $f(n)g(n) \leq c\, n^4$ for all $n \geq n_0$. By definition, $f(n) \cdot g(n) \in O(n^4)$. Hence $O(n^3) \cdot O(n) = O(n^4)$. □

Q19. Write the main three reasons why we usually concentrate on finding the worst-case running time of an algorithm. **(5 marks)** [CSE 203 | 2011–12]

Answer

1. **Guaranteed upper bound:** The worst-case gives an absolute guarantee on performance; we know the algorithm will *never* take longer than this bound for any input of size n.
2. **Worst case often occurs in practice:** For many algorithms the worst case arises frequently (e.g. searching for an element not in a list triggers worst-case sequential search every time).
3. **Average case is hard to compute:** Computing average-case complexity requires knowing the probability distribution of inputs, which is often unknown or difficult to model. Worst-case analysis avoids this assumption.

Q20. Why is $n/2 \neq o(n)$? **(5 marks)** [CSE 207 | 2009–10]

Answer

By definition, $f(n) = o(g(n))$ iff $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$.
For $f(n) = n/2$ and $g(n) = n$:

$$\lim_{n\to\infty} \frac{n/2}{n} = \lim_{n\to\infty} \frac{1}{2} = \frac{1}{2} \neq 0$$

Since the limit is a nonzero constant, $n/2$ is **not** $o(n)$. In fact, $n/2 = \Theta(n)$ — it is a tight bound, not a strict lower-order term. Little-o requires the ratio to go to zero; here it is constant.

Q21. Write down an algorithm for searching z sequentially in array A with n elements. Discuss its average-case, worst-case, and best-case complexities. How can this algorithm be improved? **(5+10+5=20 marks)** [CSE 207 | 2008–09]

Answer

Algorithm 8 SequentialSearch(A, n, z)

```
1: for i = 1 to n do
2:     if A[i] = z then
3:         return i
4:     end if
5: end for
6: return Not_Found
```

Best case: $z = A[1] \Rightarrow 1$ comparison $= \Theta(1)$. **Worst case:** $z = A[n]$ or $z \notin A \Rightarrow n$ comparisons $= \Theta(n)$. **Average case (successful):** $\frac{1}{n}\sum_{i=1}^{n} i = \frac{n+1}{2} = \Theta(n)$.
Improvements:

(i) **Self-organising search:** Move the found element to the front (move-to-front or transpose heuristic). Frequently accessed elements migrate forward, reducing average search time over repeated queries.

(ii) **Sentinel search:** Place z at position $A[n+1]$ (sentinel) and remove the end-of-array check, halving the number of comparisons per iteration.

(iii) **Sort + Binary search:** If multiple searches are needed on static data, sort in $O(n \log n)$ then search in $O(\log n)$ each.

Q22. Define big O-notation, Ω-notation and Θ-notation.

Answer

Big-O: $f(n) = O(g(n))$ iff $\exists c > 0, n_0 \geq 0$ such that $f(n) \leq cg(n)\ \forall n \geq n_0$.
Big-Ω: $f(n) = \Omega(g(n))$ iff $\exists c > 0, n_0 \geq 0$ such that $f(n) \geq cg(n)\ \forall n \geq n_0$.
Big-Θ: $f(n) = \Theta(g(n))$ iff $f = O(g)$ and $f = \Omega(g)$.

BUET · CSE 105

DSA

Correctness Proof & Algorithm Analysis

Master Theorem, Recurrence, Substitution, Loop Invariant

T3 — Correctness Proof & Algorithm Analysis

Q1. Analyze the following recurrences using the Master Theorem:

(a) $T(n) = 2T(n/2) + n \log n$

(b) $T(n) = 2T(n/2) + n^2$

(5+5=10 marks) [CSE 105 | 2023–24]

Answer

Master Theorem: For $T(n) = aT(n/b) + f(n)$, compare $f(n)$ with $n^{\log_b a}$:

(a) $T(n) = 2T(n/2) + n \log n$: $a = 2, b = 2$, $n^{\log_b a} = n^1 = n$. $f(n) = n \log n$. We need to check if $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some $k \geq 0$. Here $f(n) = n \log n = n^1 \cdot \log^1 n$, so $k = 1$. This satisfies Case 2 (extended): $T(n) = \Theta(n \log^2 n)$.

(b) $T(n) = 2T(n/2) + n^2$: $a = 2, b = 2$, $n^{\log_b a} = n$. $f(n) = n^2$. Since $n^2 = \Omega(n^{1+\varepsilon})$ for $\varepsilon = 1$, and regularity holds ($2(n/2)^2 = n^2/2 \leq cn^2$), Case 3 applies: $T(n) = \Theta(n^2)$.

Q2. Using the Master Theorem, prove that Strassen's algorithm improves the runtime of multiplying two $n \times n$ matrices compared to the naive approach with time complexity $O(n^3)$. **(10 marks)** [CSE 105 | 2023–24]

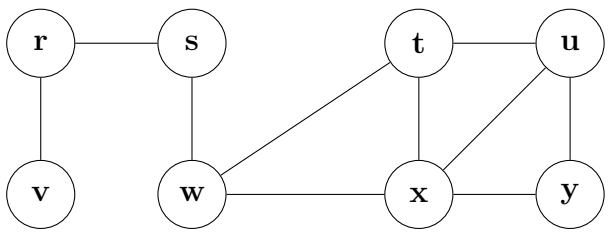
Answer

Strassen's recurrence: $T(n) = 7T(n/2) + \Theta(n^2)$. $a = 7$, $b = 2$, $n^{\log_2 7} \approx n^{2.807}$. Since $f(n) = \Theta(n^2) = O(n^{2.807-\varepsilon})$ for $\varepsilon = 0.807$, Case 1 applies:

$$T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$$

Since $2.807 < 3$, we have $\lim_{n\to\infty} \frac{n^{2.807}}{n^3} = 0$, i.e. $n^{2.807} = o(n^3)$. Therefore Strassen's algorithm is **asymptotically faster** than the naive $\Theta(n^3)$ approach. □

Q3. State the correctness theorem of Breadth-First Search (BFS). Use this theorem to judge whether the value $u.d$ assigned to a vertex u in BFS depends on the order of appearance of the vertices in the corresponding adjacency list. Demonstrate with an example graph whether the breadth-first tree produced by BFS depends on the ordering of vertices in the adjacency lists if the same source vertex is used in every ordering. **(17 marks)** [CSE 105 | 2022–23]



Answer

1. Correctness Theorem of BFS

Theorem (CLRS Theorem 22.5): Let $G = (V, E)$ be a directed or undirected graph, and suppose BFS is run on G from a given source vertex $s \in V$. Then, upon termination, for every reachable vertex $v \in V$, the value $v.d$ computed by BFS satisfies $v.d = \delta(s, v)$, where $\delta(s, v)$ is the shortest-path distance from s to v. Furthermore, the predecessor subgraph forms a breadth-first tree rooted at s.

Proof Outline (by Contradiction): Assume, for the sake of contradiction, that BFS computes an incorrect distance for some vertex. Let v be the reachable vertex with the minimum true shortest-path distance $\delta(s, v)$ that receives an incorrect distance value. Since $v \neq s$ (as $s.d = 0$ is trivially correct), v must have some predecessor u on the actual shortest path from s to v. Thus, $\delta(s, v) = \delta(s, u) + 1$.

Because $\delta(s, u) < \delta(s, v)$, by our assumption of v, vertex u must have received the correct distance: $u.d = \delta(s, u)$. Consider the moment when u is dequeued and its adjacency list is explored. What happens to v?

- If v is **white** (undiscovered), it is immediately assigned $v.d = u.d + 1 = \delta(s, u) + 1 = \delta(s, v)$, which contradicts our assumption that $v.d$ is incorrect.
- If v is **gray or black**, it means v was already discovered by some other node before or during u's processing. This implies v was assigned a distance $v.d \leq u.d + 1 = \delta(s, v)$. Since a path can never be shorter than the shortest path ($\delta(s, v)$), $v.d$ must be exactly equal to $\delta(s, v)$, which again contradicts our assumption.

Hence, BFS correctly computes $v.d = \delta(s, v)$ for all reachable vertices. (Proved)

---

2. Does $u.d$ depend on the adjacency list order?

No, $u.d$ does not depend on the order. According to the correctness theorem, upon termination, $u.d$ strictly equals $\delta(s, u)$. The shortest-path distance $\delta(s, u)$ is an inherent mathematical property of the graph G and the source s. It represents the minimum number of edges required to reach u from s. Because the topological structure of the graph does not change regardless of how the edges are stored or ordered in the adjacency list, the value of $\delta(s, u)$, and consequently $u.d$, remains absolutely fixed.

---

3. Does the BFS tree depend on the adjacency list order?

Yes, the BFS tree completely depends on the ordering. When multiple shortest paths exist to a vertex u, the parent $u.\pi$ (which forms the tree edges) is determined by whichever path discovers u *first*. This discovery order is directly controlled by the adjacency list ordering.

Demonstration with the given graph: Let us start BFS from source vertex **s**. Notice that s connects to w. Vertex w connects to t and x. Both t and x connect to u. The distances will be: $w.d = 1$, $t.d = 2$, $x.d = 2$, and $u.d = 3$. Let's see what happens to the parent of u ($u.\pi$) under two different orderings of $Adj[w]$:

Case 1: $Adj[w] = [t, x]$
- When w is dequeued, t is enqueued before x.
- The queue state becomes: $[\ldots, t, x]$.
- t is dequeued before x. Since $u \in Adj[t]$ and u is undiscovered (white), t discovers u.
- **Result:** $u.\pi = \mathbf{t}$

Case 2: $Adj[w] = [x, t]$
- When w is dequeued, x is enqueued before t.
- The queue state becomes: $[\ldots, x, t]$.
- x is dequeued before t. Since $u \in Adj[x]$ and u is undiscovered, x discovers u.
- **Result:** $u.\pi = \mathbf{x}$

Conclusion: Because the parent pointers (π) differ based on the adjacency list order, the resulting BFS trees are structurally different, even though the source s and all distance values (d) remain exactly the same.

Q4. "It is asymptotically faster to square an n-bit integer than to multiply two n-bit integers." Do you agree with this statement? Justify your answer. **(10 marks)** **[CSE 203 | 2021–22]**

Answer

Disagree. Squaring and general multiplication are asymptotically equivalent.
Squaring $\leq$ Multiplication: $A^2 = A \times A$, so $T_{\text{sq}}(n) \leq T_{\text{mult}}(n)$.
Multiplication $\leq$ Squaring: Using the identity $A \times B = \frac{(A+B)^2-(A-B)^2}{4}$, any multiplication can be reduced to a constant number of squarings plus $O(n)$ additions. So $T_{\text{mult}}(n) = O(T_{\text{sq}}(n))$.
Combining: $T_{\text{sq}}(n) = \Theta(T_{\text{mult}}(n))$. They are asymptotically the same complexity class.

Q5. Using the substitution method, solve the following recurrence:

$$T(n) = 4T\left(\frac{n}{2}\right) + 100n^2$$

(11 marks) **[CSE 203 | 2020–21]**

Answer

Guess: $T(n) = O(n^2 \log n)$. We guess $T(n) \leq cn^2 \log n$ for some constant $c > 0$.
Inductive step: Assume $T(k) \leq ck^2 \log k$ for all $k < n$. Then:

$$\begin{aligned} T(n) &= 4T(n/2) + 100n^2 \\ &\leq 4 \cdot c(n/2)^2 \log(n/2) + 100n^2 \\ &= 4 \cdot \frac{cn^2}{4}(\log n - 1) + 100n^2 \\ &= cn^2 \log n - cn^2 + 100n^2 \\ &= cn^2 \log n + (100 - c)n^2 \end{aligned}$$

For $c \geq 100$: $(100 - c)n^2 \leq 0$, so $T(n) \leq cn^2 \log n$. ✓
Lower bound: By symmetric argument, $T(n) = \Omega(n^2 \log n)$.
Conclusion: $T(n) = \Theta(n^2 \log n)$.
Note: This aligns with Master Theorem Case 2 since $n^{\log_2 4} = n^2$ and $f(n) = 100n^2 = \Theta(n^2)$.

Q6. Explain whether we can apply the Master Method to any recurrence of the form $T(n) = aT(n/b) + f(n)$. If not, explain a case where the theorem cannot be applied. **(8 marks)** **[CSE 203 | 2019–20]**

Answer

No, the Master Method cannot be applied to all recurrences of the form $T(n) = aT(n/b) + f(n)$.
The Master Theorem requires $a \geq 1$, $b > 1$, and that $f(n)$ be asymptotically *comparable* to $n^{\log_b a}$ in one of three specific ways. It fails when there is a **gap** between $f(n)$ and $n^{\log_b a}$.
Example where it fails: $T(n) = 2T(n/2) + n \log n$. Here $n^{\log_2 2} = n$ and $f(n) = n \log n$. $f(n)/n^{\log_b a} = \log n$, which is not a polynomial factor. $f(n) = \Omega(n^{1+\varepsilon})$ is not satisfied for any $\varepsilon > 0$. $f(n) = \Theta(n)$ is also not satisfied. So none of the three cases apply. (Solution is $T(n) = \Theta(n \log^2 n)$ via the Akra-Bazzi method or extended Master Theorem.)

Q7. Traditional insertion sort uses a linear search to scan (backward) through the sorted subarray $A[1 \ldots j-1]$, where $A[j]$ is the element to be inserted. Prove or disprove the claim that we can use binary search instead to improve the overall worst-case running time of insertion sort to $\Theta(n \log n)$. **(8 marks)** **[CSE 203 | 2019–20]**

Answer

Claim is FALSE. Using binary search reduces the number of *comparisons* to $O(\log j)$ for the j-th insertion, giving $\sum_{j=2}^{n} O(\log j) = O(n \log n)$ comparisons total. However, Insertion Sort's bottleneck in the worst case is **element shifting (swaps)**, not comparisons.

After finding the correct position via binary search, we still need to shift all larger elements one position right to make room. In the worst case (reverse-sorted input), inserting element j requires shifting $j-1$ elements. Total shifts: $\sum_{j=2}^{n}(j-1) = \frac{n(n-1)}{2} = \Theta(n^2)$. Therefore, the worst-case running time of Insertion Sort with binary search is still $\Theta(n^2)$. The claim is **disproved**.

Q8. Prove or disprove the following: if we choose the element at the middle index as pivot during partition in Quicksort, the algorithm will never have a runtime of $\Theta(n^2)$. **(10+8 marks)** [CSE 203 | 2019–20]

Answer

Claim is FALSE. The middle-index pivot does *not* guarantee $O(n \log n)$.
Counterexample: Consider the array that causes worst-case behaviour for middle-index pivot. One such family: the "median-of-three killer" sequence. For example, for $n = 8$, the array $[1, 5, 3, 7, 2, 6, 4, 8]$ constructed so that middle-index selection always picks the maximum or minimum of the current subarray.
General construction: For any deterministic pivot-selection rule (including middle index), there exists an adversarial permutation that causes the partition to always split as $(n-1, 0)$ or $(0, n-1)$. This gives the recurrence $T(n) = T(n-1) + \Theta(n)$, which solves to $T(n) = \Theta(n^2)$.
Therefore middle-index pivot selection still admits $\Theta(n^2)$ worst-case for adversarially constructed inputs. Only **randomised** pivot selection avoids this with high probability.

Q9. Consider a divide-and-conquer algorithm: if the problem size is ≤ 5, it solves the problem in constant time; otherwise, it divides the problem of size n into 4 subproblems each of size $n/3$, solves them recursively, and merges in $O(n)$ time. The division itself takes $O(\sqrt{n} \log n)$ time. Write the recurrence relation for $T(n)$ and solve the following recurrences using the Master Theorem:

(a) $T(n) = 9T(n/3) + 5n + n^2 + 8 \log n$

(b) $T(n) = 3T(n/4) + 3n^2 + 4n + 5$

(c) $T(n) = 4T(n/2) + 5n^3 + 7n$

(10 marks) [CSE 203 | 2018–19]

Answer

Recurrence: $T(n) = 4T(n/3) + O(\sqrt{n} \log n) + O(n) = 4T(n/3) + O(n)$ (since n dominates $\sqrt{n} \log n$).
$n^{\log_3 4} \approx n^{1.26}$. $f(n) = O(n) = O(n^{1.26-0.26})$, Case 1: $T(n) = \Theta(n^{\log_3 4})$.
Sub-recurrences:

(a) $T(n) = 9T(n/3) + 5n + n^2 + 8 \log n$. Dominant $f(n) = n^2$. $n^{\log_3 9} = n^2$. $f(n) = \Theta(n^2)$, Case 2: $T(n) = \Theta(n^2 \log n)$.

(b) $T(n) = 3T(n/4) + 3n^2 + 4n + 5$. $f(n) = 3n^2$. $n^{\log_4 3} \approx n^{0.79}$. $n^2 = \Omega(n^{0.79+\varepsilon})$, regularity holds, Case 3: $T(n) = \Theta(n^2)$.

(c) $T(n) = 4T(n/2) + 5n^3 + 7n$. $f(n) = 5n^3$. $n^{\log_2 4} = n^2$. $n^3 = \Omega(n^{2+1})$, regularity: $4(n/2)^3 = n^3/2 \leq cn^3$, Case 3: $T(n) = \Theta(n^3)$.

Q10. Suppose you are choosing among three algorithms for multiplying two $n \times n$ matrices:

(a) Algorithm A (Strassen's) divides each $n \times n$ matrix into four $n/2 \times n/2$ matrices, makes seven recursive calls, and combines them in $\Theta(n^2)$ time.

(b) Algorithm B divides each matrix into twelve $n/3 \times n/3$ matrices, makes nine recursive calls, and combines them in $\Theta(n^2)$ time.

(c) Algorithm C first multiplies two $(n-1) \times (n-1)$ matrices, then combines the result with two vectors of size n in $\Theta(n^2)$ time.

Determine the asymptotic running time of each algorithm (use the Master Theorem where applicable) and determine which algorithm you would choose. **(15 marks)** [CSE 203 | 2016–17]

Answer

(a) **Algorithm A (7 calls, $n/2$, combine $\Theta(n^2)$):** $T(n) = 7T(n/2) + \Theta(n^2)$. $n^{\log_2 7} \approx n^{2.807}$. Since $n^2 = O(n^{2.807-\varepsilon})$, Case 1: $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.

(b) **Algorithm B (9 calls, $n/3$):** $T(n) = 9T(n/3) + \Theta(n^2)$. $n^{\log_3 9} = n^2$. $f(n) = \Theta(n^2) = \Theta(n^{\log_3 9})$, Case 2: $T(n) = \Theta(n^2 \log n)$.

(c) **Algorithm C ($T(n-1)$ once, combine $\Theta(n^2)$):** $T(n) = T(n-1) + \Theta(n^2)$. Master Theorem does not apply (not of form $T(n/b)$). Unrolling: $T(n) = \sum_{k=1}^{n} k^2 = \Theta(n^3)$.

Best choice: Algorithm A with $\Theta(n^{2.807})$ (Strassen's), faster than $\Theta(n^3)$ naively and $\Theta(n^3)$ of C.

Q11. Suppose you are choosing among three algorithms for multiplying two $n \times n$ matrices:

(a) Algorithm A divides each $n \times n$ matrix into four $n/2 \times n/2$ matrices, makes 8 recursive calls, and combines in $\Theta(n^2)$ time.

(b) Algorithm B (Strassen's) divides each $n \times n$ matrix into four $n/2 \times n/2$ matrices, makes 7 recursive calls, and combines in $\Theta(n^2)$ time.

(c) Algorithm C divides each $n \times n$ matrix into nine $n/3 \times n/3$ matrices, makes 9 recursive calls, and combines in $\Theta(n^2)$ time.

Use the Master Theorem to determine asymptotic running times of all three algorithms and choose the best one. **(15 marks)** [CSE 207 | 2015–16]

Answer

(a) $T(n) = 8T(n/2) + \Theta(n^2)$: $n^{\log_2 8} = n^3$. $n^2 = O(n^{3-1})$ (Case 1): $T(n) = \Theta(n^3)$.

(b) $T(n) = 7T(n/2) + \Theta(n^2)$: $n^{\log_2 7} \approx n^{2.807}$. Case 1: $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$.

(c) $T(n) = 9T(n/3) + \Theta(n^2)$: $n^{\log_3 9} = n^2$. Case 2: $T(n) = \Theta(n^2 \log n)$.

Best: Algorithm B (Strassen's) at $\Theta(n^{2.807})$ — faster than n^3 and $n^2 \log n$.

Q12. State the Master Theorem. Explain why the Master Method cannot be used to solve the recurrence $T(n) = 2T(n/2) + n \log n$. Using the substitution method, prove that $T(n)$ is $O(n \log^2 n)$. **(6+6+6 marks)** [CSE 207 | 2014–15]

Answer

Master Theorem: For $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$:

Case 1: If $f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$: $T(n) = \Theta(n^{\log_b a})$.

Case 2: If $f(n) = \Theta(n^{\log_b a})$: $T(n) = \Theta(n^{\log_b a} \log n)$.

Case 3: If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ and regularity holds: $T(n) = \Theta(f(n))$.

Why Master Method fails for $T(n) = 2T(n/2) + n \log n$: $n^{\log_2 2} = n$. $f(n) = n \log n$. For Case 2: $n \log n \neq \Theta(n)$. For Case 3: we need $n \log n = \Omega(n^{1+\varepsilon})$, but $\log n$ is not $\Omega(n^\varepsilon)$ for any $\varepsilon > 0$. The ratio $f(n)/n^{\log_b a} = \log n$ is not polynomial. None of the three cases apply.

Substitution proof that $T(n) = O(n \log^2 n)$: Guess $T(n) \leq cn \log^2 n$. Assume true for $n/2$:

$$\begin{aligned} T(n) &\leq 2c\frac{n}{2}\log^2\frac{n}{2} + n\log n = cn(\log n - 1)^2 + n\log n \\ &= cn\log^2 n - 2cn\log n + cn + n\log n \\ &= cn\log^2 n + n\log n(1-2c) + cn \\ &\leq cn\log^2 n \quad \text{for } c \geq 1 \text{ and large } n \end{aligned}$$

Hence $T(n) = O(n \log^2 n)$. □

Q13. Using the Master Method, prove that the recurrence $T(n) = 3T(n/4) + n \lg n$ solves to $T(n) = \Theta(n \lg n)$. **(12 marks)** [CSE 207 | 2009–10]

Answer

For $T(n) = 3T(n/4) + n \lg n$: $a = 3$, $b = 4$, $f(n) = n \lg n$.
$n^{\log_b a} = n^{\log_4 3} = n^{0.792\ldots}$.
Since $f(n) = n \lg n = \Omega(n^{0.792+\varepsilon})$ for $\varepsilon \leq 0.208$ (as $n \lg n = \Omega(n^{0.9})$ for example since $\lg n = \Omega(n^{0.1})$ is false…let's be precise):
$f(n)/n^{\log_4 3} = n^{1-\log_4 3} \cdot \lg n = n^{0.208} \cdot \lg n \to \infty$. So $f(n) = \Omega(n^{\log_4 3 + \varepsilon})$ for $\varepsilon = 0.1$.
Regularity: $af(n/b) = 3 \cdot \frac{n}{4} \lg \frac{n}{4} = \frac{3n}{4}(\lg n - 2) \leq \frac{3}{4} n \lg n = cf(n)$ for $c = 3/4 < 1$. ✓
Case 3 applies: $T(n) = \Theta(f(n)) = \Theta(n \lg n)$. □

Q14. Write the pseudocode of Strassen's algorithm for multiplying two $n \times n$ matrices. Derive the running time. **(8+10=18 marks)** [CSE 207 | 2009–10]

Answer

Algorithm 9 STRASSEN(A, B, n)

1: **if** $n = 1$ **then**
2: **return** $A[1][1] \times B[1][1]$
3: **end if**
4: $A_{11}, A_{12}, A_{21}, A_{22} \leftarrow$ PARTITION(A)
5: $B_{11}, B_{12}, B_{21}, B_{22} \leftarrow$ PARTITION(B)
6: $M_1 \leftarrow$ STRASSEN($A_{11} + A_{22},\ B_{11} + B_{22},\ n/2$)
7: $M_2 \leftarrow$ STRASSEN($A_{21} + A_{22},\ B_{11},\ n/2$)
8: $M_3 \leftarrow$ STRASSEN($A_{11},\ B_{12} - B_{22},\ n/2$)
9: $M_4 \leftarrow$ STRASSEN($A_{22},\ B_{21} - B_{11},\ n/2$)
10: $M_5 \leftarrow$ STRASSEN($A_{11} + A_{12},\ B_{22},\ n/2$)
11: $M_6 \leftarrow$ STRASSEN($A_{21} - A_{11},\ B_{11} + B_{12},\ n/2$)
12: $M_7 \leftarrow$ STRASSEN($A_{12} - A_{22},\ B_{21} + B_{22},\ n/2$)
13: $C_{11} \leftarrow M_1 + M_4 - M_5 + M_7$
14: $C_{12} \leftarrow M_3 + M_5$
15: $C_{21} \leftarrow M_2 + M_4$
16: $C_{22} \leftarrow M_1 - M_2 + M_3 + M_6$
17: **return** ASSEMBLE($C_{11}, C_{12}, C_{21}, C_{22}$)

Recurrence: $T(n) = 7T(n/2) + \Theta(n^2)$. $n^{\log_2 7} \approx n^{2.807}$. Case 1 of Master Theorem: $T(n) = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$, improving on naive $\Theta(n^3)$.

BUET · CSE 105

DSA

Arrays, Linked Lists, Stacks & Queues

Linear Data Structures, Implementation, Operations

T4 — Arrays, Linked Lists, Stacks & Queues

Q1. You are asked to merge two singly linked lists whose elements are sorted in ascending order. The output list will also be sorted in ascending order. Sketch an algorithm to solve the problem. Analyze that your algorithm runs in linear time on the number of elements of the output list. **(15 marks)** [CSE 105 | 2023–24]

Answer — Merge Two Sorted Linked Lists

Idea. Maintain a pointer into each list; at each step, advance the pointer pointing to the smaller current node and append that node to the output list.

Algorithm 10 MergeSorted(L_1, L_2)

```
Create a dummy head node; tail ← dummy
p1 ← L1.head;   p2 ← L2.head
while p1 ≠ null and p2 ≠ null do
    if p1.data ≤ p2.data then
        tail.next ← p1
        p1 ← p1.next
    else
        tail.next ← p2
        p2 ← p2.next
    end if
    tail ← tail.next
end while
if p1 ≠ null then
    tail.next ← p1
end if
if p2 ≠ null then
    tail.next ← p2
end if
return dummy.next
```

Trace ($L_1 = [1, 3, 5, 7]$, $L_2 = [2, 4, 6, 8, 9]$):

Step	p_1	p_2	Output so far
1	1	2	1
2	3	2	$1 \to 2$
3	3	4	$1 \to 2 \to 3$
4	5	4	$1 \to 2 \to 3 \to 4$
5	5	6	$1 \to 2 \to 3 \to 4 \to 5$
6	7	6	$1 \to 2 \to 3 \to 4 \to 5 \to 6$
7	7	8	$1 \to 2 \to 3 \to 4 \to 5 \to 6 \to 7$
attach	NULL	$8 \to 9$	$1 \to 2 \to \cdots \to 7 \to 8 \to 9$

Correctness. At each step the smallest remaining element across both lists is appended, preserving sorted order. When one list empties the other is already sorted, so appending its tail is safe.

Time complexity analysis.
Let $m = |L_1|$, $n = |L_2|$, and $N = m + n$ = size of the output list.

- Each iteration of the `while` loop appends *exactly one* node to the output and advances *exactly one* pointer.
- The loop runs at most $m + n - 1$ times before one list empties; then the tail-attach takes $O(1)$.
- Total work $= O(m + n) = O(N)$.

$$\boxed{T(m, n) = O(m + n) = O(N).}$$

The algorithm is **linear in the size of the output list**. ✓

Q2. You are given two linked lists of unsorted elements. Investigate whether you can develop a constant time algorithm to concatenate them if both of the lists are (i) singly linked lists (SLL), (ii) doubly linked list (DLL), (iii) circular SLL, (iv) circular DLL. If yes, sketch the algorithm and examine its correctness; otherwise demonstrate why a constant time algorithm is not possible. **(20 marks)** [CSE 105 | 2023–24]

Answer — Constant-Time Concatenation of Two Lists

Key insight. Concatenation is $O(1)$ whenever we can reach the *last node of the first list* and the *first node of the second list* in $O(1)$.

(i) Singly Linked List (SLL).

Without a tail pointer: We must traverse L_1 from `head1` to its last node — $O(n)$. **Constant time is not possible.**
With a tail pointer: $O(1)$ is achievable.

Algorithm 11 CONCATENATE_SLL(L_1, L_2)

1: $tail_1.next \leftarrow head_2$
2: $tail_1 \leftarrow tail_2$

Correctness: `tail1.next` was `NULL`; now points to `head2`.

(ii) Doubly Linked List (DLL).
Without a tail pointer: Similar to SLL, finding the last node takes $O(n)$. **Constant time is not possible.**
With a tail pointer: $O(1)$ is achievable.

Algorithm 12 CONCATENATE_DLL(L_1, L_2)

1: $tail_1.next \leftarrow head_2$
2: $head_2.prev \leftarrow tail_1$
3: $tail_1 \leftarrow tail_2$

Correctness: The bidirectional chain is restored at the join point.

(iii) Circular SLL.
In a circular SLL, we typically maintain only a **tail pointer** because `tail.next` gives us the head in $O(1)$. Thus, $O(1)$ concatenation is possible.

Algorithm 13 CONCATENATE_CSLL(L_1, L_2)

1: $head_1 \leftarrow tail_1.next$
2: $tail_1.next \leftarrow tail_2.next$
3: $tail_2.next \leftarrow head_1$

Correctness: The new circle is: `head1` $\rightarrow \cdots \rightarrow$ `tail1` $\rightarrow$ `head2` $\rightarrow \cdots \rightarrow$ `tail2` $\rightarrow$ `head1`.

(iv) Circular DLL.
Constant time is ALWAYS possible, even with just a `head` pointer! In a CDLL, the tail is simply `head.prev`. We do not need to maintain a separate tail pointer.

Algorithm 14 CONCATENATE_CDLL(L_1, L_2)

1: $tail_1 \leftarrow head_1.prev$
2: $tail_2 \leftarrow head_2.prev$
3: $tail_1.next \leftarrow head_2$
4: $head_2.prev \leftarrow tail_1$
5: $tail_2.next \leftarrow head_1$
6: $head_1.prev \leftarrow tail_2$

Correctness: All four pointers at the join and closure points are correctly updated, preserving the circular doubly-linked property.

List Type	$O(1)$ possible?	Key requirement
SLL	**No** (w/o tail) / **Yes** (w/ tail)	Tail pointer needed
DLL	**No** (w/o tail) / **Yes** (w/ tail)	Tail pointer needed
Circular SLL	**Yes**	Tail pointer (gives head via `tail.next`)
Circular DLL	**Yes**	Just `head` pointer (gives tail via `head.prev`)

Q3. Assume that a string contains only left and right parentheses which may or may not be balanced and properly nested. For example, the string ``(((()))()'' contains properly nested pairs of parentheses, but the string ``)()('' does not, and the string ``())'' does not contain properly matching parentheses. Using a stack, construct an algorithm that returns `true` if a string contains properly nested and balanced parentheses, and `false` otherwise. Modify the algorithm so that it returns the position in the string of the first offending parenthesis if the string is not properly nested and balanced — if an excess right parenthesis is found, return its position; if there are too many left parentheses, return the position of the first excess left parenthesis. Return -1 if the string is properly balanced and nested. Assume indices start from 0. **(25 marks)** [CSE 105 | 2023–24]

Answer — Balanced Parentheses via Stack

Part A: Return `true`/`false`.

Algorithm 15 isBalanced(s)

```
1:  stack ← empty stack
2:  for each character ch in s do
3:      if ch = '(' then
4:          stack.Push(ch)
5:      else
6:          if stack.isEmpty() then
7:              return false
8:          end if
9:          stack.Pop()
10:     end if
11: end for
12: return stack.isEmpty()
```

Part B: Return position of first offending parenthesis (or -1).
Key insight: Push the *index* (not the character) so we can report positions.

Algorithm 16 checkPosition(s)

```
1:  stack ← empty stack
2:  for i = 0 to len(s) − 1 do
3:      ch ← s[i]
4:      if ch = '(' then
5:          stack.Push(i)
6:      else
7:          if stack.isEmpty() then
8:              return i
9:          end if
10:         stack.Pop()
11:     end if
12: end for
13: if stack is not empty then
14:     return stack.Bottom()
15: end if
16: return −1
```

Why `stack.bottom()`? Unmatched `'(' accumulate bottom-up in push order. The *first* excess left parenthesis is the one pushed earliest, i.e., at the bottom. In practice, use an auxiliary variable `firstLeft` updated only when the stack was empty before this push.

Trace on `"((()())()"` (indices 0–8):

i	`ch`	Action	Stack (indices)
0	(	push 0	[0]
1	(	push 1	[0,1]
2	(	push 2	[0,1,2]
3	)	pop 2	[0,1]
4	(	push 4	[0,1,4]
5	)	pop 4	[0,1]
6	)	pop 1	[0]
7	(	push 7	[0,7]
8	)	pop 7	[0]
End		stack not empty	bottom = 0

Return 0 (the `'(' at index 0 is the first excess left parenthesis). ✓

Test cases:

String	Result	Reason
`"((()())()"`	0	Excess `'(' at position 0
`")()("`	0	Excess `')' at position 0
`"())"`	2	Excess `')' at position 2
`"(()())()"`	-1	Balanced

Time complexity: $O(n)$ — each character processed exactly once. **Space complexity:** $O(n)$ — stack holds at most $n/2$ indices.

Q4. Given an integer k and a queue of integers, how do you reverse the order of the first k elements of the queue, leaving the other elements in the same relative order? For example, if $k = 4$ and the queue has the elements $[10, 20, 30, 40, 50, 60, 70, 80, 90]$, the output should be $[40, 30, 20, 10, 50, 60, 70, 80, 90]$. **(10 marks)** **[CSE 105 | 2023–24]**

Answer

Idea: Use an auxiliary stack to reverse the first k elements, then re-enqueue them. The remaining $n-k$ elements are rotated to the back and then brought to the front.

Algorithm 17 REVERSEFIRSTK(Q, k)

```
S ← empty stack
for i = 1 to k do
    S.PUSH(Q.DEQUEUE())
end for
while S is not empty do
    Q.ENQUEUE(S.POP())
end while
for i = 1 to Q.size − k do
    Q.ENQUEUE(Q.DEQUEUE())
end for
return Q
```

Trace ($k = 4$, queue $= [10, 20, 30, 40, 50, 60, 70, 80, 90]$):

(a) Dequeue 4 elements → Stack: $[40, 30, 20, 10]$ (top=40). Queue: $[50, 60, 70, 80, 90]$

(b) Pop stack back to queue → Queue: $[50, 60, 70, 80, 90, 40, 30, 20, 10]$

(c) Rotate remaining $9 - 4 = 5$ elements: move $50, 60, 70, 80, 90$ to back
→ Queue: $[40, 30, 20, 10, 50, 60, 70, 80, 90]$ ✓

Time complexity: $O(n)$ where n is the total number of elements.
Space complexity: $O(k)$ for the auxiliary stack.

Q5. Implement a Circular Queue using an array. Include: `enqueue(x)`, `dequeue()`, `is_empty()`, `is_full()`, `front()`. **(15 marks)** [CSE 105 | 2022–23]

Answer

Representation. Use an array `arr[0..capacity-1]`, two indices `front` and `rear`, and a size counter `sz`. Indices wrap around modulo `capacity`.

```
class CircularQueue {
    int *arr;
    int front, rear, capacity, sz;

public:
    CircularQueue(int cap)
        : capacity(cap), front(0), rear(-1), sz(0) {
        arr = new int[capacity];
    }

    bool is_empty() { return sz == 0; }

    bool is_full()  { return sz == capacity; }

    // Add element x at the rear
    void enqueue(int x) {
        if (is_full()) {
            // handle overflow (resize or error)
            return;
        }
        rear = (rear + 1) % capacity;
        arr[rear] = x;
        sz++;
    }

    // Remove and return front element
    int dequeue() {
        if (is_empty()) return -1;   // underflow
        int val = arr[front];
        front = (front + 1) % capacity;
        sz--;
        return val;
    }

    // Return front element without removing
    int front_elem() {
```

```
        if (is_empty()) return -1;
        return arr[front];
    }

    ~CircularQueue() { delete[] arr; }
};
```

Why circular? Without wraparound, after several enqueue/dequeue pairs the `rear` pointer would fall off the end of the array even though slots near the front are free. The modulo arithmetic reuses those slots, giving a *drifting-free* implementation.

Complexity Analysis. Every operation is $\boldsymbol{O(1)}$ — no loops, no shifts.

Q6. Write a function to delete a node with a given key from a doubly linked list (unique keys, access to both `head` and `tail`). **(10 marks)**
[CSE 105 | 2022–23]

Answer

```
struct Node {
    int  key;
    Node *prev, *next;
};

void deleteNode(Node* &head, Node* &tail, int key) {
    // Search for the node
    Node* cur = head;
    while (cur != nullptr && cur->key != key)
        cur = cur->next;

    if (cur == nullptr) return;   // key not found

    // Relink previous node
    if (cur->prev != nullptr)
        cur->prev->next = cur->next;
    else
        head = cur->next;         // deleting head

    // Relink next node
    if (cur->next != nullptr)
        cur->next->prev = cur->prev;
    else
        tail = cur->prev;         // deleting tail

    delete cur;
}
```

Cases handled:

(a) **Middle node** — both `prev` and `next` relinked.

(b) **Head node** (`cur->prev == nullptr`) — `head` updated to `cur->next`.

(c) **Tail node** (`cur->next == nullptr`) — `tail` updated to `cur->prev`.

(d) **Only node** — both `head` and `tail` become `nullptr`.

Complexity Analysis. $O(n)$ for the search; the relinking itself is $O(1)$.

Q7. How would you reverse a singly linked list (without duplicating the list)? Write the steps (pseudocode) conceptually. **(4 marks)**
[CSE 105 | 2022–23]

Answer

Idea. Traverse the list once, reversing each `next` pointer in place using three pointers: `prev`, `cur`, `nxt`.

Algorithm 18 REVERSE(*head*)

```
prev ← nil
cur ← head
while cur ≠ nil do
    nxt ← cur.next                ▷ save next
    cur.next ← prev               ▷ reverse link
    prev ← cur                    ▷ advance prev
    cur ← nxt                     ▷ advance cur
end while
return prev                       ▷ new head
```

Trace on `1->2->3->null`:

Step	`prev`	`cur`	list so far
init	`null`	1	1→2→3
iter 1	1	2	1→`null`, 2→3
iter 2	2	3	2→1→`null`, 3
iter 3	3	`null`	3→2→1→`null`

Complexity Analysis. $O(n)$ time, $O(1)$ extra space — no new nodes allocated.

Q8. How can you remove duplicate values from a sorted singly linked list? **(5 marks)** [CSE 105 | 2022–23]

Answer

Because the list is *sorted*, all duplicates of any value are adjacent. One pass suffices.

```
void removeDuplicates(Node* head) {
    Node* cur = head;
    while (cur != nullptr && cur->next != nullptr) {
        if (cur->val == cur->next->val) {
            Node* dup  = cur->next;
            cur->next  = dup->next;  // skip duplicate
            delete dup;
            // do NOT advance cur: next node might also be duplicate
        } else {
            cur = cur->next;
        }
    }
}
```

Example. `1->1->2->3->3->3->4` ⟶ `1->2->3->4`

Complexity Analysis. $O(n)$ time (single pass), $O(1)$ extra space.

Q9. How can you check if a singly linked list is a palindrome? Write the steps involved conceptually. **(10 marks)** [CSE 105 | 2022–23]

Answer

Method (Stack-based, $O(n)$ time, $O(n)$ space).

Algorithm 19 IS-PALINDROME(*head*)

```
S ← empty stack
cur ← head
while cur ≠ nil do                    ▷ push all values
    PUSH(S, cur.val)
    cur ← cur.next
end while
cur ← head
while cur ≠ nil do                    ▷ compare with list
    if cur.val ≠ POP(S) then
        return false
    end if
    cur ← cur.next
end while
return true
```

Alternative ($O(1)$ space).

(a) Find the midpoint using slow/fast pointers.

(b) Reverse the second half of the list in place.

(c) Compare first half with reversed second half node by node.

(d) Restore the second half (optional).

Example. `1->2->3->2->1`: stack gives `[1,2,3,2,1]` (top=1). Comparing forward `1,2,3,2,1` with pops `1,2,3,2,1` — all match ⇒ palindrome.

Complexity Analysis. Stack-based: $O(n)$ time, $O(n)$ space.
Two-pointer + in-place reverse: $O(n)$ time, $O(1)$ space.

Q10. Given an array of integers (with both negative and positive numbers), write a program to find the K-th largest sum of a contiguous subarray within the array. **(10 marks)** [CSE 203 | 2021–22]

Answer

Approach: Enumerate all $O(n^2)$ contiguous subarray sums, maintain a min-heap of size K to track the K largest.

Algorithm 20 KthLargestSubarraySum(A, n, K)

```
minHeap ← empty min-heap of capacity K
for i = 0 to n − 1 do
    sum ← 0
    for j = i to n − 1 do
        sum ← sum + A[j]                          ▷ sum of A[i .. j]
        if |minHeap| < K then
            Push(minHeap, sum)
        else if sum > Top(minHeap) then
            Pop(minHeap)
            Push(minHeap, sum)
        end if
    end for
end for
return Top(minHeap)                               ▷ K-th largest sum
```

Correctness: The min-heap always holds the K largest sums seen so far. After processing all $\binom{n}{2} + n$ subarray sums, the top of the heap is the K-th largest.
Complexity:

- $O(n^2)$ subarrays, each heap operation is $O(\log K)$.
- Total: $O(n^2 \log K)$.

Q11. Suppose the numbers $0, 1, 2, \ldots, 9$ were pushed onto a stack in that order, but pops occurred at random points between the various pushes. Determine which of the following two sequences is a valid pop sequence and explain why:

$$S_1 : 3, 2, 6, 5, 7, 4, 1, 0, 9, 8$$

$$S_2 : 3, 2, 6, 4, 7, 5, 1, 0, 9, 8$$

(3 marks) [CSE 105 | 2021–22]

Answer

S_1 is VALID. S_2 is INVALID.

Simulation of S_1:

Action	Stack (bottom→top)	Popped
push 0,1,2,3	$[0, 1, 2, 3]$	—
pop 3	$[0, 1, 2]$	3
pop 2	$[0, 1]$	2
push 4,5,6	$[0, 1, 4, 5, 6]$	—
pop 6	$[0, 1, 4, 5]$	6
pop 5	$[0, 1, 4]$	5
push 7	$[0, 1, 4, 7]$	—
pop 7	$[0, 1, 4]$	7
pop 4	$[0, 1]$	4
pop 1	$[0]$	1
pop 0	$[\,]$	0
push 8,9	$[8, 9]$	—
pop 9	$[8]$	9
pop 8	$[\,]$	8

Output: $3, 2, 6, 5, 7, 4, 1, 0, 9, 8 = S_1$ ✓

Why S_2 is INVALID:
$S_2 = 3, 2, 6, \mathbf{4}, 7, 5, \ldots$
After popping $3, 2$: stack $= [0, 1]$. To pop 6, we must push $4, 5, 6$ first, giving stack $= [0, 1, 4, 5, 6]$. Pop 6: stack $= [0, 1, 4, 5]$. Next in S_2 is **4**, but the top of the stack is **5**. We cannot pop 4 without first popping 5, and 5 has not appeared yet in the sequence before 4. Since we can only push in increasing order $0, 1, 2, \ldots$, there is no way to get 4 on top while 5 is still in the stack. Hence S_2 is invalid.

Q12. A stack is being used to parse matching parentheses, brackets, and braces — `(`, `[`, and `{`. Show the state of the stack at the end of the given code fragment. **(10 marks)** [CSE 105 | 2021–22]

```
#include <iostream>
using namespace std;

int main()
{
    const int M = 30;
    int N = 1024;
    int parent[N], height[N], depth[N];
    int maxheights[100];
    for (int i = 0; 1 < 100; ++i)
    {
        maxheights[i] = 0;
    }
    double minmeandepth = 1e300;
    double meandepths = 0;
    double maxmeandepth = 0;
    for (int j = 0; j < M; ++j)
    {
    for (int i = 0; i < N; ++i)
    {
        parent[i] = -1;
        height[i] = 0;
    }
    for (int i = 0; i < N - 1; ++i)
    while (true)
    {
    int p1 = rand() & (N - 1);
    int p2 = rand() & (N - 1);

    int s1 = pi1;
    int s2 = p2;

    while ( parent[
```

Answer

We scan the code fragment left-to-right, pushing every opening bracket/brace/paren and popping on every matching closer. The code is *truncated* midway (ends at `while ( parent[`), so we track every unmatched opener still on the stack at the point the fragment ends.

Step-by-step trace of unmatched openers:

Token encountered	Action	Stack (bottom → top)
`{` after `main()`	push `{`	`{`
`{` after first `for`	push `{`	`{ {`
`}` closes inner for-body	pop	`{`
`{` after second `for (j...)`	push `{`	`{ {`
`{` after `for (i=0; i<N)`	push `{`	`{ { {`
`}` closes that for-body	pop	`{ {`
`{` after `while (true)`	push `{`	`{ { {`
`(` in `while ( parent[`	push `(`	`{ { { (`
`[` in `parent[`	push `[`	`{ { { ( [`

Final stack state (bottom to top):

`[`	← top
`(`	
`{`	
`{`	
`{`	← bottom

The stack contains 5 unmatched openers: `{`, `{`, `{`, `(`, `[` (bottom to top), indicating the code fragment is incomplete/unbalanced at the point it was cut off.

Q13. You are given access to the following functions of a list data structure:

- `init(n)` — Initialize an array $L[0..n-1]$ of length n for integers. Current position 'points to' $L[0]$. All the functions below are applied on this array which is not accessible directly.
- `insert(item)` — Inserts an element at the current position and current position is 'incremented' (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k+1]$). Here, *item* is the element to be inserted. In case the list is full, it returns ERR (a large negative constant).
- `remove()` — Removes the element at the current position and current position is 'decremented' (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k-1]$). In case the list is empty, it returns ERR (a large negative constant).
- `value()` — Returns the value of the element at the current position. Current position remains unchanged. In case the list is empty, it returns ERR (a large negative constant).
- `moveToStart()` — Sets the current position to $L[0]$.
- `moveToEnd()` — Sets the current position to $L[n-1]$.
- `next()` — Increments the current position (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k+1]$). If $k = n-1$, it returns ERR (a large negative constant).
- `prev()` — Decrements the current position (so if current position pointed to $L[k]$ before the call, after the call it points to $L[k-1]$). If $k = 0$, it returns ERR (a large negative constant).

Now you have to build one queue and one stack in one List data structure as defined above. The queue should grow from the beginning of the list (i.e., first element of the queue should be at $L[0]$) and the stack should grow from the end of the list (i.e., the first element of the stack should be at $L[n-1]$). You have no access to the code of the above implementation and can only call these functions in your code. You need to write appropriate codes (no specific programming language is required; pseudo code is fine) for implementing the queue and stack simultaneously as described above. Please explain your code by giving appropriate comments. In particular, you need to implement the following four functions:

- `push(item)` — push a positive integer, *item* in the stack. If there is an error (e.g., no more space available for growing the stack in the list), it should output an appropriate error message and return with -1.
- `pop()` — pop an element from the stack. If there is an error, it should output an appropriate error message and return with -1.
- `enqueue(item)` — enqueue a positive integer, *item* in the queue. If there is an error (e.g., no more space available for growing the queue in the list), it should output an appropriate error message and return with -1.
- `dequeue()` — dequeue an element from the queue. If there is an error, it should output an appropriate error message and return with -1.

(25 marks) [CSE 105 | 2021–22]

Answer

Layout: Queue grows from $L[0]$ (front = $L[0]$, rear grows right). Stack grows from $L[n-1]$ (top grows left). Both meet in the middle; if they collide the list is full.
We maintain two global counters:

- `qSize` — number of elements currently in the queue.
- `sSize` — number of elements currently in the stack.

The list is full when `qSize + sSize == n`.

Algorithm 21 PUSH(*item*) — stack on shared list

```
1: if qSize + sSize = n then
2:     print "Error: stack overflow";   return −1
3: end if
4: MoveToEnd()
5: for i = 1 to sSize do
6:     Prev()                                   ▷ move to L[n − 1 − sSize]
7: end for
8: Insert(item)
9: sSize ← sSize + 1
10: return 0
```

Algorithm 22 POP() — stack on shared list

```
1: if sSize = 0 then
2:     print "Error: stack underflow";  return −1
3: end if
4: MoveToEnd()
5: for i = 1 to sSize − 1 do
6:     Prev()                                  ▷ move to L[n − sSize], the stack top
7: end for
8: val ← Value()
9: Remove()
10: sSize ← sSize − 1
11: return val
```

Algorithm 23 ENQUEUE(*item*) — queue on shared list

```
1: if qSize + sSize = n then
2:     print "Error: queue overflow";  return −1
3: end if
4: MoveToStart()
5: for i = 1 to qSize do
6:     Next()                                  ▷ move to L[qSize], first free slot
7: end for
8: Insert(item)
9: qSize ← qSize + 1
10: return 0
```

Algorithm 24 DEQUEUE() — queue on shared list

```
1: if qSize = 0 then
2:     print "Error: queue empty";  return −1
3: end if
4: MoveToStart()                               ▷ L[0] = front of queue
5: val ← Value()
6: Remove()
7: qSize ← qSize − 1
8: return val
```

Correctness notes:

- Queue front is always at $L[0]$; `dequeue` removes $L[0]$.
- Stack top is always at $L[n - \texttt{sSize}]$; `pop` returns that element.
- Overflow is detected by checking `qSize + sSize == n`.
- All operations are $O(n)$ in the worst case due to traversal.

Q14. Convert a Binary Tree to a Circular Doubly Linked List using inorder traversal (left/right pointers become prev/next). **(15 marks)** **[CSE 203 | 2021–22]**

Answer

Idea. Perform an inorder DFS. Maintain a global `prev` pointer to the last processed node. Link each node to `prev` in both directions. After traversal, connect head and tail to make it circular.

```
struct Node { int val; Node *left, *right; };

Node *head = nullptr;   // first inorder node
Node *prev = nullptr;   // last processed node

void inorderConvert(Node* root) {
    if (root == nullptr) return;

    inorderConvert(root->left);

    // Process current node
    if (prev == nullptr) {
        head = root;           // leftmost = head
    } else {
        prev->right = root;    // prev's next = cur
        root->left  = prev;    // cur's prev  = prev
    }
    prev = root;

```

```
    inorderConvert(root->right);
}

Node* bTreeToCircularDLL(Node* root) {
    head = prev = nullptr;
    inorderConvert(root);

    // Make it circular: connect last <-> head
    if (prev != nullptr && head != nullptr) {
        prev->right = head;
        head->left  = prev;
    }
    return head;
}
```

Example. BST with inorder $[1, 2, 3, 4, 5, 6, 7]$:

$$\ldots \longleftrightarrow 1 \longleftrightarrow 2 \longleftrightarrow 3 \longleftrightarrow 4 \longleftrightarrow 5 \longleftrightarrow 6 \longleftrightarrow 7 \longleftrightarrow \text{(back to 1)}$$

Complexity Analysis. $O(n)$ time (each node visited once), $O(h)$ recursion stack.

Q15. Create `KStacks`: two stacks in a single array efficiently. Implement push1,pop1, push2,pop2. **(10 marks)** [CSE 203 | 2021–22, 2016–17]

Answer

Strategy (grow-from-ends). Stack 1 grows left-to-right from index 0; Stack 2 grows right-to-left from index $N-1$. They overflow only when their tops meet.

```
const int N = 100;
int arr[N];
int top1 = -1;      // Stack 1: grows rightward
int top2 = N;       // Stack 2: grows leftward

void push1(int x) {
    if (top1 + 1 == top2) { /* overflow */ return; }
    arr[++top1] = x;
}
int pop1() {
    if (top1 == -1) return -1;    // underflow
    return arr[top1--];
}

void push2(int x) {
    if (top2 - 1 == top1) { /* overflow */ return; }
    arr[--top2] = x;
}
int pop2() {
    if (top2 == N) return -1;     // underflow
    return arr[top2++];
}
```

Complexity Analysis. All four operations — `push1`, `pop1`, `push2`, `pop2` — run in $\boldsymbol{O(1)}$.

Space advantage. Neither stack has a fixed pre-allocation. Stack 1 can use all N slots if Stack 2 is empty, and vice versa — unlike a naïve split (first $N/2$ for Stack 1, rest for Stack 2) which wastes space.

Q16. Given a string consisting of opening and closing parentheses, find the length of the longest valid parenthesis substring. **(10 marks)** [CSE 203 | 2021–22]

Answer

Stack-based $O(n)$ algorithm. Push *indices* onto the stack. Initialise the stack with a sentinel -1. For each `'('` push its index; for each `')'` pop the top and use the current index minus the new top as the current valid length.

```
#include <stack>
#include <algorithm>
using namespace std;

int longestValidParentheses(string s) {
    stack<int> st;
    st.push(-1);        // sentinel
    int maxLen = 0;
```

```
    for (int i = 0; i < (int)s.size(); i++) {
        if (s[i] == '(') {
            st.push(i);
        } else {           // s[i] == ')'
            st.pop();
            if (st.empty()) {
                st.push(i);   // new base for next valid segment
            } else {
                maxLen = max(maxLen, i - st.top());
            }
        }
    }
    return maxLen;
}
```

Trace on `")()())"` (expected: 4):

i	s[i]	Stack	maxLen
0	)	pop -1; empty $\Rightarrow$ push 0; [0]	0
1	(	push 1; [0,1]	0
2	)	pop 1; top=0; len=$2-0=2$; [0]	2
3	(	push 3; [0,3]	2
4	)	pop 3; top=0; len=$4-0=4$; [0]	**4**
5	)	pop 0; empty $\Rightarrow$ push 5; [5]	4

Complexity Analysis. $O(n)$ time, $O(n)$ stack space.

Q17. Given a Queue data structure that supports `enqueue()` and `dequeue()`, implement a Stack using only instances of Queue and Queue operations. **(10 marks)** [CSE 203 | 2021–22]

Answer

Design: push is $O(n)$, pop is $O(1)$. Keep one queue Q. On `push(x)`: enqueue x, then rotate all previously enqueued elements to the back so x is at the front.

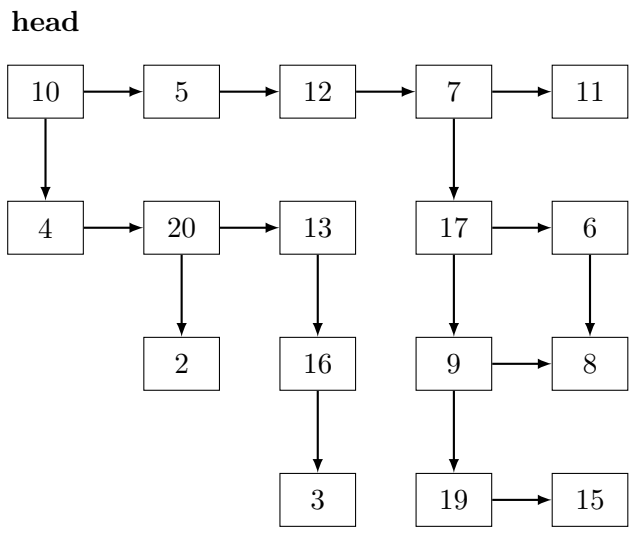
```
#include <queue>
using namespace std;

class Stack {
    queue<int> q;
public:
    void push(int x) {
        int sz = q.size();
        q.push(x);
        // Rotate: bring x to the front
        for (int i = 0; i < sz; i++) {
            q.push(q.front());
            q.pop();
        }
    }
    int pop() {
        if (q.empty()) return -1;
        int val = q.front();
        q.pop();
        return val;
    }
    int top() { return q.front(); }
    bool empty() { return q.empty(); }
};
```

Trace. push(1), push(2), push(3), pop(), push(4), pop():

Op	**Queue state (front→rear)**	**Return**
push(1)	[1]	–
push(2)	[2,1]	–
push(3)	[3,2,1]	–
pop()	[2,1]	3
push(4)	[4,2,1]	–
pop()	[2,1]	4

Complexity Analysis. `push`: $O(n)$. `pop`: $O(1)$.

Q18. Given a linked list where in addition to the next pointer, each node has a child pointer, which may or may not point to a separate list. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure, as shown in Figure . You are given the head of the first level of the list. Flatten the list so that all the nodes appear in a single-level linked list. You need to flatten the list in a way that all nodes at the first level should come first, then nodes of second level, and so on.



The above list should be converted to $10 \to 5 \to 12 \to 7 \to 11 \to 4 \to 20 \to 13 \to 17 \to 6 \to 2 \to 16 \to 9 \to 8 \to 3 \to 19 \to 15$. **(15 marks)** [CSE 203 | 2021–22]

Answer

Key Insight. The required output is in *BFS (level-by-level)* order: all level-1 nodes first, then level-2, etc. A queue naturally produces BFS order.

```cpp
#include <queue>
using namespace std;

struct Node { int val; Node *next, *child; };

Node* flatten(Node* head) {
    if (!head) return nullptr;

    queue<Node*> bfsQ;

    // Enqueue all level-1 nodes in order
    for (Node* c = head; c != nullptr; c = c->next)
        bfsQ.push(c);

    Node *dummy = new Node{0, nullptr, nullptr};
    Node *tail  = dummy;

    while (!bfsQ.empty()) {
        Node* cur = bfsQ.front();
        bfsQ.pop();

        // If cur has a child list, enqueue its nodes
        if (cur->child != nullptr) {
            for (Node* c = cur->child; c != nullptr; c = c->next)
                bfsQ.push(c);
            cur->child = nullptr;
        }

        // Append cur to result list
        cur->next = nullptr;
        tail->next = cur;
        tail = cur;
    }
    return dummy->next;
}
```

Verification on given figure:

Level 1: `10 5 12 7 11`
Level 2: `4 20 13` (children of 10) + `17 6` (children of 7)
Level 3: `2` (child of 20), `16` (child of 13), `9 8` (children of 17)
Level 4: `3` (child of 16), `19 15` (children of 9)

Result: `10 5 12 7 11 4 20 13 17 6 2 16 9 8 3 19 15` ✓

Complexity Analysis. $O(N)$ time and space, where N = total number of nodes.

Q19. Write a program to reverse a stack using recursion, without using any loop. **(10 marks)** [CSE 203 | 2021–22]

Answer

Two mutual recursions:

(a) `insertAtBottom(st, x)` — inserts x at the bottom of stack `st` without a loop.
(b) `reverseStack(st)` — pops the top, reverses the rest, then inserts the popped element at the bottom.

```cpp
#include <stack>
using namespace std;

// Insert x at the bottom of stack st (recursively)
void insertAtBottom(stack<int>& st, int x) {
    if (st.empty()) {
        st.push(x);
        return;
    }
    int top = st.top();
    st.pop();
    insertAtBottom(st, x);    // recurse with smaller stack
    st.push(top);             // restore saved top
}

// Reverse the stack (recursively)
void reverseStack(stack<int>& st) {
    if (st.empty()) return;
    int top = st.top();
    st.pop();
    reverseStack(st);         // reverse the rest
    insertAtBottom(st, top); // put top at the bottom
}
```

Trace on $[1, 2, 3, 4, 5]$ (top $= 5$):

1. `reverseStack` pops 5, reverses $[1, 2, 3, 4]$, inserts 5 at bottom.
2. After full recursion unwinds: stack becomes $[5, 4, 3, 2, 1]$ (top $= 1$).

Complexity Analysis. `insertAtBottom`: $O(n)$ recursive calls.
`reverseStack`: calls `insertAtBottom` n times $\Rightarrow O(n^2)$ total time.
Call-stack depth: $O(n)$.

Q20. Why is Binary Search preferred over Ternary Search? **(10 marks)** [CSE 203 | 2021–22]

Answer

Although Ternary Search splits the array into 3 parts and eliminates $\frac{2}{3}$ of the array per iteration (vs. $\frac{1}{2}$ for Binary Search), Binary Search is preferred because of the **number of comparisons per iteration**:

- **Binary Search** requires **1 comparison** per iteration (just check if target $\leq$ mid), reducing search space by half. Worst-case comparisons: $\lceil \log_2 n \rceil$.
- **Ternary Search** requires **2 comparisons** per iteration (check vs. mid_1 and mid_2), reducing search space to one-third. Worst-case comparisons: $2\lceil \log_3 n \rceil$.

Comparison:

$$2 \log_3 n = 2 \cdot \frac{\log_2 n}{\log_2 3} = \frac{2}{\log_2 3} \log_2 n \approx \frac{2}{1.585} \log_2 n \approx 1.26 \log_2 n$$

So Ternary Search performs approximately **26% more comparisons** than Binary Search in the worst case. Since comparisons are the dominant cost in search algorithms, Binary Search is more efficient in practice.

Conclusion: The gain in iterations from splitting into 3 parts does not offset the cost of the extra comparison per iteration. Binary Search achieves a better total comparison count.

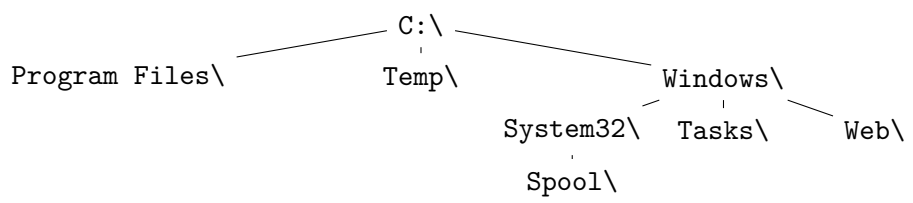
Q21. Write a code snippet to implement a queue using a circular array, where each element contains ID, name, and department of a student. **(13 marks)** [CSE 203 | 2019–20]

Answer

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Student {
    int    id;
    string name;
    string department;
};

class StudentQueue {
    Student *arr;
    int front, rear, capacity, sz;

public:
    StudentQueue(int cap) : capacity(cap), front(0), rear(-1), sz(0) {
        arr = new Student[capacity];
    }

    bool isEmpty() { return sz == 0; }
    bool isFull()  { return sz == capacity; }

    void enqueue(int id, string name, string dept) {
        if (isFull()) { cout << "Queue full\n"; return; }
        rear = (rear + 1) % capacity;
        arr[rear] = {id, name, dept};
        sz++;
    }

    Student dequeue() {
        if (isEmpty()) { cout << "Queue empty\n"; return {-1,"",""}; }
        Student s = arr[front];
        front = (front + 1) % capacity;
        sz--;
        return s;
    }

    Student frontElement() {
        if (isEmpty()) { cout << "Queue empty\n"; return {-1,"",""}; }
        return arr[front];
    }

    ~StudentQueue() { delete[] arr; }
};

int main() {
    StudentQueue q(3);
    q.enqueue(101, "Alice", "CSE");
    q.enqueue(102, "Bob",   "EEE");
    q.enqueue(103, "Carol", "ME");

    Student s = q.dequeue();
    cout << s.id << " " << s.name << " " << s.department << "\n";

    q.enqueue(104, "Dave", "CE");   // reuses slot (circular)
    return 0;
}
```

Complexity Analysis. `enqueue`, `dequeue`, `frontElement`: all $O(1)$.

Q22. Consider the following hierarchical structure of a file system. Write a program to print the directory hierarchy of the file system, where all children of a parent will be placed one tab-space right from the parent. Use depth-first tree traversal technique to perform the above task.



(12 marks) [CSE 203 | 2019–20]

Answer

Data structure. Represent the file system as an n-ary tree where each node has a name and a list of children.

```cpp
#include <iostream>
#include <string>
#include <vector>
using namespace std;

struct FSNode {
    string name;
    vector<FSNode*> children;
};

// DFS print: depth controls the indentation level
void printHierarchy(FSNode* node, int depth = 0) {
    // Print current node with 'depth' tabs
    for (int i = 0; i < depth; i++) cout << "\t";
    cout << node->name << "\n";

    // Recurse on each child
    for (FSNode* child : node->children)
        printHierarchy(child, depth + 1);
}

int main() {
    // Build the given file system tree
    FSNode *root = new FSNode{"C:\\"};
    FSNode *pf   = new FSNode{"Program Files\\"};
    FSNode *tmp  = new FSNode{"Temp\\"};
    FSNode *win  = new FSNode{"Windows\\"};
    FSNode *sys  = new FSNode{"System32\\"};
    FSNode *spool= new FSNode{"Spool\\"};
    FSNode *tasks= new FSNode{"Tasks\\"};
    FSNode *web  = new FSNode{"Web\\"};

    sys->children.push_back(spool);
    win->children = {sys, tasks, web};
    root->children = {pf, tmp, win};

    printHierarchy(root);
    return 0;
}
```

Expected output:

```
C:\
    Program Files\
    Temp\
    Windows\
        System32\
            Spool\
        Tasks\
        Web\
```

Complexity Analysis. $O(n)$ time and $O(h)$ stack space where n = total nodes, h = height of the tree.

Q23. Give the algorithm for converting an infix expression to postfix, and for evaluating a postfix expression using a stack. **(10 marks)** [CSE 203 | 2018–19]

Answer

Part A — Infix to Postfix (Shunting-Yard):

Algorithm 25 INFIX-TO-POSTFIX(*expr*)

```
S ← empty stack                                              ▷ operator stack
output ← ""
for each token t in expr do
    if t is an operand then
        output ← output + t
    else if t = '(' then
        PUSH(S, t)
    else if t = ')' then
        while TOP(S) ≠ '(' do
            output ← output + POP(S)
        end while
        POP(S)                                               ▷ discard '('
    else                                                     ▷ t is an operator
        while S ≠ ∅ and PREC(TOP(S)) ≥ PREC(t) and TOP(S) ≠ '(' do
            output ← output + POP(S)
        end while
        PUSH(S, t)
    end if
end for
while S ≠ ∅ do
    output ← output + POP(S)
end while
return output
```

Examples:

- `A+B*C` ⟶ `ABC*+`
- `(A+B)*(C-D)` ⟶ `AB+CD-*`

Part B — Evaluate Postfix:

Algorithm 26 EVAL-POSTFIX(*expr*)

```
S ← empty stack
for each token t in expr do
    if t is an operand then
        PUSH(S, VALUE(t))
    else                                                     ▷ t is an operator
        b ← POP(S)
        a ← POP(S)
        PUSH(S, APPLY(a, t, b))
    end if
end for
return POP(S)                                                ▷ final result
```

Example trace. Postfix `"231*+9-"` (i.e., $2 + (3 \times 1) - 9$):

Token	Stack (bottom→top)	Action
2	[2]	push
3	[2,3]	push
1	[2,3,1]	push
*	[2,3]	pop 1,3; push $3 \times 1 = 3$
+	[5]	pop 3,2; push $2 + 3 = 5$
9	[5,9]	push
-	[**-4**]	pop 9,5; push $5 - 9 = -4$

Complexity Analysis. Both algorithms: $O(n)$ time, $O(n)$ stack space.

Q24. Ternary search, like binary search, is a divide-and-conquer algorithm. It is mandatory for the array (in which you will search for an element) to be sorted before you begin the search. In this search, after each iteration it neglects $\frac{2}{3}$ part of the array and repeats the same operations on the remaining $\frac{1}{3}$. Write an algorithm for ternary search and show its complexity. **(12 marks)** [CSE 203 | 2018–19]

Answer

Idea. Split the search range into three equal thirds using two midpoints m_1 and m_2. Compare the target with $A[m_1]$ and $A[m_2]$ to decide which third to recurse into.

Algorithm 27 TERNARY-SEARCH(A, lo, hi, x)

```
if lo > hi then
    return −1
end if
m1 ← lo + ⌊(hi − lo)/3⌋
m2 ← hi − ⌊(hi − lo)/3⌋
if A[m1] = x then
    return m1
end if
if A[m2] = x then
    return m2
end if
if x < A[m1] then
    return TERNARY-SEARCH(A, lo, m1 − 1, x)
else if x > A[m2] then
    return TERNARY-SEARCH(A, m2 + 1, hi, x)
else
    return TERNARY-SEARCH(A, m1 + 1, m2 − 1, x)
end if
```

Complexity Analysis.
Each call reduces the search space by a factor of 3, giving the recurrence:

$$T(n) = T\left(\frac{n}{3}\right) + O(1)$$

Solving: $T(n) = O(\log_3 n)$.

Comparison with Binary Search.

$$\log_3 n = \frac{\log_2 n}{\log_2 3} \approx 0.631 \cdot \log_2 n.$$

Ternary search makes *fewer iterations* but **two comparisons per iteration** versus one for binary search, so the total number of comparisons is:

$$2\log_3 n \approx 1.26 \log_2 n \;>\; \log_2 n.$$

Remark. Binary search is preferred because it uses fewer total comparisons despite more iterations ($\log_2 n$ comparisons vs $\approx 1.26 \log_2 n$). Ternary search is not an improvement for comparison-based search.

Q25. Describe a situation where storing items in an array is clearly better than storing items in a linked list. **(8 marks)** [CSE 203 | 2018–19]

Answer

Scenario: Repeated random-access / index-based lookups on a fixed dataset.
Example. You store $n = 10^6$ sensor readings and repeatedly answer queries of the form "what is the k-th reading?" With an array, the answer is simply `arr[k]`, i.e., $O(1)$ time. With a linked list, you must traverse k nodes from the head: $O(k) = O(n)$ worst-case.

Why arrays win in this scenario:

(a) $O(1)$ **random access.** The address of element i is `base + i*sizeof(element)`, computed without any traversal.

(b) **Cache efficiency.** Array elements are contiguous in memory; a cache line loaded for element i also brings $i{+}1, i{+}2, \ldots$ into cache. Linked list nodes may be scattered, causing cache misses on every step.

(c) **Lower memory overhead.** Each array element stores only the datum; a linked list node additionally stores one or two pointers (8–16 bytes each on 64-bit systems), wasting significant memory for small data types (e.g. `int`).

(d) **Simpler iteration.** A for-loop `for(i=0;i<n;i++)` with an array runs faster than pointer chasing in a linked list due to branch prediction and pipelining.

Conclusion. Arrays excel when (a) the size is known in advance, (b) random access is frequent, and (c) insertions/deletions are rare.

Q26. Write two versions of binary search. Simulate on $[23, 56, 8, 10, 12, 15, 20, 21, 23]$ for keys $1, 6, 23, 24$. **(10+10=20 marks)** [CSE 203 | 2017–18]

Answer

Note. Binary search requires a *sorted* array. Sorting the given array yields [**8**, **10**, **12**, **15**, **20**, **21**, **23**, **23**, **56**] (indices 0–8).

Version 1 — Iterative:

```
// Returns index of x in sorted arr[0..n-1], or -1 if not found
int binarySearchIter(int arr[], int n, int x) {
    int lo = 0, hi = n - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;    // avoids overflow
        if (arr[mid] == x) return mid;
        if (arr[mid] <  x) lo = mid + 1;
        else               hi = mid - 1;
    }
    return -1;
}
```

Version 2 — Recursive:

```
int binarySearchRec(int arr[], int lo, int hi, int x) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;
    if (arr[mid] == x) return mid;
    if (arr[mid] <  x) return binarySearchRec(arr, mid + 1, hi,  x);
    else               return binarySearchRec(arr, lo,  mid - 1, x);
}
```

Simulation on sorted array [8, 10, 12, 15, 20, 21, 23, 23, 56]**:**

Key	lo	hi	mid (arr[mid])	Result
1	0	8	4 (20)	$1 < 20 \Rightarrow$ hi=3
	0	3	1 (10)	$1 < 10 \Rightarrow$ hi=0
	0	0	0 (8)	$1 < 8 \Rightarrow$ hi=$-1 \Rightarrow$ **NOT FOUND**
6	0	8	4 (20)	$6 < 20 \Rightarrow$ hi=3
	0	3	1 (10)	$6 < 10 \Rightarrow$ hi=0
	0	0	0 (8)	$6 < 8 \Rightarrow$ hi=$-1 \Rightarrow$ **NOT FOUND**
23	0	8	4 (20)	$23 > 20 \Rightarrow$ lo=5
	5	8	6 (23)	**FOUND at index 6**
24	0	8	4 (20)	$24 > 20 \Rightarrow$ lo=5
	5	8	6 (23)	$24 > 23 \Rightarrow$ lo=7
	7	8	7 (23)	$24 > 23 \Rightarrow$ lo=8
	8	8	8 (56)	$24 < 56 \Rightarrow$ hi=7 $\Rightarrow$ **NOT FOUND**

Complexity Analysis. $O(\log n)$ time, $O(1)$ space (iterative) / $O(\log n)$ stack space (recursive).

Q27. How do we cope with hard problems? Discuss. **(10 marks)** [CSE 207 | 2017–18]

Answer

When a problem is NP-hard (no known polynomial-time algorithm), we use one or more of the following strategies:

1. **Identify tractable special cases.** Restrict inputs so the problem becomes polynomial. E.g., 2-SAT is polynomial though 3-SAT is NP-complete; TSP on trees is $O(n)$.

2. **Approximation algorithms.** Find a solution guaranteed to be within a factor ρ of optimal in polynomial time. E.g., the 2-approximation for vertex cover; PTAS for knapsack achieving $(1-\varepsilon)$-optimal in $O(n/\varepsilon)$ time.

3. **Randomized algorithms.** Use randomness to get correct or near-optimal answers with high probability. E.g., randomized quicksort, Monte Carlo methods, simulated annealing.

4. **Fixed-parameter tractability (FPT).** If the problem has a natural parameter k that is small in practice, design algorithms exponential only in k but polynomial in n. E.g., vertex cover in $O(2^k \cdot n)$.

5. **Exponential algorithms with pruning.** Branch-and-bound, backtracking with aggressive pruning, and memoization can solve moderate-sized instances exactly (e.g., TSP with $n \leq 20$ via Held-Karp DP in $O(2^n n^2)$).

6. **Heuristics and metaheuristics.** No worst-case guarantee but work well in practice: greedy heuristics, genetic algorithms, tabu search, ant-colony optimization.

7. **Integer programming / SAT solvers.** Modern ILP solvers (CPLEX, Gurobi) and SAT solvers handle many practically-sized NP-hard instances efficiently through branch-and-cut and clause learning.

Summary: The right strategy depends on whether we need exact solutions, how large n is, and whether a performance guarantee is required.

Q28. Given a singly linked list and integers m and k, find the node with value m then delete the next k nodes. **(10 marks)** [CSE 203 | 2016–17]

Answer

```cpp
struct Node { int item; Node* next; };
Node* head;

void deleteNextK(int m, int k) {
    // Step 1: find node with value m
    Node* cur = head;
    while (cur != nullptr && cur->item != m)
        cur = cur->next;

    if (cur == nullptr) return;   // m not found

    // Step 2: delete the next k nodes
    for (int i = 0; i < k; i++) {
        if (cur->next == nullptr) break;  // fewer than k nodes remain
        Node* toDelete = cur->next;
        cur->next = toDelete->next;
        delete toDelete;
    }
}
```

Example. List: $1 \to 2 \to 3 \to 4 \to 5 \to 6 \to 7$, $m = 3$, $k = 2$.
Find node 3. Delete node 4 (`toDelete`), then delete node 5.
Result: $1 \to 2 \to 3 \to 6 \to 7$.

Complexity Analysis. $O(n)$ to find m; $O(k)$ for deletions; $O(n)$ total.

Q29. Write `enqueue` and `dequeue` functions for a FIFO queue implemented using a singly linked list. **(10 marks)** [CSE 203 | 2016–17]

Answer

Maintain two pointers: `front` (for dequeue) and `rear` (for enqueue). Both operations are $O(1)$.

```cpp
struct Node { int val; Node* next; };
Node *front = nullptr, *rear = nullptr;

// Add element at the rear
void enqueue(int x) {
    Node* n = new Node{x, nullptr};
    if (rear == nullptr) {          // empty queue
        front = rear = n;
    } else {
        rear->next = n;
        rear = n;
    }
}

// Remove and return front element
int dequeue() {
    if (front == nullptr) return -1;    // underflow
    Node* tmp = front;
    int val   = tmp->val;
    front     = front->next;
    if (front == nullptr) rear = nullptr; // queue became empty
    delete tmp;
    return val;
}
```

Complexity Analysis. `enqueue`: $O(1)$ — append at tail.
`dequeue`: $O(1)$ — remove from head.

Q30. Deduce the condition under which an array-based implementation of a list would be better in space utilisation than a linked-based

implementation. **(10 marks)** [CSE 203 | 2015–16]

Answer

Let:

- d = size (bytes) of one *data item*,
- p = size of one *pointer* (typically 8 bytes on 64-bit systems),
- n = current number of elements,
- c = allocated array capacity ($c \geq n$; worst-case $c = 2n$ after a doubling).

Space used:

$$S_{\text{array}} = c \cdot d, \qquad S_{\text{linked}} = n \cdot (d + p) \quad \text{(singly linked)}.$$

Array beats linked list when $S_{\text{array}} < S_{\text{linked}}$:

$$c \cdot d < n(d + p) \implies \frac{c}{n} < 1 + \frac{p}{d}.$$

With worst-case $c = 2n$ (just doubled):

$$2 < 1 + \frac{p}{d} \implies \frac{p}{d} > 1 \implies \boxed{p > d}.$$

Conclusion. An array uses less space than a singly linked list whenever the pointer size p exceeds the data item size d. In practice, with $p = 8$ bytes (64-bit pointer), arrays are more space-efficient for data types smaller than 8 bytes (e.g., `char`, `short`, `int`). For large structs ($d \gg p$), the extra pointer overhead becomes negligible and the linked list can be competitive.

Remark. For a *doubly* linked list the condition becomes $2p > d$.

Q31. Given input string x, check whether x is of the form $w\#w'$, where w' is the reverse of w. Suggest a data structure and show how to solve it efficiently. **(10 marks)** [CSE 203 | 2015–16]

Answer

Data structure: Stack.
Algorithm:

Algorithm 28 CHECK-W-HASH-REVERSE(x)

```
1:  hashPos ← index of '#' in x
2:  if hashPos not found then
3:      return false
4:  end if
5:  w ← x[0 .. hashPos − 1]
6:  w' ← x[hashPos + 1 .. end]
7:  if |w| = 0 or |w| ≠ |w'| then
8:      return false
9:  end if
10: S ← empty stack
11: for each char c in w do
12:     PUSH(S, c)
13: end for
14: for each char c in w' do
15:     if S is empty or TOP(S) ≠ c then
16:         return false
17:     end if
18:     POP(S)
19: end for
20: return S is empty
```

Why it works. Pushing w onto the stack gives a reversed w at the top. Comparing pops (which produce w in reverse) character by character with w' (forward) checks exactly that $w' = \text{reverse}(w)$.
Example. $x =$ ``abc#cba'': push a, b, c; compare pops c, b, a with $w' =$ `cba` ⇒ match, return `true`.

Complexity Analysis. $O(|x|)$ time, $O(|w|)$ stack space.

Q32. Given input k and n, return the key of the n-th node before the one containing k. (All keys distinct.) **(15 marks)** [CSE 203 | 2015–16]

Answer

Two-pointer technique. Advance a `fast` pointer n steps ahead, then move both `slow` and `fast` together until `fast` reaches the node with key k. At that moment `slow` is exactly n nodes behind.

```
1 struct listNode { int key; listNode* next; };
2 listNode* head;
```

```
int nthNodeBeforeKey(int k, int n) {
    listNode *fast = head, *slow = head;

    // Advance fast by n steps
    for (int i = 0; i < n; i++) {
        if (fast == nullptr) return -1;  // list shorter than n
        fast = fast->next;
    }

    // Move both until fast->key == k
    while (fast != nullptr && fast->key != k) {
        fast = fast->next;
        slow = slow->next;
    }

    if (fast == nullptr) return -1;      // k not in list
    return slow->key;
}
```

Example. List: $10 \to 20 \to 30 \to 40 \to 50 \to 60$, $k = 50$, $n = 2$.
`fast` starts at 10, advances 2 steps to 30.
Move together: fast=40, slow=20; fast=50 (found k), slow=30.
Return 30. ✓

Complexity Analysis. $O(L)$ time where L is the length of the list; single pass. $O(1)$ extra space.

Q33. Your group proposes that the beginning (index 0) of the array be the top of the stack. A friend claims this is inefficient. Do you agree? Justify. **(8 marks)** [CSE 203 | 2015–16]

Answer

Yes, the friend is correct — this is inefficient.

If `arr[0]` is the top:

- **`push(x)`:** Must shift every existing element one position to the right to make room at index 0. Cost: $O(n)$.
- **`pop()`:** Removes `arr[0]`, then must shift every remaining element one position left. Cost: $O(n)$.

Efficient alternative — use the end as the top. Maintain a `topIndex` pointing to the last-filled slot.

- **`push(x)`:** `arr[++topIndex] = x` — $O(1)$.
- **`pop()`:** `return arr[topIndex--]` — $O(1)$.

Conclusion. Using index 0 as the top degrades both `push` and `pop` from $O(1)$ to $O(n)$, making the implementation as slow as a list-based approach while forfeiting the simplicity of the array. The end of the array should always be the top.

Q34. Discuss the Drifting Queue Problem with an illustrative example. Propose a solution. Address empty/full recognition. **(5+5+5+4=19 marks)** [CSE 203 | 2015–16]

Answer

The Drifting Queue Problem. In a naive linear-array queue with `front` and `rear` indices, each dequeue increments `front` and each enqueue increments `rear`. Over time both indices drift rightward until `rear` hits the end of the array — even though positions $0 \ldots$ `front` $- 1$ are free.

Example. Capacity = 4.

Op	[0]	[1]	[2]	[3]	front	rear	Status
init	–	–	–	–	0	-1	empty
enqueue(A)	A	–	–	–	0	0	
enqueue(B)	A	B	–	–	0	1	
dequeue()	–	B	–	–	1	1	
dequeue()	–	–	–	–	2	1	empty but front drifted
enqueue(C)	–	–	C	–	2	2	
enqueue(D)	–	–	C	D	2	3	
enqueue(E)	–	–	C	D	2	3	**FULL?** rear = 3 = cap-1 but slots 0,1 are free!

Proposed Solution: Circular (Ring) Queue. Wrap indices modulo `capacity`:

$$\texttt{rear} \leftarrow (\texttt{rear} + 1) \bmod \texttt{capacity}, \quad \texttt{front} \leftarrow (\texttt{front} + 1) \bmod \texttt{capacity}.$$

Empty/Full ambiguity. When using only `front` and `rear`, both empty and full give `front == (rear+1) mod capacity` (if one slot is sacrificed) or `front == rear` — these are ambiguous. **Solution:** maintain a `size` counter (or a boolean `isFull` flag).

Algorithm 29 ENQUEUE(x) — circular queue

```
1: if sz = capacity then
2:     error "Queue Overflow"
3: end if
4: rear ← (rear + 1) mod capacity
5: arr[rear] ← x
6: sz ← sz + 1
```

Algorithm 30 DEQUEUE() — circular queue

```
1: if sz = 0 then
2:     error "Queue Underflow"
3: end if
4: val ← arr[front]
5: front ← (front + 1) mod capacity
6: sz ← sz − 1
7: return val
```

This completely eliminates drifting and correctly distinguishes empty from full.

Q35. What is an application of prefix sums? What are prefix sums of an array $A[i]$, $1 \le i \le n$? Write a parallel algorithm that finds prefix sums of the array A. Illustrate for $[15, 35, 40, 25, 20, 50, 10, 30]$ showing bottom-up and top-down traversals. **(15 marks)** [CSE 207 | 2015–16]

Answer

Application: Range-sum queries — given $A[1..n]$, answer $\sum_{i=l}^{r} A[i]$ in $O(1)$ after $O(n)$ preprocessing. Other uses: parallel scan, histogram equalization, load balancing.

Definition: The prefix sum array P is defined as:

$$P[i] = \sum_{j=1}^{i} A[j], \quad P[0] = 0$$

Parallel Prefix Sum (Blelloch Scan):
Uses a complete binary tree over n leaves. Two phases:

Algorithm 31 PARALLEL-PREFIX-SUM(T, n)

```
1: for d = 0 to log n − 1 do                                   ▷ Bottom-up reduce
2:     for each i = 0 to n − 1, step 2^(d+1), in parallel do
3:         T[i + 2^(d+1) − 1] ← T[i + 2^(d+1) − 1] + T[i + 2^d − 1]
4:     end for
5: end for
6: T[n − 1] ← 0                                                ▷ Top-down downsweep
7: for d = log n − 1 downto 0 do
8:     for each i = 0 to n − 1, step 2^(d+1), in parallel do
9:         left ← T[i + 2^d − 1]
10:        T[i + 2^d − 1] ← T[i + 2^(d+1) − 1]
11:        T[i + 2^(d+1) − 1] ← T[i + 2^(d+1) − 1] + left
12:    end for
13: end for
```

Complexity: $O(\log n)$ parallel steps, $O(n)$ total work.

Illustration for $A = [15, 35, 40, 25, 20, 50, 10, 30]$, $n = 8$:
Bottom-up (reduce) — each node stores sum of its subtree:

Level	Tree nodes (left to right)							
Leaves (d=0)	15	35	40	25	20	50	10	30
d=1 (pairs)	50	–	65	–	70	–	40	–
d=2 (quads)	115	–	–	–	110	–	–	–
d=3 (root)				225				

Top-down (downsweep) — root set to 0, propagate exclusive prefix:

Level	Node values after downsweep							
Root					0			
d=2	0	–	–	–	115	–	–	–
d=1	0	–	50	–	115	–	185	–
Leaves	0	15	50	90	115	135	185	195

Result (exclusive prefix sums): $P = [0, 15, 50, 90, 115, 135, 185, 195]$
Inclusive prefix sums (shift by one): $[15, 50, 90, 115, 135, 185, 195, 225]$

Q36. Implement `void delete(int N)` that deletes all Mth nodes such that $M \bmod N = 0$ from the doubly linked list (head is the 1st node). **(15 marks)** [CSE 203 | 2014–15]

Answer

```
struct listNode {
    int item;
    listNode *next, *prev;
};
listNode *head, *tail;

void deleteEveryNth(int N) {
    listNode* cur = head;
    int pos = 1;

    while (cur != nullptr) {
        listNode* nxt = cur->next;      // save before possible delete

        if (pos % N == 0) {
            // Relink prev side
            if (cur->prev != nullptr)
                cur->prev->next = cur->next;
            else
                head = cur->next;       // deleting head

            // Relink next side
            if (cur->next != nullptr)
                cur->next->prev = cur->prev;
            else
                tail = cur->prev;       // deleting tail

            delete cur;
        }
        pos++;
        cur = nxt;
    }
}
```

Example. List: $1 \to 2 \to 3 \to 4 \to 5 \to 6 \to 7 \to 8 \to 9$, $N = 3$:
Delete positions 3, 6, 9 $\Rightarrow 1 \to 2 \to 4 \to 5 \to 7 \to 8$.

Complexity Analysis. $O(n)$ — single pass; each deletion is $O(1)$ pointer manipulation.

Q37. Compare worst-case run times of ArrayList and LinkedList (doubly, with head and tail) for the listed operations. **(4×3=12 marks)** [CSE 203 | 2014–15]

Answer

Operation	ArrayList	LinkedList
Insert at N-th position (preserve order)	$O(n)$ — shift elements right	$O(n)$ — traverse to pos N, then $O(1)$ relink
Remove first item (preserve order)	$O(n)$ — shift all elements left	$O(1)$ — update `head`
Remove N-th item (preserve order)	$O(n)$ — traverse + shift	$O(n)$ — traverse to pos N, then $O(1)$ relink
Get (without removing) N-th item	$O(1)$ — direct index access	$O(n)$ — traverse from head

Key insight. ArrayList excels at random access; LinkedList excels at front insertions/deletions. Both are $O(n)$ for arbitrary position insert/delete, but ArrayList shifts data while LinkedList traverses pointers.

Q38. Give two advantages and disadvantages of using array-based lists over linked lists. **(8 marks)** [CSE 203 | 2014–15]

Answer

Advantages of array-based lists:

(a) $O(1)$ **random access.** Index arithmetic computes the address directly: `addr = base + i * size`. Linked lists require $O(n)$ traversal.

(b) **Better cache performance.** Elements are stored contiguously in memory, so prefetching and cache lines are used efficiently. Linked list nodes may be scattered across the heap, causing frequent cache misses.

Disadvantages of array-based lists:

(a) **Fixed / costly resizing.** The capacity must be declared upfront. Dynamic resizing (doubling) is amortised $O(1)$, but incurs a one-time $O(n)$ copy and wastes up to half the array at any time.

(b) **Costly insertions/deletions.** Inserting or removing an element at position i requires shifting up to $n - i$ elements, giving $O(n)$ in the worst case. Linked lists do this in $O(1)$ once the position is known.

Q39. Implement a **stack** using a singly linked list (`push`, `pop`, `size`). Only `head` and `tail` pointers; no other global/static variables. **(15 marks)** [CSE 203 | 2014–15]

Answer

Strategy. Use the *head* as the top of the stack. `push` inserts at head; `pop` removes from head. This makes both operations $O(1)$. `size` is encoded implicitly: we infer it by traversal, but to satisfy "no extra global variables" we walk the list.

```
struct listNode {
    int item;
    listNode* next;
};
listNode* head = nullptr;
listNode* tail = nullptr;  // tail unused by stack, kept for ADT

void push(int x) {
    listNode* n = new listNode{x, head};
    head = n;
    if (tail == nullptr) tail = n;   // first element
}

int pop() {
    if (head == nullptr) return -1;  // underflow sentinel
    listNode* tmp = head;
    int val = tmp->item;
    head = head->next;
    if (head == nullptr) tail = nullptr;
    delete tmp;
    return val;
}

// O(n) size by traversal (no extra variable allowed)
int size() {
    int count = 0;
    listNode* cur = head;
    while (cur != nullptr) { count++; cur = cur->next; }
    return count;
}
```

Complexity Analysis. `push`: $O(1)$. `pop`: $O(1)$. `size`: $O(n)$.

Remark. If an $O(1)$ `size` is required, one would typically maintain an integer counter — but the problem forbids extra global variables. Using the linked list head-insertion approach still gives optimal push/pop.

Q40. Suppose we double the memory of an array-based list every time it is full during insertion. Show the average time per insertion for copying is constant. **(12 marks)** [CSE 203 | 2014–15]

Answer

Setup. Start with capacity 1; double whenever full. After n insertions the doublings occur at sizes $1, 2, 4, 8, \ldots, 2^k$ where $2^k \leq n$.

Total copy cost. When the array doubles to size s, we copy $s/2$ elements. Summing over all doublings:

$$\text{Total copies} = 1 + 2 + 4 + \cdots + 2^{k-1} + 2^k \;=\; 2^{k+1} - 1 \;<\; 2n.$$

(Geometric series with ratio 2.)

Amortised cost.

$$\text{Average copies per insertion} = \frac{\text{Total copies}}{n} < \frac{2n}{n} = 2 = O(1).$$

Accounting argument. Charge each element \$2 at insertion time: \$1 pays for its own insertion, and \$1 is saved. When the array doubles, the saved credit of each "old" element pays for copying it. Hence every doubling is fully paid for, so the amortised cost per insertion is $O(1)$.

Complexity Analysis. Amortised $O(1)$ per insertion, $O(n)$ total for n insertions.

Q41. Give two reasons why we should use restrictive data structures (e.g., Stack, Queue) instead of general-purpose ones (e.g., List). **(4 marks)** [CSE 203 | 2014–15]

Answer

(a) **Semantic clarity and correctness.** A stack enforces LIFO discipline; a queue enforces FIFO. Using a general list allows any element to be accessed/modified, making it easy to accidentally violate the intended order. Restricted interfaces prevent such bugs at the API level (e.g. a function call stack must never allow arbitrary middle-frame deletion).

(b) **Optimised implementation and encapsulation.** Because the interface is constrained, the implementation can be tuned for exactly those operations (e.g. array-based stack has $O(1)$ push/pop with no overhead). It also hides internal details, making the code easier to reason about, maintain, and replace.

Q42. Given a doubly linked list and an integer n, write a function `deleteLastNElements()` that deletes the last n nodes from the given linked list. **(8 marks)** [CSE 203 | 2013–14]

Answer

Algorithm 32 DeleteLastNElements($head$, $tail$, n)

```
1: if n ≤ 0 then
2:     return
3: end if
4: curr ← tail;   count ← 0
5: while curr ≠ nil and count < n do
6:     toDelete ← curr
7:     curr ← curr.prev
8:     count ← count + 1
9:     Free(toDelete)
10: end while
11: if curr = nil then                          ▷ deleted all nodes
12:     head ← nil;   tail ← nil
13: else                                        ▷ curr is the new tail
14:     curr.next ← nil;   tail ← curr
15: end if
```

Time complexity: $O(n)$ — traverse n nodes from the tail.
Space complexity: $O(1)$ — no extra space used.
Edge cases handled:

- $n \geq$ list size: all nodes deleted, head and tail set to NULL.
- $n = 0$: no deletion.

Q43. How do you implement a Queue using a Stack? What are the running times of `enqueue` and `dequeue` operations of such an implementation? **(8+2=10 marks)** [CSE 203 | 2013–14]

Answer

Idea: Use **two stacks** S_1 (inbox) and S_2 (outbox).

Algorithm 33 Enqueue($item$) — two-stack queue

```
1: S1.Push(item)                                ▷ always push to inbox
```

Algorithm 34 DEQUEUE() — two-stack queue

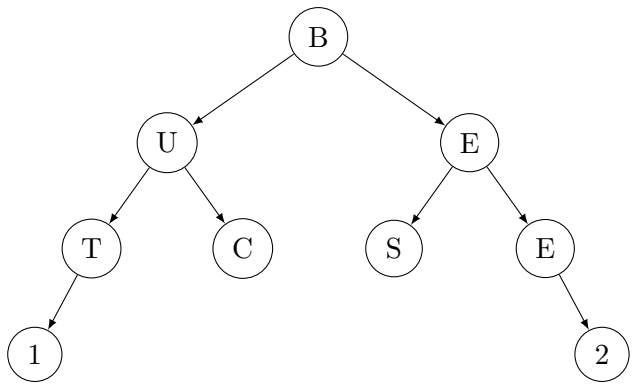
```
1: if S1 is empty and S2 is empty then
2:     print "Error: queue empty";
3:     return −1
4: end if
5: if S2 is empty then
6:     while S1 is not empty do                          ▷ transfer to outbox
7:         S2.PUSH(S1.POP())
8:     end while
9: end if
10: return S2.POP()                                      ▷ top of S2 is oldest element
```

Example: Enqueue 1, 2, 3 → $S_1 = [1, 2, 3]$ (top=3). `dequeue`: move all to $S_2 = [3, 2, 1]$ (top=1), pop 1. ✓

Running times:

- `enqueue`: $O(1)$ always (single push to S_1).
- `dequeue`: $O(n)$ **worst case** (when S_2 is empty and all n elements must be transferred from S_1 to S_2); **amortized** $O(1)$ — each element is pushed and popped at most twice total.

Q44. Which type of traversal will traverse the binary tree `BUETCSE12` (as shown in figure)? Write an algorithm for such traversal of a binary tree using a Stack or Queue. **(2+10=12 marks)** [CSE 203 | 2013–14]



Answer

Traversal type: Level-order (Breadth-First) traversal.
Reading the tree level by level from left to right: Level 0: B, Level 1: U, E, Level 2: T, C, S, E, Level 3: 1, 2 ⇒ B, U, E, T, C, S, E, 1, 2 = `BUETCSE12` ✓

Algorithm using a Queue:

Algorithm 35 LEVELORDERTRAVERSAL($root$)

```
1: if root = nil then
2:     return
3: end if
4: Q ← empty queue
5: ENQUEUE(Q, root)
6: while Q ≠ ∅ do
7:     node ← DEQUEUE(Q)
8:     print node.data
9:     if node.left ≠ nil then
10:        ENQUEUE(Q, node.left)
11:    end if
12:    if node.right ≠ nil then
13:        ENQUEUE(Q, node.right)
14:    end if
15: end while
```

Trace on BUETCSE12 tree:

Dequeued	Printed	Queue after
B	B	[U, E]
U	U	[E, T, C]
E	E	[T, C, S, E]
T	T	[C, S, E, 1]
C	C	[S, E, 1]
S	S	[E, 1]
E	E	[1, 2]
1	1	[2]
2	2	[]

Output: B U E T C S E 1 2 = `BUETCSE12` ✓
Time complexity: $O(n)$. **Space complexity:** $O(w)$ where w is the maximum width of the tree.

Q45. Give 2 (two) real-life applications each for: (i) Stack, (ii) Queue, (iii) Priority Queue. **(2+2+2=6 marks)** [CSE 203 | 2013–14]

Answer

(i) **Stack:**

1. **Function call stack:** The OS uses a stack to manage function calls and returns in program execution (each call pushes a frame, return pops it).
2. **Undo/Redo in text editors:** Each edit operation is pushed; pressing Undo pops the most recent operation.

(ii) **Queue:**

1. **CPU process scheduling (FCFS):** Processes waiting for the CPU are stored in a queue; the first to arrive is the first to be served.
2. **Printer spooler:** Print jobs are queued; the first job submitted is the first to be printed.

(iii) **Priority Queue:**

1. **Hospital emergency room:** Patients are treated based on severity of illness, not arrival order.
2. **Dijkstra's shortest path algorithm:** Vertices are extracted in order of their current shortest distance estimate, stored in a min-priority queue.

Q46. Write pseudocodes for `enqueue` and `dequeue` operations for the array implementation of a circular queue. **(7 marks)** [CSE 203 | 2013–14]

Answer

Variables: array `Q[0..MAX-1]`, integers `front`, `rear`, `size` (all initially 0).

Algorithm 36 ENQUEUE($item$) — array circular queue

```
1: if size = MAX then
2:     print "Error: Queue is full";  return −1
3: end if
4: Q[rear] ← item
5: rear ← (rear + 1) mod MAX
6: size ← size + 1
7: return 0
```

Algorithm 37 DEQUEUE() — array circular queue

```
1: if size = 0 then
2:     print "Error: Queue is empty";  return −1
3: end if
4: val ← Q[front]
5: front ← (front + 1) mod MAX
6: size ← size − 1
7: return val
```

Why circular? Using modular arithmetic on `front` and `rear` avoids the "drifting queue" problem where usable space at the beginning of the array goes to waste after dequeues.
Time complexity: Both operations are $O(1)$.

Q47. Give a comparison of array and linked list. **(6 marks)** [CSE 203 | 2013–14]

Answer

Property	Array	Linked List
Memory allocation	Static (compile time) or dynamic but contiguous	Dynamic, non-contiguous nodes
Access time	$O(1)$ random access via index	$O(n)$ sequential access
Insertion/Deletion (middle)	$O(n)$ (shifting required)	$O(1)$ if pointer to node is known
Memory usage	No extra overhead per element	Extra pointer(s) per node
Cache performance	Excellent (spatial locality)	Poor (scattered memory)
Size flexibility	Fixed (static) or expensive to resize	Easily grows/shrinks

Use array when: frequent random access, size known in advance, cache performance matters.
Use linked list when: frequent insertions/deletions, size unknown or highly variable.

Q48. Write down a linear-time algorithm for finding the maximum sum of a consecutive subsequence of an array. **(15 marks)** [CSE 207 | 2013–14]

Answer

Algorithm (Kadane's):

Algorithm 38 MAX-SUBARRAY(A, n)

```
1:  maxsum ← −∞
2:  currentsum ← 0
3:  start ← 0;   end ← 0;   tempstart ← 0
4:  for i = 1 to n do
5:      currentsum ← currentsum + A[i]
6:      if currentsum > maxsum then
7:          maxsum ← currentsum
8:          start ← tempstart
9:          end ← i
10:     end if
11:     if currentsum < 0 then
12:         currentsum ← 0
13:         tempstart ← i + 1
14:     end if
15: end for
16: return maxsum, start, end
```

Correctness: At each index i, `currentsum` holds the maximum subarray sum ending at i. If this drops below 0, any future subarray is better off starting fresh from $i+1$.
Key invariant:

$$\texttt{currentsum}_i = \max_{j \le i} \sum_{k=j}^{i} A[k] \quad \text{and} \quad \texttt{maxsum} = \max_{i} \texttt{currentsum}_i$$

Complexity: One pass $\Rightarrow O(n)$ time, $O(1)$ extra space.
Edge cases:

- All elements negative: return the single largest element (initialize `maxsum` $= -\infty$).
- All elements non-negative: entire array is the answer.

Q49. Write two functions for the insertion and deletion operations in a circular linked list. **(5+5=10 marks)** [CSE 203 | 2012–13]

Answer

We maintain a single `tail` pointer. `tail->next` is always the head, so the list is never broken.

```
struct Node { int val; Node* next; };

// Insert val at the END of the circular list
void insertEnd(Node* &tail, int val) {
    Node* n = new Node{val, nullptr};
    if (tail == nullptr) {
```

```
        n->next = n;          // points to itself
        tail = n;
    } else {
        n->next    = tail->next;   // n -> old head
        tail->next = n;            // old tail -> n
        tail       = n;            // update tail
    }
}

// Delete first occurrence of val in the circular list
void deleteVal(Node* &tail, int val) {
    if (tail == nullptr) return;          // empty list

    Node* cur  = tail->next;              // start at head
    Node* prev = tail;

    do {
        if (cur->val == val) {
            if (cur == tail && cur->next == cur) {
                // only one node
                delete cur;
                tail = nullptr;
                return;
            }
            prev->next = cur->next;
            if (cur == tail) tail = prev; // deleted tail
            delete cur;
            return;
        }
        prev = cur;
        cur  = cur->next;
    } while (cur != tail->next);          // full circle
}
```

Complexity Analysis. `insertEnd`: $O(1)$.
`deleteVal`: $O(n)$ (search), $O(1)$ relinking.

Q50. Explain the advantages and disadvantages of arrays and linked lists. **(5 marks)** [CSE 203 | 2011–12]

Answer

	Array	**Linked List**
Advantages	Random access $O(1)$ via index; cache-friendly (contiguous memory); no extra pointer overhead	Dynamic size (no pre-allocation); $O(1)$ insert/delete at known pointer; no unused memory waste
Disadvantages	Fixed size (may waste or overflow); $O(n)$ insert/delete (shifting); resizing is costly	No random access ($O(n)$ search); extra memory per node (pointer fields); poor cache locality

Q51. Write pseudocodes for the `Insert` and `Delete` operations of a circular queue. **(6 marks)** [CSE 203 | 2011–12]

Answer

Algorithm 39 INSERT(x) — circular queue

1: **if** $sz = capacity$ **then**
2: **error** "Queue Overflow"
3: **end if**
4: $rear \leftarrow (rear + 1) \bmod capacity$
5: $arr[rear] \leftarrow x$
6: $sz \leftarrow sz + 1$

Algorithm 40 DELETE() — circular queue

```
1: if sz = 0 then
2:     error "Queue Underflow"
3: end if
4: val ← arr[front]
5: front ← (front + 1) mod capacity
6: sz ← sz − 1
7: return val
```

Remark. The `sz` counter is the cleanest way to distinguish full (`sz` = `capacity`) from empty (`sz` = 0). Without it one must sacrifice one slot or use a boolean flag to tell apart these two states when `front == (rear+1) mod capacity`.

Q52. Write down a linear-time algorithm for computing the maximum sum of a consecutive subsequence. Simulate the algorithm on the following array, showing values of `maxsum` and `suffixmax`:

$$2, -1, 3, 1, 2, -4, 5, 2, -3, 7, 2, 4, -1, 2, -3$$

(15 marks) [CSE 207 | 2007–08]

Answer

Algorithm (Kadane's / suffix-max variant):

Algorithm 41 MAX-SUBARRAY-SIMPLE(A, n)

```
1: maxsum ← 0
2: suffixmax ← 0
3: for i = 1 to n do
4:     suffixmax ← max(0, suffixmax + A[i])
5:     maxsum ← max(maxsum, suffixmax)
6: end for
7: return maxsum
```

Idea: `suffixmax` tracks the maximum subarray sum ending at index i. If it goes negative we reset to 0 (start a new subarray). `maxsum` records the global best seen so far.
Time: $O(n)$. **Space:** $O(1)$.

Simulation on $A = [2, -1, 3, 1, 2, -4, 5, 2, -3, 7, 2, 4, -1, 2, -3]$:

i	$A[i]$	`suffixmax`	`maxsum`
1	2	2	2
2	-1	1	2
3	3	4	4
4	1	5	5
5	2	7	7
6	-4	3	7
7	5	8	8
8	2	10	10
9	-3	7	10
10	7	14	14
11	2	16	16
12	4	20	20
13	-1	19	20
14	2	21	**21**
15	-3	18	**21**

Maximum subarray sum = 21, achieved by the subarray $[2, -1, 3, 1, 2, -4, 5, 2, -3, 7, 2, 4, -1, 2]$ (indices 1–14), with sum = 21.

Q53. Suppose you are given an implementation of a list Abstract Data Type (ADT) which has the functions listed in Table A. Implement a stack which has the functionality shown in Table B. In your implementation, you can only use the list implementation of Table A. Assume that the elements are integers.**(15 marks)**

Answer

Strategy. Use the *end* of the list as the top of the stack so that `push` ≡ `append` and `pop` ≡ move-to-end then remove — both $O(1)$.

Algorithm 42 CLEAR()

```
1: while list.LENGTH() > 0 do
2:     list.MOVETOEND()
3:     list.REMOVE()
4: end while
```

Algorithm 43 PUSH(*item*) — list-based stack

```
1: list.APPEND(item)
```

Algorithm 44 POP() — list-based stack

```
1: if list.LENGTH() = 0 then
2:     error "Stack underflow"
3: end if
4: list.MOVETOEND()
5: list.PREV()
6: return list.REMOVE()
```

Algorithm 45 MAX() — list-based stack

```
1: if list.LENGTH() = 0 then
2:     error "Empty stack"
3: end if
4: list.MOVETOSTART()
5: maxVal ← list.GETVALUE()
6: for i = 1 to list.LENGTH() − 1 do
7:     list.NEXT()
8:     if list.GETVALUE() > maxVal then
9:         maxVal ← list.GETVALUE()
10:    end if
11: end for
12: return maxVal
```

Complexity Analysis. `push`: $O(1)$. `pop`: $O(1)$. `length`: $O(1)$. `max`: $O(n)$ (must scan all elements).

Q54. Your friend uses your above stack implementation regularly. A bug has been reported for the `max()` function of your stack implementation. Now your friend needs to implement a separate independent `max()` function which uses the stack data structure (avoiding its own `max()` function). He does not have access to the list data structure of Table A. How will he implement the `max()` function? He cannot use any data structure other than your stack implementation. **(10 marks)**

Answer

Idea. Pop all elements into a temporary stack (reversing their order), track the maximum while doing so, then push everything back to restore the original stack.

Algorithm 46 MAX(S)

```
1: if S.length() = 0 then error "Empty"
2: end if
3: temp ← new Stack
4: maxVal ← −∞
5: while S.length() > 0 do
6:     val ← S.pop()
7:     if val > maxVal then
8:         maxVal ← val
9:     end if
10:    temp.push(val)
11: end while
12: while temp.length() > 0 do
13:    S.push(temp.pop())
14: end while
15: return maxVal
```

Remark. After `max()` the original stack is fully restored. Two passes are needed because a single pop-and-push into a second stack reverses the order; popping back reverses again, giving the original order.

Complexity Analysis. $O(n)$ time (two passes of n pops/pushes). $O(n)$ space for the temporary stack.

Q55. We use a doubly linked list (DLL) to keep records of some books. In the list, all the books that belong to the same publisher are placed in consecutive nodes. Now, write a function `addBook(title, publisher)` that inserts a book into the DLL as the first book for its publisher. So, if the list already has three books for a particular publisher, the 4th book will be inserted in front of the three books already in the DLL. For a new publisher, the book should be inserted at the end of the DLL. Assume that each DLL node has 'prev' and 'next' pointers and stores the title and publisher of a book as Strings. **(10 marks)**

Answer

```cpp
#include <string>
using namespace std;

struct DLLNode {
    string title, publisher;
    DLLNode *prev, *next;
};
DLLNode *head = nullptr, *tail = nullptr;

void addBook(string title, string publisher) {
    DLLNode* newNode = new DLLNode{title, publisher,
                                   nullptr, nullptr};

    if (head == nullptr) {            // empty list
        head = tail = newNode;
        return;
    }

    // Search for the first node of this publisher
    DLLNode* cur = head;
    while (cur != nullptr && cur->publisher != publisher)
        cur = cur->next;

    if (cur == nullptr) {
        // Publisher not found: insert at END
        tail->next  = newNode;
        newNode->prev = tail;
        tail = newNode;
    } else {
        // Insert BEFORE cur (front of publisher block)
        newNode->next = cur;
        newNode->prev = cur->prev;
        if (cur->prev != nullptr)
            cur->prev->next = newNode;
        else
            head = newNode;           // becomes new head
        cur->prev = newNode;
    }
}
```

Example.

(a) `addBook("A","P1")` ⇒ `A(P1)`
(b) `addBook("B","P2")` ⇒ `A(P1)` ↔ `B(P2)`
(c) `addBook("C","P1")` ⇒ `C(P1)` ↔ `A(P1)` ↔ `B(P2)`
(d) `addBook("D","P2")` ⇒ `C(P1)` ↔ `A(P1)` ↔ `D(P2)` ↔ `B(P2)`

Complexity Analysis. $O(n)$ to find the publisher block; $O(1)$ insertion once found.

Q56. Suppose Figure 8.b-I gives a snapshot of the memory at one point. You are running your program for a linked list based list implementation. In your implementation each node has two variables: an integer *key* and a pointer *ptr*. The current situation of the list is illustrated in Figure 8.b-II. You have further implemented the concept of freelist so as to minimize the number of calls to 'new' and 'delete'. The first, second, third and fourth (i.e., last) node correspond to address 1, 2, 3 and 4 in the memory respectively. Suppose at this point a delete operation is done for the node containing 29 followed by two append operations of two nodes containing values 1111 and 2222 respectively.

Address	Values at Memory Location
0	−31
1	12
2	14
3	29
4	100
5	112
6	4
...	...

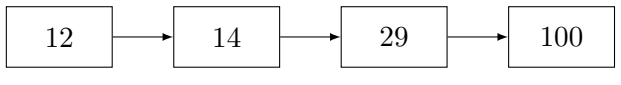


Figure 8.b-II

(i) Draw the updated list after each operation.

(ii) Provide the value of *ptr* variable for each node in Figure 8.b-II before and after the delete operation. If a node remains in the memory after your deletion operation, you would need to provide the value of the *ptr* variable of that node as well.

(iii) Provide the updated values of each memory cell after the two operations (delete and append) are complete.

You can assume that the next call of C++ 'new' operator will return the address 5. **(5+10+10=25 marks)**

Answer

Setup. The memory before any operations is:

Addr	Value	Meaning
0	−31	freelist head pointer (addr 0 stores head of freelist; −31 means no free node yet, or sentinel)
1	12	node key (1st list node)
2	14	node key (2nd list node)
3	29	node key (3rd list node)
4	100	node key (4th / last node)
5	112	*not yet allocated*
6	4	*not yet allocated*

Each node occupies **two consecutive** memory words: `[key, ptr]`. So node at address a stores its key at a and its next pointer at a+? — but from the snapshot only one word per address is shown, so we interpret the list pointers as follows (deduced from Figure 8.b-II):

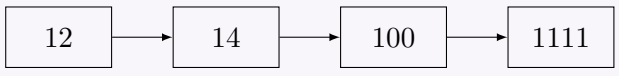
$$1 \xrightarrow{ptr} 2 \xrightarrow{ptr} 3 \xrightarrow{ptr} 4 \xrightarrow{ptr} \texttt{null}$$

(i) Updated list after each operation.

Op 1: Delete node 29 (address 3). Node 2 (key=14) must now point to node 4 (key=100). Node 3 is returned to the freelist; the freelist head points to address 3.

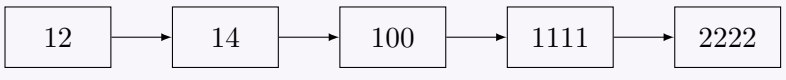
(node 29 removed; addr 3 in freelist)

Op 2: Append 1111. The freelist is non-empty (head = 3); reuse address 3. Set key at addr 3 to 1111, ptr to null. Node 4 (key=100) now points to addr 3.



(addr 3 reused for 1111)

Op 3: Append 2222. Freelist is now empty; call `new`, which returns address 5 (given). Set key at addr 5 to 2222, ptr to null. Node at addr 3 (key=1111) points to addr 5.



(ii) Value of `ptr` for each node — before and after delete.

Node (addr)	Key	ptr before delete	ptr after delete
1	12	2	2 (unchanged)
2	14	3	4 (skip deleted node)
3	29	4	freelist ptr (e.g. `null` or previous freelist head)
4	100	`null`	`null` (unchanged)

Node 3 remains in memory (it becomes the new freelist head) but is no longer part of the active list.

(iii) Updated memory cells after all three operations (delete 29, append 1111, append 2222).

Addr	New value	Explanation
0	`null`	freelist head = null (addr 3 was reused, freelist empty)
1	12	unchanged
2	14	unchanged
3	1111	reused for first append (key changed from 29 to 1111)
4	100	unchanged; its ptr now points to 3
5	2222	newly allocated via `new` for second append
6	`null`	ptr of node 5 (end of list)

Remark. The freelist pattern avoids repeated `new`/`delete` calls. Deleted nodes are chained via their `ptr` field; the next allocation pops from this chain in $O(1)$ instead of calling the OS allocator.

BUET · CSE 105

DSA

Trees & Tree Traversals

Binary Trees, General Trees, Traversal Algorithms

T5 — Trees & Tree Traversals

Q1. Analyze the maximum and minimum number of nodes in a binary tree of height h. **(10 marks)** [CSE 105 | 2023–24]

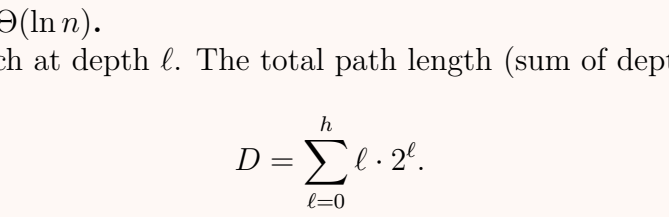
Answer

Let h denote the height of a binary tree (number of edges on the longest root-to-leaf path).
Minimum nodes. A binary tree of height h has at least $h+1$ nodes. The extreme case is a *skewed* (degenerate) tree in which every internal node has exactly one child, forming a single path of $h+1$ nodes.

$$N_{\min}(h) = h + 1$$

Maximum nodes. A binary tree of height h has at most $2^{h+1} - 1$ nodes. This maximum is achieved by a *perfect* binary tree, where every level $\ell \in \{0, 1, \ldots, h\}$ is completely filled with 2^ℓ nodes, giving

$$N_{\max}(h) = \sum_{\ell=0}^{h} 2^\ell = 2^{h+1} - 1.$$



Left: perfect tree ($h = 2$, $N = 7$). Right: right-skewed tree ($h = 2$, $N = 3$).

Complexity Analysis. Summary: for a binary tree of height h, $h + 1 \le N \le 2^{h+1} - 1$.

Q2. Prove the following theorems that describe the properties of perfect binary trees:

(a) A perfect tree has $2^{h+1} - 1$ nodes.

(b) The average depth of a node is $\Theta(\ln n)$.

(4+6=10 marks) [CSE 105 | 2022–23]

Answer

(a) A perfect binary tree of height h has $2^{h+1} - 1$ nodes.
Proof by induction on h.
Base case ($h = 0$): the tree is a single root node, so $N = 1 = 2^{0+1} - 1 = 1$. ✓
Inductive step: assume every perfect binary tree of height $h - 1$ has $2^h - 1$ nodes. A perfect binary tree of height h consists of a root plus two perfect subtrees of height $h - 1$.

$$N(h) = 1 + 2 \cdot N(h-1) = 1 + 2(2^h - 1) = 1 + 2^{h+1} - 2 = 2^{h+1} - 1. \qquad \square$$

(b) The average depth of a node is $\Theta(\ln n)$.
Level ℓ ($0 \le \ell \le h$) contains 2^ℓ nodes, each at depth ℓ. The total path length (sum of depths) is

$$D = \sum_{\ell=0}^{h} \ell \cdot 2^\ell.$$

Using the identity $\sum_{\ell=0}^{h} \ell\, 2^\ell = (h-1)2^{h+1} + 2$,

$$D = (h-1)\, 2^{h+1} + 2.$$

With $n = 2^{h+1} - 1$ nodes, $h = \log_2(n+1) - 1 = \Theta(\log n)$, so $2^{h+1} = n + 1$.

$$\text{Average depth} = \frac{D}{n} = \frac{(h-1)(n+1) + 2}{n} \approx h - 1 = \Theta(\log n) = \Theta(\ln n). \qquad \square$$

(Since $\log_2 n = \frac{\ln n}{\ln 2}$, $\Theta(\log_2 n) = \Theta(\ln n)$.)

Q3. You are given a tree of size n as an array `parent[0..n-1]`, where every index i represents a node and `parent[i]` is its immediate parent (-1 for the root). Write an algorithm to find the height of the generic tree.

```
Input: parent[] = {-1, 0, 0, 0, 3, 1, 1, 2}   Output: 2
```

(15 marks) [CSE 203 | 2021–22]

Answer

Algorithm (BFS / level-order from root):

Algorithm 47 HeightFromParentArray($parent[0..n-1]$)

```
Build adjacency list: for each i, add i to children[parent[i]]
Find root r where parent[r] = −1
Enqueue r; h ← −1
while queue not empty do
    sz ← queue.size()
    for j ← 1 to sz do
        v ← dequeue()
        for each c ∈ children[v] do
            enqueue c
        end for
    end for
    h ← h + 1
end while
return h
```

Simulation on `parent[] = {-1,0,0,0,3,1,1,2}`:
Adjacency list: $0 \to [1,2,3]$, $1 \to [5,6]$, $2 \to [7]$, $3 \to [4]$; root $= 0$.

- Level 0: $\{0\} \Rightarrow h = 0$
- Level 1: $\{1,2,3\} \Rightarrow h = 1$
- Level 2: $\{5,6,7,4\} \Rightarrow h = 2$; queue empty.

Output: **2** ✓

Complexity Analysis. Building adjacency list: $O(n)$. BFS visits each node once: $O(n)$. Total: $\Theta(n)$.

Q4. Write two procedures for finding the successor element and the predecessor element in a binary search tree. Write the preorder, inorder, and postorder traversal sequences of the binary tree whose Euler tour traversal sequence is `a e h i h e a c b f b g b c d j d c a`. Also draw the binary tree. **(10+15 marks)** [CSE 203 | 2016–17]

Answer

BST Successor and Predecessor.

```
// Returns the node with the smallest key >= node->key
treeNode* successor(treeNode* node) {
    if (node->right != NULL)          // Case 1: right subtree exists
        return minimum(node->right);  //   leftmost node there
    // Case 2: go up until we go right
    treeNode* parent = node->parent;
    while (parent != NULL && node == parent->right) {
        node   = parent;
        parent = parent->parent;
    }
    return parent;
}

// Returns the node with the largest key <= node->key
treeNode* predecessor(treeNode* node) {
    if (node->left != NULL)           // Case 1: left subtree exists
        return maximum(node->left);   //   rightmost node there
    // Case 2: go up until we go left
    treeNode* parent = node->parent;
    while (parent != NULL && node == parent->left) {
        node   = parent;
        parent = parent->parent;
    }
    return parent;
}
```

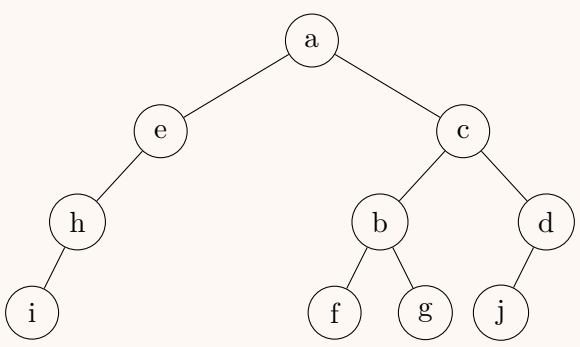
Euler Tour → traversals.
Euler tour: `a e h i h e a c b f b g b c d j d c a`
Rules:

- **Preorder**: first visit of each node.
- **Inorder**: middle visit (between left and right subtrees).
- **Postorder**: last visit of each node.

Annotating the tour ($1^{st}/2^{nd}/3^{rd}$ visit for each node):

Node	Pre (1st)	In (2nd)	Post (3rd)
a	1st pos: a	7th: a	19th: a
e	2nd: e	6th: e	— (only 2 visits ⇒ leaf's 1st=in=post)
…			

Working through the tour step by step to build the tree:



- **Preorder**: `a e h i c b f g d j`
- **Inorder**: `i h e a f b g c j d`
- **Postorder**: `i h e f g b j d c a`

Q5. Suppose the tree is an unordered tree (i.e., the order of children is not relevant). Determine whether each of the following breadth-first traversals is valid or not, with justification:

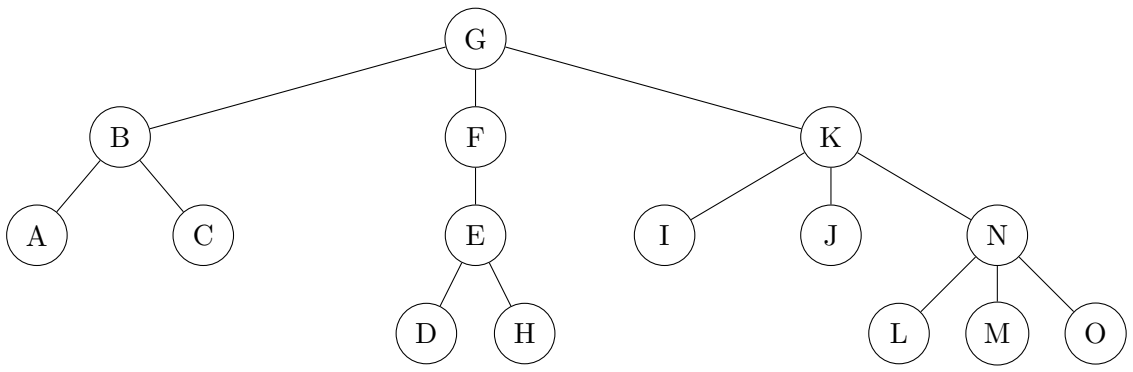
(a) `G F B K E C A I N J D H M L O`

(b) `G F B K E C A I N J D H M L O`

(c) `G K F B I J N E A C O M L H D`

(d) `G F B K E C A I N J O M L H D`

(16 marks) [CSE 203 | 2015–16]



Answer

In a **BFS (level-order)** traversal of an *unordered* tree, the following rules must hold:

- The root appears first.
- Every node appears *after* its parent.
- All nodes at depth d must appear before any node at depth $d+1$ (no interleaving of levels).
- Since the tree is *unordered*, siblings may appear in any relative order.

From the given tree, the depth-level partition is:

$$\underbrace{\{G\}}_{\text{depth 0}} \quad \underbrace{\{B,\, F,\, K\}}_{\text{depth 1}} \quad \underbrace{\{A,\, C,\, E,\, I,\, J,\, N\}}_{\text{depth 2}} \quad \underbrace{\{D,\, H,\, L,\, M,\, O\}}_{\text{depth 3}}$$

(a) `G F B K E C A I N J D H M L O`

Valid. The depth-1 block `F B K` is a permutation of $\{B, F, K\}$ ✓. The depth-2 block `E C A I N J` is a permutation of $\{A, C, E, I, J, N\}$ ✓. The depth-3 block `D H M L O` is a permutation of $\{D, H, L, M, O\}$ ✓. No level is interleaved with another, and every node appears after its parent. All BFS conditions are satisfied.

(b) `G F B K E C A I N J D H M L O`

Valid. This sequence is identical to (i); the same argument applies.

(c) `G K F B I J N E A C O M L H D`

Valid. Depth-1 block `K F B` is a permutation of $\{B, F, K\}$ ✓. Depth-2 block `I J N E A C` is a permutation of $\{A, C, E, I, J, N\}$ ✓ and each child appears after its parent (I, J, N follow K; E follows F; A, C follow B) ✓. Depth-3 block `O M L H D` is a permutation of $\{D, H, L, M, O\}$ ✓. No level interleaving occurs. All BFS conditions are satisfied.

(d) `G F B K E C A I N J O M L H D`

Invalid. Consider the relative positions of `J` and `O`: `J` is at depth 2, while `O` is at depth 3. In the sequence, `O` appears at position 11 whereas `J` appears at position 10 — a depth-3 node (`O`) is listed *before* the depth-2 node (`J`). This violates the level-order requirement that all depth-2 nodes must be exhausted before any depth-3 node appears.

Q6. Suppose that we are given the following definition of a rooted binary tree (**not binary search tree**):

```
struct treeNode
{
    int item ;               //stores the value
    struct treeNode * left ; //points to left child
    struct treeNode * right ;//points to right child
} ;
struct treeNode * root ; //global variable to store tree root node
```

Implement the following function *clone* (**must be recursive**) that makes a copy of a binary tree (keeping the original tree intact). The function will be called using the original root pointer as input argument. The function should return the root node of the new copy. For example calling $x = clone(root)$ will return the new root (of the tree copy) into variable x.

```
struct treeNode * clone(struct treeNode * node);
```

(12 marks) [CSE 203 | 2014–15]

Answer

```
struct treeNode* clone(struct treeNode* node)
{
    // Base case: empty subtree
    if (node == NULL)
        return NULL;

    // Allocate a new node and copy the value
    struct treeNode* newNode =
        (struct treeNode*)malloc(sizeof(struct treeNode));
    newNode->item = node->item;

    // Recursively clone left and right subtrees
    newNode->left  = clone(node->left);
    newNode->right = clone(node->right);

    return newNode;
}
```

Complexity Analysis. Each node is visited exactly once (post-order fashion). Time: $\Theta(n)$. Space (stack): $O(h)$, where h is the tree height.

Q7. What is a proper binary tree? Construct the proper binary tree whose inorder and postorder traversals are given as:

- **Inorder:** `BUETCSE12`
- **Postorder:** `BEUCTES21`

(2+8=10 marks) [CSE 203 | 2013–14]

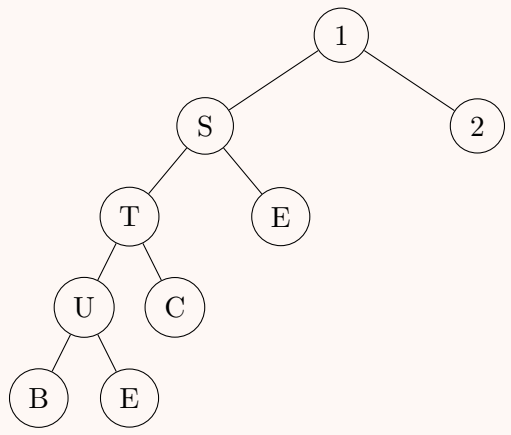
Answer

A **proper binary tree** is one where every internal node has exactly 2 children (no node has exactly 1 child).

Reconstruction (last of postorder = root; split inorder):

Step	Root	Left Inorder	Right Inorder
1	**1**	`B U E T C S E`	`2`
2	**S** (last of remaining postorder `B E U C T E S 2`)	`B U E T C`	`E`
3	**T**	`B U E`	`C`
4	**U**	`B`	`E`

Tree:

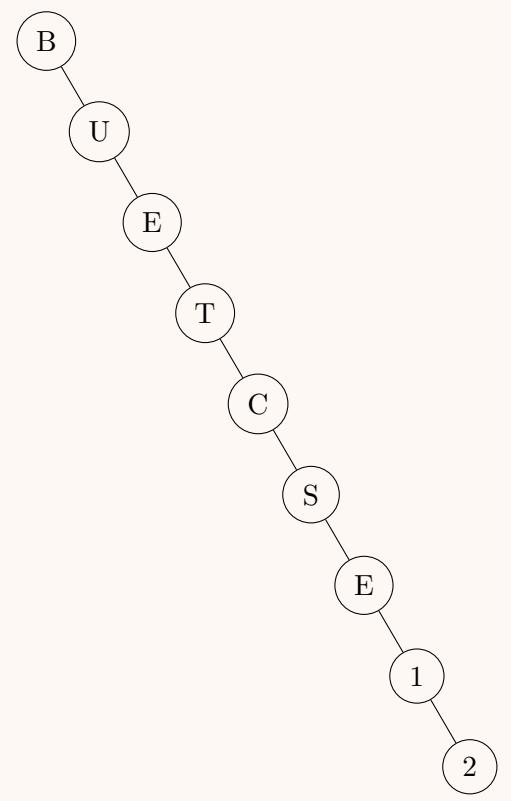


Remark. Verify: Inorder (left-root-right): B U E T C S E 1 2 ✓ Postorder (left-right-root): B E U C T E S 2 1 ✓

Q8. Draw a binary tree whose preorder and inorder traversals both yield `BUETCSE12`. **(7 marks)** [CSE 203 | 2013–14]

Answer

If **preorder** = **inorder** = `B U E T C S E 1 2`, then at every step the root is the first element of the preorder list, and it must also be the first element of the inorder list (otherwise there would be a non-empty left subtree, contradicting preorder = inorder). This forces every node to have an *empty left subtree*, i.e. the tree is a **right-skewed chain**.



Remark. Preorder (root, right): B U E T C S E 1 2 ✓ Inorder (left=empty, root, right): B U E T C S E 1 2 ✓

Q9. What is a tree? Write a linear-time algorithm for finding the height of a tree. Analyze the time complexity of the algorithm. **(1+4+4=9 marks)** [CSE 203 | 2012–13 , 2011–12]

Answer

Tree definition (same as above — connected, acyclic graph with a root).
Linear-time algorithm using BFS (iterative level-order):

Algorithm 48 HeightBFS(*root*)

```
1: if root is null then
2:     return −1
3: end if
4: Enqueue root; h ← −1
5: while queue is not empty do
6:     size ← queue.size()
7:     for i ← 1 to size do
8:         v ← dequeue()
9:         for each child c of v do
10:            enqueue c
11:        end for
12:    end for
13:    h ← h + 1
14: end while
15: return h
```

Complexity Analysis. Each node is enqueued and dequeued exactly once ⇒ time $\Theta(n)$, space $O(w)$ where w is the maximum width of the tree (worst case $O(n)$).

Q10. Let T be a proper binary tree with height h. Prove that the number of internal nodes in T is at least h and at most $2^h - 1$. **(4+4=8 marks)** [CSE 203 | 2012–13 , 2011–12]

Answer

A **proper** (full) binary tree is one in which every node has either 0 or 2 children (no node has exactly one child).
Let I = number of internal nodes, L = number of leaves. For any proper binary tree: $L = I + 1$ and $N = 2I + 1$.
Lower bound: $I \geq h$.
The path from root to a deepest leaf has length h; each node on that path except the leaf is an internal node. Hence there are at least h internal nodes.
Upper bound: $I \leq 2^h - 1$.
Proof by induction on h.
Base ($h = 0$): a single node is a leaf, so $I = 0 \leq 2^0 - 1 = 0$. ✓
Step: assume proper binary trees of height $\leq h - 1$ have at most $2^{h-1} - 1$ internal nodes. A proper binary tree of height h has a root (1 internal node) and two proper subtrees of height $\leq h - 1$, so

$$I \leq 1 + (2^{h-1} - 1) + (2^{h-1} - 1) = 2^h - 1. \qquad \square$$

Q11. Construct the binary tree whose in-order and post-order traversals are given as:

- **Inorder:** `i f d j g c a b k h m e`
- **Postorder:** `i j g d f k m h e b a c`

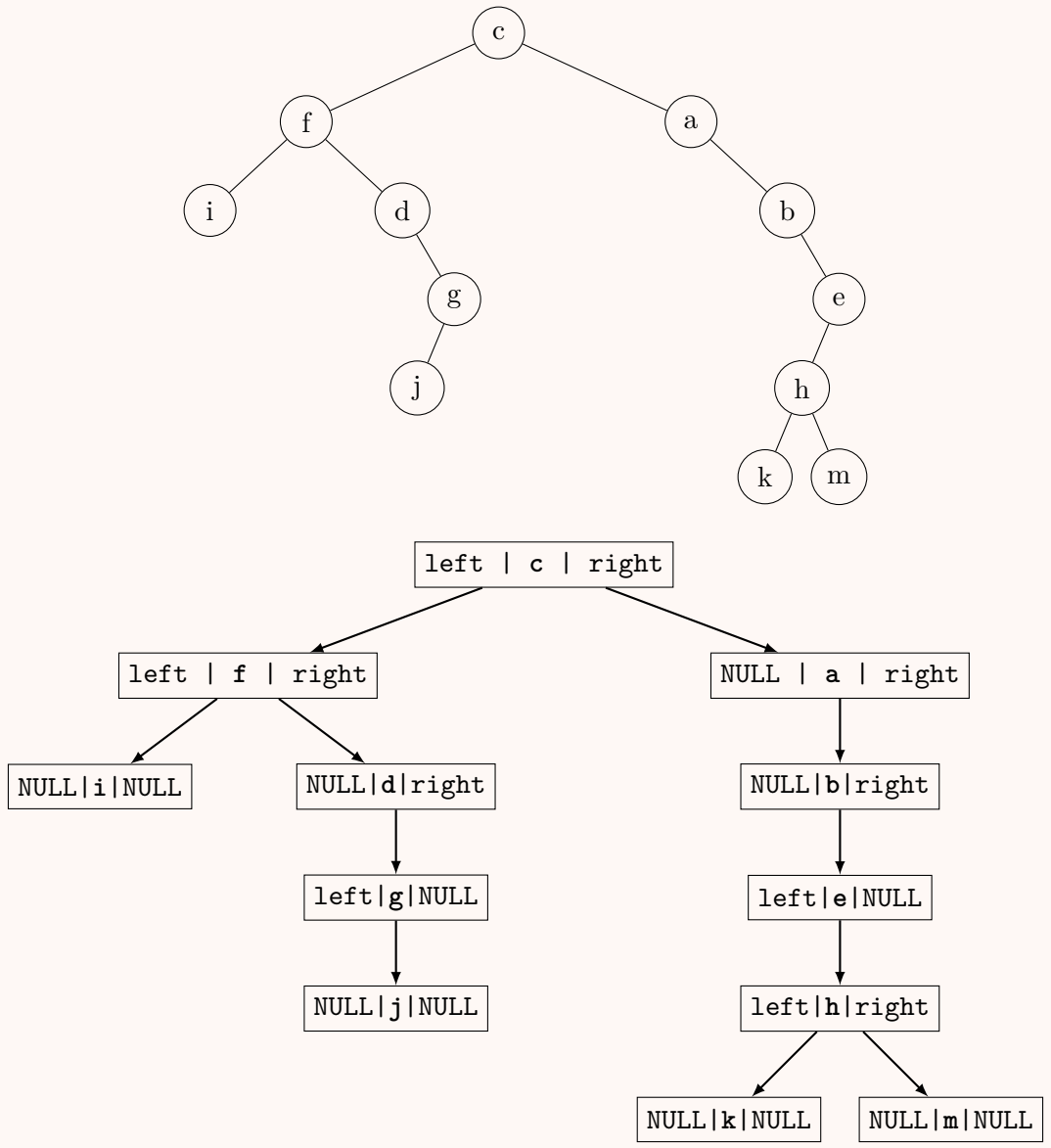
Draw the linked structure for the resulting binary tree. **(10+6=16 marks)** [CSE 203 | 2012–13]

Answer

Key idea: the last element of postorder is the root; find it in inorder to split left/right subtrees; recurse.

Step	Root	Left inorder / Right inorder
1	**c** (last of postorder)	`i f d j g` / `a b k h m e`
2	Left subtree root: **f**	`i` / `d j g`
3	Left of f: **i** (leaf)	—
4	Right of f root: **d**	/ `j g`
5	Left of d: (empty)	
6	Right of d root: **g**	`j` / (wait—inorder `j g` ⇒ `j` left of g) left: **j** (leaf), right: empty
7	Right subtree of c root: **a**	/ `b k h m e`
8	Right of a root: **b**	/ `k h m e`
9	Right of b root: **e**	`k h m` /
10	Left of e root: **h** left: **k** (leaf), right: **m** (leaf)	`k` / `m`

Resulting tree:



NULL pointers not drawn as arrows (indicated by NULL in node).

BUET · CSE 105

DSA

Heaps & Binary Search Trees

BST Operations, Max/Min Heap, Priority Queue, Build-Heap

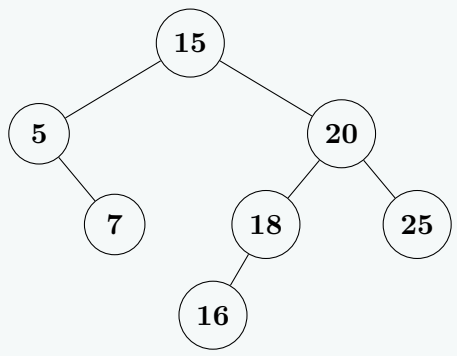
T6 — Heaps & Binary Search Trees

Q1. Show the binary search tree (BST) that results from inserting the values 15, 20, 25, 18, 16, 5, and 7 (in that order). Write the enumerations of the tree that result from pre-order, in-order, and post-order traversal of the tree. Write a recursive algorithm that takes as input the pointer to the root of a BST and a value X, and returns `true` if the value X appears in the tree and `false` otherwise. Modify the algorithm if the tree is a generalized binary tree. In both cases, analyze the time complexity of your algorithms. **(25 marks)** [CSE 105 | 2023–24]

Answer

Step 1: Build the BST by inserting values in order.
Insert 15: root. Insert 20: $20 > 15$, goes right of 15. Insert 25: $25 > 15$, go right; $25 > 20$, goes right of 20. Insert 18: $18 > 15$, go right; $18 < 20$, goes left of 20. Insert 16: $16 > 15$, go right; $16 < 20$, go left; $16 < 18$, goes left of 18. Insert 5: $5 < 15$, goes left of 15. Insert 7: $7 < 15$, go left; $7 > 5$, goes right of 5.

Final BST:



Step 2: Tree Traversals.

- **Pre-order** (Root → Left → Right): 15, 5, 7, 20, 18, 16, 25
- **In-order** (Left → Root → Right): 5, 7, 15, 16, 18, 20, 25 *(always sorted!)*
- **Post-order** (Left → Right → Root): 7, 5, 16, 18, 25, 20, 15

Step 3: Recursive BST Search Algorithm.

Algorithm 49 BstSearch($root$, X)

```
1: if root = nil then
2:     return false
3: end if
4: if root.key = X then
5:     return true
6: end if
7: if X < root.key then
8:     return BstSearch(root.left, X)
9: else
10:    return BstSearch(root.right, X)
11: end if
```

How it works: At each node we exploit the BST property. If X equals the current key, we are done. If X is smaller, the answer can only lie in the left subtree; if larger, in the right subtree. This halves the search space at each level.

Step 4: Modification for a General Binary Tree.
In a general (non-BST) binary tree, there is no ordering property, so we cannot prune. We must search *both* subtrees:

Algorithm 50 GeneralSearch($root$, X)

```
1: if root = nil then
2:     return false
3: end if
4: if root.key = X then
5:     return true
6: end if
7: return GeneralSearch(root.left, X) or GeneralSearch(root.right, X)
```

Complexity Analysis. **BST Search:**

- **Best case:** $O(1)$ — X is the root.
- **Average case:** $O(\log n)$ — for a balanced BST with n nodes, height $= O(\log n)$.
- **Worst case:** $O(n)$ — for a completely skewed BST (degenerate tree resembling a linked list), height $= n - 1$.

General Binary Tree Search: Always $\Theta(n)$ worst case, since every node may need to be visited.

Q2. Assume you are given a set of integer numbers. Design a data structure and associated algorithms that support finding the minimum and maximum of the numbers in constant time, and insertion, deletion of maximum, and deletion of minimum in $O(\log n)$ time. *Hint: use two different heaps on the same set of numbers; however, remember their mutual dependencies.* **(15 marks)** [CSE 105 | 2023–24]

Answer

Idea. Maintain the *same* set of n numbers simultaneously in two heap structures:

- A **Max-Heap** H_{max}: root always holds the **maximum**.
- A **Min-Heap** H_{min}: root always holds the **minimum**.

Each element has a pointer in both heaps so that its position in one heap is known from the other. This is the key *mutual dependency*.

Operations:
(a) `findMax()` — $O(1)$**:** Return `H_max.root`.
(b) `findMin()` — $O(1)$**:** Return `H_min.root`.
(c) `insert(x)` — $O(\log n)$**:**

Algorithm 51 INSERT(x)
1: Insert x into H_{max}
2: Insert x into H_{min}
3: Record cross-pointers between x's positions in both heaps

(d) `deleteMax()` — $O(\log n)$**:**

Algorithm 52 DELETEMAX()
1: $m \leftarrow H_{max}.root$
2: Use cross-pointer to locate m's position p in H_{min}
3: Remove m from H_{max}
4: Remove m from H_{min} at position p via pointer **return** m

(e) `deleteMin()` — $O(\log n)$**:** Symmetric to `deleteMax`.

Complexity Analysis.

Operation	Time	Reason
`findMin`, `findMax`	$O(1)$	Read heap root
`insert`	$O(\log n)$	Bubble-up in both heaps
`deleteMax`, `deleteMin`	$O(\log n)$	Heapify in both heaps

Remark. The cross-pointer is the critical design choice. Without it, finding an arbitrary element in a heap takes $O(n)$. With it, deletion at a known position takes $O(\log n)$ using heap fix-up operations.

Q3. The key at index 6 of the following max priority queue has been increased to 32. Re-establish the max-heap property of the queue and illustrate the steps using the array only. Justify whether you can use the `max_heapify` algorithm to re-establish the heap property.

1	2	3	4	5	6	7	8	9	10	11	12	13
25	23	20	15	18	16	12	14	7	8	11	4	14

(12 marks) [CSE 105 | 2022–23]

Answer — Max-Heap: Increase Key at Index 6

Original heap (1-indexed):

$$[25,\ 23,\ 20,\ 15,\ 18,\ \mathbf{16},\ 12,\ 14,\ 7,\ 8,\ 11,\ 4,\ 14]$$

Index 6 (value 16) increased to **32**:

$$[25,\ 23,\ 20,\ 15,\ 18,\ \mathbf{32},\ 12,\ 14,\ 7,\ 8,\ 11,\ 4,\ 14]$$

Why not `max_heapify`? `max_heapify` fixes violations by pushing a node *downward*. Here the key *increased*, so the violation is upward (node may be larger than its parent). We need **HEAP-INCREASE-KEY** (bubble up).

Bubble-up steps:
Step 1: heap[6] $= 32 >$ heap[3] $= 20$ (parent of 6 is $\lfloor 6/2 \rfloor = 3$). **Swap.**

$$[25,\ 23,\ \mathbf{32},\ 15,\ 18,\ \mathbf{20},\ 12,\ 14,\ 7,\ 8,\ 11,\ 4,\ 14]$$

Step 2: heap[3] $= 32 >$ heap[1] $= 25$ (parent of 3 is 1). **Swap.**

$$[\mathbf{32},\ 23,\ \mathbf{25},\ 15,\ 18,\ 20,\ 12,\ 14,\ 7,\ 8,\ 11,\ 4,\ 14]$$

Step 3: Node is now at root (index 1). **Stop.**
Final valid max-heap:

$$\boxed{[32,\ 23,\ 25,\ 15,\ 18,\ 20,\ 12,\ 14,\ 7,\ 8,\ 11,\ 4,\ 14].}\ \checkmark$$

Q4. Given two max-priority queues represented by two arrays, write an efficient algorithm to merge them into a single max-priority queue. Analyse the time complexity of your algorithm. **(17 marks)** [CSE 105 | 2022–23]

Answer

Key Idea. Concatenate both arrays into one, then run the standard BUILD-MAX-HEAP procedure bottom-up. This avoids inserting elements one by one (which would cost $O((m+n)\log(m+n))$) and instead exploits the linear build-heap fact.

Algorithm 53 MAX-HEAPIFY(A, i, n)

```
1: left ← 2i + 1;   right ← 2i + 2;   largest ← i
2: if left < n and A[left] > A[largest] then
3:     largest ← left
4: end if
5: if right < n and A[right] > A[largest] then
6:     largest ← right
7: end if
8: if largest ≠ i then
9:     SWAP(A[i], A[largest])
10:    MAX-HEAPIFY(A, largest, n)
11: end if
```

Algorithm 54 MERGE-MAX-HEAPS(A, m, B, n)

```
1: C ← new array of size m + n
2: for i = 0 to m − 1 do
3:     C[i] ← A[i]
4: end for
5: for i = 0 to n − 1 do
6:     C[m + i] ← B[i]
7: end for
8: total ← m + n
9: for i = ⌊total/2⌋ − 1 downto 0 do
10:    MAX-HEAPIFY(C, i, total)
11: end for
12: return C
```

Correctness. After Step 1 the combined array C contains all $m+n$ elements. BUILD-MAX-HEAP (the bottom-up loop) restores the max-heap property for every subtree, guaranteeing that C is a valid max-heap upon return.

Complexity Analysis.

- **Concatenation (Step 1):** $O(m+n)$
- **Build-Max-Heap (Step 2):** $O(m+n)$ — each call to MAX-HEAPIFY costs $O(\log(m+n))$, but the sum over all $\lfloor (m+n)/2 \rfloor$ calls is $O(m+n)$ by the standard geometric series argument.
- **Total: $O(m+n)$**

Remark. An alternative using a Fibonacci heap supports UNION in $O(1)$ amortised, but requires a more complex structure. The array-based BUILD-MAX-HEAP approach above is simpler and optimal for array-represented heaps.

Q5. Write a code snippet to traverse the binary tree in an order so that you can store it in an array. Show the placement of the node numbers of the given tree in an array. Write a code to access the parent of a given node. What will be the index no of the array for node 12?**(10+3+8+4=25 marks)** [CSE 105 | 2022–23]

Answer

Key Idea. Use **level-order (BFS) traversal** to fill the array. For a 1-indexed array, node at index i has:

- Left child at $2i$, right child at $2i+1$.
- Parent at $\lfloor i/2 \rfloor$.

Level-order traversal code:

```cpp
#include <queue>
#include <vector>
using namespace std;
struct Node { int val; Node *left, *right; };

// Returns 1-indexed level-order array
vector<int> levelOrderArray(Node* root) {
    vector<int> arr = {0};          // index 0 unused
    if (!root) return arr;
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* cur = q.front(); q.pop();
        arr.push_back(cur->val);
        if (cur->left)  q.push(cur->left);
        if (cur->right) q.push(cur->right);
    }
    return arr;
}

// Access parent of node at index i (1-indexed)
int parentIndex(int i) { return i / 2; }
int parentValue(vector<int>& arr, int i) {
    if (i <= 1) return -1;          // root has no parent
    return arr[i / 2];
}
```

Tree from figure (level-order placement):

Index	1	2	3	4	5	6	7	8	9	10	11	12
Value	3	9	5	14	10	6	8	17	15	13	23	12

Verification of structure:

- Root (idx 1) = 3; left child = arr[2]=9; right child = arr[3]=5
- 9 (idx 2): left=arr[4]=14, right=arr[5]=10
- 5 (idx 3): left=arr[6]=6, right=arr[7]=8
- 14 (idx 4): left=arr[8]=17, right=arr[9]=15
- 10 (idx 5): left=arr[10]=13, right=arr[11]=23
- 6 (idx 6): left=arr[12]=**12**

Node 12 is at index 12.
Parent of index 12 = $\lfloor 12/2 \rfloor = 6 \Rightarrow$ arr[6] = **6**.

Parent access code:

```cpp
// Given 1-indexed array arr[] of size n+1
// Find parent of the node at index i
int getParent(int arr[], int i) {
    if (i <= 1) return -1;        // root has no parent
    return arr[i / 2];            // O(1) direct access
}
// Example: getParent(arr, 12) = arr[6] = 6
```

Complexity Analysis. Level-order traversal: $O(n)$. Parent access: $O(1)$.

Q6. You are given the following numbers to insert into an empty Binary Search Tree (BST): $5, 7, 8, 12, 15, 27$. Determine which insertion order below would yield the tree with the least height:

(a) $15, 5, 27, 8, 7, 12$

(b) $12, 7, 15, 27, 5, 8$

(c) $8, 27, 7, 5, 15, 12$

(d) $7, 5, 12, 8, 15, 27$

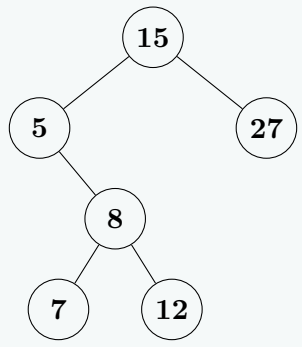
(10 marks) [CSE 105 | 2021–22]

Detailed Answer

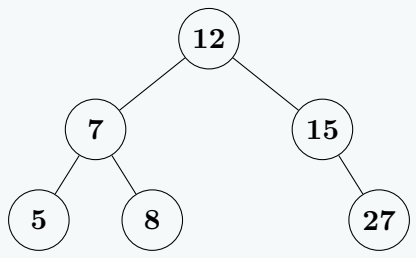
Insert values $\{5, 7, 8, 12, 15, 27\}$ in each given order to construct the Binary Search Trees and measure their resulting heights. (Assuming height is defined as the number of edges on the longest path from root to a leaf, i.e., root is at level 0).

Option	Insertion Order	Height	
(a)	$15, 5, 27, 8, 7, 12$	3	
(b)	$\mathbf{12, 7, 15, 27, 5, 8}$	**2**	← **minimum**
(c)	$8, 27, 7, 5, 15, 12$	3	
(d)	$7, 5, 12, 8, 15, 27$	3	

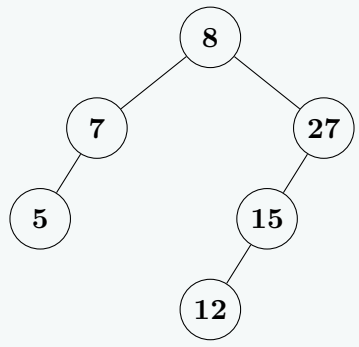
Tree for Option (a): $15, 5, 27, 8, 7, 12$
Root is 15. The longest path is $15 \to 5 \to 8 \to 7$ (or 12).
Height = 3.



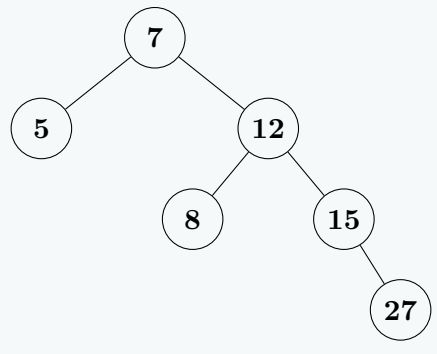
Tree for Option (b) [Minimum Height]: $12, 7, 15, 27, 5, 8$
Root is 12. The longest path reaches level 2 (e.g., $12 \to 7 \to 5$).
Height = 2. ✓



Tree for Option (c): $8, 27, 7, 5, 15, 12$
Root is 8. The longest path is $8 \to 27 \to 15 \to 12$.
Height = 3.



Tree for Option (d): $7, 5, 12, 8, 15, 27$
Root is 7. The longest path is $7 \to 12 \to 15 \to 27$.
Height = 3.



Conclusion: Inserting the median value (12) first as the root (Option b) creates a roughly balanced split between the left and right subtrees, minimising skewness. Options (a), (c), and (d) all produce trees with a height of 3 due to poorer balance.

Q7. A perfect balanced BST requires all interior nodes to contain two children and all leaf nodes to be on the same level. A perfect balanced BST of $h = 0$ would have $n = 1$ node, $h = 1$ would have $n = 3$ nodes, $h = 2$ would have $n = 7$ nodes, and so on. Providing appropriate arguments, derive the worst-case running time, in terms of n, for successfully searching for an element in a perfect balanced BST. **(10 marks)** **[CSE 105 | 2021–22]**

Answer

Structure of a perfect balanced BST.
A perfect balanced BST of height h has:

$$n = 2^{h+1} - 1 \quad \text{nodes.}$$

For example: $h = 0 \Rightarrow n = 1$; $h = 1 \Rightarrow n = 3$; $h = 2 \Rightarrow n = 7$.
Worst-case search.
In a successful search, the deepest level visited is the leaf level at depth h. Thus the maximum number of comparisons $= h + 1$ (one per level including root).
Derivation in terms of n:
From $n = 2^{h+1} - 1$:

$$n + 1 = 2^{h+1} \implies h + 1 = \log_2(n+1) \implies h = \log_2(n+1) - 1.$$

Maximum comparisons $= h + 1 = \log_2(n+1)$.

Complexity Analysis.

$$T_{\text{worst}}(n) = \log_2(n+1) = \Theta(\log n).$$

This is the number of nodes visited on the path from root to the deepest leaf. In a perfect balanced BST each comparison eliminates half the remaining search space, leading to the logarithmic bound. For an unsuccessful search (not found), the same bound holds since we reach a null pointer at depth $h + 1$.

Q8. Present a recursive method `countInRange(root, low, high)`, without any helper functions, to count in a BST all keys that are within a given range. Assume keys in the BST are unique (no duplicates). For example, if the keys in a BST are 4, 7, 10, 14, 15, 17, 20, 30, then `countInRange(root, 8, 17)` returns 4 and
`countInRange(root, 21, 29)` returns 0. For your convenience a code snippet in C++is given below , but you need not follow any particular programming language . **(10 marks)** [CSE 105 | 2021–22]

```
public class BSTNode {
    int key;
    Value value;
    BSTNode left;
    BSTNode right;
    ...
}
// counts number of keys in BST that are in the range low to high, inclusive
// i.e. all keys k such that low <= k <= high
public static int countInRange(BSTNode root, int low, int high) {
    // COMPLETE THIS METHOD
}
```

Answer

Key insight: The BST ordering property lets us prune the search. If `root.key` is below `low`, the entire left subtree is also below `low` (skip it). If `root.key` is above `high`, the entire right subtree is also above (skip it).

```
public static int countInRange(BSTNode root, int low, int high) {
    if (root == null) return 0;              // base case: empty subtree

    // Current key is within range: count it + recurse on both subtrees
    if (low <= root.key && root.key <= high) {
        return 1
             + countInRange(root.left,  low, high)
             + countInRange(root.right, low, high);
    }

    // Current key < low: left subtree entirely out of range, go right only
    if (root.key < low) {
        return countInRange(root.right, low, high);
    }

    // Current key > high: right subtree entirely out of range, go left only
    return countInRange(root.left, low, high);
}
```

Trace on example (keys: $4, 7, 10, 14, 15, 17, 20, 30$):
`countInRange(root, 8, 17)`: Keys in $[8, 17]$ are $10, 14, 15, 17 \Rightarrow$ returns **4**. ✓
`countInRange(root, 21, 29)`: No key in $[21, 29] \Rightarrow$ returns **0**. ✓

Complexity Analysis.

- **Best case:** $O(\log n)$ — range is outside tree, we prune at every node.
- **Worst case:** $O(k + \log n)$, where k is the number of keys in range. We visit k matching nodes plus $O(\log n)$ boundary nodes.
- If all n keys are in range: $O(n)$.

Q9. Present an algorithm to delete a full node in a binary search tree. Explain how it works with illustrative examples. **(10 marks)** [CSE 105 | 2021–22]

Answer

A **full node** in a BST is a node with exactly **two children**. Deletion of such a node uses the *in-order successor* (or predecessor) strategy.
Algorithm — `deleteFullNode(N):`

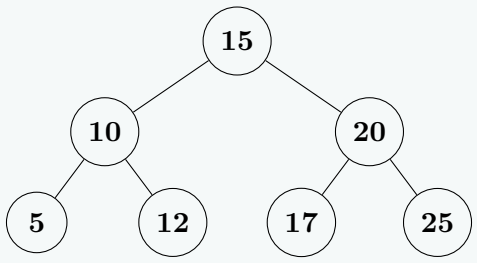
Algorithm 55 DeleteFullNode(N)

1: $successor \leftarrow$ FindMin($N.right$)
2: $N.key \leftarrow successor.key$
3: DeleteNode($N.right$, $successor.key$)

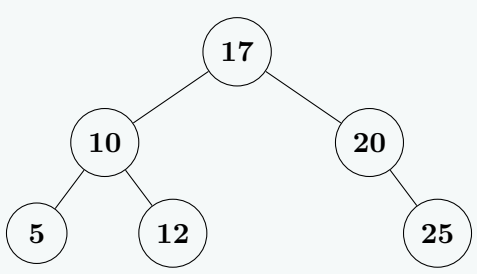
Why does this work? The in-order successor S is the *smallest* value greater than N. Replacing N's key with S's key maintains the BST property:

- All keys in N's left subtree $< S.key$ (they were $< N.key < S.key$). ✓
- All keys in N's right subtree $> S.key$ (since S was the minimum there). ✓

Illustrative Example. Delete node 15 from the BST below.



Step 1: Find in-order successor of 15 = **17** (leftmost in right subtree).
Step 2: Copy 17 into node 15. Delete 17 from right subtree (17 has no children — simple leaf deletion).
Result:



In-order before: $5, 10, 12, 15, 17, 20, 25$. In-order after: $5, 10, 12, 17, 20, 25$. ✓

Complexity Analysis. Finding the successor: $O(h)$ where h is tree height. Deleting the successor (at most one child): $O(h)$. Total: $O(h)$. For a balanced BST, $O(\log n)$; worst case $O(n)$ for skewed tree.

Q10. Consider the following array A of integers:(index start from 1)

$$A = [16,\ 4,\ 10,\ 7,\ 14,\ 9,\ 3,\ 2,\ 8,\ 1]$$

Verify whether the array is a max heap. Derive the complexity of your verification algorithm. Justify whether the algorithm `max_heapify(A, i)` will establish the max heap property of the array if applied with index $i = 2$. Analyze the time complexity of the `max_heapify` algorithm. **(20 marks)** [CSE 105 | 2021–22]

Answer

Array (1-indexed): $A = [16, 4, 10, 7, 14, 9, 3, 2, 8, 1]$
Step 1: Verify Max-Heap Property.
A max-heap requires $A[\lfloor i/2 \rfloor] \geq A[i]$ for all $i > 1$ (every parent $\geq$ its children).

Node i	Value	Children	Violation?
1	16	$A[2] = 4,\ A[3] = 10$	None
2	4	$A[4] = 7,\ A[5] = 14$	**Yes**: $4 < 7$ and $4 < 14$
3	10	$A[6] = 9,\ A[7] = 3$	None
4	7	$A[8] = 2,\ A[9] = 8$	**Yes**: $7 < 8$
5	14	$A[10] = 1$	None

Violations found at $i = 2$ and $i = 4$. The array is *not* a valid max-heap.

Complexity Analysis. **Heap Verification Algorithm:** Check all nodes $i = 1$ to $\lfloor n/2 \rfloor$ (internal nodes). Each check is $O(1)$. Total: $O(n)$.

Step 2: Apply `max_heapify(A, 2)`.

`max_heapify(A, i)` fixes a potential max-heap violation at node i by repeatedly swapping the node with its largest child and moving down.
Starting at $i = 2$, $A[2] = 4$. Children: $A[4] = 7$, $A[5] = 14$.

- **Iteration 1:** Largest child $= A[5] = 14 > A[2] = 4$. Swap $A[2] \leftrightarrow A[5]$.

  $$\text{Array: } [16, \underline{14}, 10, 7, \underline{4}, 9, 3, 2, 8, 1]$$

  Move down to $i = 5$.
- **Iteration 2:** At $i = 5$, $A[5] = 4$. Only child: $A[10] = 1$. $4 \geq 1$: no swap. **Stop.**

After `max_heapify(A, 2)`: $A = [16, 14, 10, 7, 4, 9, 3, 2, 8, 1]$
Is the result a full max-heap? Check $i = 4$: $A[4] = 7$, $A[9] = 8$: $7 < 8$ — still a violation! `max_heapify(A,2)` only fixes the subtree rooted at node 2, not violations elsewhere in the array. The full array is **still not** a max-heap.

Complexity Analysis. `max_heapify` descends at most h levels where $h = \lfloor \log_2 n \rfloor$ is the height of the heap. Each level: $O(1)$ comparisons. **Time complexity:** $O(\log n)$**.**

Q11. Calculate the number of swap operations (data interchanges) required if you apply the `Build_Max_Heap` algorithm on the array $A = \{5, 3, 17, 10, 84, 19, 6, 22, 9\}$. Justify whether the max heap produced can be used as a priority queue. Show that `Heap_Extract_Max` algorithm is essentially a part of Heapsort. Derive and compare the time complexities of Heapsort and `Heap_Extract_Max`. **(17 marks)** [CSE 105 | 2021–22]

Answer

Step 1: Build_Max_Heap — counting swaps.
Array (1-indexed): $A = [5, 3, 17, 10, 84, 19, 6, 22, 9]$, $n = 9$.
Call `max_heapify` for $i = \lfloor 9/2 \rfloor = 4$ down to 1:

i	Array before	Array after	Swaps
4	$[5, 3, 17, \mathbf{10}, 84, 19, 6, 22, 9]$	$[5, 3, 17, \mathbf{22}, 84, 19, 6, \mathbf{10}, 9]$	1
3	$[5, 3, \mathbf{17}, 22, 84, 19, 6, 10, 9]$	$[5, 3, \mathbf{19}, 22, 84, \mathbf{17}, 6, 10, 9]$	1
2	$[5, \mathbf{3}, 19, 22, 84, 17, 6, 10, 9]$	$[5, \mathbf{84}, 19, 22, \mathbf{3}, 17, 6, 10, 9]$	1
1	$[\mathbf{5}, 84, 19, 22, 3, 17, 6, 10, 9]$	$[\mathbf{84}, 22, 19, \mathbf{10}, 3, 17, 6, \mathbf{5}, 9]$	3

$$\boxed{\textbf{Total swaps} = 1 + 1 + 1 + 3 = 6}$$

Final max-heap: $[84, 22, 19, 10, 3, 17, 6, 5, 9]$ — verified valid. ✓

Step 2: Can this heap be used as a priority queue?
Yes. A max-heap is the standard implementation of a **max-priority queue**. The three required operations are all supported:

- `Insert(key)`: Add at the end, bubble up — $O(\log n)$.
- `Extract-Max()`: Remove root, replace with last, heapify down — $O(\log n)$.
- `Maximum()`: Return root — $O(1)$.

Step 3: Heap_Extract_Max is part of Heapsort.

Algorithm 56 HEAPSORT(A)

```
1: BUILD-MAX-HEAP(A)
2: for i = n downto 2 do
3:     SWAP(A[1], A[i])
4:     heap-size ← heap-size − 1
5:     MAX-HEAPIFY(A, 1)
6: end for
```

Each iteration of the loop is exactly `Heap_Extract_Max`: swap the maximum (root) to the end, reduce heap size, and call `Max-Heapify`. Thus Heapsort performs $n - 1$ repeated extract-max operations.

Complexity Analysis.

Algorithm	Time Complexity	Space
`Heap_Extract_Max` (single call)	$O(\log n)$	$O(1)$
`Heapsort` ($n - 1$ extract-max + Build-Max-Heap)	$O(n \log n)$	$O(1)$

Heapsort detail: `Build-Max-Heap` takes $O(n)$; the loop calls `Max-Heapify` $n - 1$ times, each $O(\log n)$; total $= O(n) + O(n \log n) = O(n \log n)$.

Q12. You are given two balanced binary search trees with m and n elements respectively. Write a function that merges them into a balanced BST in $O(m + n)$ time. **(15 marks)** [CSE 203 | 2021–22]

Answer

Algorithm (3 steps):

Algorithm 57 MergeBSTs($root_1$, $root_2$)

```
arr1 ← InorderToArray(root1)
arr2 ← InorderToArray(root2)
merged ← MergeSortedArrays(arr1, arr2)
return SortedArrayToBST(merged, 0, |arr1| + |arr2| − 1)
```

Algorithm 58 InorderToArray($root$)

```
if root = nil then
    return []
end if
return InorderToArray(root.left) + [root.data] + InorderToArray(root.right)
```

Algorithm 59 MergeSortedArrays(A, B)

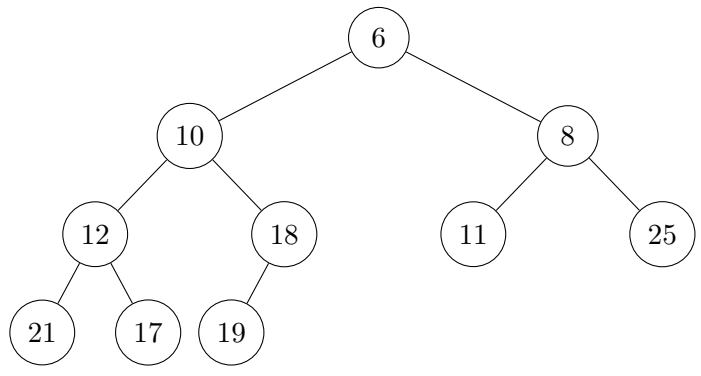
```
result ← []; i ← 0; j ← 0
while i < |A| and j < |B| do
    if A[i] ≤ B[j] then
        result.Append(A[i]); i ← i + 1
    else
        result.Append(B[j]); j ← j + 1
    end if
end while
Append remaining elements of A or B to result return result
```

Algorithm 60 SortedArrayToBST(arr, lo, hi)

```
if lo > hi then
    return nil
end if
mid ← ⌊(lo + hi)/2⌋
node ← new Node(arr[mid])
node.left ← SortedArrayToBST(arr, lo, mid − 1)
node.right ← SortedArrayToBST(arr, mid + 1, hi)
return node
```

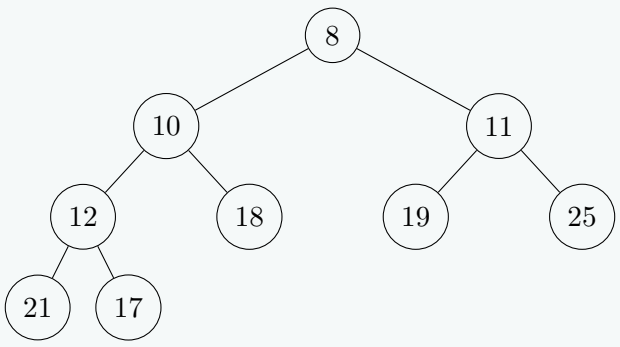
Correctness: In-order traversal of a BST gives sorted order. Merging two sorted arrays gives a globally sorted array. Building a BST from the middle element of a sorted array recursively produces a balanced BST.
Time complexity: $O(m) + O(n) + O(m+n) + O(m+n) = O(m+n)$ ✓
Space complexity: $O(m+n)$ for the merged array.

Q13. Calculate for the min heap shown in the given figure. Show the resulting min heap after deleting the minimum element (10), including intermediate steps. **(10 marks)** [CSE 203 | 2021–22]



Answer

Original min-heap array (level-order): `[6,10,8,12,18,11,25,21,17,19]`. Valid min-heap ✓.
Step 1: replace root (6) with last (19); $n = 9$. Array: `[19,10,8,12,18,11,25,21,17]`.
Step 2: heapify at 0: children 10(idx1), 8(idx2). Min=8 → swap 19↔8. Array: `[8,10,19,12,18,11,25,21,17]`.
Step 3: heapify at 2 (val 19): children 11(idx5), 25(idx6). Min=11 → swap 19↔11. Array: `[8,10,11,12,18,19,25,21,17]`.
Step 4: heapify at 5 (val 19): no children. Done.
Final min-heap: `[8,10,11,12,18,19,25,21,17]` ✓



Q14. While preparing for this exam, you have found an interesting question in a very old book with missing pages. It talks about a Binary Search Tree (BST) that stores integers in the range from 1 to 100 and proposes to search 22 and print the values encountered on the search path through the BST. Then it provides the following sequences of numbers (Sequences 1–3) and asks the following question: "Are these sequences possible search paths through the BST?"

Sequence 1 : 100, 40, 20, 30, 23, 35, 22

Sequence 2 : 100, 35, 15, 32, 20, 22

Sequence 3 : 50, 60, 90, 40, 75

You have to answer the above question. If your answer is 'yes', please show the path (with branches); if 'no', explain why.
[Hint: you were thinking whether there was a missing page with a figure showing the BST, but your genius little brother smiled and hinted that no need to check for a figure as the answer lied already within the sequences provided.] **(9 marks)** [CSE 203 | 2020–21]

Answer

Search target: 22, keys in range $[1, 100]$.
Validity Rule: A BST search path for value X maintains implicit bounds (ℓ, u) on every visited node. Initially $\ell = -\infty$, $u = +\infty$.

- Going **left** from node v (because $X < v$): $u \leftarrow v$ (all future nodes must be $< v$).
- Going **right** from node v (because $X > v$): $\ell \leftarrow v$ (all future nodes must be $> v$).
- Each node visited must satisfy $\ell <$ node $< u$.

Sequence 1: [100, 40, 20, 30, 23, 35, 22] — **INVALID**

Step	Node	Bounds (ℓ, u)	Valid?	Direction
1	100	$(-\infty, +\infty)$	✓	$22 < 100$: go left, $u \leftarrow 100$
2	40	$(-\infty, 100)$	✓	$22 < 40$: go left, $u \leftarrow 40$
3	20	$(-\infty, 40)$	✓	$22 > 20$: go right, $\ell \leftarrow 20$
4	30	$(20, 40)$	✓	$22 < 30$: go left, $u \leftarrow 30$
5	23	$(20, 30)$	✓	$22 < 23$: go left, $u \leftarrow 23$
6	35	$(20, 23)$	×	$35 \notin (20, 23)$ — **VIOLATION**

After visiting 23 going left, all future nodes must be in $(20, 23)$. But $35 > 23$: **impossible in any BST**.

Sequence 2: [100, 35, 15, 32, 20, 22] — **VALID**

Step	Node	Bounds (ℓ, u)	Valid?	Direction
1	100	$(-\infty, +\infty)$	✓	$22 < 100$: go left, $u \leftarrow 100$
2	35	$(-\infty, 100)$	✓	$22 < 35$: go left, $u \leftarrow 35$
3	15	$(-\infty, 35)$	✓	$22 > 15$: go right, $\ell \leftarrow 15$
4	32	$(15, 35)$	✓	$22 < 32$: go left, $u \leftarrow 32$
5	20	$(15, 32)$	✓	$22 > 20$: go right, $\ell \leftarrow 20$
6	22	$(20, 32)$	✓	$22 =$ target: **FOUND**

Every node satisfies its bounds. Path with branches: $100 \xrightarrow{L} 35 \xrightarrow{L} 15 \xrightarrow{R} 32 \xrightarrow{L} 20 \xrightarrow{R} 22$.

Sequence 3: [50, 60, 90, 40, 75] — **INVALID**

Step	Node	Bounds (ℓ, u)	Valid?	Direction
1	50	$(-\infty, +\infty)$	✓	$22 < 50$: go left … wait
Actually: next node is 60 > 50, so we go RIGHT at 50.				
1	50	$(-\infty, +\infty)$	✓	$60 > 50$: go right, $\ell \leftarrow 50$
2	60	$(50, +\infty)$	✓	$90 > 60$: go right, $\ell \leftarrow 60$
3	90	$(60, +\infty)$	✓	$40 < 90$: go left, $u \leftarrow 90$
4	40	$(60, 90)$	×	$40 \notin (60, 90)$ — **VIOLATION**

After going right from 50 and 60, all future nodes must be > 60. But $40 < 60$: **impossible**.

Summary: Only **Sequence 2** is a valid BST search path. Sequences 1 and 3 violate the BST ordering invariant.

Q15. In a Binary Search Tree (BST), the successor of a node N is defined as the node N_{suc} with the smallest key greater than the key of N. Similarly, the predecessor of N is defined as the node N_{pre} with the largest key smaller than the key of N. Now answer the following questions (i)–(iii):

(i) Write an efficient algorithm that will store the keys of the successor and predecessor of each node. Assume that you have two variables, `suc` and `pre`, respectively in the node class/data structure for that purpose. **(7 marks)**

(ii) Discuss an efficient implementation strategy of `removeFullNode(`N`,` N_{suc}`)` (as defined below) with the help of appropriate illustrative examples. Analyze the time complexity of your strategy.

`removeFullNode(`N`,` N_{suc}`)`	N is a full node that has to be removed from the BST. N_{suc} is the successor of N.

(12 marks)

(iii) Your genius little brother suggests that an equivalent strategy is available if we use N_{pre} instead of N_{suc}. Do you agree with him? Justify your answer using the same illustrative example you showed in part 2.b(ii). **(7 marks)**

[CSE 203 | 2020–21]

Answer

(i) Algorithm to store successor and predecessor for all nodes:
Use a single **in-order traversal**. In-order gives nodes in sorted order; each node's predecessor is the previous node visited and successor is the next.

Algorithm 61 StoreSuccPred(*node*)

```
1: global prev_node                                   ▷ initially nil
2: if node = nil then return
3: end if
4: StoreSuccPred(node.left)
5: if prev_node ≠ nil then
6:     prev_node.suc ← node
7:     node.pre ← prev_node
8: end if
9: prev_node ← node
10: StoreSuccPred(node.right)
```

Time: $O(n)$ (single traversal). **Space:** $O(h)$ for recursion stack.

(ii) removeFullNode(N, N_{suc}):
N is a full node (has two children). N_{suc} is its in-order successor (leftmost node in N's right subtree, so N_{suc} has *no left child*).
Strategy:
1. Copy N_{suc}'s key into N (replace N's data with N_{suc}'s data).
2. Delete N_{suc} from its original position (easy: N_{suc} has at most one child — its right child).
3. Update `suc`/`pre` pointers: N_{pre}`.suc` $= N_{suc}$`.suc` and N_{suc}`.suc.pre` $= N$ (now carrying N_{suc}'s old key).

Example: Delete node 5 from BST with nodes 3, 5, 7, 6, 9. N_{suc} of 5 is 6 (leftmost of right subtree).
- Copy 6 into node 5's position.
- Delete the original node 6 (no left child — simply replace with its right child if any).

Time complexity: $O(h)$ where h is the height of the BST ($O(\log n)$ for balanced, $O(n)$ worst case for skewed tree).

(iii) Using N_{pre} instead of N_{suc}:
Yes, it is equivalent. N_{pre} is the in-order predecessor of N (rightmost node in N's left subtree), which has no right child.
Strategy: Copy N_{pre}'s key into N, then delete N_{pre} from its original position (at most one child — its left child).
Using the same example: delete node 5. $N_{pre} = 3$ (rightmost of left subtree). Copy 3 into node 5's position. Delete original node 3 (no right child).
Both strategies produce a valid BST and have identical $O(h)$ time complexity. The choice between N_{suc} and N_{pre} is a matter of implementation preference; both maintain the BST property.

Q16. Assume that a binary heap is stored in an array called `treeNodes`, which has a capacity of 100 array elements, with array index 0 not being used. If the binary heap currently contains 86 keys, answer the following questions with proper explanation:

(i) Is `treeNodes[44]` a leaf node?

(ii) How many children does `treeNodes[43]` have?

(iii) Is the subtree rooted at `treeNodes[8]` a full binary tree?

(iv) How many levels are there in the subtree rooted at `treeNodes[8]` including `treeNodes[8]`?

(v) Including the root node (`treeNodes[1]`), how many levels are there in the binary heap?

(vi) How many leaf nodes are there?

(12 marks) [CSE 203 | 2020–21]

Answer

Heap with $n = 86$ elements, 1-indexed. First leaf $= \lfloor 86/2 \rfloor + 1 = 44$.

(i) **`treeNodes[44]` a leaf?** $44 \geq 44 \rightarrow$ **Yes**. Children at $88, 89 > 86$.

(ii) **Children of `treeNodes[43]`?** Left=86 (valid), right=87 > 86. $\rightarrow$ **One child**.

(iii) **Subtree at `treeNodes[8]` a full binary tree?** Node 43 (one child) lies in this subtree $\rightarrow$ **No**.

(iv) **Levels in subtree at `treeNodes[8]`?** $\lfloor \log_2(86/8) \rfloor + 1 = \lfloor \log_2 10.75 \rfloor + 1 = 3 + 1 = \mathbf{4}$.

(v) **Total levels in heap?** $\lfloor \log_2 86 \rfloor + 1 = 6 + 1 = \mathbf{7}$.

(vi) **Number of leaves?** $86 - 43 = \mathbf{43}$ leaves.

Q17. Show the steps of building a max-heap by inserting the numbers $45, 36, 27, 14, 86, 19, 69, 10$ one by one. Show the memory representation of the heap. Then write down the steps of sorting these numbers using heap sort. Show the analysis of how `Build-Max-Heap` can be done in $\Theta(n)$ time. **(10+5+10+10 marks)** [CSE 203 | 2019–20]

Answer

Step-by-step insertion:

Insert	Array
45	[45]
36	[45, 36]
27	[45, 36, 27]
14	[45, 36, 27, 14]
86	[86, 45, 27, 14, 36]
19	[86, 45, 27, 14, 36, 19]
69	[86, 45, 69, 14, 36, 19, 27]
10	[86, 45, 69, 14, 36, 19, 27, 10]

Memory representation:

Index	1	2	3	4	5	6	7	8
Value	86	45	69	14	36	19	27	10

Heap Sort (extract-max, then heapify):

Step	Heap | Sorted
1	[69,45,27,14,36,19,10] | {86}
2	[45,36,27,14,10,19] | {69,86}
3	[36,19,27,14,10] | {45,69,86}
4	[27,19,10,14] | {36,45,69,86}
5	[19,14,10] | {27,36,45,69,86}
6	[14,10] | {19,27,36,45,69,86}
7	[10] | {14,19,27,36,45,69,86}

Sorted: **10, 14, 19, 27, 36, 45, 69, 86** ✓
Build-Max-Heap in $\Theta(n)$: (see proof in [2014–15] answer).

Q18. Assume you are given a Max Priority Queue of n elements. Write a function that first finds and then deletes the minimum element of the priority queue. **(10 marks)** [CSE 203 | 2018–19]

Answer

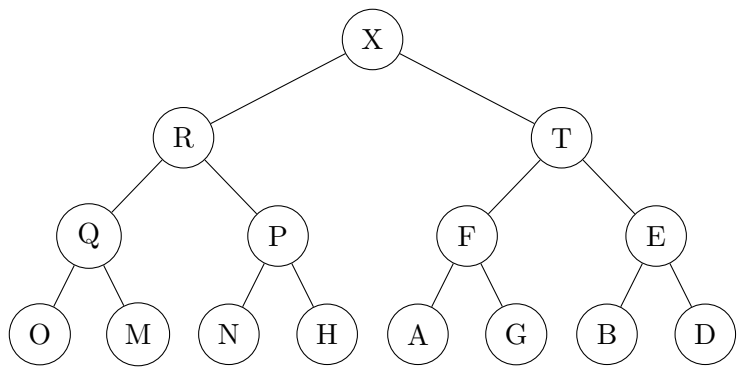
Using the standard Priority Queue ADT (e.g., C++ `std::priority_queue`), we cannot directly access the leaves. We must extract all elements, track the minimum, and re-insert everything except the minimum element.

```cpp
#include <queue>
#include <vector>
using namespace std;

void deleteMin(priority_queue<int>& pq) {
    if (pq.empty()) return;

    vector<int> temp;
    int min_val = pq.top();

    // Step 1: Extract all elements to find the minimum
    while (!pq.empty()) {
        int current = pq.top();
        pq.pop();
```

```
        if (current < min_val) {
            min_val = current;
        }
        temp.push_back(current);
    }

    // Step 2: Re-insert all elements except one instance of the minimum
    bool is_deleted = false;
    for (int val : temp) {
        if (val == min_val && !is_deleted) {
            is_deleted = true; // Skip pushing this element
            continue;
        }
        pq.push(val);
    }
}
```

Complexity Analysis. Extracting all elements: $O(n \log n)$. Re-inserting $n-1$ elements: $O(n \log n)$. Extra Space: $O(n)$ for the temporary vector. **Total Time:** $O(n \log n)$.

Q19. Consider the following max-heap of letters:



Draw the results of inserting S into the above max-heap. **(7 marks)** [CSE 203 | 2018–19]

Detailed Answer

Original Heap Array (0-indexed):
Total 15 nodes. Indices 0 to 14.
`[X, R, T, Q, P, F, E, O, M, N, H, A, G, B, D]`

Step 1: Insert S
The new node 'S' must be inserted at the next available position, which is index 15. The parent of index 15 is $\lfloor (15-1)/2 \rfloor = 7$. The node at index 7 is 'O'. So, 'S' is initially placed as the **left child of O**.

Step 2: Bubble-up (Heapify-up) trace
Letter values for comparison: S=19, O=15, Q=17, R=18, X=24.

(a) **S(19) vs Parent O(15) at idx 7:** $19 > 15 \rightarrow$ Swap! 'S' moves to idx 7, 'O' moves to idx 15.

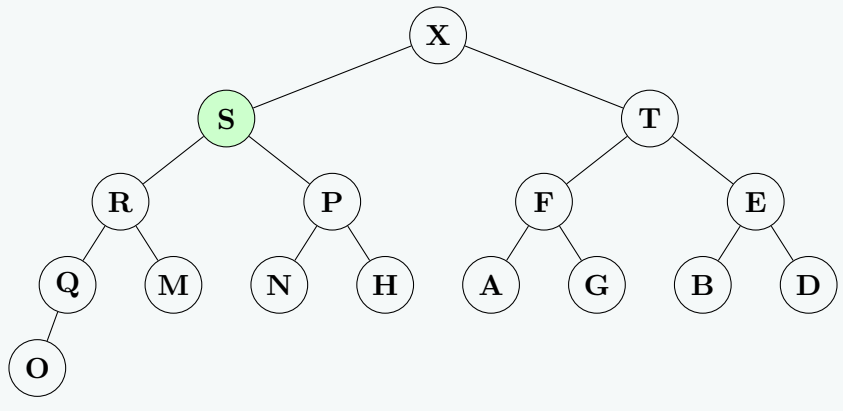
(b) **S(19) vs Parent Q(17) at idx 3:** $19 > 17 \rightarrow$ Swap! 'S' moves to idx 3, 'Q' moves to idx 7.

(c) **S(19) vs Parent R(18) at idx 1:** $19 > 18 \rightarrow$ Swap! 'S' moves to idx 1, 'R' moves to idx 3.

(d) **S(19) vs Parent X(24) at idx 0:** $19 < 24 \rightarrow$ **Stop**. Max-heap property restored.

Final Heap Array:
`[X, S, T, R, P, F, E, Q, M, N, H, A, G, B, D, O]`

Final Max-Heap Tree:



Remark. In the final tree, 'S' successfully bubbled up to replace 'R', pushing 'R' down to replace 'Q', 'Q' down to replace 'O', and 'O' becoming the left child of 'Q'.

Q20. Suppose you have a set of numbers where you do a constant number of insertions and a linear number of lookups of the maximum over some time period. Compare using a heap vs. a binary search tree for maintaining this set:

(a) What is the asymptotic upper bound on the *average-case* runtime for each, and which structure should you choose?

(b) What is the asymptotic upper bound on the *worst-case* runtime for each, and which structure should you choose?

(12 marks) [CSE 203 | 2018–19]

Answer

Operation	Max-Heap Avg	Max-Heap Worst	BST Avg	BST Worst
c Insertions	$O(c \log n)$	$O(c \log n)$	$O(c \log n)$	$O(cn)$
n Find-Max	$O(n)$	$O(n)$	$O(n \log n)$	$O(n^2)$
Total	$O(n)$	$O(n)$	$O(n \log n)$	$O(n^2)$

(a) Average case: Heap $O(n)$ vs BST $O(n \log n)$ $\rightarrow$ **choose heap**.
(b) Worst case: Heap $O(n)$ vs BST $O(n^2)$ $\rightarrow$ **choose heap**.

Q21. Write a function `int treeHeight()` for a Binary Search Tree that computes the height of the tree. Give a worst-case Big-O running time in terms of n. **(8 marks)** [CSE 203 | 2018–19]

Answer

```
// Height: O(n)
int treeHeight(treeNode* root){
    if(!root) return -1;
    return 1 + max(treeHeight(root->left), treeHeight(root->right));
}
```

Q22. Write a function `void deleteMax()` for a Binary Search Tree that deletes the maximum element (or does nothing if the tree is empty). Give a worst-case Big-O running time in terms of n. Do not assume the tree is balanced. **(7 marks)** [CSE 203 | 2018–19]

Answer

```
// Delete max (rightmost node): O(h) worst O(n)
void deleteMax(treeNode* &root){
    if(!root) return;
    if(!root->right){
        treeNode* tmp = root;
        root = root->left;
        delete tmp; return;
    }
    treeNode* parent = root, *curr = root->right;
    while(curr->right){ parent = curr; curr = curr->right; }
    parent->right = curr->left;
    delete curr;
}
```

Q23. Write a method `isHeap()` that determines whether a particular binary tree is a heap. What is the Big-O bound on the running time of this algorithm? **(8 marks)** [CSE 203 | 2018–19]

Answer

```
// isHeap (array, 1-indexed): O(n)
bool isHeap(int A[], int n){
    for(int i = 1; i <= n/2; i++){
        if(2*i   <= n && A[i] < A[2*i])   return false;
        if(2*i+1 <= n && A[i] < A[2*i+1]) return false;
    }
    return true;
}
```

Q24. Construct a max-heap with the data $[6, 1, 7, 2, 8, 3, 9, 4, 10, 5, 11, 17, 4, 5]$ by insertion and adjustment. Show the changes in values at each step. Carry out complexity analysis for both algorithms. **(5+5+10 marks)** [CSE 203 | 2017–18]

Answer

Build by successive insertion + bubble-up:

Insert	Array after bubble-up
6	[6]
1	[6, 1]
7	[7, 1, 6]
2	[7, 2, 6, 1]
8	[8, 7, 6, 1, 2]
3	[8, 7, 6, 1, 2, 3]
9	[9, 7, 8, 1, 2, 3, 6]
4	[9, 7, 8, 4, 2, 3, 6, 1]
10	[10, 9, 8, 7, 2, 3, 6, 1, 4]
5	[10, 9, 8, 7, 5, 3, 6, 1, 4, 2]
11	[11, 10, 8, 7, 9, 3, 6, 1, 4, 2, 5]
17	[17, 10, 11, 7, 9, 8, 6, 1, 4, 2, 5, 3]
4	[17, 10, 11, 7, 9, 8, 6, 1, 4, 2, 5, 3, 4]
5	[17, 10, 11, 7, 9, 8, 6, 1, 4, 2, 5, 3, 4, 5]

Complexity Analysis. Insertion method: $\Theta(n \log n)$. Bottom-up build: $\Theta(n)$.

Q25. By writing a procedure, explain the three cases that may arise for deleting a node from a binary search tree. **(15 marks)** [CSE 203 | 2016–17, 2015–16]

Answer

Case 1 — leaf: remove; parent pointer set to NULL.
Case 2 — one child: splice; parent points directly to child.
Case 3 — two children: find in-order successor s (leftmost in right subtree); copy s.key into node; delete s (which has ≤ 1 child, so Case 1 or 2).

```
treeNode* deleteNode(treeNode* root, int key){
    if(!root) return NULL;
    if(key < root->item)
        root->left = deleteNode(root->left, key);
    else if(key > root->item)
        root->right = deleteNode(root->right, key);
    else {
        if(!root->left){ treeNode* t=root->right; delete root; return t; }
        if(!root->right){ treeNode* t=root->left; delete root; return t; }
        treeNode* s = minimum(root->right);
        root->item  = s->item;
        root->right = deleteNode(root->right, s->item);
    }
    return root;
}
```

Complexity Analysis. $O(h)$; average $O(\log n)$, worst $O(n)$.

Q26. Write a function to find the smallest element in a max binary heap. **(8 marks)** [CSE 203 | 2016–17]

Answer

In a max binary heap, the smallest element is always a *leaf node*. Using `std::priority_queue` (max-heap by default), we cannot access internal elements directly. Instead, we copy all elements into a `min_heap` and return the top, or simply scan all leaves.

Using STL `priority_queue` (min-heap trick):

```
#include <queue>
#include <vector>
using namespace std;

int findMin(priority_queue<int> maxHeap) {
    // Copy into a min-heap to find the smallest
    priority_queue<int, vector<int>, greater<int>> minHeap;
    while (!maxHeap.empty()) {
        minHeap.push(maxHeap.top());
        maxHeap.pop();
    }
    return minHeap.top();  // smallest element
}
```

Efficient approach — scan leaves only (underlying container):

```
#include <queue>
```

```cpp
#include <vector>
#include <algorithm>
using namespace std;

// Expose underlying container via inheritance trick
struct MinFromHeap : priority_queue<int> {
    int findMin() const {
        int n = c.size();
        // Leaves are at indices n/2 to n-1 (0-indexed)
        return *min_element(c.begin() + n/2, c.end());
    }
};
```

Complexity Analysis. $O(n)$ for both approaches — the first rebuilds the entire heap; the second scans only the $\lceil n/2 \rceil$ leaf nodes.

Q27. Write the differences between a standard queue and a priority queue. Write two functions that implement the DECREASE-KEY and INCREASE-KEY operations in a max-heap. **(10 marks)** [CSE 203 | 2016–17]

Answer

Queue vs. Priority Queue: (see table in [2012–13] answer).

Algorithm 62 MAX-HEAP-INCREASE-KEY(A, x, k)

1: **if** $k < x.key$ **then**
2: **error** "new key is smaller than current key"
3: **end if**
4: $x.key \leftarrow k$
5: Find the index i in array A where object x occurs
6: **while** $i > 1$ **and** $A[\text{PARENT}(i)].key < A[i].key$ **do**
7: Exchange $A[i]$ with $A[\text{PARENT}(i)]$, updating the information that maps priority queue objects to array indices
8: $i \leftarrow \text{PARENT}(i)$
9: **end while**

Algorithm 63 MAX-HEAP-INSERT(A, x, n)

1: **if** $A.heap\text{-}size == n$ **then**
2: **error** "heap overflow"
3: **end if**
4: $A.heap\text{-}size \leftarrow A.heap\text{-}size + 1$
5: $k \leftarrow x.key$
6: $x.key \leftarrow -\infty$
7: $A[A.heap\text{-}size] \leftarrow x$
8: Map x to index $heap\text{-}size$ in the array
9: MAX-HEAP-INCREASE-KEY(A, x, k)

Complexity Analysis. Both $O(\log n)$.

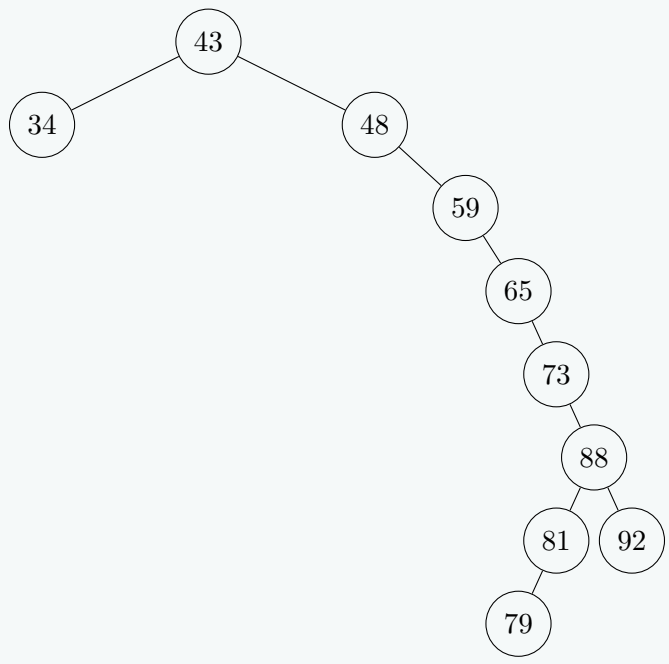
Q28. Write an algorithm that sorts the keys of a binary search tree. Draw the BST that results after inserting the keys $43, 34, 48, 59, 65, 73, 88, 81, 92, 79$ (in that order) into an initially empty BST. Also draw a BST of height three using these keys. **(10 marks)** [CSE 203 | 2016–17]

Answer

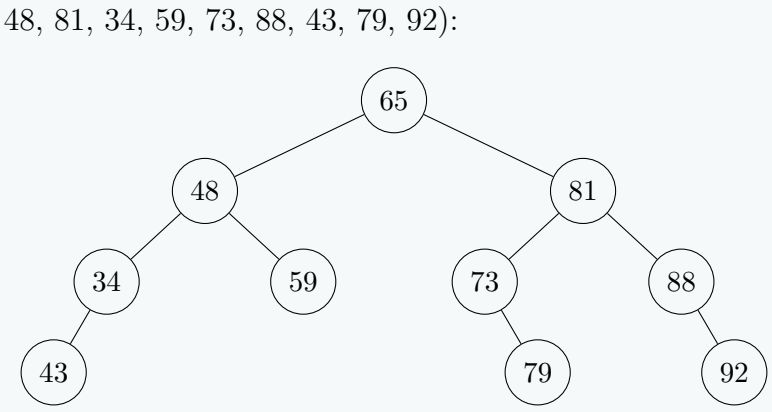
In-order traversal visits nodes in ascending order:

```cpp
void inorder(treeNode* root){
    if(!root) return;
    inorder(root->left);
    cout << root->item << " ";
    inorder(root->right);
}
```

BST for 43,34,48,59,65,73,88,81,92,79 (height = 7):



BST of height 3 (insert order: 65, 48, 81, 34, 59, 73, 88, 43, 79, 92):



Q29. Write a function to find the largest element in a min binary heap. Assume the array representing a min binary heap contains the values $2, 8, 3, 10, 16, 7, 18, 13, 15$. Show the contents of the array after deleting the minimum element. **(10 marks)** [CSE 203 | 2015–16]

Answer

Part 1 — Largest element in a min-heap (STL).
The largest element is always a *leaf node*. Using STL's `priority_queue` (min-heap), the underlying container can be exposed via inheritance to scan only the leaf nodes.

```cpp
#include <queue>
#include <vector>
#include <algorithm>
using namespace std;

struct MaxFromMinHeap : priority_queue<int, vector<int>, greater<int>> {
    int findMax() const {
        int n = c.size();
        // Leaves occupy indices n/2 .. n-1 (0-indexed)
        return *max_element(c.begin() + n/2, c.end());
    }
};

int main() {
    MaxFromMinHeap h;
    for (int x : {2,8,3,10,16,7,18,13,15}) h.push(x);
    cout << "Max = " << h.findMax() << endl; // 18
}
```

Complexity Analysis. $O(n)$ — scans only the $\lceil n/2 \rceil$ leaf nodes.

Part 2 — Delete minimum from `[2,8,3,10,16,7,18,13,15]`.

```cpp
#include <queue>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    // Build min-heap from given values
    priority_queue<int, vector<int>, greater<int>> minHeap;
    for (int x : {2,8,3,10,16,7,18,13,15})
```

```
        minHeap.push(x);

    cout << "Min (root) = " << minHeap.top() << endl; // 2
    minHeap.pop();   // delete minimum element

    // Print remaining elements (heap order)
    // Result: [3, 8, 7, 10, 16, 15, 18, 13]
    priority_queue<int, vector<int>, greater<int>> temp = minHeap;
    while (!temp.empty()) {
        cout << temp.top() << " ";
        temp.pop();
    }
}
```

Trace of `pop()` internally:

(a) Root 2 removed; last element 15 placed at root.
Array: `[15, 8, 3, 10, 16, 7, 18, 13]`

(b) Sift-down at index 0: min child = 3 (idx 2) → swap.
Array: `[3, 8, 15, 10, 16, 7, 18, 13]`

(c) Sift-down at index 2: min child = 7 (idx 5) → swap.
Array: `[3, 8, 7, 10, 16, 15, 18, 13]`

(d) Index 5 has no children. Done.

Final array after delete-min: `[3, 8, 7, 10, 16, 15, 18, 13]` ✓

Complexity Analysis. $O(\log n)$ — `pop()` sifts down at most $\lfloor \log_2 n \rfloor$ levels.

Q30. Prove that a min binary heap can be built in linear time. **(10 marks)** [CSE 203 | 2015–16]

Answer

At height h there are $\leq \lceil n/2^{h+1} \rceil$ nodes, each costing $O(h)$:

$$T(n) \leq cn \sum_{h=0}^{\infty} \frac{h}{2^h} = cn \cdot 2 = O(n). \qquad \square$$

(Using $\sum_{h=0}^{\infty} hx^h = x/(1-x)^2$ with $x = 1/2$.)

Q31. Write two procedures for finding the successor element and the predecessor element in a binary search tree. **(10 marks)** [CSE 203 | 2015–16]

Answer

```
treeNode* minimum(treeNode* n){
    while(n->left) n = n->left; return n;
}
treeNode* maximum(treeNode* n){
    while(n->right) n = n->right; return n;
}
treeNode* successor(treeNode* n){
    if(n->right) return minimum(n->right);
    treeNode* p = n->parent;
    while(p && n == p->right){ n = p; p = p->parent; }
    return p;
}
treeNode* predecessor(treeNode* n){
    if(n->left) return maximum(n->left);
    treeNode* p = n->parent;
    while(p && n == p->left){ n = p; p = p->parent; }
    return p;
}
```

Complexity Analysis. $O(h)$ each.

Q32. What are the main operations of a priority queue? Explain how one can implement these operations using a heap. **(15 marks)** [CSE 203 | 2014–15, 2011–12]

Answer

Main operations of a Priority Queue:

- INSERT(S, x): insert element x into set S.
- MAXIMUM(S): return element with the largest key ($O(1)$).
- EXTRACT-MAX(S): remove and return element with the largest key.
- INCREASE-KEY(S, x, k): increase value of element x's key to $k \geq x$.key.

Heap-based implementation (Max-Heap, 1-indexed array A):
Heap property: $A[\lfloor i/2 \rfloor] \geq A[i]$ for all $i > 1$.

```
// O(log n): restore heap property downward
void maxHeapify(int A[], int i, int n) {
    int l = 2*i, r = 2*i+1, largest = i;
    if (l <= n && A[l] > A[largest]) largest = l;
    if (r <= n && A[r] > A[largest]) largest = r;
    if (largest != i) {
        swap(A[i], A[largest]);
        maxHeapify(A, largest, n);
    }
}
int heapMaximum(int A[]) { return A[1]; }          // O(1)

int heapExtractMax(int A[], int &n) {               // O(log n)
    int max = A[1];
    A[1] = A[n--];
    maxHeapify(A, 1, n);
    return max;
}
void heapIncreaseKey(int A[], int i, int key) {     // O(log n)
    A[i] = key;
    while (i > 1 && A[i/2] < A[i]) {
        swap(A[i], A[i/2]);
        i /= 2;
    }
}
void maxHeapInsert(int A[], int &n, int key) {      // O(log n)
    A[++n] = INT_MIN;
    heapIncreaseKey(A, n, key);
}
```

Complexity Analysis. MAXIMUM: $O(1)$. EXTRACT-MAX, INSERT, INCREASE-KEY: $O(\log n)$.

Q33. Write an algorithm for building a heap. Show that an n-element heap can be built in $O(n)$ time. **(10 marks)** [CSE 203 | 2014–15]

Answer

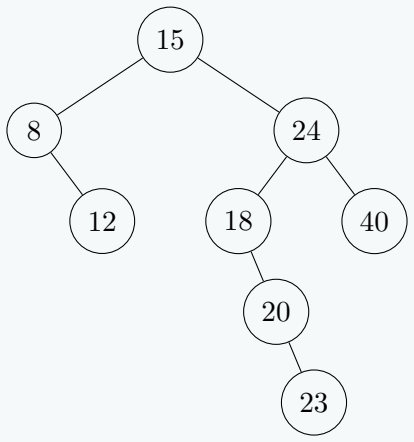
BUILD-MAX-HEAP (same algorithm as above).
$O(n)$ **proof**: a node at height h costs $O(h)$ in HEAPIFY; there are $\leq \lceil n/2^{h+1} \rceil$ such nodes.

$$T(n) \leq cn \sum_{h=0}^{\infty} \frac{h}{2^h} = cn \cdot \frac{1/2}{(1-1/2)^2} = 2cn = O(n). \qquad \square$$

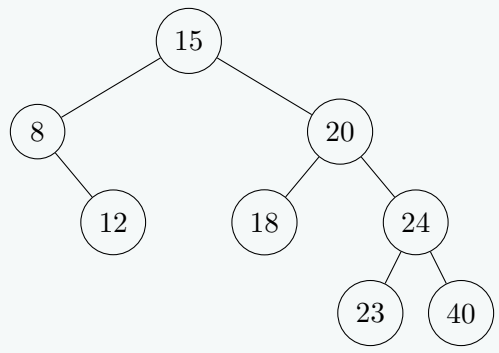
Q34. Draw the binary search tree that results after successively inserting the keys $15, 8, 24,$ $12, 40, 18, 20, 23$ into an initially empty BST. Then, by using necessary rotations, redraw the BST so that it becomes an AVL tree. **(10 marks)** [CSE 203 | 2014–15]

Answer

BST after inserting 15, 8, 24, 12, 40, 18, 20, 23:



Converting to AVL — detecting imbalance:
Node 24: BF = height(18 subtree) − height(40) = $2 - 0 = 2 \rightarrow$ unbalanced (Right-Left case, since 20>18 but 20<24).
Fix: Left-rotate at 18, then Right-rotate at 24. After left-rotate at 18: 20 takes 18's position; 18 becomes 20's left child; 23 becomes 20's right (wait — 23 is right of 20, after rotation 23 stays right of 20 and 24 becomes root of right part).
After full LR rotation at 24: 20 becomes subroot; 18 is 20's left; 24 is 20's right with 23 as 24's left and 40 as 24's right.
Final AVL tree:



Remark. All balance factors $\in \{-1, 0, +1\}$. ✓

Q35. Suppose we create a binary search tree by inserting the following values in the given order: $50, 10, 13, 45, 55, 110, 5, 31, 64, 47$. Answer the following questions:

(a) Draw the binary search tree.

(b) Show the output values if we visit the tree using pre-order traversal.

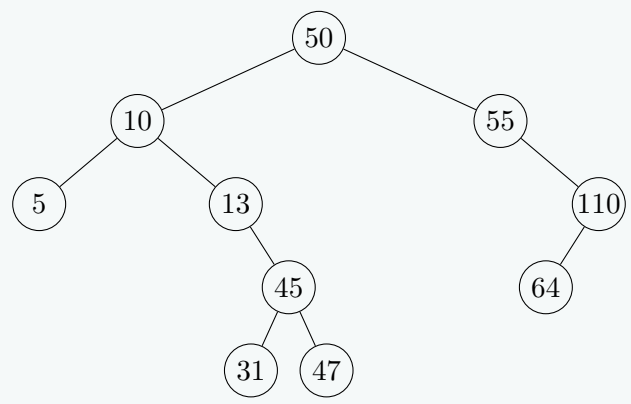
(c) Show the output values if we visit the tree using post-order traversal.

(d) Show the resulting trees after deleting 47, 110, and 50 (each deletion applied on the original tree).

(5+4+4+6=19 marks) [CSE 203 | 2014–15]

Answer

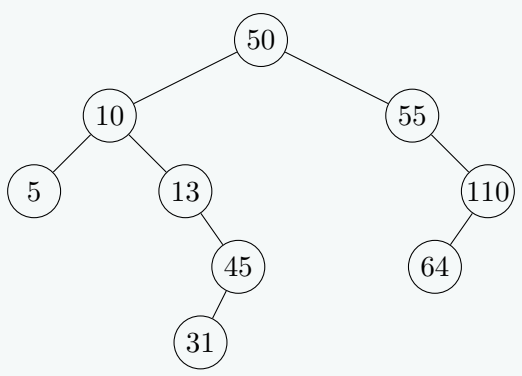
(a) Original BST:



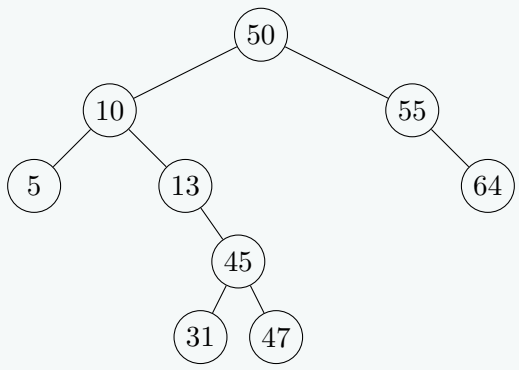
(b) Preorder: `50 10 5 13 45 31 47 55 110 64` ✓
(c) Postorder: `5 31 47 45 13 10 64 110 55 50` ✓

(d) Deletions (each applied independently on the original tree):

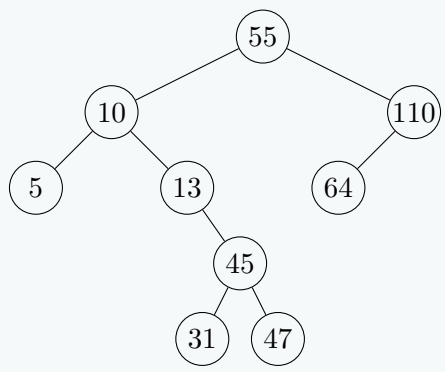
1. Delete 47 (Case 1: Leaf node). Simply remove node 47.



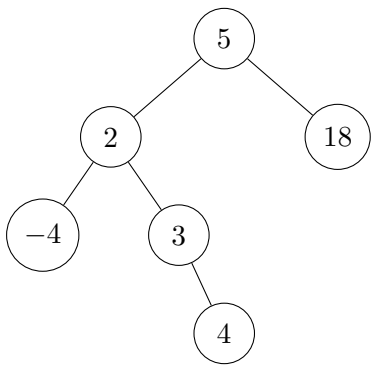
2. Delete 110 (Case 2: Node with one child). Replace 110 with its left child 64.



3. Delete 50 (Case 3: Node with two children). Replace 50 with its inorder successor (which is 55). Then delete the original node 55. The right child of the new root 55 becomes 110.



Q36. Give the idea of a data structure that can be used to implement a general tree where nodes can have more than two children. Re-draw the following binary search tree using your proposed data structure:



(8 marks) **[CSE 203 | 2014–15]**

Answer

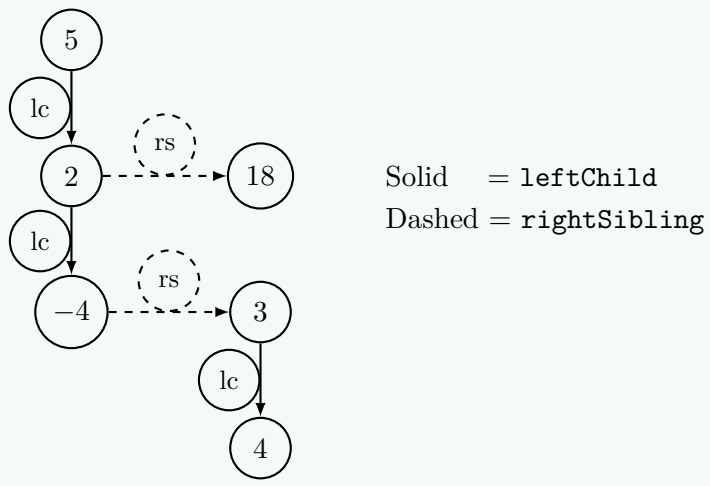
Left-Child Right-Sibling (LCRS) Representation:
The optimal data structure to represent a general tree (where nodes can have more than two children) using a binary structure is the **Left-Child Right-Sibling (LCRS)** tree.
Each node stores exactly two pointers:

- `leftChild` — points to the node's **first (leftmost) child**.
- `rightSibling` — points to the node's **immediate next sibling** to its right.

This effectively transforms any general tree into a binary tree representation, as each node has at most two outgoing pointers.

Re-drawn BST using LCRS representation:
Treating the given binary search tree as a general tree, we apply the LCRS rules:

- **Node 5:** First child is 2 (`leftChild` = 2). No siblings (`rightSibling` = NULL).
- **Node 2:** First child is −4 (`leftChild` = −4). Next sibling is 18 (`rightSibling` = 18).
- **Node 18:** No children (`leftChild` = NULL). No next sibling (`rightSibling` = NULL).
- **Node −4:** No children (`leftChild` = NULL). Next sibling is 3 (`rightSibling` = 3).
- **Node 3:** First (and only) child is 4 (`leftChild` = 4). No next sibling (`rightSibling` = NULL).
- **Node 4:** No children (`leftChild` = NULL). No next sibling (`rightSibling` = NULL).



Q37. Assume an object which has two elements {key, element}. An array of such objects is given. Then:

(a) Convert the array into a Max-heap.

(b) Insert $(36, B)$ in the resultant Max-heap.

(c) Show each step of Heap Sort over the resultant Max-heap.

Array: [14/U, 11/E, 13/T, 12/C, 26/S, 19/E, 20/W, 24/P, 18/O, 17/L, 7/A, 8/S, 9/H, 10/I] **(8+8+6=22 marks)** [CSE 203 | 2013–14]

Detailed Answer

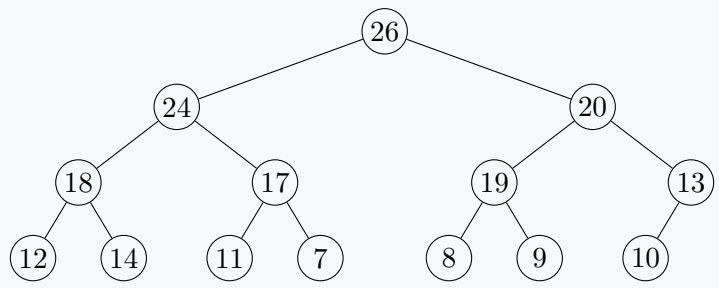
Given Array: $[14/U, 11/E, 13/T, 12/C, 26/S, 19/E, 20/W, 24/P, 18/O, 17/L, 7/A, 8/S, 9/H, 10/I]$
Total elements, $n = 14$. For clarity in diagrams and steps, only the **keys** are shown.

(a) Convert the array into a Max-Heap

We use the bottom-up heapify method. We start from the last non-leaf node at index $\lfloor n/2 \rfloor - 1 = \lfloor 14/2 \rfloor - 1 = 6$.

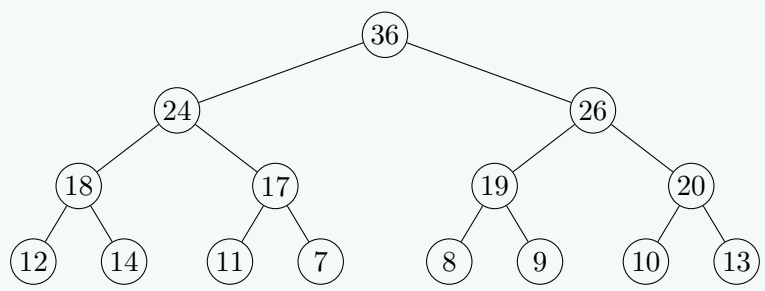
Node Idx	Heapify Action	Array State (Keys)
idx 6 (20)	$20 \geq 10$ (child). No swap.	[14, 11, 13, 12, 26, 19, 20, 24, 18, 17, 7, 8, 9, 10]
idx 5 (19)	$19 \geq 8, 9$ (children). No swap.	unchanged
idx 4 (26)	$26 \geq 17, 7$ (children). No swap.	unchanged
idx 3 (12)	$12 < 24$. Swap 12 ↔ 24.	[14, 11, 13, **24**, 26, 19, 20, **12**, 18, 17, 7, 8, 9, 10]
idx 2 (13)	$13 < 20$. Swap 13 ↔ 20.	[14, 11, **20**, 24, 26, 19, **13**, 12, 18, 17, 7, 8, 9, 10]
idx 1 (11)	$11 < 26$. Swap 11 ↔ 26.	[14, **26**, 20, 24, **11**, 19, 13, 12, 18, 17, 7, 8, 9, 10]
	↪ Heapify idx 4: Swap 11 ↔ 17.	[14, 26, 20, 24, **17**, 19, 13, 12, 18, **11**, 7, 8, 9, 10]
idx 0 (14)	$14 < 26$. Swap 14 ↔ 26.	[**26**, **14**, 20, 24, 17, 19, 13, 12, 18, 11, 7, 8, 9, 10]
	↪ Heapify idx 1: Swap 14 ↔ 24.	[26, **24**, 20, **14**, 17, 19, 13, 12, 18, 11, 7, 8, 9, 10]
	↪ Heapify idx 3: Swap 14 ↔ 18.	[26, 24, 20, **18**, 17, 19, 13, 12, **14**, 11, 7, 8, 9, 10]

Max-Heap Tree after Step (a):



(b) Insert (36, B) in the resultant Max-Heap

We insert 36 at the very end of the array (index 14) and "bubble-up" to restore the Max-Heap property.
1. Insert at idx 14. Array ends with $[\ldots, 10, \mathbf{36}]$. 2. Parent is idx 6 (key 13). $36 > 13 \rightarrow$ **Swap 36 and 13**. 3. Parent is idx 2 (key 20). $36 > 20 \rightarrow$ **Swap 36 and 20**. 4. Parent is idx 0 (key 26). $36 > 26 \rightarrow$ **Swap 36 and 26**.
Array after insertion: $[36, 24, 26, 18, 17, 19, 20, 12, 14, 11, 7, 8, 9, 10, 13]$
Max-Heap Tree after Step (b):



(c) Heap Sort over the resultant Max-Heap

Heap sort extracts the maximum element (root), swaps it with the last element of the heap, reduces the heap size by 1, and heapifies the root. The extracted elements build the sorted array at the end.

- **Initial Heap:** $[36, 24, 26, 18, 17, 19, 20, 12, 14, 11, 7, 8, 9, 10, 13]$
- **Extract 36:** Swap 36 & 13. Heapify root (13 → swaps with 26, 20, 10).
  Heap: $[26, 24, 20, 18, 17, 19, 10, 12, 14, 11, 7, 8, 9, 13]$ | **36**
- **Extract 26:** Swap 26 & 13. Heapify root (13 → swaps with 24, 18, 14).
  Heap: $[24, 18, 20, 14, 17, 19, 10, 12, 13, 11, 7, 8, 9]$ | **26, 36**
- **Extract 24:** Swap 24 & 9. Heapify root (9 → swaps with 20, 19).
  Heap: $[20, 18, 19, 14, 17, 9, 10, 12, 13, 11, 7, 8]$ | **24, 26, 36**
- **Extract 20:** Swap 20 & 8. Heapify root (8 → swaps with 19, 10).
  Heap: $[19, 18, 10, 14, 17, 8, 9, 12, 13, 11, 7]$ | **20 . . . 36**
- **Extract 19:** Swap 19 & 7. Heapify root (7 → swaps with 18, 17, 11).
  Heap: $[18, 17, 10, 14, 11, 8, 9, 12, 13, 7]$ | **19 . . . 36**
- **Extract 18:** Swap 18 & 7. Heapify root (7 → swaps with 17, 14, 13).
  Heap: $[17, 14, 10, 13, 11, 8, 9, 12, 7]$ | **18 . . . 36**
- **Extract 17:** Swap 17 & 7. Heapify root (7 → swaps with 14, 13, 12).
  Heap: $[14, 13, 10, 12, 11, 8, 9, 7]$ | **17 . . . 36**
- **Extract 14:** Swap 14 & 7. Heapify root (7 → swaps with 13, 12).
  Heap: $[13, 12, 10, 7, 11, 8, 9]$ | **14 . . . 36**
- **Extract 13:** Swap 13 & 9. Heapify root (9 → swaps with 12, 11).
  Heap: $[12, 11, 10, 7, 9, 8]$ | **13 . . . 36**
- **Extract 12:** Swap 12 & 8. Heapify root (8 → swaps with 11, 9).
  Heap: $[11, 9, 10, 7, 8]$ | **12 . . . 36**
- **Extract 11:** Swap 11 & 8. Heapify root (8 → swaps with 10).
  Heap: $[10, 9, 8, 7]$ | **11 . . . 36**
- **Extract 10:** Swap 10 & 7. Heapify root (7 → swaps with 9).
  Heap: $[9, 7, 8]$ | **10 . . . 36**
- **Extract 9:** Swap 9 & 8. Heapify root (8 → no swap).
  Heap: $[8, 7]$ | **9 . . . 36**
- **Extract 8:** Swap 8 & 7.
  Heap: $[7]$ | **8 . . . 36**

Final Sorted Array (Ascending): $[7, 8, 9, 10, 11, 12, 13, 14, 17, 18, 19, 20, 24, 26, 36]$

Q38. Write down two algorithms for constructing a max-heap and deduce their complexities. Discuss how Carlsson reduces the leading coefficient in the complexity of the Heapsort algorithm. **(10 marks)** **[CSE 207 | 2013–14]**

Answer

Algorithm 1: Repeated Insertion (top-down)
Insert elements one by one into an initially empty heap using Heapify-Up.

Algorithm 64 Build-Heap-Insertion(A, n)

```
heap ← []
for i = 1 to n do
    heap.Append(A[i])
    Heapify-Up(heap, |heap|)
end for
```

Complexity: Each HeapifyUp takes $O(\log k)$ for the k-th insertion. Total: $\sum_{k=1}^{n} O(\log k) = O(n \log n)$.

Algorithm 2: Build-Max-Heap (bottom-up, CLRS)
Start from the last non-leaf and call Max-Heapify downward.

Algorithm 65 Build-Max-Heap(A, n)

```
for i = ⌊n/2⌋ downto 1 do
    Max-Heapify(A, i, n)
end for
```

Complexity: $O(n)$ — nodes at height h take $O(h)$ each, and $\sum_{h=0}^{\lfloor \log n \rfloor} \frac{n}{2^{h+1}} \cdot O(h) = O(n)$.

Carlsson's improvement to Heapsort:
Standard Heapsort uses 2 comparisons per Max-Heapify step (compare both children, then compare winner with parent). Carlsson (1987) observed that during the sifting-down phase, the element being sifted almost always goes to a leaf. His optimization:

(a) **Sift-down to a leaf** using only 1 comparison per level (just pick the larger child, without comparing against the element being sifted).

(b) **Then sift back up** to find the correct position.
This reduces the average number of comparisons from $\approx 2n \log n$ to $\approx n \log n + O(n)$, nearly halving the leading coefficient. The asymptotic complexity remains $O(n \log n)$ but the constant factor is smaller, making it faster in practice.

Q39. Write the differences between a standard queue and a priority queue. Write the property of a Max-Heap. Write an algorithm for building a Max-Heap. Analyze the time complexity of the algorithm. **(3+3+7+7=20 marks)** [CSE 203 | 2012–13]

Answer

Standard Queue vs. Priority Queue:

Property	Standard Queue	Priority Queue
Order	FIFO	By priority/key
Dequeue	Front element	Highest-priority element
Implementation	Array/Linked list	Heap
Extract time	$O(1)$	$O(\log n)$

Max-Heap property: for every node $i \neq$ root, $A[\lfloor i/2 \rfloor] \geq A[i]$. The root holds the maximum.
Build-Max-Heap:

Algorithm 66 BUILD-MAX-HEAP(A, n)

```
1: for i ← ⌊n/2⌋ downto 1 do
2:     MAX-HEAPIFY(A, i, n)
3: end for
```

Complexity Analysis. Naïve: $O(n \log n)$. Tight: $O(n)$, since $\sum_{h=0}^{\lfloor \log n \rfloor} \lceil n/2^{h+1} \rceil \cdot O(h) = O(n \sum_{h=0}^{\infty} h/2^h) = O(2n) = O(n)$.

Q40. Sort the array $[15, 13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 1, 26, 23]$ using Heapsort, showing the values at the right of the nodes at each step. **(8 marks)** [CSE 203 | 2011–12]

Answer

Heapsort works in two phases: (1) build a max-heap from the array, then (2) repeatedly extract the maximum.

Phase 1 — Build max-heap (starting from last non-leaf, $i = \lfloor 14/2 \rfloor - 1 = 6$, down to 0):
After BUILDMAXHEAP:

$$[26,\ 13,\ 23,\ 10,\ 12,\ 9,\ 15,\ 4,\ 5,\ 6,\ 2,\ 1,\ 8,\ 7]$$

Phase 2 — Extract-max repeatedly (swap root with last, reduce heap size, heapify):

Extract	Max	Heap (remaining)	Sorted suffix
1	26	[23, 13, 15, 10, 12, 9, 7, 4, 5, 6, 2, 1, 8]	[26]
2	23	[15, 13, 9, 10, 12, 8, 7, 4, 5, 6, 2, 1]	[23, 26]
3	15	[13, 12, 9, 10, 6, 8, 7, 4, 5, 1, 2]	[15, 23, 26]
4	13	[12, 10, 9, 5, 6, 8, 7, 4, 2, 1]	[13, 15, 23, 26]
5	12	[10, 6, 9, 5, 1, 8, 7, 4, 2]	[12, 13, 15, 23, 26]
6	10	[9, 6, 8, 5, 1, 2, 7, 4]	[10, 12, 13, 15, 23, 26]
7	9	[8, 6, 7, 5, 1, 2, 4]	[9, 10, 12, 13, 15, 23, 26]
8	8	[7, 6, 4, 5, 1, 2]	[8, 9, 10, 12, 13, 15, 23, 26]
9	7	[6, 5, 4, 2, 1]	[7, 8, 9, 10, 12, 13, 15, 23, 26]
10	6	[5, 2, 4, 1]	[6, 7, 8, 9, 10, 12, 13, 15, 23, 26]
11	5	[4, 2, 1]	[5, 6, 7, 8, 9, 10, 12, 13, 15, 23, 26]
12	4	[2, 1]	[4, 5, 6, 7, 8, 9, 10, 12, 13, 15, 23, 26]
13	2	[1]	[2, 4, 5, 6, 7, 8, 9, 10, 12, 13, 15, 23, 26]
Sorted		[1, 2, 4, 5, 6, 7, 8, 9, 10, 12, 13, 15, 23, 26]	

Complexity Analysis. BUILDMAXHEAP: $O(n)$. Each EXTRACTMAX + HEAPIFY: $O(\log n)$, done $n - 1$ times. Total: $\Theta(n \log n)$ worst, average, and best case.

Q41. What is a priority queue? Write three applications of a priority queue. **(2+3=5 marks)** [CSE 203 | 2011–12]

Answer

Definition.
A **priority queue** is an abstract data type (ADT) where each element is associated with a *priority key*. The element with the **highest** (or lowest) priority is always served first, regardless of the order in which elements were inserted. It supports:

- `insert(element, priority)`: add an element.
- `extractMax()` or `extractMin()`: remove and return the highest/lowest priority element.

- `peek()`: view the top element without removing.

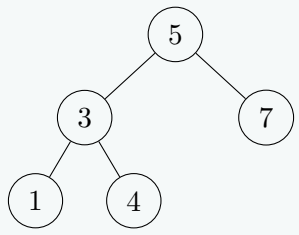
Three Applications:

1. **CPU Scheduling (OS):** Processes are assigned priorities. The scheduler always runs the highest-priority ready process first (preemptive priority scheduling).
2. **Dijkstra's Shortest Path Algorithm:** A min-priority queue is used to always process the unvisited vertex with the current shortest known distance, enabling $O((V+E)\log V)$ time.
3. **Huffman Encoding (Data Compression):** Characters are inserted with their frequencies as priorities. The two lowest-frequency nodes are repeatedly extracted and merged to build the optimal prefix-free code tree.

Q42. Explain the insertion and deletion operations in a binary search tree with illustrative examples. **(14 marks)** [CSE 203 | 2011–12]

Answer

Insertion. Start at root; go left if key < current, right if key > current; insert at the first `NULL` position.
Example: insert $\{5,3,7,1,4\}$:




```
treeNode* insert(treeNode* root, int key) {
    if (!root) return new treeNode(key);
    if (key < root->item) root->left  = insert(root->left,  key);
    else if (key > root->item) root->right = insert(root->right, key);
    return root;
}
```

Deletion — three cases:
Case 1 (leaf): remove directly.
Case 2 (one child): splice — parent points to the single child.
Case 3 (two children): copy in-order successor's key into node; delete the successor (which has ≤ 1 child).

```
treeNode* minNode(treeNode* n){
    while(n->left) n=n->left; return n;
}
treeNode* deleteNode(treeNode* root, int key){
    if(!root) return root;
    if(key < root->item) root->left = deleteNode(root->left, key);
    else if(key > root->item) root->right = deleteNode(root->right, key);
    else {
        if(!root->left){ treeNode* t=root->right; delete root; return t; }
        if(!root->right){ treeNode* t=root->left;  delete root; return t; }
        treeNode* s = minNode(root->right);
        root->item  = s->item;
        root->right = deleteNode(root->right, s->item);
    }
    return root;
}
```

Complexity Analysis. $O(h)$; average $O(\log n)$, worst $O(n)$ for skewed tree.

BUET · CSE 105

DSA

Graphs & Graph Representations

Adjacency Matrix, Adjacency List, Graph Properties

T7 — Graphs & Graph Representations

Q1. Given a graph $G = (V, E)$ with unique edge weights, let $(S, V - S)$ be a cut in G such that S is non-empty and $S \neq V$. Show that the minimum weighted crossing edge between any such cut in G is included in every minimum spanning tree. **(15 marks)** [CSE 105 | 2023–24]

Answer

Theorem (Cut Property). Let $G = (V, E)$ have unique edge weights, let $(S, V \setminus S)$ be any cut ($S \neq \emptyset$, $S \neq V$), and let $e^* = (u, v)$ be the unique minimum-weight edge crossing the cut. Then e^* belongs to *every* MST of G.

Proof (by contradiction).

Let T be an MST of G that does *not* contain e^*.

Since T is a spanning tree it is connected, so there exists a path P from u to v in T. Because $u \in S$ and $v \in V \setminus S$, this path must cross the cut at least once; let $e' = (x, y)$ be any such crossing edge on P (with $x \in S$, $y \in V \setminus S$).

Since e^* is the *unique* minimum crossing edge, $w(e^*) < w(e')$.

Construct $T' = T - \{e'\} + \{e^*\}$.

- T' is still spanning (same vertices).
- T' is still connected: removing e' splits T into two components; adding e^* reconnects them (since e^* also crosses the cut, linking the same two components).
- T' is acyclic: if a cycle existed it would have to use e^*; but then e' would complete a cycle in T, contradicting T being a tree.
- $w(T') = w(T) - w(e') + w(e^*) < w(T)$.

This contradicts T being an MST. Therefore every MST must contain e^*. □

Q2. Given an adjacency list representation of a directed graph $G = (V, E)$, sketch an efficient algorithm to find its transpose graph $G^T = (V, E^T)$ where $E^T = \{(v, u) \in V \times V : (u, v) \in E\}$. Analyze the time complexity of your algorithm. Also evaluate how long it takes to compute the in-degree and out-degree of every vertex of G. **(18 marks)** [CSE 105 | 2022–23]

Answer

Transpose Graph G^T: reverse every edge $(u, v) \to (v, u)$.

Algorithm 67 TRANSPOSE(G)

```
1: Initialize G^T with same vertex set V, empty edge set
2: for each vertex u ∈ V do
3:     for each vertex v in Adj[u] do
4:         Add edge (v, u) to G^T                ▷ reverse direction
5:     end for
6: end for
7: return G^T
```

Complexity Analysis. We visit every vertex once and every edge once: $\Theta(V + E)$.

In-degree and Out-degree of every vertex:

Algorithm 68 COMPUTEDEGREES(G)

```
1: outdeg[u] ← 0 for all u;   indeg[u] ← 0 for all u
2: for each vertex u ∈ V do
3:     for each v ∈ Adj[u] do
4:         outdeg[u] += 1
5:         indeg[v] += 1
6:     end for
7: end for
```

Complexity Analysis. Each edge is scanned once: $\Theta(V + E)$.

Remark. Using an adjacency *matrix*: out-degree of u = sum of row u ($O(V)$ per vertex, $O(V^2)$ total); in-degree of u = sum of column u ($O(V^2)$ total). So adjacency list is faster for sparse graphs.

Q3. A DAG is given to you. Find the maximum number of edges that can be added to this DAG such that the resulting graph is still a DAG (i.e., adding even one more edge would create a cycle). **(10 marks)** [CSE 203 | 2021–22]

Answer

Key insight: a directed graph is a DAG if and only if it has a topological ordering $v_1, v_2, \ldots, v_n$. In any DAG, all edges go from v_i to v_j with $i < j$ in *some* topological order. The maximum number of such directed forward edges is $\binom{n}{2} = \frac{n(n-1)}{2}$ (one for every

pair $i < j$).
Algorithm:

(a) Run topological sort on the given DAG to obtain order $v_1, v_2, \ldots, v_n$. Time: $O(V + E)$.

(b) For every pair (i, j) with $i < j$, if edge (v_i, v_j) does not already exist, add it. These are the only edges we can add without creating a cycle.

Maximum edges that can be added:

$$\frac{n(n-1)}{2} - |E|$$

where $|E|$ is the current number of edges.
The resulting graph is a *DAG tournament* (every pair of vertices has exactly one directed edge between them), and adding any further edge would create a cycle.

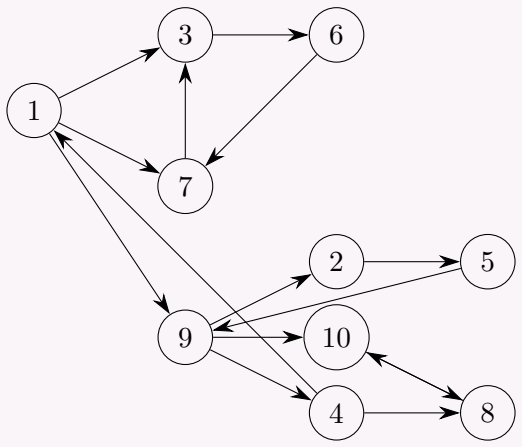
Remark. Different topological orderings may allow different sets of edges to be added, but the *count* $\binom{n}{2} - |E|$ is the same for all valid topological orderings.

Q4. Consider the adjacency matrix representation of a directed graph G. Draw the graph and find its adjacency list representation considering the vertices in ascending order. Find the shortest path distance of every vertex from vertex 1 using BFS. Show the queue status when vertex 10 is just enqueued. Prove that the distance of the most recent vertex (tail in the queue) is at most one larger than the distance of the oldest vertex (head in the queue). **(20 marks)** [CSE 105 | 2021–22]

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Answer

Graph (directed edges from the matrix):



Adjacency List (vertices in ascending order):

Vertex	Neighbours
1	3, 7, 9
2	5
3	6
4	1, 8
5	9
6	7
7	3
8	10
9	2, 4, 10
10	8

BFS from vertex 1 (neighbours processed in ascending order):

Step	Dequeue	Enqueue	Distances updated
1	1	3, 7, 9	$d[3]=1,\ d[7]=1,\ d[9]=1$
2	3	6	$d[6]=2$
3	7	(3 visited)	—
4	9	2, 4, 10	$d[2]=2,\ d[4]=2,\ d[10]=2$
5	6	(7 visited)	—
6	2	5	$d[5]=3$
7	4	8	$d[8]=3$
8	10	(8 not yet — enqueued next)	$d[8]=3$

Shortest distances from vertex 1:

Vertex	1	2	3	4	5	6	7	8	9	10
Distance	0	2	1	2	3	2	1	3	1	2

Queue state when vertex 10 is just enqueued (10 is enqueued while processing vertex 9):

$$\text{Queue (front to back):}\quad \underbrace{6}_{\text{head}},\ 2,\ 4,\ \underbrace{10}_{\text{tail}}$$

$d[\text{head}] = d[6] = 2, \quad d[\text{tail}] = d[10] = 2.$
Proof: $d[\text{tail}] \le d[\text{head}] + 1.$
Claim: At any point in BFS, if the queue contains $v_1, v_2, \ldots, v_k$ (front to back), then $d[v_k] \le d[v_1] + 1$ and $d[v_i] \le d[v_{i+1}]$ for all i.
Proof by induction on BFS steps.
Base: initially only the source with $d = 0$; trivially true.
Step: suppose the property holds. We dequeue v_1 and enqueue its unvisited neighbours w with $d[w] = d[v_1] + 1$.

- New head is v_2 with $d[v_2] \ge d[v_1]$.
- New tail w satisfies $d[w] = d[v_1] + 1 \le d[v_2] + 1 \le d[\text{new head}] + 1$. ✓

Hence at all times $d[\text{tail}] \le d[\text{head}] + 1$. □

Q5. Write a Graph (directed) data structure using a linked list. Write a delete-node operation for the data structure. **(8+7 marks)** [CSE 203 | 2019–20]

Answer

Data structures:

```
// Edge node in adjacency list
struct EdgeNode {
    int dest;
    EdgeNode* next;
};

// Vertex node in vertex list
struct VertexNode {
    int id;
    EdgeNode* edges;   // head of adjacency list
    VertexNode* next;  // next vertex in vertex list
};

struct Graph {
    VertexNode* vertices; // head of vertex linked list
};
```

Delete a vertex v from the graph:

(a) Remove all edges *from* v (free its edge list).

(b) Remove all edges *to* v from every other vertex's list.

(c) Remove v from the vertex list.

```
void deleteNode(Graph* g, int v) {
    VertexNode* prev = NULL;
    VertexNode* curr = g->vertices;

    // Step 1 & 3: find v, free its edge list, remove from vertex list
    while (curr && curr->id != v) { prev = curr; curr = curr->next; }
    if (!curr) return;  // vertex not found

    // Free edge list of v
```

```
    EdgeNode* e = curr->edges;
    while (e) { EdgeNode* tmp = e; e = e->next; free(tmp); }

    // Remove v from vertex list
    if (prev) prev->next = curr->next;
    else g->vertices = curr->next;
    free(curr);

    // Step 2: remove all edges pointing TO v from other vertices
    for (VertexNode* u = g->vertices; u; u = u->next) {
        EdgeNode* ep = NULL, *en = u->edges;
        while (en) {
            if (en->dest == v) {
                EdgeNode* tmp = en;
                if (ep) ep->next = en->next;
                else u->edges = en->next;
                en = en->next;
                free(tmp);
            } else { ep = en; en = en->next; }
        }
    }
}
```

Complexity Analysis. $O(V + E)$ — we scan all vertices and their edge lists once.

Q6. Suppose you are going to represent a very dense graph and want to optimize both space and time for accessing an edge. Which representation (adjacency list or adjacency matrix) would you prefer? Justify your answer. **(7 marks)** [CSE 203 | 2017–18]

Answer

For a **very dense graph**, prefer the **adjacency matrix**.
Justification:

- **Space**: a dense graph has $E = \Theta(V^2)$ edges, so the adjacency list uses $O(V + E) = O(V^2)$ space — the same asymptotic space as the matrix. No space saving from the list.
- **Edge access**: the matrix gives $O(1)$ lookup for edge (u, v), whereas the list requires $O(\deg(u)) = O(V)$ time.
- **Cache performance**: the matrix is a contiguous 2-D array; traversal is cache-friendly compared to pointer chasing in a list.

Therefore, for a very dense graph, the adjacency matrix is preferred for both space efficiency (same asymptotic cost) and faster edge access.

Q7. Compare adjacency matrix and adjacency list representations of a graph. **(10 marks)** [CSE 203 | 2016–17 ,2013–14]

Answer

Property	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Edge lookup (u, v)	$O(1)$	$O(\deg(u))$
Add edge	$O(1)$	$O(1)$
Remove edge	$O(1)$	$O(\deg(u))$
Enumerate neighbours of u	$O(V)$	$O(\deg(u))$
BFS / DFS total	$O(V^2)$	$O(V + E)$
Best for	Dense graphs	Sparse graphs

When to prefer which: For a *dense* graph ($E \approx V^2$) the matrix uses comparable space to the list but gives $O(1)$ edge lookup. For a *sparse* graph ($E \ll V^2$) the list saves space and graph traversal is faster.

Q8. Define graph, path, cycle, and subgraph with examples. **(10 marks)** [CSE 203 | 2015–16]

Answer

Graph $G = (V, E)$: a set of vertices V and edges $E \subseteq V \times V$.
Example: $V = \{1, 2, 3\}$, $E = \{(1, 2), (2, 3)\}$.
Path: a sequence of distinct vertices $v_0, v_1, \ldots, v_k$ where $(v_{i-1}, v_i) \in E$ for each i.
Example: $1 \to 2 \to 3$ is a path of length 2.
Cycle: a path $v_0, v_1, \ldots, v_k$ with $k \geq 1$ and $v_0 = v_k$ (start = end, all intermediate vertices distinct).
Example: $1 \to 2 \to 3 \to 1$ is a cycle of length 3.
Subgraph $H = (V', E')$ of G: $V' \subseteq V$, $E' \subseteq E$, and every edge in E' has both endpoints in V'.
Example: $V' = \{1, 2\}$, $E' = \{(1, 2)\}$ is a subgraph of the above.

Q9. For each of the following operations in an undirected graph, find the worst-case running times in Big-O notation using (i) an adjacency matrix and (ii) an adjacency list:

(a) Determine whether a given vertex is connected to no other vertex.

(b) Determine whether the graph has a vertex that is connected to no other vertex.

(c) Determine whether a given vertex is connected to every other vertex. Give the idea of your algorithm for the adjacency list case.

(4+4+6=14 marks) [CSE 203 | 2014–15]

Answer

(a) Is vertex v isolated (connected to no other vertex)?

- **Adjacency matrix**: scan row v for any $1 \to O(V)$.
- **Adjacency list**: check if Adj$[v]$ is empty $\to O(1)$.

(b) Does the graph have any isolated vertex?

- **Adjacency matrix**: scan all rows $\to O(V^2)$.
- **Adjacency list**: scan all lists (total work $= V + E$) $\to O(V + E)$.

(c) Is vertex v connected to every other vertex?

- **Adjacency matrix**: scan row v, check all $V - 1$ non-diagonal entries are $1 \to O(V)$.
- **Adjacency list**: maintain a boolean array `seen[1..V]`, initialise to `false`; traverse Adj$[v]$ and mark each neighbour `true`; finally scan the array checking all $u \neq v$ are marked $\to O(V + \deg(v))$. Worst case $O(V + E)$.

Remark. For the adjacency list case in (c): we cannot simply check $\deg(v) = V - 1$ because v might have multi-edges or self-loops in a general graph. The boolean array approach handles the general case correctly in $O(V + \deg(v))$.

Q10. Give a brief description of: (i) Graph, (ii) Forest, (iii) Tree, (iv) Spanning subgraph. **(1.5+1.5+1.5+2.5=7 marks)** [CSE 203 | 2013–14]

Answer

(i) Graph: A graph $G = (V, E)$ consists of a finite set of vertices V and a set of edges $E \subseteq V \times V$. Edges may be directed or undirected, weighted or unweighted.
(ii) Forest: An undirected acyclic graph (a graph with no cycles). A forest may be disconnected; each connected component is a tree.
(iii) Tree: A connected, acyclic, undirected graph. Equivalently, a connected graph on n vertices with exactly $n-1$ edges. A forest where every component is connected.
(iv) Spanning Subgraph: A subgraph $H = (V, E')$ of $G = (V, E)$ that contains *all* vertices of G ($V(H) = V(G)$) and a subset of edges ($E' \subseteq E$). If H is also a tree, it is called a *spanning tree*.

Q11. Show the adjacency matrix and adjacency list for the following graph. Calculate the number of bytes required in both representations. Assume that the storage requirement of a pointer and an integer is 4 bytes.

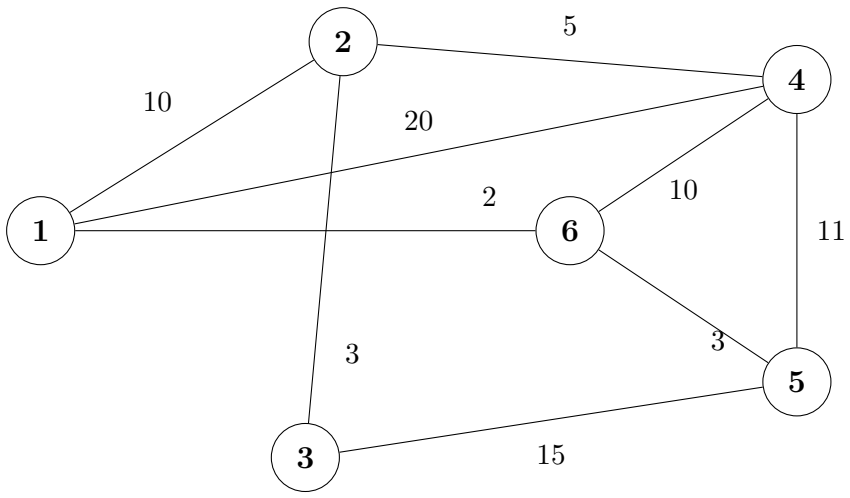


Figure 1: Weighted Graph

Answer

Graph: $V = \{1, 2, 3, 4, 5, 6\}$, 9 undirected weighted edges.
Adjacency Matrix (entry = weight, 0 = no edge):

	1	2	3	4	5	6
1	0	10	0	20	0	2
2	10	0	3	5	0	0
3	0	3	0	0	15	0
4	20	5	0	0	11	10
5	0	0	15	11	0	3
6	2	0	0	10	3	0

Byte count (matrix): $6 \times 6 = 36$ integers $\times$ 4 bytes = **144** bytes.
Adjacency List:

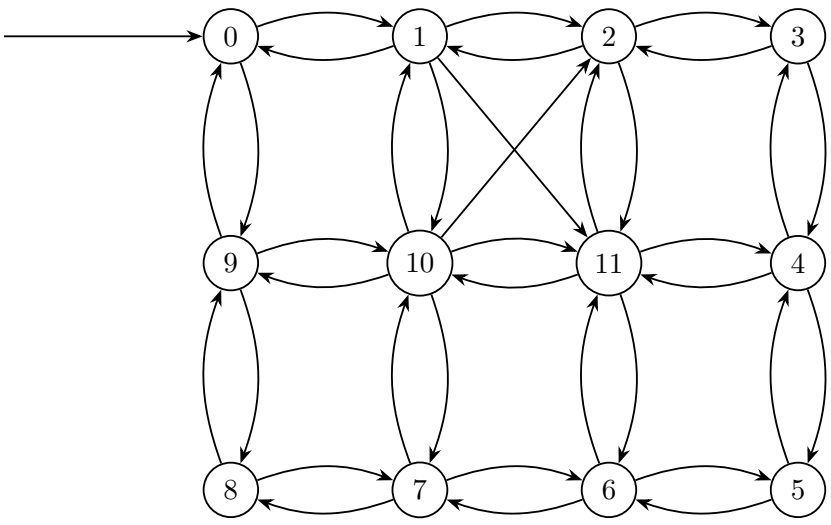
Vertex	List (neighbour, weight)
1	→(2,10)→(4,20)→(6,2)→NULL
2	→(1,10)→(3,3)→(4,5)→NULL
3	→(2,3)→(5,15)→NULL
4	→(1,20)→(2,5)→(5,11)→(6,10)→NULL
5	→(3,15)→(4,11)→(6,3)→NULL
6	→(1,2)→(4,10)→(5,3)→NULL

Byte count (list): Each list node stores: `dest` (4B) + `weight` (4B) + `next*` (4B) = 12 bytes.
For an undirected graph, each edge is stored *twice*: total list nodes $= 2 \times 9 = 18$.

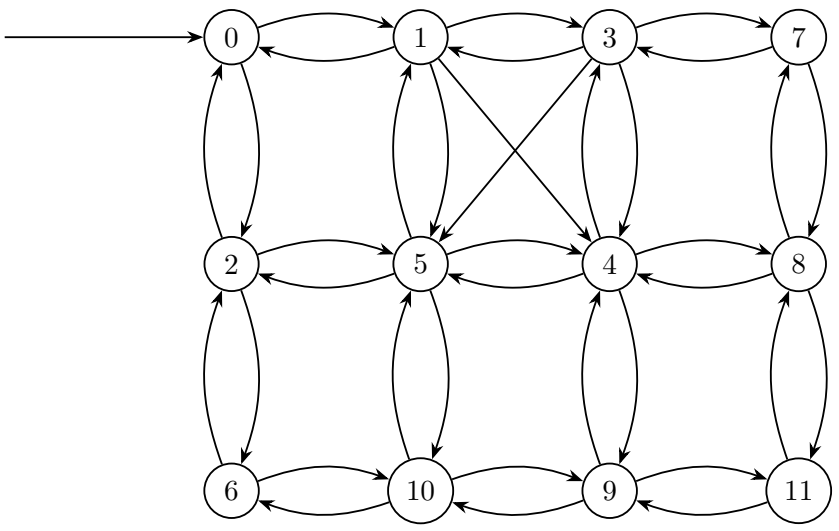
$$\underbrace{6 \times 4}_{\text{head pointers}} + \underbrace{18 \times 12}_{\text{list nodes}} = 24 + 216 = \textbf{240} \text{ bytes.}$$

Remark. The adjacency matrix (144 B) uses *less* space than the list (240 B) for this particular graph because the graph is relatively dense ($E = 9$ out of a possible $\binom{6}{2} = 15$ edges).

Q12. Your friend has employed two different traversal algorithms, *Algo I* and *Algo II*, on a graph and shown the output in Figure Q.5(c)-I and Figure Q.5(c)-II respectively. In both cases, the traversal starts at the node shown by the block right arrow. In the output, nodes are labeled with the order they are traversed. Identify the traversal algorithm for Figure Q.5(c)-I and Figure Q.5(c)-II and justify your answer. **(10 marks)**



$Figure Q.5(c) - I$



$Figure Q.5(c) - II$

Answer

Figure Q.5(c)-I: DFS (Depth-First Search)
Justification: The problem specifies that the node labels represent the *order* of traversal, not fixed node IDs. If we follow the numeric sequence ($0 \to 1 \to 2 \to 3 \to \cdots \to 11$), we trace the exact path the algorithm took. Starting from node 0, the algorithm moves to an adjacent neighbor (1) and immediately continues to plunge deeper into the graph to unvisited neighbors (2, then 3, then 4), rather than pausing to visit node 0's other immediate neighbor (which ends up being labeled 9 much later). The path forms a continuous snake that wraps around the perimeter and spirals inward. This strategy of exploring a single branch "as deep as possible" before backtracking is the exact definition of Depth-First Search.

Figure Q.5(c)-II: BFS (Breadth-First Search)
Justification: By following the numeric labels $(0, 1, 2, 3, \dots)$, we can observe that the nodes are visited in a wave-like, layer-by-layer pattern based on their shortest distance (number of edges) from the start node.

- **Distance 0:** The start node is labeled **0**.
- **Distance 1:** The immediate neighbors of the start node are labeled **1** and **2**.
- **Distance 2:** The unvisited neighbors of nodes 1 and 2 are visited next, receiving the labels **3, 4, 5,** and **6**.
- **Distance 3:** The unvisited neighbors of the previous layer receive the labels **7, 8, 9,** and **10**.
- **Distance 4:** The furthest node in the graph is finally labeled **11**.

Because the algorithm completely exhausts all nodes at a distance k from the source before moving to any node at distance $k+1$, this strictly layer-by-layer expansion is the hallmark of Breadth-First Search.

BUET · CSE 105

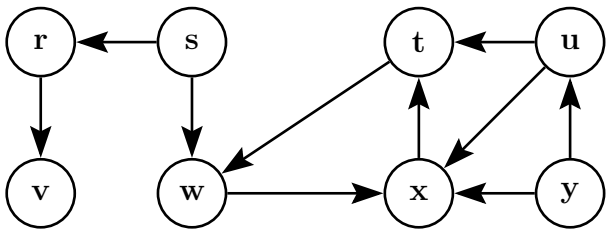
DSA

Graph Traversals: DFS & BFS

DFS Algorithm, BFS Algorithm, Edge Classification

T8 — Graph Traversals: DFS & BFS

Q1. Using a stack, construct an iterative (non-recursive) algorithm for depth first search (DFS) of a graph. Show how the state of the stack changes over time using the graph given in the figure. Analyze the complexity. Modify the algorithm to print the type of each edge it encounters. **(20 marks)** **[CSE 105 | 2023–24]**



Answer

Algorithm 69 ITERATIVE-DFS(G)

```
1: for each v ∈ V[G] do
2:     color[v] ← WHITE;   parent[v] ← nil
3: end for
4: time ← 0
5: for each s ∈ V[G] do
6:     if color[s] = WHITE then
7:         PUSH(S, s);   color[s] ← GRAY
8:         time ← time + 1;   d[s] ← time
9:         while S ≠ ∅ do
10:            u ← PEEK(S)
11:            found_white ← false
12:            for each v ∈ Adj[u] in order do
13:                if color[v] = WHITE then
14:                    color[v] ← GRAY;   time ← time + 1;   d[v] ← time
15:                    parent[v] ← u;   PUSH(S, v)
16:                    found_white ← true;   break
17:                end if
18:            end for
19:            if not found_white then
20:                POP(S);   color[u] ← BLACK
21:                time ← time + 1;   f[u] ← time
22:            end if
23:        end while
24:    end if
25: end for
```

Stack trace (Adj lists: r:[v], s:[r, w], t:[w], u:[t, x], v:[], w:[x], x:[t], y:[u, x]):

Action	**Stack** (bot→top)	**Event**	$d[\,]$	$f[\,]$
Start r	$[r]$	Push r	$d[r] = 1$	
$r \to v$	$[r, v]$	Tree: $r \to v$	$d[v] = 2$	
v done	$[r]$	Pop v		$f[v] = 3$
r done	[]	Pop r		$f[r] = 4$
Start s	$[s]$	Push s	$d[s] = 5$	
$s \to r$	$[s]$	Cross: $s \to r$ (r BLACK)		
$s \to w$	$[s, w]$	Tree: $s \to w$	$d[w] = 6$	
$w \to x$	$[s, w, x]$	Tree: $w \to x$	$d[x] = 7$	
$x \to t$	$[s, w, x, t]$	Tree: $x \to t$	$d[t] = 8$	
$t \to w$	$[s, w, x, t]$	Back: $t \to w$ (w GRAY)		
t done	$[s, w, x]$	Pop t		$f[t] = 9$
x done	$[s, w]$	Pop x		$f[x] = 10$
w done	$[s]$	Pop w		$f[w] = 11$
s done	[]	Pop s		$f[s] = 12$
Start u	$[u]$	Push u	$d[u] = 13$	
$u \to t$	$[u]$	Cross: $u \to t$ (t BLACK, $d[u] > d[t]$)		
$u \to x$	$[u]$	Cross: $u \to x$ (x BLACK, $d[u] > d[x]$)		
u done	[]	Pop u		$f[u] = 14$
Start y	$[y]$	Push y	$d[y] = 15$	
$y \to u$	$[y]$	Cross: $y \to u$ (u BLACK, $d[y] > d[u]$)		
$y \to x$	$[y]$	Cross: $y \to x$ (x BLACK)		
y done	[]	Pop y		$f[y] = 16$

Final timestamps:

Vertex	r	s	t	u	v	w	x	y
d	1	5	8	13	2	6	7	15
f	4	12	9	14	3	11	10	16

Edge classification: Tree: $(r,v),(s,w),(w,x),(x,t)$; Back: (t,w); Cross: $(s,r),(u,t),(u,x),(y,u),(y,x)$; Forward: none.

Complexity: $O(V+E)$. Each vertex is pushed/popped once ($O(V)$); each edge is examined once ($O(E)$).

Modified algorithm to print edge types — add inside the neighbor loop:

Algorithm 70 DFS-Visit-Classify(G, u)

```
1:  color[u] ← GRAY;  time ← time + 1;  d[u] ← time
2:  for each v ∈ Adj[u] do
3:      if color[v] = WHITE then
4:          print "Tree edge: u → v"
5:          parent[v] ← u
6:          DFS-Visit-Classify(G, v)
7:      else if color[v] = GRAY then
8:          print "Back edge: u → v"
9:      else
10:         if d[u] < d[v] then
11:             print "Forward edge: u → v"
12:         else
13:             print "Cross edge: u → v"
14:         end if
15:     end if
16: end for
17: color[u] ← BLACK;  time ← time + 1;  f[u] ← time
```

Q2. Define the four types of edges that a DFS on a directed graph yields. Modify the pseudocode of the original DFS algorithm to print all the edges along with their types. Justify whether your algorithm will work correctly. **(18 marks)** [CSE 105 | 2022–23]

Answer

Four DFS edge types (directed graph):

Type	**Condition when (u,v) examined**	**Example**
Tree edge	v is WHITE (undiscovered)	builds DFS tree
Back edge	v is GRAY (open ancestor)	$v.d < u.d$; indicates cycle
Forward edge	v is BLACK, $u.d < v.d$	v is non-tree descendant
Cross edge	v is BLACK, $u.d > v.d$	v in different branch/tree

Modified DFS pseudocode:

Algorithm 71 DFS(G)

```
1: for each u ∈ V[G] do
2:     color[u] ← WHITE; parent[u] ← nil
3: end for
4: time ← 0
5: for each u ∈ V[G] do
6:     if color[u] = WHITE then
7:         DfsVisit(G, u)
8:     end if
9: end for
```

Algorithm 72 DfsVisit(G, u)

```
1: color[u] ← GRAY; time ← time + 1; d[u] ← time
2: for each v ∈ Adj[u] do
3:     if color[v] = WHITE then
4:         parent[v] ← u
5:         print "Tree edge: (u, v)"
6:         DfsVisit(G, v)
7:     else if color[v] = GRAY then
8:         print "Back edge: (u, v)"
9:     else
10:        if d[u] < d[v] then
11:            print "Forward edge: (u, v)"
12:        else
13:            print "Cross edge: (u, v)"
14:        end if
15:    end if
16: end for
17: color[u] ← BLACK; time ← time + 1; f[u] ← time
```

Justification of correctness:
At the moment edge (u, v) is examined, u is GRAY (being processed). The color of v and the timestamps uniquely determine the edge type:

- v WHITE $\Rightarrow$ not yet visited; DFS will now discover it via $u \Rightarrow$ tree edge by definition.
- v GRAY $\Rightarrow$ v is an ancestor still open on the DFS stack $\Rightarrow$ back edge (going backward in the current path).
- v BLACK, $d[u] < d[v] \Rightarrow$ v was discovered after u but finished before u examined this edge; by Parenthesis Theorem v is a descendant of u (nested interval) $\Rightarrow$ forward edge.
- v BLACK, $d[u] > d[v] \Rightarrow$ v was discovered and fully finished before u was even discovered; the intervals are disjoint $\Rightarrow$ cross edge.

Note: For undirected graphs, only tree and back edges occur — the check $d[u] < d[v]$ vs. $d[u] > d[v]$ is unnecessary.

Q3. Classify the edges of the directed graph G after running DFS on the graph considering the vertices in ascending order. Prove that an edge (u, v) is a tree edge if $u.d < v.d < v.f < u.f$. **(15 marks)** **[CSE 105 | 2021–22]**

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

Answer

Adjacency list (from matrix): 1:[3, 7, 9], 2:[5], 3:[6], 4:[1, 8], 5:[9], 6:[7], 7:[3], 8:[10], 9:[2, 4, 10], 10:[8].

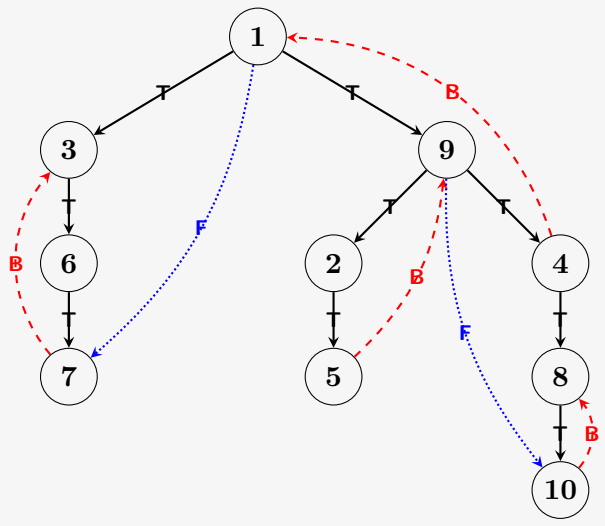
DFS timestamps (ascending order; start from 1):

Vertex	1	2	3	4	5	6	7	8	9	10
d	1	9	2	13	10	3	4	14	8	15
f	20	12	7	18	11	6	5	17	19	16
parent	—	9	1	9	2	3	6	4	1	8

Edge classification:

- **Tree edges (T):** $(1, 3), (3, 6), (6, 7), (1, 9), (9, 2), (2, 5), (9, 4), (4, 8), (8, 10)$
- **Back edges (B):** $(7, 3), (5, 9), (4, 1), (10, 8)$
- **Forward edges (F):** $(1, 7), (9, 10)$
- **Cross edges (C):** none

DFS Graph Representation:



Proof: (u, v) **is a tree edge** $\Leftrightarrow u.d < v.d < v.f < u.f$

($\Rightarrow$) Suppose (u, v) is a tree edge. By definition, v was WHITE when u examined it, and DFS called DFS-Visit(v) immediately. Thus:

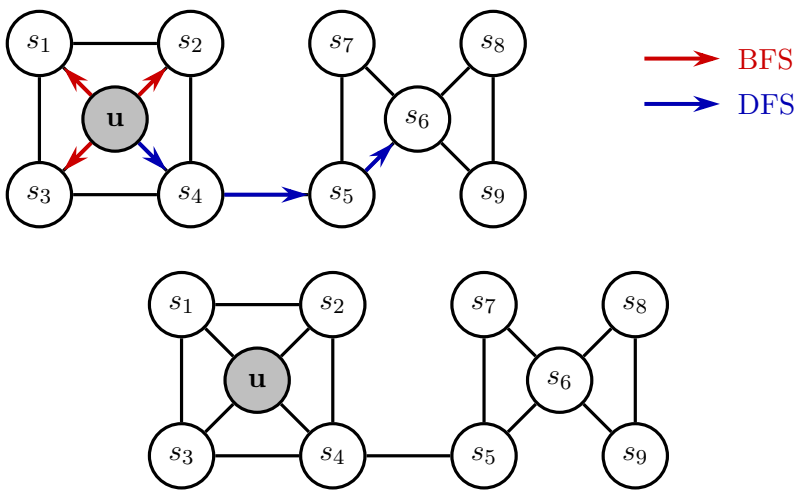
1. u was already GRAY before v was discovered $\Rightarrow u.d < v.d$.
2. v's entire subtree finishes before DFS returns to $u \Rightarrow v.f < u.f$.
3. v is discovered then finished $\Rightarrow v.d < v.f$.

Hence $u.d < v.d < v.f < u.f$.

($\Leftarrow$) Suppose $u.d < v.d < v.f < u.f$. By the **Parenthesis Theorem**, the interval $[v.d, v.f] \subset [u.d, u.f]$, so v is a descendant of u in the DFS tree. Now suppose for contradiction (u, v) is not a tree edge but a forward edge. Then v was already BLACK when (u, v) was examined — but this contradicts v being in u's subtree with $v.d > u.d$ (DFS would have made it a tree edge on first encounter). Hence (u, v) must be a tree edge. $\square$

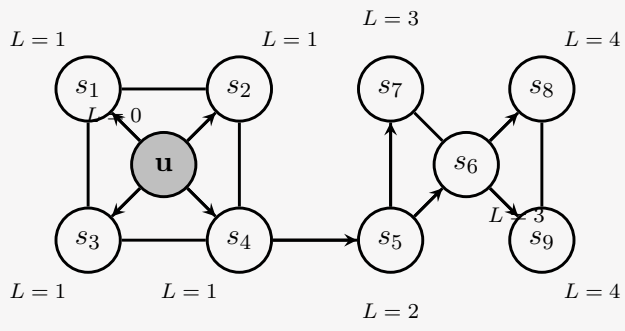
Verification: $(1, 3)$: $1 < 2 < 7 < 20$ ✓; $(9, 4)$: $8 < 13 < 18 < 19$ ✓.

Q4. You are given the following undirected graph to traverse using both BFS and DFS starting from vertex u. Show the simulation steps for labeling the vertices in order of visits. In BFS, the label of a vertex is its level; in DFS, label each vertex with its discovery and finishing times. Also write the pseudocodes for BFS and DFS. **(10+10 marks)** **[CSE 203 | 2019–20]**



Detailed Answer with Simulation Graphs

1. BFS Traversal starting from u (Alphabetical neighbor order)



Simulation Steps (Queue & Labeling):

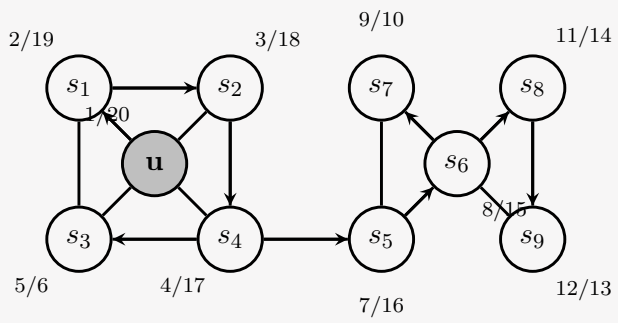
- **Init:** Enqueue u. Label Level $L(u) = 0$. Queue: $[u]$
- **Step 1:** Dequeue u. Unvisited neighbors: s_1, s_2, s_3, s_4. Label $L = 1$. Enqueue all. Queue: $[s_1, s_2, s_3, s_4]$
- **Step 2-4:** Dequeue s_1, s_2, s_3. None have new unvisited neighbors. Queue: $[s_4]$
- **Step 5:** Dequeue s_4. Unvisited neighbor: s_5. Label $L(s_5) = 2$. Enqueue s_5. Queue: $[s_5]$
- **Step 6:** Dequeue s_5. Unvisited neighbors: s_6, s_7. Label $L = 3$. Enqueue s_6, s_7. Queue: $[s_6, s_7]$
- **Step 7:** Dequeue s_6. Unvisited neighbors: s_8, s_9. Label $L = 4$. Enqueue s_8, s_9. Queue: $[s_7, s_8, s_9]$
- **Step 8-10:** Dequeue s_7, s_8, s_9. No unvisited neighbors left. Queue empty. Done.

Vertex	u	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9
Level	0	1	1	1	1	2	3	3	4	4
Parent	—	u	u	u	u	s_4	s_5	s_5	s_6	s_6

BFS order: $u,\ s_1,\ s_2,\ s_3,\ s_4,\ s_5,\ s_6,\ s_7,\ s_8,\ s_9$

---

2. DFS Traversal starting from u (Alphabetical neighbor order)



Simulation Steps (Discovery d / Finish f times):

- Visit u ($d = 1$). Neighbors: s_1, s_2, s_3, s_4. Go to s_1.
- Visit s_1 ($d = 2$). Neighbors: s_2, s_3, u. Go to s_2.
- Visit s_2 ($d = 3$). Neighbors: s_1, s_4, u. Go to s_4.
- Visit s_4 ($d = 4$). Neighbors: s_2, s_3, s_5, u. Go to s_3.
- Visit s_3 ($d = 5$). Neighbors visited. **Finish** s_3 ($f = 6$). Backtrack to s_4.
- At s_4: Next unvisited neighbor is s_5. Go to s_5.
- Visit s_5 ($d = 7$). Neighbors: s_4, s_6, s_7. Go to s_6.
- Visit s_6 ($d = 8$). Neighbors: s_5, s_7, s_8, s_9. Go to s_7.
- Visit s_7 ($d = 9$). Neighbors visited. **Finish** s_7 ($f = 10$). Backtrack to s_6.
- At s_6: Next unvisited is s_8. Go to s_8.
- Visit s_8 ($d = 11$). Neighbors: s_6, s_9. Go to s_9.
- Visit s_9 ($d = 12$). Neighbors visited. **Finish** s_9 ($f = 13$). Backtrack to s_8.
- **Finish** s_8 ($f = 14$) → **Finish** s_6 ($f = 15$) → **Finish** s_5 ($f = 16$) → **Finish** s_4 ($f = 17$) → **Finish** s_2 ($f = 18$) → **Finish** s_1 ($f = 19$) → **Finish** u ($f = 20$).

Vertex	u	s_1	s_2	s_3	s_4	s_5	s_6	s_7	s_8	s_9
d	1	2	3	5	4	7	8	9	11	12
f	20	19	18	6	17	16	15	10	14	13

DFS order (by discovery): $u,\ s_1,\ s_2,\ s_4,\ s_3,\ s_5,\ s_6,\ s_7,\ s_8,\ s_9$

---

3. Pseudocodes

Algorithm 73 BFS(G, s)

```
1:  for each u ∈ V[G] \ {s} do
2:      color[u] ← WHITE;   d[u] ← ∞;   parent[u] ← nil
3:  end for
4:  color[s] ← GRAY;   d[s] ← 0;   parent[s] ← nil
5:  Q ← empty queue;   ENQUEUE(Q, s)
6:  while Q ≠ ∅ do
7:      u ← DEQUEUE(Q)
8:      for each v ∈ Adj[u] do
9:          if color[v] = WHITE then
10:             color[v] ← GRAY
11:             d[v] ← d[u] + 1
12:             parent[v] ← u
13:             ENQUEUE(Q, v)
14:         end if
15:     end for
16:     color[u] ← BLACK
17: end while
```

DFS Pseudocode:

Algorithm 74 DFS(G)

```
for each u ∈ V[G] do
    color[u] ← WHITE; parent[u] ← nil
end for
time ← 0
for each u ∈ V[G] do
    if color[u] = WHITE then
        DfsVisit(G, u)
    end if
end for
```

Algorithm 75 DfsVisit(G, u)

```
color[u] ← GRAY; time ← time + 1; d[u] ← time
for each v ∈ Adj[u] do
    if color[v] = WHITE then
        parent[v] ← u
        DfsVisit(G, v)
    end if
end for
color[u] ← BLACK; time ← time + 1; f[u] ← time
```

Q5. Explain when there is no back edge if DFS is performed on an undirected graph. **(8 marks)** [CSE 203 | 2018–19]

Answer

There is no back edge in undirected DFS if and only if the graph is acyclic (i.e., a forest/tree).

- In undirected DFS, every edge (u, v) is either a *tree edge* (when v is WHITE) or a *back edge* (when v is GRAY, i.e., v is an open ancestor). Forward and cross edges do not occur in undirected graphs.
- A back edge (u, v) with v GRAY means v is an ancestor of u, creating a cycle $v \rightsquigarrow u \to v$.
- If the graph has no cycles (is a forest), every neighbor of u is either its parent (already seen but handled as parent, not a back edge) or undiscovered — so no back edges arise.
- Conversely, every cycle produces at least one back edge during DFS.

Summary: No back edges $\Leftrightarrow$ graph is a forest (acyclic).

Q6. Give the running times of DFS using adjacency lists and adjacency matrix. **(8 marks)** [CSE 203 | 2018–19]

Answer

Representation	Running Time	Reason
Adjacency list	$\Theta(V + E)$	Neighbor scan: $\sum \lvert \text{Adj}[u] \rvert = E$
Adjacency matrix	$\Theta(V^2)$	Each row scanned fully: $V \times V$

For sparse graphs ($E \ll V^2$), adjacency list is preferred. For dense graphs ($E = \Theta(V^2)$), both representations give the same asymptotic bound.

Q7. Would you use BFS or DFS to find any path from a start vertex to a goal in a graph with many long paths? Justify. **(10 marks)** [CSE 203 | 2018–19]

Answer

Use DFS.

- **DFS follows one long path immediately** without expanding all neighbors, likely hitting the goal quickly.
- **Memory:** DFS uses $O(\text{path length})$ stack space. BFS stores all vertices at each level: $O(b^d)$ where b = branching factor, d = distance. For long paths in a wide graph, BFS memory is prohibitive.
- **Optimality not needed:** We only need *any* path, so DFS's non-shortest path is acceptable.

Trade-off: BFS finds the *shortest* path; DFS finds a path faster with less memory. Since optimality is not required, **DFS is the better choice**.

Q8. "Given a directed graph G, both BFS and DFS from a particular starting vertex s will traverse the same set of vertices." Do you agree with this statement? Justify your answer. **(8 marks)** [CSE 203 | 2017–18]

Answer

Yes — both traverse exactly the set of vertices reachable from s.
Both BFS and DFS from s only visit vertices v such that there exists a directed path $s \rightsquigarrow v$. Neither can jump to a vertex with no such path.

- BFS enqueues only reachable vertices; unreachable ones stay WHITE.
- DFS-Visit(s) only recurses into WHITE neighbors, always following directed edges from s.

However, the *order* of visitation and the resulting *tree structure* (BFS tree vs. DFS tree) differ. Also, if we run the full DFS procedure (outer loop over all vertices), it visits all vertices including unreachable ones — but that is not "from s" alone.

Q9. Given an undirected graph $G = (V, E)$, give a linear-time algorithm to check whether there is a cycle of odd length in the graph. Briefly justify its correctness. **(15 marks)** [CSE 207 | 2015–16]

Answer

Key theorem: A graph has an odd-length cycle ⇔ it is NOT bipartite.

Algorithm 76 HAS-ODD-CYCLE(G)

```
1:  for each v ∈ V[G] do
2:      color[v] ← −1
3:  end for
4:  for each s ∈ V[G] do
5:      if color[s] = −1 then
6:          Q ← empty queue
7:          color[s] ← 0;   ENQUEUE(Q, s)
8:          while Q ≠ ∅ do
9:              u ← DEQUEUE(Q)
10:             for each v ∈ Adj[u] do
11:                 if color[v] = −1 then
12:                     color[v] ← 1 − color[u]
13:                     ENQUEUE(Q, v)
14:                 else if color[v] = color[u] then
15:                     return true                          ▷ odd cycle found
16:                 end if
17:             end for
18:         end while
19:     end if
20: end for
21: return false
```

Correctness:

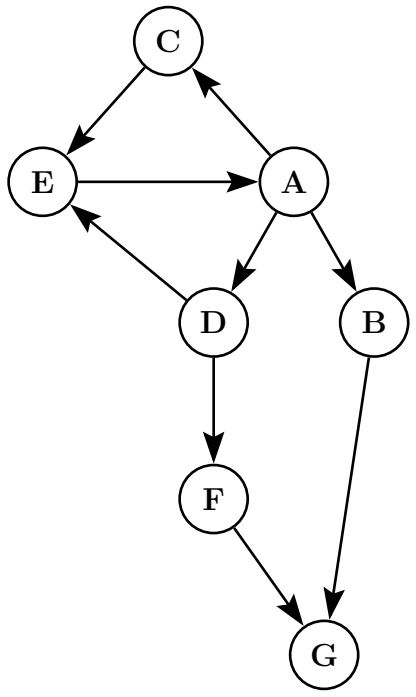
- **Bipartite ⇒ no odd cycle:** In any 2-coloring, every edge connects vertices of opposite colors. Any cycle alternates colors and must have even length.
- **Not bipartite ⇒ odd cycle:** If two adjacent vertices u, v get the same color during BFS, the BFS-level paths from their common ancestor to u and v have the same length, so the cycle ancestor$\rightsquigarrow u \to v \rightsquigarrow$ancestor has odd length.

Time: $O(V + E)$ (standard BFS).

Q10. Consider the following directed graph. Answer the following questions:

(a) Perform depth first search on the graph starting from vertex A. Choose the smallest (in alphabetical order) vertex when there is a choice. Find the discovery time and finishing time of each vertex.

(b) Perform breadth first search on the graph starting from vertex A. Choose the smallest (in alphabetical order) vertex when there is a choice. Find the distance and parent of each vertex.

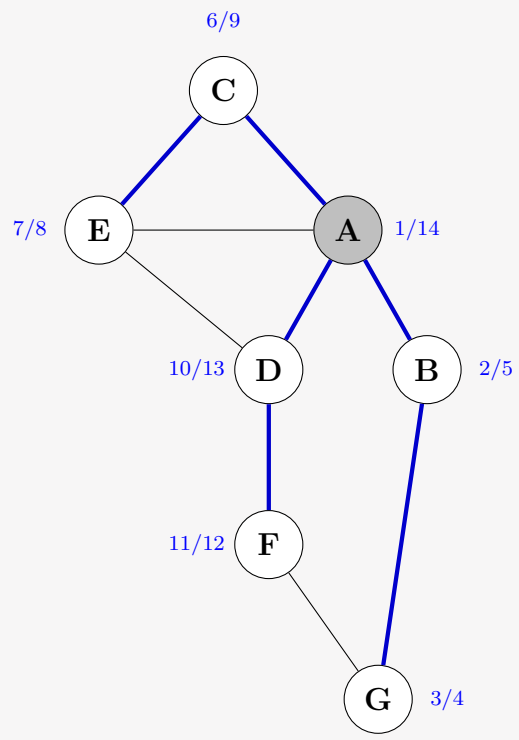


(6+6=12 marks) **[CSE 203 | 2014–15]**

Answer

Adjacency List: A:$[B, C, D]$; B:$[G]$; C:$[E]$; D:$[E, F]$; E:$[A]$; F:$[G]$; G:$[]$

(a) DFS Traversal starting from A (Alphabetical order)



Simulation Steps:

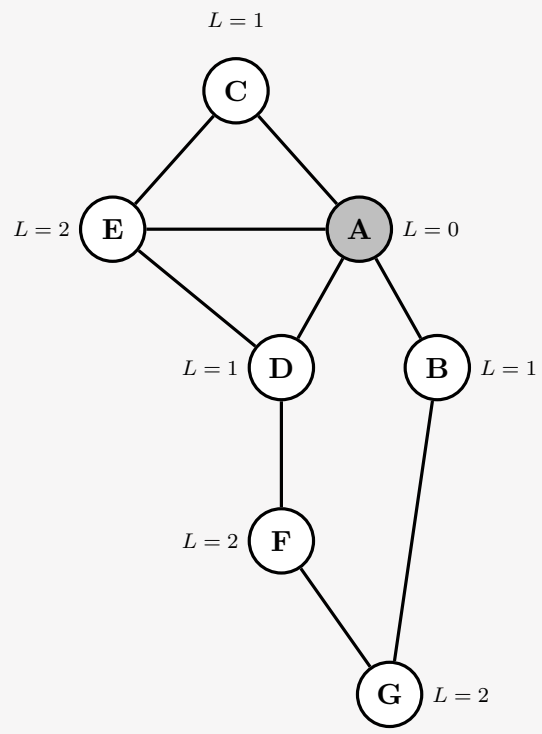
- Start at A ($d = 1$). Neighbors: B, C, D. Go to B.
- Visit B ($d = 2$). Neighbors: G. Go to G.
- Visit G ($d = 3$). No neighbors. **Finish** G ($f = 4$). Backtrack to B.
- **Finish** B ($f = 5$). Backtrack to A. Next neighbor C.
- Visit C ($d = 6$). Neighbors: E. Go to E.
- Visit E ($d = 7$). Neighbor A is already GRAY (Back edge). **Finish** E ($f = 8$). Backtrack to C.
- **Finish** C ($f = 9$). Backtrack to A. Next neighbor D.
- Visit D ($d = 10$). Neighbors: E, F. E is BLACK (Cross edge). Go to F.
- Visit F ($d = 11$). Neighbor G is BLACK (Cross edge). **Finish** F ($f = 12$). Backtrack to D.
- **Finish** D ($f = 13$). Backtrack to A. **Finish** A ($f = 14$).

Vertex	A	B	C	D	E	F	G
d	1	2	6	10	7	11	3
f	14	5	9	13	8	12	4

Edge types:

- **Tree edges:** $(A, B), (B, G), (A, C), (C, E), (A, D), (D, F)$
- **Back edges:** (E, A)
- **Cross edges:** $(D, E), (F, G)$

(b) BFS Traversal starting from A (Alphabetical order)



Simulation Steps (Queue):

- **Init:** Enqueue A. Level $L(A) = 0$. **Q:** $[A]$
- **Step 1:** Dequeue A. Unvisited neighbors: B, C, D. Label $L = 1$, parent A. Enqueue B, C, D. **Q:** $[B, C, D]$
- **Step 2:** Dequeue B. Unvisited neighbors: G. Label $L(G) = 2$, parent B. Enqueue G. **Q:** $[C, D, G]$
- **Step 3:** Dequeue C. Unvisited neighbors: E. Label $L(E) = 2$, parent C. Enqueue E. **Q:** $[D, G, E]$
- **Step 4:** Dequeue D. Neighbors: E, F. E is visited. Label $L(F) = 2$, parent D. Enqueue F. **Q:** $[G, E, F]$
- **Step 5-7:** Dequeue G, E, F. No unvisited neighbors left. **Q:** []. Done.

BFS Order: $A,\ B,\ C,\ D,\ G,\ E,\ F$

Vertex	A	B	C	D	E	F	G
dist (Level)	0	1	1	1	2	2	2
parent	—	A	A	A	C	D	B

Q11. Suggest an algorithm based on DFS that counts the number of connected components in an undirected graph. **(8 marks)** [CSE 203 | 2014–15]

Answer

Algorithm 77 COUNT-COMPONENTS(G)

```
1: for each u ∈ V[G] do
2:     color[u] ← WHITE
3: end for
4: count ← 0
5: for each u ∈ V[G] do
6:     if color[u] = WHITE then
7:         count ← count + 1
8:         DFS-VISIT(G, u)
9:     end if
10: end forreturn count
```

Justification: A single DFS-VISIT(u) marks exactly all vertices reachable from u — the complete connected component containing u. Each new WHITE vertex in the outer loop belongs to a previously unseen component.
Time: $O(V + E)$.

Q12. Consider the graph G. Perform the following:

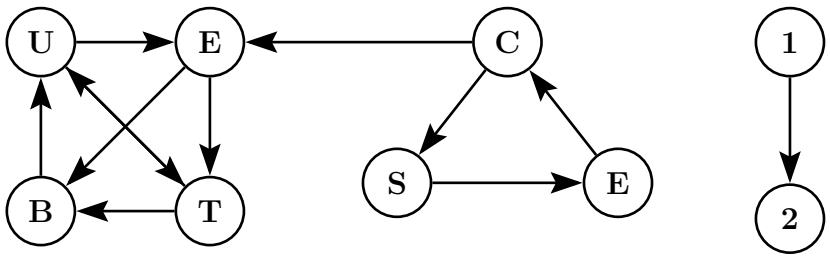
(a) Write the adjacency matrix and adjacency list of graph G.

(b) Draw the BFS forest of graph G assuming B as the starting vertex.

(c) Write the sequence of vertices if you explore graph G starting from vertex B in DFS.

(d) Write the tree edges, back edges, forward edges, and cross edges.

(6+6+4+6=22 marks) [CSE 203 | 2013–14]



Detailed Answer with Simulation Graphs

Graph Analysis: The complete graph G has 9 vertices: $\{1, 2, B, C, E_1, E_2, S, T, U\}$. Note: There are two nodes labeled 'E' in the figure. To avoid confusion, the one in the first component is denoted as E_1, and the one in the second as E_2.

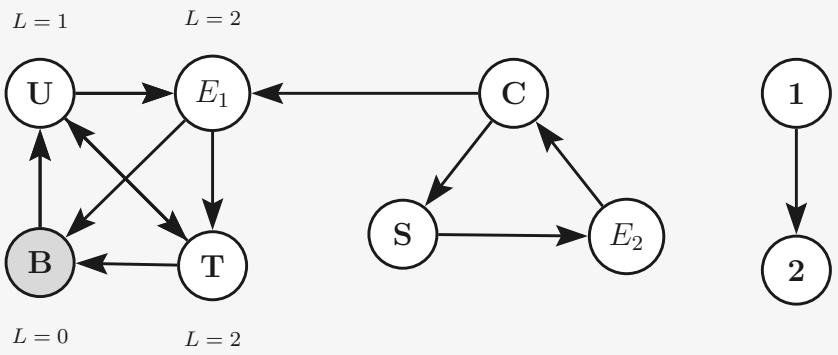
(a) Adjacency Matrix and List of entire graph G:

	1	2	B	C	E_1	E_2	S	T	U
1	0	1	0	0	0	0	0	0	0
2	0	0	0	0	0	0	0	0	0
B	0	0	0	0	0	0	0	0	1
C	0	0	0	0	1	0	1	0	0
E_1	0	0	1	0	0	0	0	1	0
E_2	0	0	0	1	0	0	0	0	0
S	0	0	0	0	0	1	0	0	0
T	0	0	1	0	0	0	0	0	1
U	0	0	0	0	1	0	0	1	0

Adjacency List:

- 1: [2]
- 2: []
- B: $[U]$
- C: $[E_1, S]$
- E_1: $[B, T]$
- E_2: $[C]$
- S: $[E_2]$
- T: $[B, U]$
- U: $[E_1, T]$

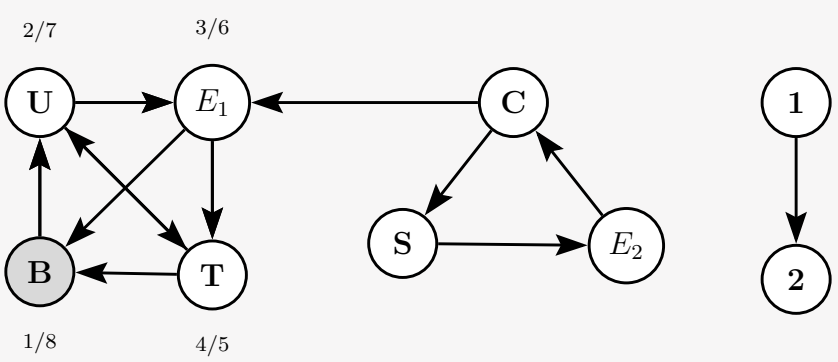
(b) BFS forest of graph G starting from vertex B:



Simulation Queue: $[B] \to [U] \to [E_1, T] \to [T] \to []$
BFS Order from B: $B,\ U,\ E_1,\ T$ (Other components remain unvisited).

Vertex	B	E_1	T	U
dist (Level)	0	2	2	1
parent	—	U	U	B

(c) DFS sequence starting from vertex B:



Simulation Steps: Start at $B(1) \xrightarrow{\text{tree}} U(2) \xrightarrow{\text{tree}} E_1(3)$. From E_1, B is GRAY (Back edge). Next $\xrightarrow{\text{tree}} T(4)$. From T, B and U are GRAY (Back edges). Finish $T(5) \rightarrow$ Finish $E_1(6)$. Back to U, T is BLACK (Forward edge). Finish $U(7) \rightarrow$ Finish $B(8)$.
DFS Order from B: $B,\ U,\ E_1,\ T$ (Other components remain unvisited).

Vertex	B	E_1	T	U
d	1	3	4	2
f	8	6	5	7

(d) Edge Classification (for the traversed component):

- **Tree Edges:** $(B, U),\ (U, E_1),\ (E_1, T)$
- **Back Edges:** $(E_1, B),\ (T, B),\ (T, U)$
- **Forward Edges:** (U, T)
- **Cross Edges:** None

Q13. Write an algorithm for DFS traversal. Analyze time complexity. **(8+7=15 marks)** [CSE 203 | 2012–13]

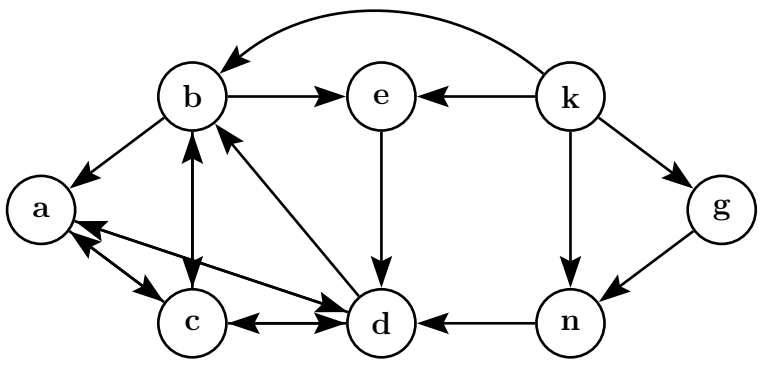
Answer

(Same pseudocode as Q4. See above.)
Complexity: $\Theta(V + E)$ with adjacency list; $\Theta(V^2)$ with adjacency matrix. Each vertex is discovered and finished exactly once; each edge is examined once per endpoint.

Q14. Consider the graph G. Perform the following:

(a) Write the adjacency matrix and adjacency list of the graph G.
(b) Draw the BFS forest of the graph G assuming b as the starting vertex.
(c) Write the sequence of vertices if you explore the graph G starting from vertex b in DFS.
(d) Write the tree edges, back edges, forward edges, and cross edges.

(6+4+4+6=20 marks) [CSE 203 | 2012–13]



Detailed Answer with Simulation Graphs

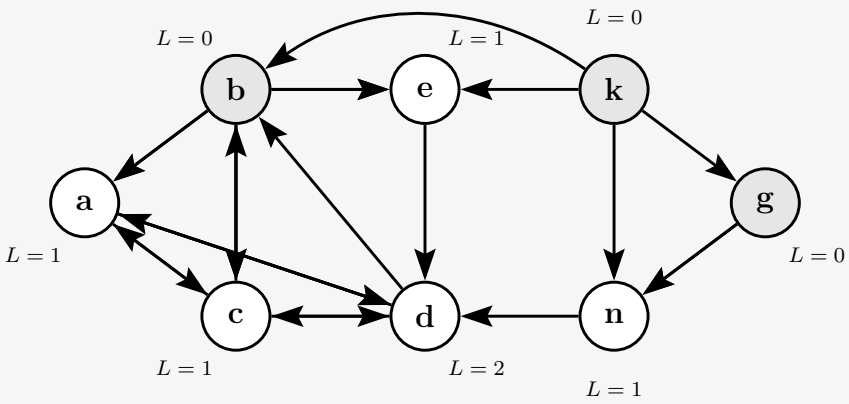
(a) Adjacency matrix and list (vertices: a, b, c, d, e, g, k, n):

	a	b	c	d	e	g	k	n
a	0	0	1	1	0	0	0	0
b	1	0	1	0	1	0	0	0
c	1	1	0	1	0	0	0	0
d	1	1	1	0	0	0	0	0
e	0	0	0	1	0	0	0	0
g	0	0	0	0	0	0	0	1
k	0	1	0	0	1	1	0	1
n	0	0	0	1	0	0	0	0

Adj list:

- a: $[c, d]$
- b: $[a, c, e]$
- c: $[a, b, d]$
- d: $[a, b, c]$
- e: $[d]$
- g: $[n]$
- k: $[b, e, g, n]$
- n: $[d]$

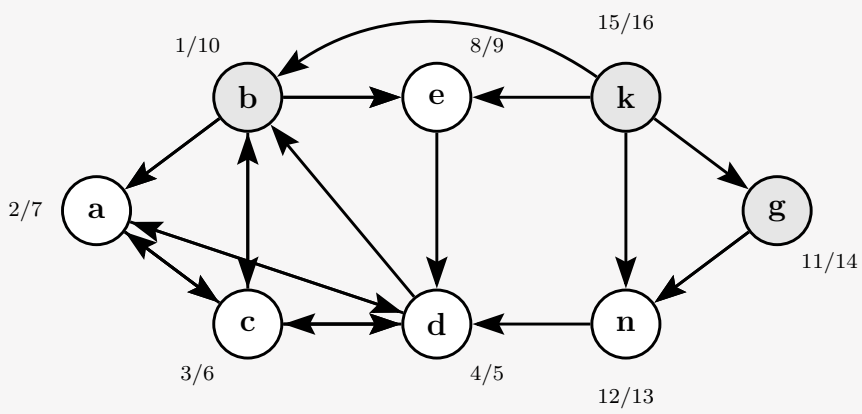
(b) BFS forest of graph G starting from b:



BFS order (Forest): $b,\ a,\ c,\ e,\ d,\ g,\ n,\ k$

Vertex	b	a	c	e	d	g	n	k
dist (L)	0	1	1	1	2	0	1	0
parent	—	b	b	b	a	—	g	—

(c) DFS sequence starting from b:



Simulation Traversal: Start at $b \to a \to c \to d$. From d, all neighbors (a, b, c) are already visited (Back edges). Finish $d \to c \to a$. From b, visit e. From e, d is visited (Cross edge). Finish $e \to b$. Next unvisited is $g \to n$. From n, d is visited (Cross edge). Finish $n \to g$. Finally, visit k. All neighbors of k are visited (Cross edges). Finish k.
DFS order (by discovery): $b,\ a,\ c,\ d,\ e,\ g,\ n,\ k$

Vertex	b	a	c	d	e	g	n	k
d	1	2	3	4	8	11	12	15
f	10	7	6	5	9	14	13	16

(d) Edge classification (Based on DFS from b):

- **Tree:** $(b, a),\ (a, c),\ (c, d),\ (b, e),\ (g, n)$
- **Back:** $(c, a),\ (c, b),\ (d, a),\ (d, b),\ (d, c)$
- **Forward:** $(a, d),\ (b, c)$
- **Cross:** $(e, d),\ (n, d),\ (k, b),\ (k, e),\ (k, g),\ (k, n)$

Q15. Write an algorithm for DFS traversal of a given graph. Analyze the time complexity of the algorithm. **(10+5=15 marks)** [CSE 203 | 2011–12]

Answer

Algorithm 78 DFS(G)

```
1: for each u ∈ V[G] do
2:     color[u] ← WHITE;   parent[u] ← nil
3: end for
4: time ← 0
5: for each u ∈ V[G] do
6:     if color[u] = WHITE then
7:         DFS-VISIT(G, u)
8:     end if
9: end for
```

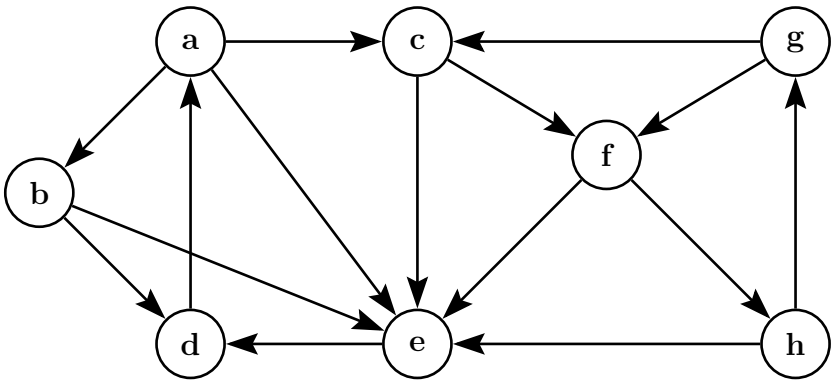
Algorithm 79 DFS-Visit(G, u)

```
1: color[u] ← GRAY;   time ← time + 1;   d[u] ← time
2: for each v ∈ Adj[u] do
3:     if color[v] = WHITE then
4:         parent[v] ← u
5:         DFS-VISIT(G, v)
6:     end if
7: end for
8: color[u] ← BLACK;   time ← time + 1;   f[u] ← time
```

Time Complexity:

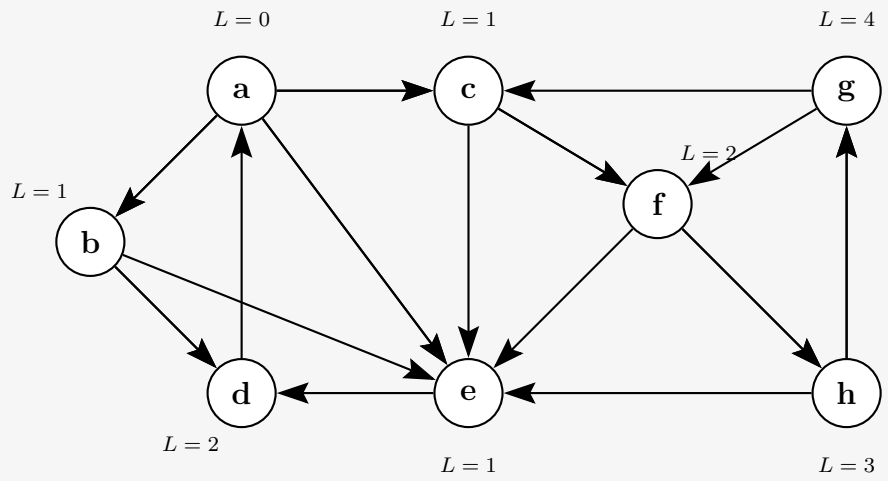
- Initialization loop in DFS: $\Theta(V)$.
- DFS-Visit(u) is called exactly once per vertex (only when WHITE).
- Total work for adjacency-list scanning across all calls: $\sum_u |\text{Adj}[u]| = |E|$.
- **Total:** $\Theta(V + E)$ with adjacency list; $\Theta(V^2)$ with adjacency matrix.

Q16. Write the sequences of the vertices if you explore the given graph G using BFS and DFS. Classify the edges of G for your DFS traversal. **(4+6=10 marks)** [CSE 203 | 2011–12]



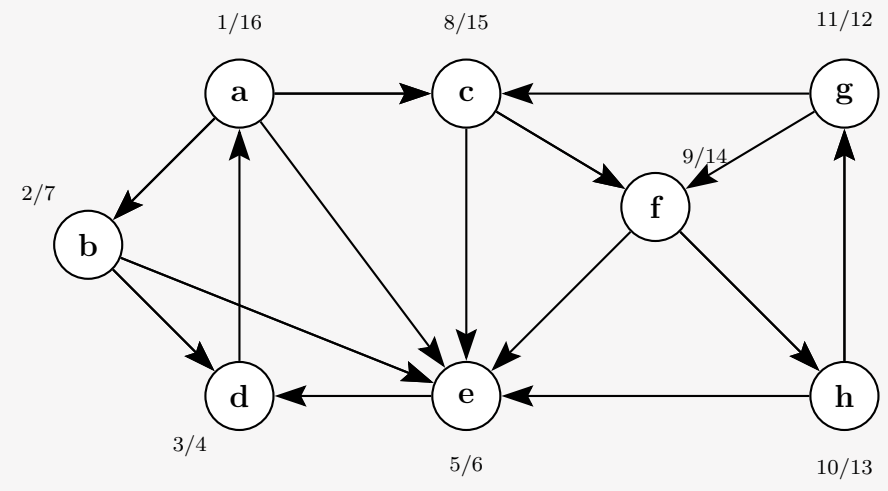
Detailed Answer with Simulation Graphs

(a) BFS Traversal starting from a (Alphabetical Order):



BFS Order: $a,\ b,\ c,\ e,\ d,\ f,\ h,\ g$
Distances: $a : 0,\ \ b, c, e : 1,\ \ d, f : 2,\ \ h : 3,\ \ g : 4$

(b) DFS Traversal starting from a (Alphabetical Order):



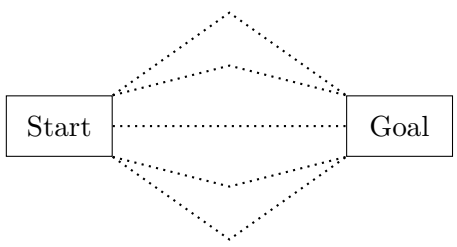
DFS Order: $a,\ b,\ d,\ e,\ c,\ f,\ h,\ g$

Vertex	a	b	c	d	e	f	g	h
d	1	2	8	3	5	9	11	10
f	16	7	15	4	6	14	12	13

Edge classification:

- **Tree:** $(a, b),\ (b, d),\ (b, e),\ (a, c),\ (c, f),\ (f, h),\ (h, g)$
- **Back:** $(d, a),\ (g, c),\ (g, f)$
- **Forward:** (a, e)
- **Cross:** $(e, d),\ (c, e),\ (f, e),\ (h, e)$

Q17. Suppose you have a graph like the one below. The dots signify some part of the graph that you don't know exactly, not a straight link. You know however, that there are many paths from the start to the goal. You also know that they tend to be rather long. Suppose that you want to implement a program that searches for a path and returns the first one it can find. You have no need for finding the optimal path in any sense, just any path will do. Would you want to use BFS or DFS as the basis for your program? Justify your answer.



(10 marks)

Answer

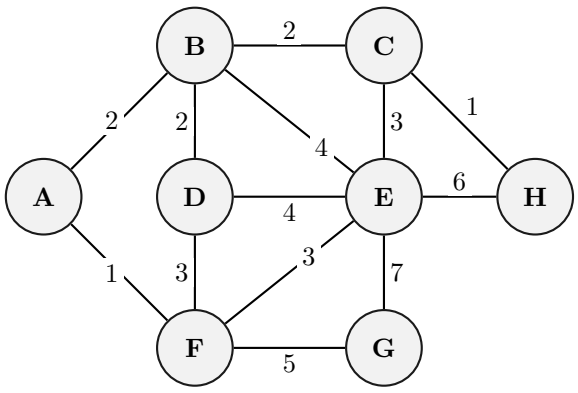
Use DFS.

Justification:
The graph has *many long paths* from Start to Goal. In this setting:

- **BFS** explores nodes *level by level*. Since the paths are long, BFS would need to store *all* nodes at each level in its queue before advancing. With many paths, the queue grows very large, consuming significant memory. BFS is ideal when the goal is *close* to the start, not far away.
- **DFS** dives deep along *one path at a time*. Since we only need *any* path (not the shortest or optimal), DFS will quickly reach the Goal by following one of the many long paths without storing all alternatives. Memory usage is proportional to the *depth* of the path ($O(d)$ stack space), which is far more efficient here.

Conclusion: Since the goal is far, paths are long, and any path will do, **DFS** is the better choice — it finds a solution faster and uses much less memory than BFS in this scenario.

Q18. Consider the weighted graph G as shown in Figure 1. Then draw the adjacency lists of the weighted graph G. Whenever there is a choice of vertices to explore, always pick the one that is alphabetically first.



Answer

Adjacency List of weighted graph G (neighbours listed alphabetically with edge weights):

Vertex	Adjacency List (neighbour, weight)
A	→ B(2) → F(1)
B	→ A(2) → C(2) → D(2) → E(4)
C	→ B(2) → E(3) → H(1)
D	→ B(2) → E(4) → F(3)
E	→ B(4) → C(3) → D(4) → F(3) → G(7) → H(6)
F	→ A(1) → D(3) → E(3) → G(5)
G	→ E(7) → F(5)
H	→ C(1) → E(6)

Each entry vertex → $u(w)$ means there is an edge between the vertex and u with weight w. Since the graph is undirected, each edge appears in both endpoints' lists.

BUET · CSE 105

DSA

Applications of DFS & BFS

Topological Sort, SCC, Shortest Paths, Dijkstra

T8a — Applications of DFS & BFS

Q1. Explain why the topological sort algorithm cannot run on a directed graph that contains cycle(s). Derive the component graph of the directed graph G represented by the given adjacency matrix. Then find the topological sorting of the vertices of the component graph. Show every step. **(15 marks)** [CSE 105 | 2021–22]

	1	2	3	4	5	6	7	8	9	10
1	0	0	1	0	0	0	1	0	1	0
2	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0
4	1	0	0	0	0	0	0	1	0	0
5	0	0	0	0	0	0	0	0	1	0
6	0	0	0	0	0	0	1	0	0	0
7	0	0	1	0	0	0	0	0	0	0
8	0	0	0	0	0	0	0	0	0	1
9	0	1	0	1	0	0	0	0	0	1
10	0	0	0	0	0	0	0	1	0	0

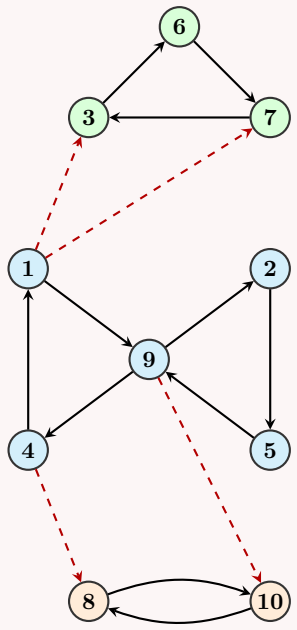
Answer

1. Why topological sort fails on a cyclic directed graph:
Topological sort requires a *linear ordering* of vertices such that for every directed edge (u, v), vertex u appears before v. If the graph contains a cycle (e.g., $v_1 \to v_2 \to \cdots \to v_k \to v_1$), we would need v_1 to appear before v_k and simultaneously v_k before v_1, which is a logical contradiction. Thus, a valid linear ordering is impossible.

2. Adjacency List and Original Graph (G):
From the given adjacency matrix, we can derive the adjacency list and visualize the graph cleanly.

Adjacency List:

- $1 \to 3, 7, 9$
- $2 \to 5$
- $3 \to 6$
- $4 \to 1, 8$
- $5 \to 9$
- $6 \to 7$
- $7 \to 3$
- $8 \to 10$
- $9 \to 2, 4, 10$
- $10 \to 8$



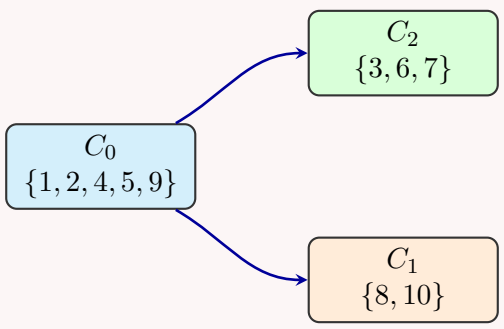
3. Finding Strongly Connected Components (Kosaraju's Algorithm):
Step 1: DFS on G to compute finish times.
Starting DFS from vertex 1, we visit recursively and record the finish times:

- Exploring from 1: $1 \to 3 \to 6 \to 7$ (finish 7,6,3), then $1 \to 9 \to 2 \to 5$ (finish 5,2), then $9 \to 4 \to 8 \to 10$ (finish 10,8,4), then finish 9, 1.
- **Finish order (first to last):** $7, 6, 3, 5, 2, 10, 8, 4, 9, 1$.
- **Reverse finish order:** $1, 9, 4, 8, 10, 2, 5, 3, 6, 7$.

Step 2: DFS on the Transpose Graph G^T in reverse finish order.

- Start at **1**: Visits $\{1, 4, 9, 5, 2\}$. This forms **SCC** C_0.
- Next unvisited is **8**: Visits $\{8, 10\}$. This forms **SCC** C_1.
- Next unvisited is **3**: Visits $\{3, 7, 6\}$. This forms **SCC** C_2.

4. Component Graph (G^{SCC}):
By collapsing each SCC into a single node, we get the directed acyclic graph (DAG) G^{SCC}. Based on the original graph's cross-edges, C_0 has outgoing edges to both C_1 and C_2.



5. Topological Sorting of G^{SCC}:
To find the topological sort, we compute the in-degrees of each node in G^{SCC}:

- Indegree(C_0) = 0. (Process first, remove its outgoing edges).
- Indegree(C_1) becomes 0.
- Indegree(C_2) becomes 0.

Since C_1 and C_2 both have an in-degree of 0 after C_0 is processed, they can be placed in any order.
Final Topological Order:

$$\boxed{C_0 \to C_1 \to C_2} \quad \text{or} \quad \boxed{C_0 \to C_2 \to C_1}$$

Q2. How do you determine whether a given graph is bipartite? **(10 marks)** [CSE 203 | 2019–20]

Answer

A graph is **bipartite** iff it contains no odd-length cycle, equivalently iff its vertices can be 2-colored such that no two adjacent vertices share a color.
Algorithm (BFS 2-coloring):

Algorithm 80 IsBipartite(G)

```
color[v] ← −1 for all v
for each uncolored vertex s do
    color[s] ← 0; enqueue s
    while queue not empty do
        u ← dequeue
        for each neighbor v of u do
            if color[v] = −1 then
                color[v] ← 1 − color[u]; enqueue v
            else if color[v] = color[u] then
                return false                                  ▷ odd cycle found
            end if
        end for
    end while
end for
return true
```

Example: a 4-cycle $1-2-3-4-1$ is bipartite (color $\{1,3\}$ red and $\{2,4\}$ blue). A 3-cycle $1-2-3-1$ is **not** bipartite.

Complexity Analysis. $O(V+E)$.

Q3. Write a function to find the lexicographically smallest topological ordering of a directed graph with N vertices and M edges that may contain cycles. Print -1 if the graph has cycles. **(8 marks)** [CSE 203 | 2018–19]

Answer

Use **Kahn's algorithm** with a *min-heap* (priority queue) instead of a plain queue.

```cpp
vector<int> lexTopoSort(int N, vector<pair<int,int>>& edges) {
    vector<vector<int>> adj(N+1);
    vector<int> indeg(N+1, 0);
    for (auto [u,v] : edges) {
        adj[u].push_back(v);
        indeg[v]++;
    }
    priority_queue<int, vector<int>, greater<int>> pq; // min-heap
    for (int i = 1; i <= N; i++)
        if (indeg[i] == 0) pq.push(i);

    vector<int> result;
    while (!pq.empty()) {
        int u = pq.top(); pq.pop();
        result.push_back(u);
        for (int v : adj[u]) {
            if (--indeg[v] == 0) pq.push(v);
        }
    }
    if ((int)result.size() < N) return {-1}; // cycle detected
    return result;
}
```

Complexity Analysis. $O((V+E)\log V)$ due to the min-heap operations.

Q4. The following is Dijkstra's algorithm. Perform a line-by-line time-complexity analysis for an input graph G where an adjacency list is used for graph representation and an ordinary array is used for storing $d[\,]$ values.

```
1:  Q ← V[G]
2:  for each v ∈ Q do
3:      d[v] ← ∞
4:  end for
5:  d[s] ← 0; S ← ∅
6:  while Q ≠ ∅ do
7:      u ← ExtractMin(Q)
8:      S ← S ∪ {u}
9:      for each v ∈ u.Adj[ ] do
10:         if v ∈ Q and d[v] > d[u] + w(u, v) then
11:             d[v] ← d[u] + w(u, v)
12:         end if
13:     end for
14: end while
```

(15 marks) [CSE 207 | 2018–19]

Answer

Using an **ordinary array** for $d[\,]$ (no heap).

Line	Code	Cost per call / times called
1	$Q \leftarrow V[G]$	$O(V)$ once
2	init $d[v] \leftarrow \infty$	$O(V)$ once
3	$d[s] \leftarrow 0$	$O(1)$ once
4	**while** $Q \neq \emptyset$	V iterations
5	ExtractMin(Q) (scan array)	$O(V)$ per iter $\rightarrow$ $O(V^2)$ total
6	$S \leftarrow S \cup \{u\}$	$O(1)$ per iter
7	**for** $v \in u$.Adj	sum over all $u = O(E)$ total
8–9	relax + update $d[v]$	$O(1)$ per edge

Total: $O(V^2 + E) = O(V^2)$ (since $E = O(V^2)$).

Remark. For *dense* graphs ($E = \Theta(V^2)$) this plain-array implementation is asymptotically optimal. For sparse graphs ($E = O(V)$), a binary heap gives $O(V \log V)$.

Q5. Johnson's Algorithm uses Bellman-Ford and Dijkstra's algorithms to compute All-Pairs-Shortest-Paths efficiently on sparse graphs. Explain the basic clever idea of Johnson's Algorithm. **(15 marks)** [CSE 207 | 2018–19]

Answer

Problem: All-Pairs Shortest Paths (APSP) on a sparse graph that may have negative-weight edges (but no negative cycles). Dijkstra is fast ($O((V + E) \log V)$) but requires non-negative weights.
Johnson's clever idea — reweighting:

(a) Add a new vertex q with zero-weight edges to all other vertices: $w(q, v) = 0$ for all $v \in V$.

(b) Run **Bellman-Ford** from $q \rightarrow$ get $h[v] = \delta(q, v)$ for all v. Time: $O(VE)$.

(c) **Reweight** each edge: $\hat{w}(u, v) = w(u, v) + h[u] - h[v] \geq 0$. (Non-negativity follows from the triangle inequality $h[v] \leq h[u] + w(u, v)$.)

(d) Run **Dijkstra** from every vertex using $\hat{w}$. Time: $O(V(V + E) \log V)$.

(e) Recover true distances: $\delta(u, v) = \hat{\delta}(u, v) - h[u] + h[v]$.

Total: $O(VE + V(V + E) \log V)$ — faster than Floyd-Warshall's $O(V^3)$ for sparse graphs.

Q6. Briefly explain how a directed acyclic graph $G = (V, E)$ can be topologically sorted so that if (u, v) is an edge then u appears before v in the ordering. Pseudocode is not needed. **(8 marks)** [CSE 203 | 2017–18]

Answer

Idea (DFS-based topological sort):
Run DFS on G. When a vertex v is *finished* (all its descendants fully explored), prepend v to an output list. The resulting list is a valid topological ordering because: for any edge (u, v), v finishes before u finishes (since v is a descendant of u in the DFS tree, or u is already finished when v is discovered via a cross/forward edge — either way $f[u] > f[v]$).
Kahn's algorithm (BFS-based): Repeatedly remove vertices with in-degree 0, appending them to the output and decrementing neighbours' in-degrees. If the output contains fewer than $|V|$ vertices, a cycle exists.

Both approaches run in $\Theta(V + E)$.

Q7. Suppose you are developing a feature for Facebook. You have access to everyone's friend list and the goal is to show how any two users are connected through a chain of "a friend of a friend". For two users A and B, show a sequence of individuals $X_0, X_1, \ldots, X_n$ where $X_0 = A$, $X_n = B$, and X_i is friends with X_{i+1}, such that the length of the sequence is minimized. Explain how this can be formulated as a graph problem and what algorithm you would use. **(7 marks)** [CSE 203 | 2017–18]

Answer

Graph formulation:

- **Vertices**: one per Facebook user.
- **Edges**: undirected edge (X_i, X_{i+1}) if X_i and X_{i+1} are friends.
- The problem reduces to finding the **shortest path** from vertex A to vertex B in this unweighted, undirected graph.

Algorithm: **BFS** from source A.
BFS explores vertices level by level, guaranteeing the shortest (minimum-hop) path. When B is first dequeued, trace back via the BFS predecessor array to recover the chain $X_0 = A, X_1, \ldots, X_n = B$.
Time: $O(V + E)$ where V = number of users, E = number of friendship pairs.

Q8. How can BFS be adapted to solve the single-source shortest path problem in an edge-weighted graph with unequal weights? Explain. **(8 marks)** [CSE 207 | 2017–18]

Answer

Standard BFS finds shortest paths in *unweighted* graphs. For edge-weighted graphs with **unequal positive integer weights**, BFS can be adapted by **edge splitting**:
Replace each edge (u, v) of weight w with a path $u \to x_1 \to x_2 \to \cdots \to x_{w-1} \to v$ using $w - 1$ dummy vertices and w unit-weight edges. Now all edges have weight 1; run standard BFS.

Complexity Analysis. If W = max edge weight and E edges, the expanded graph has $O(E \cdot W)$ edges → BFS runs in $O(V + E \cdot W)$. For small W this is practical; for large weights, use Dijkstra instead.

Alternatively: use a **modified BFS with a deque (0-1 BFS)** for $\{0, 1\}$-weight graphs, or simply use **Dijkstra** for general non-negative integer weights in $O((V + E) \log V)$.

Q9. Explain the intuitive idea behind the Bellman-Ford algorithm. How can you detect a negative cycle using Bellman-Ford? **(6+2 marks)** [CSE 207 | 2017–18]

Answer

Intuitive idea. Any shortest path without negative cycles is a *simple* path with at most $|V| - 1$ edges. Bellman-Ford iteratively *relaxes* all edges $|V| - 1$ times: in pass i, it correctly computes shortest paths using at most i edges. After $|V| - 1$ passes, all shortest paths (if they exist) are found.
Negative cycle detection. After $|V| - 1$ passes, run one more relaxation pass over all edges. If any distance $d[v]$ decreases, a negative-weight cycle is reachable from the source — because a simple path cannot have more than $|V| - 1$ edges; any further improvement must involve repeating a vertex, i.e. traversing a negative cycle.

Q10. Write Dijkstra's algorithm for the single-source shortest path problem. Prove its correctness. Analyze the time complexity of the algorithm. What would be the running time if a Fibonacci heap is used instead of a binary heap? **(5+5+7+5 marks)** [CSE 207 | 2017–18]

Answer

Algorithm 81 DIJKSTRA(G, w, s)

```
1: d[s] ← 0; d[v] ← ∞ for v ≠ s; S ← ∅
2: Insert all vertices into min-priority queue Q keyed by d
3: while Q ≠ ∅ do
4:     u ← EXTRACT-MIN(Q); S ← S ∪ {u}
5:     for each (u, v) ∈ E do
6:         if d[v] > d[u] + w(u, v) then
7:             d[v] ← d[u] + w(u, v); DECREASE-KEY(Q, v, d[v])
8:         end if
9:     end for
10: end while
```

Correctness: proved in [CSE 207 | 2014–15] answer above.

Complexity Analysis. With a **binary heap**: $|V|$ Extract-Min at $O(\log V)$ each, $|E|$ Decrease-Key at $O(\log V)$ each. Total: $O((V+E)\log V)$.
With a **Fibonacci heap**: Extract-Min: $O(\log V)$ amortised; Decrease-Key: $O(1)$ amortised. Total: $O(V \log V + E)$ — better for dense graphs.

Q11. Design a linear-time algorithm for each of the following. Briefly describe the ideas (pseudocode is not needed):

(a) Given an undirected graph G and a particular edge (u, v) in it, determine whether G has a cycle containing (u, v).

(b) Given an undirected and unweighted graph G, a particular edge (u, v), and two vertices s and t, determine whether there is a shortest path from s to t containing the edge (u, v).

(7+8 marks) [CSE 203 | 2016–17]

Answer

(a) Does G have a cycle containing edge (u, v)?
Idea: A cycle containing (u, v) exists iff there is a path from v back to u in $G \setminus \{(u, v)\}$.
Algorithm:

(a) Temporarily remove edge (u, v) from G.

(b) Run BFS/DFS from v.

(c) If u is reachable from $v \rightarrow$ **Yes**, a cycle exists. Otherwise $\rightarrow$ **No**.

(d) Restore (u, v).

Time: $O(V + E)$ (single BFS/DFS).
(b) Is there a shortest path from s to t using edge (u, v)?
Idea: A shortest s-t path uses (u, v) iff

$$d(s, u) + 1 + d(v, t) = d(s, t),$$

where $d(a, b)$ denotes the unweighted shortest-path distance.
Algorithm:

(a) BFS from s on $G \rightarrow$ compute $d(s, \cdot)$ for all vertices. $O(V + E)$.

(b) BFS from t on G^T (transpose) $\rightarrow$ compute $d(\cdot, t)$ for all vertices. $O(V + E)$.

(c) Check: $d(s, u) + 1 + d(v, t) = d(s, t)$?

Time: $O(V + E)$ total.

Q12. Write the properties satisfied by shortest paths of a graph. Show by example that Dijkstra's algorithm gives wrong results for a graph with negative-weight edges. Write an algorithm that finds shortest paths in a DAG and analyze its time complexity. **(15 marks)** [CSE 207 | 2016–17]

Answer

Properties of shortest paths:

(a) **Optimal substructure**: any sub-path of a shortest path is itself shortest.

(b) **No negative cycles**: shortest paths are undefined if negative cycles are reachable.

(c) **Triangle inequality**: $\delta(s, v) \leq \delta(s, u) + w(u, v)$.

(d) **Upper-bound property**: $d[v] \geq \delta(s, v)$ always; equality holds once v is finalized.

Dijkstra fails on negative edges — example: Graph: $s \rightarrow a$ (weight 3), $s \rightarrow b$ (weight 4), $b \rightarrow a$ (weight -2). True $\delta(s, a) = \min(3,\ 4-2) = 2$. Dijkstra finalizes a first with $d[a] = 3$ (greedy), never revisiting it to find the shorter path $s \rightarrow b \rightarrow a = 2$.
DAG Shortest Path algorithm:

Algorithm 82 DAG-Shortest-Path(G, w, s)

```
1: Topologically sort G
2: d[s] ← 0; d[v] ← ∞ for v ≠ s
3: for each vertex u in topological order do
4:     for each (u, v) ∈ E do
5:         if d[u] + w(u, v) < d[v] then
6:             d[v] ← d[u] + w(u, v)
7:         end if
8:     end for
9: end for
```

Complexity Analysis. $\Theta(V + E)$ — works even with negative edges since there are no cycles.

Q13. Prove that the Bellman-Ford algorithm returns TRUE if the input graph contains no negative-weight cycles reachable from the source, and FALSE otherwise. **(10 marks)** [CSE 207 | 2016–17]

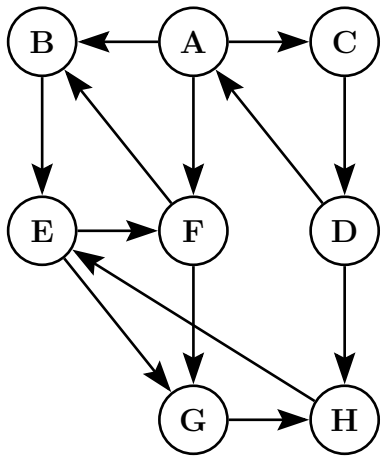
Answer

Proof.
(⇒) No negative cycle ⇒ returns TRUE.
If no negative cycle is reachable from s, all shortest paths are simple (at most $|V|-1$ edges). After $|V|-1$ relaxation passes, by the path-relaxation property, $d[v] = \delta(s,v)$ for all reachable v. For the final check: for any edge (u,v), $d[u]+w(u,v) \geq \delta(s,u)+w(u,v) \geq \delta(s,v) = d[v]$. No update occurs → return TRUE. □
(⇐) Negative cycle reachable ⇒ returns FALSE.
Suppose a negative cycle $C : v_0 \to v_1 \to \cdots \to v_k \to v_0$ is reachable from s with $\sum w(v_i, v_{i+1}) < 0$. Assume for contradiction the algorithm returns TRUE, meaning $d[v_{i+1}] \leq d[v_i] + w(v_i, v_{i+1})$ for all edges. Summing around the cycle:

$$\sum_{i=0}^{k-1} d[v_{i+1}] \leq \sum_{i=0}^{k-1} d[v_i] + \sum w(v_i, v_{i+1}).$$

The $d[\cdot]$ sums cancel (cycle), leaving $0 \leq \sum w < 0$ — a contradiction. Hence the algorithm must return FALSE. □

Q14. Run DFS on a given graph starting from vertex A and show the discovery and finishing times for each vertex as well as the edge types. Decompose the graph into strongly connected components (SCCs). **(15 marks)** [CSE 203 | 2016–17]



Answer

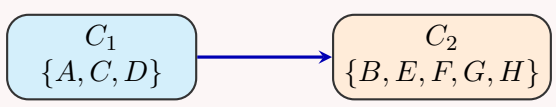
Adjacency list (alphabetical order): $A \to [B, C, F]$, $B \to [E]$, $C \to [D]$, $D \to [A, H]$, $E \to [F, G]$, $F \to [B, G]$, $G \to [H]$, $H \to [E]$.
DFS from A (neighbours in alphabetical order):

Vertex	Discovery	Finish
A	1	16
B	2	11
C	12	15
D	13	14
E	3	10
F	4	9
G	5	8
H	6	7

Edge classification:

Edge	Type
(A, B)	Tree
(B, E)	Tree
(E, F)	Tree
(F, B)	**Back** (cycle: $B \to E \to F \to B$)
(F, G)	Tree
(G, H)	Tree
(H, E)	**Back** (cycle: $E \to F \to G \to H \to E$)
(E, G)	Forward
(A, C)	Tree
(C, D)	Tree
(D, A)	**Back** (cycle: $A \to C \to D \to A$)
(D, H)	Cross
(A, F)	Forward

Strongly Connected Components (Kosaraju's):



SCC	Vertices
C_1	$\{B, E, F, G, H\}$
C_2	$\{A, C, D\}$

Remark. Back edges directly reveal the cycles that form SCCs: $F \to B$ closes the cycle B-E-F; $H \to E$ closes E-F-G-H; together these five vertices form one SCC. $D \to A$ closes A-C-D.

Q15. Suppose a CS curriculum consists of n mandatory courses. The prerequisite directed graph G has a node for each course and an edge from v to w if v is a prerequisite for w. Give a linear-time algorithm that computes the minimum number of semesters necessary to complete the curriculum (a student may take any number of courses per semester). **(15 marks)** [CSE 207 | 2015–16]

Answer

Formulation: longest path in the prerequisite DAG gives the minimum number of semesters (each level = one semester).

Algorithm 83 MinSemesters(G)

```
Topologically sort G → order v_1, v_2, ..., v_n
sem[v] ← 1 for all v
for each u in topological order do
    for each neighbor v of u do
        sem[v] ← max(sem[v], sem[u] + 1)
    end for
end for
return max_v sem[v]
```

Complexity Analysis. Topological sort: $O(V + E)$. Longest-path DP: $O(V + E)$. Total: $O(V + E)$.

Q16. Let C and C' be distinct strongly connected components in a directed graph $G = (V, E)$. Suppose there is an edge $(u, v) \in E$ where $u \in C$ and $v \in C'$. Prove that $f(C) > f(C')$, where $f(U)$ denotes the latest finishing time of any vertex in U. **(10 marks)** [CSE 207 | 2014–15]

Answer

Theorem. Let C, C' be distinct SCCs with edge (u, v), $u \in C$, $v \in C'$. Then $f(C) > f(C')$.
Proof. Let $x \in C$ be the first vertex discovered in DFS. Since C is an SCC, all of C is reachable from x via G; since (u, v) exists and C' is reachable from C, all of C' is also reachable from x.
Case 1: DFS discovers some vertex $y \in C'$ before x. Then x is discovered later with C' already fully explored (since there is no back-edge from C' to C, as that would merge them into one SCC). So $f(C') < d(x) < f(x) \leq f(C)$.
Case 2: x is discovered first. The DFS tree rooted at x reaches all of C and all of C' (via the path through (u, v)). C' has no path back to C (otherwise they'd be the same SCC), so C' finishes before the DFS returns to any vertex of C. Hence $f(C') < f(C)$. □

Q17. State and prove the convergence property of edge relaxation in the context of shortest paths. **(8 marks)** [CSE 207 | 2014–15]

Q18. Provide a correctness proof of Dijkstra's algorithm. **(10 marks)** [CSE 207 | 2014–15]

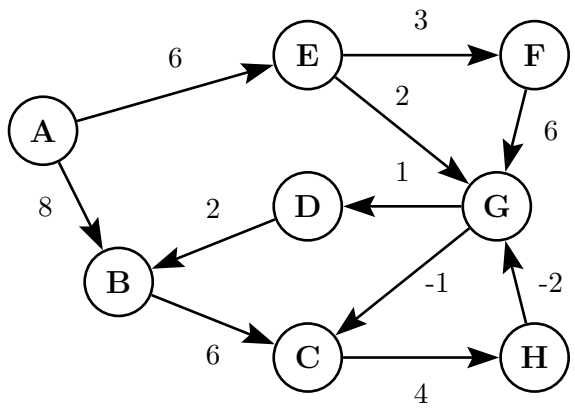
Answer

Theorem. On graphs with non-negative edge weights, when Dijkstra's algorithm finalizes vertex u (extracts u from Q), $d[u] = \delta(s, u)$.
Proof (by induction on $|S|$, the set of finalized vertices).
Base ($|S| = 0$): s is extracted first with $d[s] = 0 = \delta(s, s)$. ✓
Step: assume $d[v] = \delta(s, v)$ for all $v \in S$. Let u be the next vertex extracted.
Suppose for contradiction $d[u] > \delta(s, u)$. Let p be a true shortest path $s \xrightarrow{*} u$; let (x, y) be the first edge on p with $x \in S$, $y \notin S$.
Since all edge weights ≥ 0:

$$d[y] = \delta(s, y) \leq \delta(s, u) < d[u].$$

(The first equality holds by the inductive hypothesis applied when x was finalized and (x, y) was relaxed; the second by non-negativity.)
But $y \notin S$ and $d[y] < d[u]$ — the algorithm would have chosen y before u, contradicting that u was just extracted. □

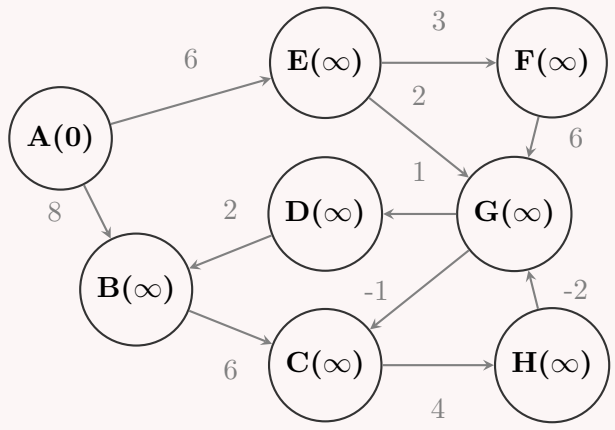
Q19. Apply the Bellman-Ford algorithm to find the shortest paths. Show all relaxation steps. **(10 marks)** [CSE 207 | 2014–15]



Answer — Bellman-Ford Algorithm Step-by-Step

Edge List & Relaxation Order: $(A \to E, 6)$, $(A \to B, 8)$, $(E \to F, 3)$, $(E \to G, 2)$, $(F \to G, 6)$, $(G \to D, 1)$, $(H \to G, -2)$, $(G \to C, -1)$, $(D \to B, 2)$, $(B \to C, 6)$, $(C \to H, 4)$.
Source Vertex = **A**. Total Vertices $|V| = 8$. Maximum required passes $= |V| - 1 = 7$.

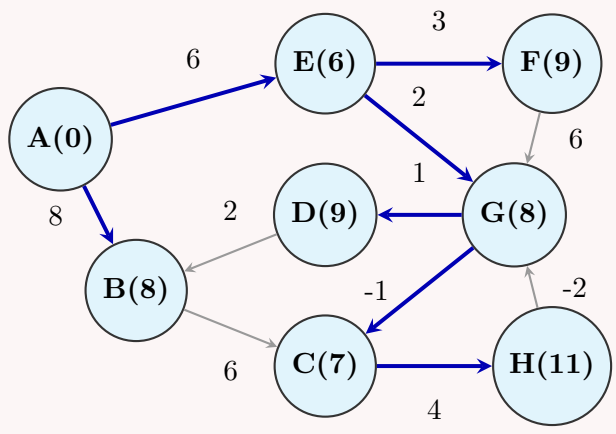
Initialization (Pass 0): Set distance to source $d[A] = 0$, and all other distances to ∞.



Pass 1: Edge Relaxations (Condition: if $d[u] + w(u, v) < d[v]$, update $d[v]$)

Edge $(u \to v)$	Wt (w)	Check $d[u] + w < d[v]$	Action	New Distance
$A \to E$	6	$0 + 6 < \infty$	**Update**	$d[E] = 6$
$A \to B$	8	$0 + 8 < \infty$	**Update**	$d[B] = 8$
$E \to F$	3	$6 + 3 < \infty$	**Update**	$d[F] = 9$
$E \to G$	2	$6 + 2 < \infty$	**Update**	$d[G] = 8$
$F \to G$	6	$9 + 6 \not< 8$	No change	-
$G \to D$	1	$8 + 1 < \infty$	**Update**	$d[D] = 9$
$H \to G$	-2	$\infty - 2 \not< 8$	No change	-
$G \to C$	-1	$8 - 1 < \infty$	**Update**	$d[C] = 7$
$D \to B$	2	$9 + 2 \not< 8$	No change	-
$B \to C$	6	$8 + 6 \not< 7$	No change	-
$C \to H$	4	$7 + 4 < \infty$	**Update**	$d[H] = 11$

State after Pass 1: *(Highlighted blue edges represent the shortest path tree)*



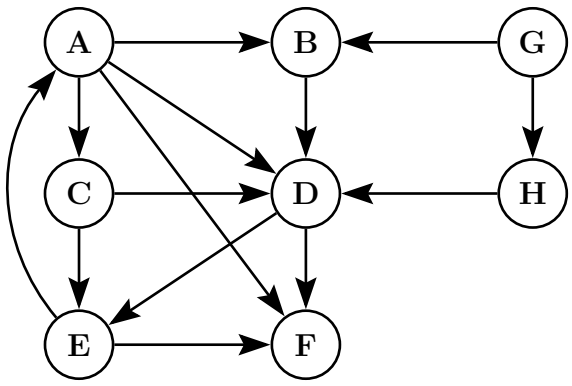
Pass 2: Verification
Iterating through all edges again, no condition $d[u] + w < d[v]$ is met. For example:

- $H \to G$: $d[H] - 2 = 11 - 2 = 9 \not< 8$ (no change).
- $D \to B$: $d[D] + 2 = 9 + 2 = 11 \not< 8$ (no change).

Since no updates occur in Pass 2, distances are optimal and the algorithm terminates early. This also confirms there are **no negative weight cycles** reachable from A.

Final shortest distances from A:
$d[B] = 8$, $d[C] = 7$, $d[D] = 9$, $d[E] = 6$, $d[F] = 9$, $d[G] = 8$, $d[H] = 11$.

Q20. Find the different types of edges if DFS is applied from vertex A. Discuss the edges' properties with respect to the graph. **(6 marks)**
[CSE 207 | 2014–15]

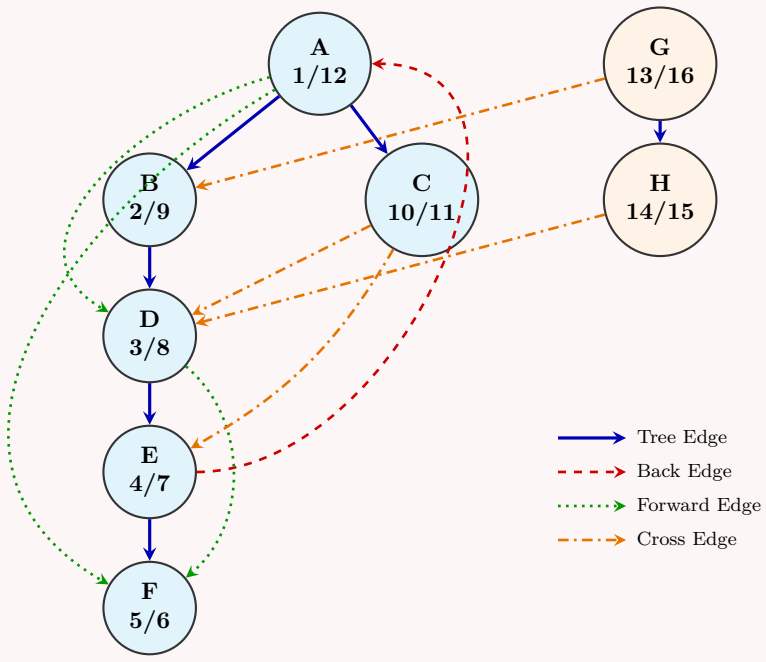


Answer — DFS Edge Classification & Forest

In DFS on a directed graph, every edge (u, v) falls into one of four categories based on discovery/finish times ($d[u]/f[u]$):

- **Tree edge:** v is first discovered via (u, v).
- **Back edge:** v is an ancestor of u ($d[v] < d[u] < f[u] < f[v]$). Indicates a cycle.
- **Forward edge:** v is a descendant of u, but not a tree edge ($d[u] < d[v] < f[v] < f[u]$).
- **Cross edge:** v is neither ancestor nor descendant ($d[v] < f[v] < d[u] < f[u]$).

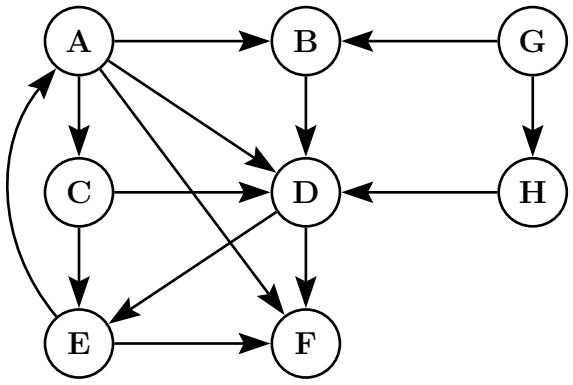
1. Depth First Forest (with Timestamps d/f & Edge Types):
Assuming standard alphabetical priority, the DFS traverses $A \to B \to D \to E \to F$, then explores remaining neighbors. The graph decomposes into two DFS trees (rooted at A and G).



2. Corrected Edge Classification Table:

Edge	Type	Reason based on timestamps (d/f)
$A \to B$	**Tree**	B is first discovered from A.
$A \to C$	**Tree**	C is first discovered from A.
$A \to D$	**Forward**	D is completely processed ($f[D] = 8 < f[A] = 12$), but is a descendant of A.
$A \to F$	**Forward**	F is completely processed ($f[F] = 6 < f[A] = 12$), but is a descendant of A.
$B \to D$	**Tree**	D is first discovered from B.
$C \to D$	**Cross**	D is processed ($f[D] = 8$) before C is even discovered ($d[C] = 10$).
$C \to E$	**Cross**	E is processed ($f[E] = 7$) before C is discovered ($d[C] = 10$).
$D \to E$	**Tree**	E is first discovered from D.
$D \to F$	**Forward**	F is a descendant of D (visited via E), discovered after D.
$E \to A$	**Back**	A is an ancestor of E and is still active ($d[A] = 1 < d[E] = 4$). **(Forms Cycle)**
$E \to F$	**Tree**	F is first discovered from E.
$G \to B$	**Cross**	B is entirely in a different tree, finished before G started.
$G \to H$	**Tree**	H is first discovered from G.
$H \to D$	**Cross**	D finished long before H was discovered.

Q21. Show the topological sorting order of a given graph with explanation of discovery and finishing times of node exploration. **(6 marks)**
[CSE 207 | 2014–15]



Answer — Topological Sorting

Crucial Observation: Topological sorting is strictly defined only for **Directed Acyclic Graphs (DAGs)**. However, this given graph contains a **cycle** (e.g., $A \to D \to E \to A$ via the back edge $E \to A$). Therefore, a strictly valid topological ordering is mathematically impossible.
If we forcefully apply the standard topological sorting algorithm (performing DFS and sorting nodes by decreasing finish times), we get the following result, which will inevitably violate the topological property for the back edge(s).

Step 1: DFS Discovery (d) and Finish (f) Times
(Assuming standard alphabetical order for unvisited vertices and neighbors)

Vertex	Discovery (d)	Finish (f)	Vertex	Discovery (d)	Finish (f)
A	1	12	E	4	7
B	2	9	F	5	6
C	10	11	G	13	16
D	3	8	H	14	15

Step 2: Sorting by Decreasing Finish Time
Arranging the vertices in descending order of their finish times ($f[u]$):

G (16) → H (15) → A (12) → C (11) → B (9) → D (8) → E (7) → F (6)

Generated Order: G → H → A → C → B → D → E → F

Verification of Failure: In a valid topological sort, for every directed edge $u \to v$, vertex u must come before v. While this order satisfies most edges (e.g., $G \to H$, $A \to C$, $D \to E$), it **fails** for the edge $E \to A$, because E appears after A in our sequence. This confirms the sequence is pseudo-topological due to the cycle.

Q22. State and prove the convergence property of shortest paths. **(12 marks)** [CSE 207 | 2012–13]

Answer

Convergence Property. Let $G = (V, E, w)$, source s, and let $s \xrightarrow{*} u \to v$ be a shortest path. If $\delta(s, u)$ is the true shortest-path distance from s to u, and $d[u] = \delta(s, u)$ at some point before edge (u, v) is relaxed, then after relaxing (u, v):

$$d[v] = \delta(s, v).$$

and $d[v]$ will not change thereafter.
Proof.
Since $s \xrightarrow{*} u \to v$ is a shortest path, $\delta(s, v) = \delta(s, u) + w(u, v)$.
By the upper-bound property, $d[v] \geq \delta(s, v)$ always holds.
After relaxing (u, v):

$$d[v] \leq d[u] + w(u, v) = \delta(s, u) + w(u, v) = \delta(s, v).$$

Combined with $d[v] \geq \delta(s, v)$, we get $d[v] = \delta(s, v)$.
Future relaxations can only set $d[v] \leftarrow \min(d[v], \ldots) \geq \delta(s, v)$, so $d[v]$ cannot decrease below $\delta(s, v)$. $\square$

Q23. Write the Bellman-Ford algorithm. Analyze the time complexity and correctness of the Bellman-Ford algorithm. **(20 marks)** [CSE 207 | 2012–13]

Answer

Algorithm 84 BELLMAN-FORD(G, w, s)

```
d[s] ← 0; d[v] ← ∞ for all v ≠ s
π[v] ← NIL for all v
for i ← 1 to |V| − 1 do
    for each edge (u, v) ∈ E do
        if d[u] + w(u, v) < d[v] then
            d[v] ← d[u] + w(u, v); π[v] ← u
        end if
    end for
end for
for each edge (u, v) ∈ E do                    ▷ check for negative cycles
    if d[u] + w(u, v) < d[v] then
        return false
    end if
end for
return true
```

Complexity Analysis. $|V| - 1$ passes, each scanning all $|E|$ edges: $\Theta(VE)$.

Correctness sketch. *Loop invariant*: after i passes, $d[v] = \delta(s, v)$ for all v reachable via a shortest path with $\leq i$ edges. After $|V| - 1$ passes (longest possible simple path), all reachable $d[v] = \delta(s, v)$ if no negative cycle. If a negative cycle is reachable, some $d[v]$ can still be reduced in the final check → return FALSE. □

Q24. Does Dijkstra's algorithm terminate if it is applied on a graph G that contains a negative-weight cycle? Why or why not? Justify your answer. **(6 marks)** [CSE 207 | 2012–13]

Answer

Yes, Dijkstra terminates, but it may give **wrong answers**.
Dijkstra's algorithm extracts each vertex from the priority queue *exactly once* (once extracted, a vertex is finalized and never re-inserted in the standard version). Since there are only $|V|$ vertices, the algorithm always terminates in $O((V + E) \log V)$ steps regardless of edge weights.
However, negative-weight edges (or cycles) violate the *greedy choice property*: once a vertex is finalized with estimate $d[u]$, the algorithm assumes no shorter path exists — but a negative edge could provide one. Therefore, the distances computed may be **incorrect**.

Remark. A negative-weight *cycle* makes shortest paths undefined ($-\infty$); both Dijkstra and Bellman-Ford cannot give meaningful answers in that case. Bellman-Ford at least *detects* such cycles; Dijkstra does not.

Q25. State the parenthesis theorem for depth-first search. **(6 marks)** [CSE 207 | 2012–13]

Answer

Parenthesis Theorem. In a DFS of graph $G = (V, E)$, for any two vertices u and v, exactly one of the following holds:

(a) $[d[u], f[u]]$ and $[d[v], f[v]]$ are *disjoint* (neither is an ancestor of the other in the DFS forest).

(b) $[d[u], f[u]] \subset [d[v], f[v]]$ (u is a descendant of v).

(c) $[d[v], f[v]] \subset [d[u], f[u]]$ (v is a descendant of u).

Proof. WLOG assume $d[u] < d[v]$ (discovered first).
Case A: $d[v] > f[u]$. v discovered after u finished → intervals disjoint. ✓
Case B: $d[v] < f[u]$ (v discovered while u is gray). Then v is a descendant of u. v must finish before u (DFS finishes a subtree before backtracking), so $f[v] < f[u]$, giving $[d[v], f[v]] \subset [d[u], f[u]]$. ✓
Overlapping without nesting is impossible: if $d[u] < d[v] < f[u] < f[v]$, then v is discovered while u is gray (so v is u's descendant), but then v must finish before u — contradiction. □

Q26. Assume that a breadth-first tree has already been computed with BFS on a graph G with source vertex s. Write an algorithm to print the vertices on a shortest path from s to v. **(8 marks)** [CSE 207 | 2012–13]

Answer

```cpp
// Assumes BFS has been run; parent[] stores BFS predecessor
void printPath(int parent[], int s, int v) {
    if (v == s) {
        cout << s;
        return;
    }
    if (parent[v] == -1) {
```

```
        cout << "No path from " << s << " to " << v;
        return;
    }
    printPath(parent, s, parent[v]);
    cout << " -> " << v;
}
```

The recursion follows predecessor pointers until the source is reached, then prints vertices on the way back. Time: $O(V)$ (path length $\leq V-1$).

Q27. Prove that the component graph of a directed graph is a DAG. **(6 marks)** [CSE 207 | 2012–13]

Answer

Claim: G^{SCC} is a DAG.
Proof (by contradiction). Suppose G^{SCC} contains a cycle $C_{i_1} \to C_{i_2} \to \cdots \to C_{i_k} \to C_{i_1}$. Then there exist directed paths in G: from every vertex in C_{i_1} to every vertex in C_{i_2}, …, and back to C_{i_1}. This means all vertices in $C_{i_1} \cup \cdots \cup C_{i_k}$ are mutually reachable — so they all belong to the *same* SCC, contradicting the assumption that $C_{i_1}, \ldots, C_{i_k}$ are *distinct* SCCs. □

Q28. Write a linear-time algorithm to compute the strongly connected components of a directed graph $G = (V, E)$ using two depth-first searches. Give an example to show every step of the algorithm. **(8+5 marks)** [CSE 207 | 2012–13]

Answer

Algorithm 85 STRONGLY-CONNECTED-COMPONENTS(G)

1: Run DFS on G; record finish times $f[v]$ for each v
2: Compute G^T (transpose of G)
3: Run DFS on G^T, processing vertices in *decreasing* $f[v]$ order
4: Each DFS tree in step 3 is one SCC

Complexity Analysis. $\Theta(V+E)$.

Example (graph $q, r, s, t, u, v, w, x, y, z$ — alphabetical start):
Pass 1 — DFS on G:

Vertex	d	f
q	1	16
s	2	7
v	3	6
w	4	5
t	8	15
x	9	12
z	10	11
y	13	14
r	17	20
u	18	19

Finish stack (last finished = top): $w, v, s, z, x, y, t, q, u, r$.
Pass 2 — DFS on G^T (reverse finish order):

SCC	Vertices
1	$\{r\}$
2	$\{u\}$
3	$\{q\}$
4	$\{t\}$
5	$\{y\}$
6	$\{x, z\}$
7	$\{s, v, w\}$

Q29. What does $d[v]$ represent at the end of the execution of Algorithm 1, if the algorithm is applied on an unweighted directed acyclic graph (DAG) G with a source s? What is the runtime complexity in terms of $|V|$ and $|E|$ of Algorithm 1? **(6+6 marks)** [CSE 207 | 2012–13]

Algorithm 86 LongestPath-DAG(G)

```
1: Compute a topological sorting of G
2: for each v ∈ V[G] do
3:     d[v] ← 0
4: end for
5: for each u in topologically sorted order do
6:     for each v ∈ Adj[u] do
7:         if d[v] < d[u] + 1 then
8:             d[v] ← d[u] + 1
9:         end if
10:    end for
11: end for
```

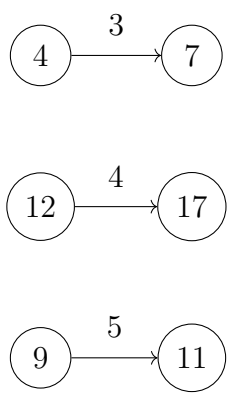
Answer

What does $d[v]$ represent?
The algorithm computes the **length of the longest path** from source s to v in the unweighted DAG (counting edges). Equivalently, $d[v]$ = the maximum number of edges on any path from s to v.
Runtime:

- Topological sort: $O(V + E)$.
- Initialisation: $O(V)$.
- Main loop: each vertex processed once, each edge relaxed once $\rightarrow O(V + E)$.

Total: $O(V + E)$.

Q30. In Figure 1, the shortest path estimate of each vertex appears within the vertex and the distance to traverse from a vertex to another appears as the weight of the corresponding directed edge. Show the updated shortest path estimates of vertices after relaxing the following edges. **(6 marks)**



[CSE 207 | 2012–13]

Answer

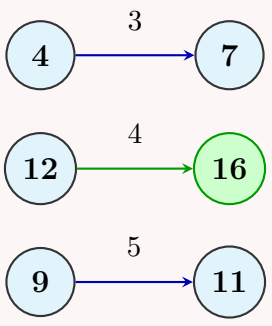
Current distance estimates (shown inside each node):
$d[4] = 4,\ d[7] = 7,\ d[12] = 12,\ d[17] = 17,\ d[9] = 9,\ d[11] = 11.$

Relax $(4 \rightarrow 7,\ w = 3)$:
$d[4] + 3 = 4 + 3 = 7 \not< d[7] \rightarrow$ **no update**.

Relax $(12 \rightarrow 17,\ w = 4)$:
$d[12] + 4 = 12 + 4 = 16 < d[17] \rightarrow$ **update** $d[17] \leftarrow 16$.

Relax $(9 \rightarrow 11,\ w = 5)$:
$d[9] + 5 = 9 + 5 = 14 \not< d[11] \rightarrow$ **no update**.

Updated Graph after Relaxation:
(Only the estimate for the middle-right vertex changes from 17 to 16)

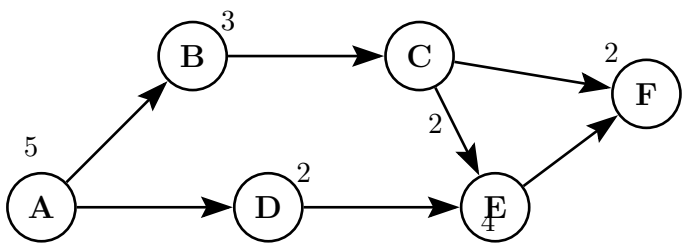


Q31. Prove the correctness of the BFS algorithm. What are "discovery time" and "finish time" in DFS? How can you find the shortest path between two nodes from a BFS tree? **(8+4+6 marks)** [CSE 207 | 2009–10]

Answer

BFS Correctness.
Claim: when BFS terminates, $d[v] = \delta(s, v)$ for all v.
Proof sketch (by induction on $\delta(s, v)$).
Base: $d[s] = 0 = \delta(s, s)$. ✓
Step: assume $d[u] = \delta(s, u)$ for all u with $\delta(s, u) < k$. Let v have $\delta(s, v) = k$ and let u be its predecessor on a shortest path. When u is dequeued, v is either already gray (so $d[v] \leq d[u] + 1 = k$) or white (so $d[v]$ is set to $d[u] + 1 = k$). By the upper-bound property, $d[v] \geq k$. Hence $d[v] = k = \delta(s, v)$. □
Discovery time ($d[v]$) in DFS: the time-stamp when v is first visited (colored gray).
Finish time ($f[v]$) in DFS: the time-stamp when v's adjacency list is fully explored (colored black).
Shortest path from BFS tree: trace predecessor pointers $\pi[v] \to \pi[\pi[v]] \to \cdots \to s$ recursively and print in reverse. (See `printPath` code in [CSE 207 | 2012–13] answer.)

Q32. A project is described by tasks A, B, C, D, E, F taking $5, 3, 2, 2, 4, 2$ days respectively, with precedence constraints (e.g., E cannot start until C and D are completed, but D can be done in parallel with B and C). Write a pseudocode that computes the total time required to complete the project using the PERT technique. **(8 marks)** **[CSE 207 | 2009–10]**



Answer — PERT Technique Simulation

Tasks: $A(5), B(3), C(2), D(2), E(4), F(2)$.
Precedences: $A \to B$, $A \to D$, $B \to C$, $C \to E$, $C \to F$, $D \to E$, $E \to F$.

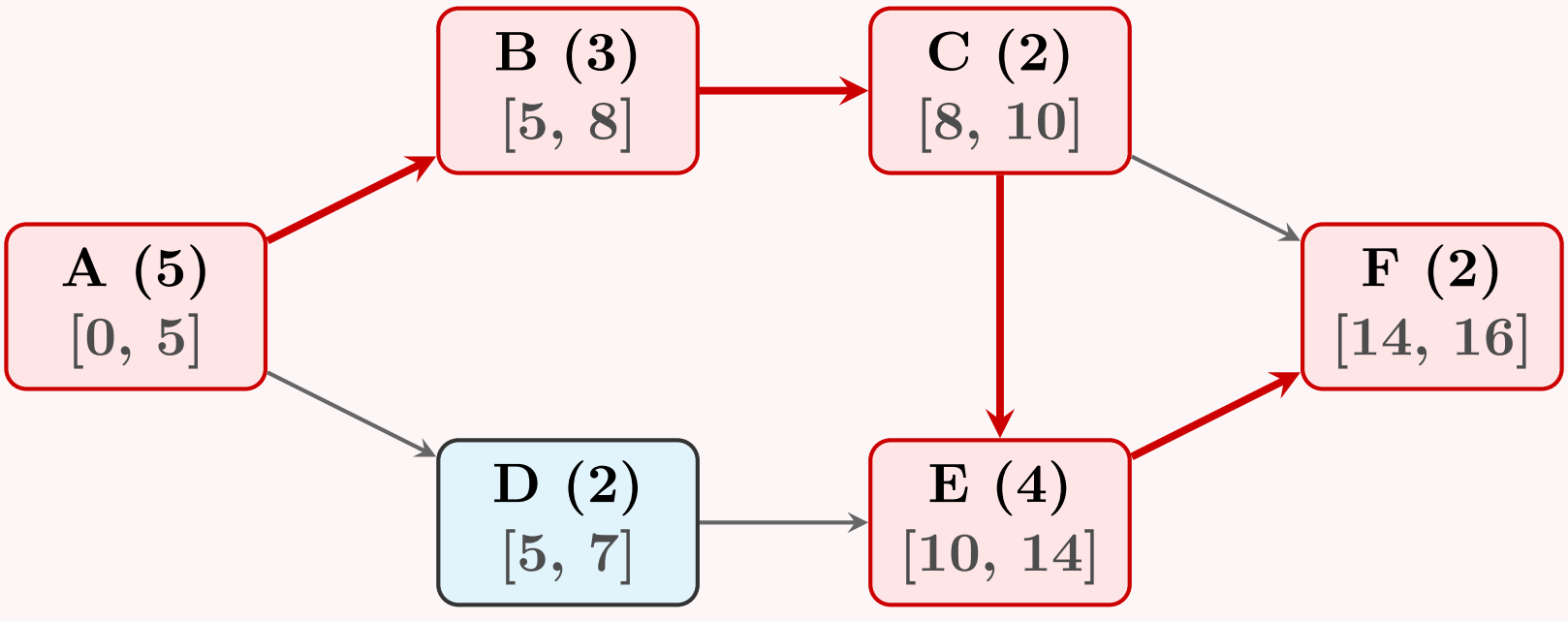
Algorithm 87 PERT(G, dur)

```
1: Topologically sort G
2: earliest[v] ← 0 for all v
3: for each u in topological order do
4:     for each successor v of u do
5:         earliest[v] ← max(earliest[v], earliest[u] + dur[u])
6:     end for
7: end for
8: return max_v (earliest[v] + dur[v])
```

Simulation (Topological order: A, B, D, C, E, F):

Task (v)	Earliest start (ES)	Earliest finish ($EF = ES + \text{dur}[v]$)
A	0	5
B	5	8
D	5	7
C	8	10
E	10	14
F	14	16

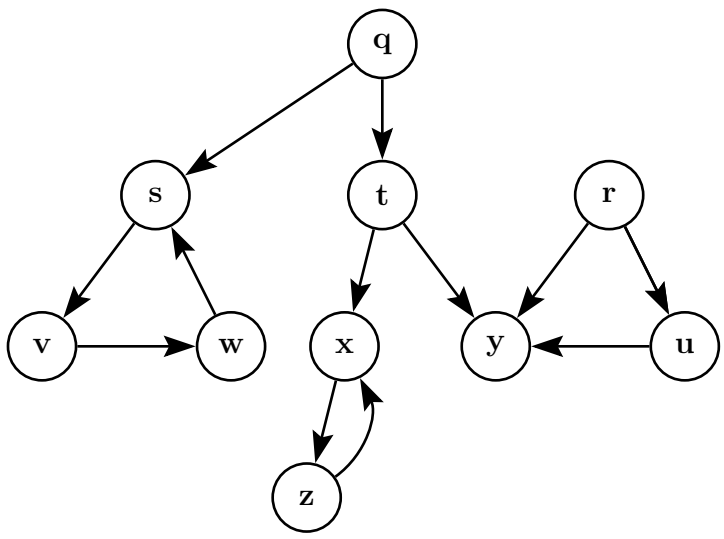
PERT Chart with Earliest Start/Finish Times:
(Format inside nodes: **Task (Duration)**
[ES, EF]. The **red arrows** represent the Critical Path.)



Minimum project completion time = 16 days. ✓
Critical path: $A \to B \to C \to E \to F$.

Complexity Analysis. $O(V + E)$, as it fundamentally relies on Topological Sorting and relaxation.

Q33. Using DFS, compute the strongly connected components and the component graph G^{SCC}. Show the discovery/finish times and the DFS forests created during computation. Start alphabetically. **(17 marks)** [CSE 207 | 2009–10]



Answer — Strongly Connected Components & Component Graph

Kosaraju's Algorithm for SCCs:
Step 1 — DFS on G to compute finish times. Starting DFS alphabetically $(q, r, s, t, u, v, w, x, y, z)$:

Vertex	Discovery (d)	Finish (f)	Vertex	Discovery (d)	Finish (f)
q	1	16	x	9	12
s	2	7	z	10	11
v	3	6	y	13	14
w	4	5	r	17	20
t	8	15	u	18	19

Finish order (decreasing): $r(20)$, $u(19)$, $q(16)$, $t(15)$, $y(14)$, $x(12)$, $z(11)$, $s(7)$, $v(6)$, $w(5)$.

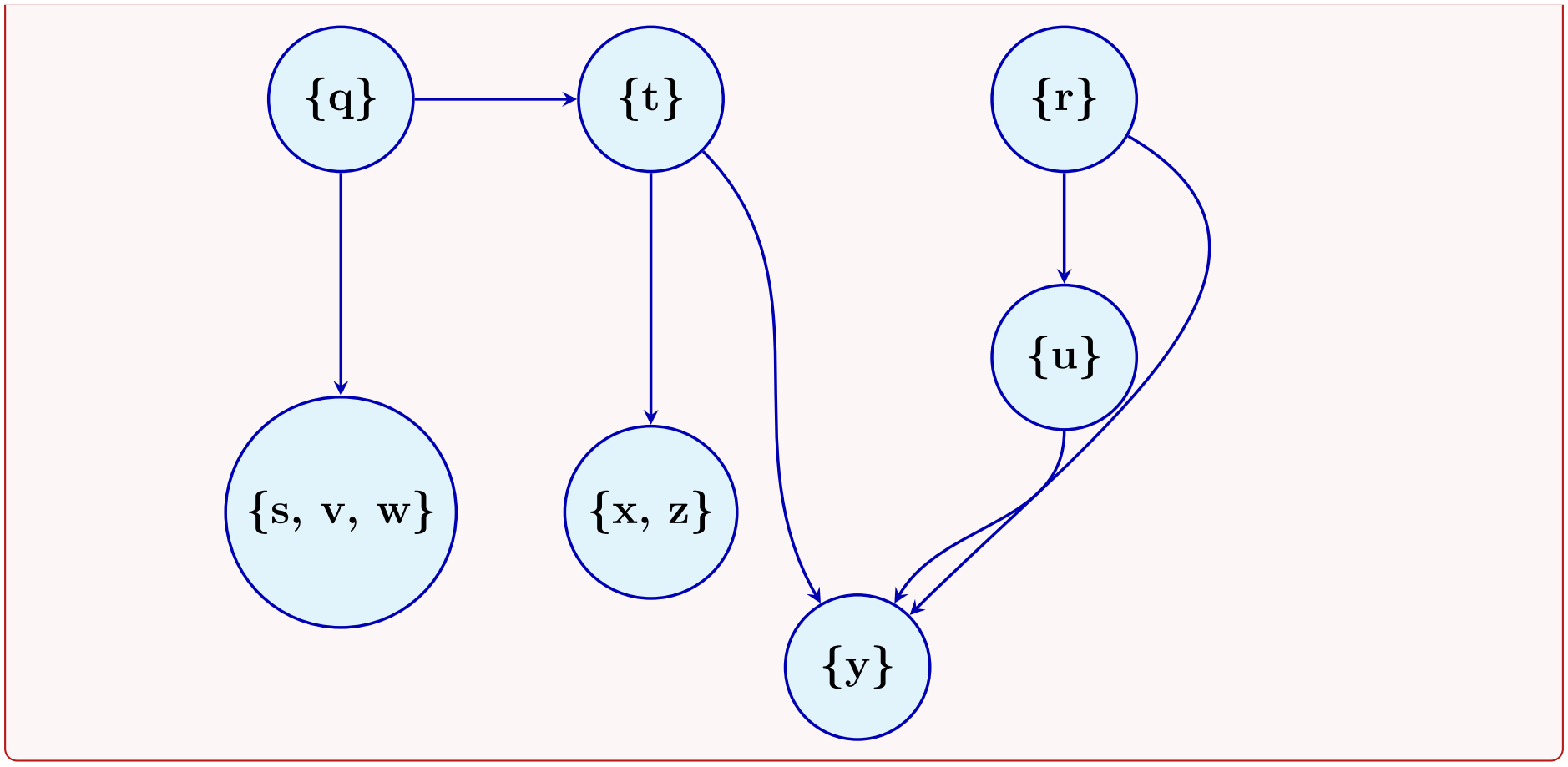
Step 2 — DFS on G^T (Transpose Graph) in decreasing finish order: G^T reverses all edges. We pop vertices from the sorted list and run DFS. Each new tree forms an SCC.

- Start **r**: visits only r. **SCC: {r}**
- Start **u**: visits only u. **SCC: {u}**
- Start **q**: visits only q. **SCC: {q}**
- Start **t**: visits only t. **SCC: {t}**
- Start **y**: visits only y. **SCC: {y}**
- Start **x**: visits z. **SCC: {x, z}**
- Start **s**: visits w, v. **SCC: {s, v, w}**

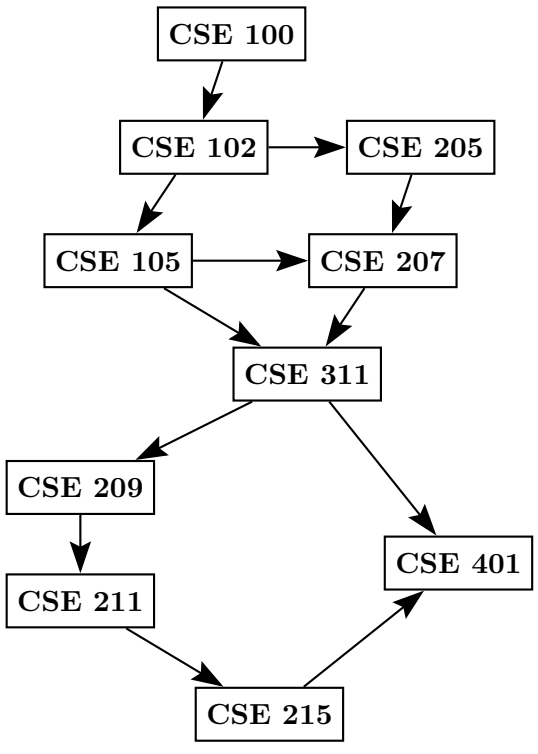
Strongly Connected Components (7 components):

$$C_1 = \{r\},\ C_2 = \{u\},\ C_3 = \{q\},\ C_4 = \{t\},\ C_5 = \{y\},\ C_6 = \{x, z\},\ C_7 = \{s, v, w\}$$

Step 3 — Component Graph G^{SCC}:
The component graph collapses each SCC into a single node. Edges exist between components if there was an original edge connecting their vertices. Edges: $C_3 \to C_7$ (via $q \to s$), $C_3 \to C_4$ (via $q \to t$), $C_4 \to C_6$ (via $t \to x$), $C_4 \to C_5$ (via $t \to y$), $C_1 \to C_2$ (via $r \to u$), $C_1 \to C_5$ (via $r \to y$), $C_2 \to C_5$ (via $u \to y$).



Q34. Find the topological order of courses for this given prerequisite graph. **(10 marks)** [CSE 207 | 2006–07]



Answer

Algorithm: Run DFS; upon finishing each vertex prepend it to the output list (or equivalently, push onto a stack and pop at the end).

In-degree method (Kahn's):

(a) Compute in-degree of every vertex.

(b) Enqueue all vertices with in-degree 0.

(c) While queue non-empty: dequeue u, append to order, decrement in-degree of each neighbor; enqueue neighbor if in-degree becomes 0.

In-degrees:

Course	In-degree
CSE 100	0
CSE 102	1
CSE 205	1
CSE 105	1
CSE 207	2
CSE 311	2
CSE 209	1
CSE 211	1
CSE 215	1
CSE 401	2

Execution trace:

Step	Dequeued	Queue after
1	CSE 100	[CSE 102]
2	CSE 102	[CSE 205, CSE 105]
3	CSE 205	[CSE 105] *(CSE 207 in-deg → 1)*
4	CSE 105	[CSE 207] *(CSE 207 in-deg → 0, CSE 311 in-deg → 1)*
5	CSE 207	[CSE 311] *(CSE 311 in-deg → 0)*
6	CSE 311	[CSE 209] *(CSE 209 in-deg → 0, CSE 401 in-deg → 1)*
7	CSE 209	[CSE 211] *(CSE 211 in-deg → 0)*
8	CSE 211	[CSE 215] *(CSE 215 in-deg → 0)*
9	CSE 215	[CSE 401] *(CSE 401 in-deg → 0)*
10	CSE 401	[]

Topological Order:

$$\text{CSE 100} \to \text{CSE 102} \to \text{CSE 205} \to \text{CSE 105} \to \text{CSE 207} \to \text{CSE 311} \to \text{CSE 209}$$

$$\to \text{CSE 211} \to \text{CSE 215} \to \text{CSE 401}$$

Time complexity: $O(V + E)$.

BUET · CSE 105

DSA

Divide & Conquer

Mergesort, Closest Pair, Inversions, Strassen

T9 — Divide & Conquer

Q1. Using the recurrence tree method, analyze the runtime complexity of the Mergesort algorithm to sort an array of size n. **(10 marks)** [CSE 105 | 2023–24]

Answer

Recurrence: $T(n) = 2\,T(n/2) + cn, \quad T(1) = \Theta(1).$

Recurrence Tree:
At each level k, there are 2^k nodes each doing $cn/2^k$ work, so the cost per level is:

$$2^k \cdot \frac{cn}{2^k} = cn$$

The tree has $\log_2 n + 1$ levels (level 0 to level $\log_2 n$), and the leaves contribute $\Theta(n)$ total.

Level	Subproblems	Cost per level
0	$1 \times cn$	cn
1	$2 \times cn/2$	cn
2	$4 \times cn/4$	cn
$\vdots$	$\vdots$	$\vdots$
$\log_2 n$	$n \times c$	cn

Total cost:

$$T(n) = cn \cdot (\log_2 n + 1) = \Theta(n \log n)$$

Since the merge step always processes all n elements at every level regardless of input, this bound holds in all cases. Hence Mergesort runs in $\Theta(n \log n)$ in the worst case.

Q2. Explain the run-time complexity of the Mergesort algorithm. First, formulate the run-time as a recurrence relation. Then solve the recurrence relation using both: (i) the recurrence tree method, and (ii) the Master theorem (distinguish the case that is applicable in your analysis). **(2+5+3=10 marks)** [CSE 105 | 2022–23]

Q3. Given a sequence of n numbers $a_1, \ldots, a_n$ (all distinct), define an *inversion* as a pair $i < j$ such that $a_i > a_j$. Present an $O(n \log n)$ divide-and-conquer algorithm to count the number of inversions. Define a *modified-inversion* as a pair where $i < j$ and $a_i > 2a_j$. What modifications are needed in the divide-and-conquer algorithm to count the number of modified-inversions? **(20 marks)** [CSE 105 | 2021–22]

Answer

Inversion Count — $O(n \log n)$ Algorithm (Modified Mergesort):

Key idea: During the merge step, whenever an element from the right half is placed before elements remaining in the left half, each such left-half element forms an inversion with it.

Algorithm 88 MergeCount(A, lo, hi)

```
if lo ≥ hi then
    return 0
end if
mid ← ⌊(lo + hi)/2⌋
left_inv ← MergeCount(A, lo, mid)
right_inv ← MergeCount(A, mid + 1, hi)
split_inv ← MergeAndCount(A, lo, mid, hi)
return left_inv + right_inv + split_inv
```

Algorithm 89 MergeAndCount(A, lo, mid, hi)

```
L ← A[lo .. mid];   R ← A[mid + 1 .. hi]
i ← 0;   j ← 0;   inv ← 0
for k = lo to hi do
    if i > |L| − 1 then
        A[k] ← R[j ++]
    else if j > |R| − 1 then
        A[k] ← L[i ++]
    else if L[i] ≤ R[j] then
        A[k] ← L[i ++]
    else
        A[k] ← R[j ++]
        inv ← inv + (mid − lo + 1) − i
    end if
end for
return inv
```

Complexity: Same recurrence as Mergesort: $T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.

Modified-Inversion ($a_i > 2a_j$):
A modified-inversion is a pair (i, j) with $i < j$ and $a_i > 2a_j$.
Modification needed: In MergeAndCount, replace the split-inversion counting step with a separate pointer-based count *before* merging:

Algorithm 90 CountModified(L, R)

```
j ← 0;   inv ← 0
for each element L[i] in L do
    while j < |R| and L[i] > 2 · R[j] do
        j ← j + 1
    end while
    inv ← inv + j
end for
return inv
```

This uses a two-pointer scan (since both L and R are sorted) and runs in $O(n)$ per merge level. The overall complexity remains $\Theta(n \log n)$.
Note: The modified count must be done *before* the merge step (which would rearrange elements).

Q4. Given n points $(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)$ in the plane, give an efficient divide-and-conquer algorithm to find the pair of points that is closest together. Prove the correctness and analyze the running time of your algorithm. **(17 marks)** [CSE 105 | 2021–22, 2016–17]

Answer

Algorithm 91 Closest-Pair($P[1..n]$)

```
Sort P by x-coordinate
return Closest-Rec(P, 1, n)
```

Algorithm 92 Closest-Rec(P, l, r)

```
if r − l ≤ 2 then
    return Brute-Force(P, l, r)
end if
m ← ⌊(l + r)/2⌋;   x_m ← P[m].x
d_L ← Closest-Rec(P, l, m)
d_R ← Closest-Rec(P, m + 1, r)
δ ← min(d_L, d_R)
strip ← { P[i] : |P[i].x − x_m| < δ }
Sort strip by y-coordinate
for each point q in strip do
    for each point p′ in strip with 0 < p′.y − q.y < δ do
        δ ← min(δ, dist(q, p′))
    end for
end for
return δ
```

Why at most 7 strip comparisons per point. Place point q at the origin. Any point r with $d(q, r) < \delta$ must lie in the $2\delta \times \delta$ rectangle centred at q. Divide this rectangle into eight $(\delta/2) \times (\delta/2)$ boxes (four on each side of the dividing line). Each box holds at most 1 point with pairwise distance $\geq \delta$ inside its half. Hence ≤ 7 other points can be within distance δ of q in the strip.
Runtime:

- With y-sorted strip maintained via merge during recursion: $T(n) = 2T(n/2) + O(n) = O(n \log n)$.
- Naive re-sorting of strip per call: $T(n) = 2T(n/2) + O(n \log n) = O(n \log^2 n)$.

Complexity Analysis. $O(n \log n)$ with pre-sorted y-arrays merged during recursion.

Q5. Provide a divide-and-conquer algorithm that takes an n-digit number A as input and efficiently computes the square of A with a faster asymptotic running time than $O(n^2)$. Analyze the running time. **(12 marks)** [CSE 203 | 2021–22]

Answer

Split: $A = A_1 \cdot 10^{n/2} + A_0$ (high and low halves).

$$A^2 = A_1^2 \cdot 10^n + 2A_1A_0 \cdot 10^{n/2} + A_0^2.$$

Use the identity $2A_1A_0 = (A_1 + A_0)^2 - A_1^2 - A_0^2$ to avoid a separate multiplication. Only **3 recursive squarings** needed.

Algorithm 93 SQUARE(A, n)

```
if n = 1 then
    return A × A
end if
A_1 ← high ⌊n/2⌋ digits of A
A_0 ← low ⌊n/2⌋ digits of A
p ← SQUARE(A_1, n/2)                          ▷ A_1^2
q ← SQUARE(A_0, n/2)                          ▷ A_0^2
r ← SQUARE(A_1 + A_0, n/2 + 1)                ▷ (A_1 + A_0)^2
cross ← r − p − q                             ▷ = 2A_1A_0
return p · 10^n + cross · 10^{n/2} + q
```

Recurrence: $T(n) = 3T(n/2) + O(n)$.
Master Theorem: $a = 3$, $b = 2$, $f(n) = n$. $n^{\log_2 3} \approx n^{1.585} > n \Rightarrow$ Case 1:

$$T(n) = \Theta(n^{\log_2 3}) \approx \Theta(n^{1.585}).$$

This beats the $O(n^2)$ schoolbook algorithm.

Complexity Analysis. $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$ vs naive $O(n^2)$.

Q6. Suppose, in a 2-dimensional co-ordinate system, you are given with n points, say $P_1, P_2, \ldots, P_n$, where the x-y co-ordinates of any point P_i is represented as (x_i, y_i). You have been assigned to find the closest pair of points using divide and conquer method.

(i) Write down the approach that you will follow to split these n points into two partitions, say *LEFT* and *RIGHT*, in such a way that each partition contains roughly $\frac{n}{2}$ points.

Now suppose, the *LEFT* and *RIGHT* partitions, respectively calculate d_L and d_R as the distances between the closest pair of points within themselves. Now your task is to check whether there exist a pair of points P_{Li} and P_{Rj} such that $P_{Li} \in LEFT$ and $P_{Rj} \in RIGHT$, and the distance between P_{Li} and P_{Rj} is less than $\min(d_L,\ d_R)$.

(ii) Write down the steps to perform this checking so that the minimum number of comparisons is ensured and the checking can be performed in $O(n)$ time.

(5+10=15 marks) [CSE 203 | 2020–21]

Answer

(i) Splitting into LEFT and RIGHT.

(a) Pre-sort all n points by x-coordinate ($O(n \log n)$, done once).
(b) Dividing line: $x = x_{\lceil n/2 \rceil}$ (the x-value of the median point).
(c) **LEFT** = first $\lceil n/2 \rceil$ points by x; **RIGHT** = remaining $\lfloor n/2 \rfloor$ points.

(ii) Strip in $O(n)$.
Let $\delta = \min(d_L, d_R)$. A cross-pair (P_{Li}, P_{Rj}) with $d(P_{Li}, P_{Rj}) < \delta$ requires $|P_{Li}.x - P_{Rj}.x| < \delta$, so both points lie within δ of the dividing line.
Collect into strip, sorted by y. For each point q:

- Only points r with $|q.y - r.y| < \delta$ can be candidates.
- The $\delta \times 2\delta$ box around q holds at most 8 points with mutual distance $\geq \delta$ (see Q10).
- So each q requires at most **7 comparisons**.

Total strip work: $O(7n) = O(n)$. ✓

Q7. In a sequence of n distinct numbers $a_1, \ldots, a_n$, a *significant inversion* is a pair $i < j$ such that $a_i > 2a_j$. Give an $O(n \log n)$ algorithm to count the number of significant inversions. Write the corresponding pseudocode. **(10 marks)** [CSE 203 | 2020–21]

Answer

Idea. Modify Merge Sort. When an element from the *right* half is chosen over an element from the *left* half during merging, it forms an inversion with *all remaining elements* of the left half.

```cpp
long long mergeCount(vector<int>& A, int l, int r) {
    if (l >= r) return 0;
    int m = (l + r) / 2;
    long long cnt = mergeCount(A, l, m)       // left inversions
                  + mergeCount(A, m+1, r);    // right inversions

    // Merge and count split inversions
    vector<int> tmp;
    int i = l, j = m + 1;
    while (i <= m && j <= r) {
        if (A[i] <= A[j]) {
```

```
            tmp.push_back(A[i++]);
        } else {
            cnt += (m - i + 1); // all left remaining > A[j]
            tmp.push_back(A[j++]);
        }
    }
    while (i <= m) tmp.push_back(A[i++]);
    while (j <= r) tmp.push_back(A[j++]);
    for (int k = l; k <= r; k++) A[k] = tmp[k - l];
    return cnt;
}
```

Simulation on $[7, 9, 1, 5, 4]$**:**

Call	Subarray	cnt	Note
mergeCount([7,9])	[7],[9]	0	$7 < 9$, no inversion
mergeCount([1])	[1]	0	base case
merge [7,9]+[1] Left=[1,7,9], cnt=2		2	$1 < 7$: adds $m-i+1 = 2$
mergeCount([5,4]) Right=[4,5], cnt=1	[5],[4]	1	$5 > 4$: adds 1
merge [1,7,9]+[4,5]			
$1 \leq 4$: take 1			cnt+=0
$7 > 4$: take 4			cnt+=$m - i + 1 = 2$
$7 > 5$: take 5			cnt+=$m - i + 1 = 2$
take 7,9		4	
Total		**7**	$0 + 2 + 1 + 4 = 7$

Inversions: $(7,1), (7,5), (7,4), (9,1), (9,5), (9,4), (5,4) = 7$ ✓

Complexity Analysis. $T(n) = 2T(n/2) + O(n) = O(n \log n)$.

Q8. Write an algorithm to find a peak element of an $n \times n$ matrix in $O(n)$ time using divide and conquer. A peak element is one whose left, right, top, and bottom neighbors (if available) are all smaller. Define the recurrence relation and derive the runtime. **(10+5 marks)** [CSE 203 | 2019–20]

Answer

Definition: An element $M[r][c]$ is a *peak* if it is greater than all its existing neighbors (left, right, top, bottom).

Algorithm: Divide and Conquer

1. If colLo = colHi, find the maximum element in column colLo and return it.
2. Let colMid = $\lfloor (\text{colLo} + \text{colHi})/2 \rfloor$.
3. Find the global maximum $M[r^*][\text{colMid}]$ in column colMid (scan all n rows). $O(n)$
4. **If** $M[r^*][\text{colMid}] \geq M[r^*][\text{colMid} - 1]$ and $M[r^*][\text{colMid}] \geq M[r^*][\text{colMid} + 1]$:
   $\Rightarrow$ **Return** $M[r^*][\text{colMid}]$ (it is a peak).
5. **Else if** $M[r^*][\text{colMid} - 1] > M[r^*][\text{colMid}]$:
   $\Rightarrow$ Recurse on left half: FindPeak$(M, \text{colLo}, \text{colMid} - 1, n)$.
6. **Else**:
   $\Rightarrow$ Recurse on right half: FindPeak$(M, \text{colMid} + 1, \text{colHi}, n)$.

Correctness Argument:
If $M[r^*][\text{colMid}]$ is the column maximum but is not a peak, a larger neighbor must exist in an adjacent column. Recursing toward that side guarantees a peak exists there — the column maximum of the new mid-column will eventually dominate both neighbors.

Recurrence Relation:

Each recursive call halves the number of columns; finding the column maximum costs $O(n)$.

$$T(n) = T\left(\frac{n}{2}\right) + O(n)$$

Solving by Master Theorem (Case 3: $f(n) = O(n)$, $a = 1$, $b = 2$, $n^{\log_b a} = n^0 = 1$, $f(n) = \Omega(n^{0+\varepsilon})$):

$$T(n) = O(n)$$

Time Complexity: $O(n)$ for an $n \times n$ matrix.

Q9. Propose the pseudocode of a merge sort algorithm where the array is divided into three equal parts at each step. Derive the asymptotic runtime of this algorithm. Compare it with standard merge sort (two equal parts) and mention cases when one outperforms the other. **(10+8+7 marks)** [CSE 203 | 2019–20]

Answer

Algorithm 94 3Way-Merge-Sort(A, l, r)

```
if l ≥ r then
    return
end if
t1 ← l + ⌊(r − l)/3⌋
t2 ← l + ⌊2(r − l)/3⌋
3Way-Merge-Sort(A, l, t1)
3Way-Merge-Sort(A, t1 + 1, t2)
3Way-Merge-Sort(A, t2 + 1, r)
3Way-Merge(A, l, t1, t2, r)        ▷ O(n) using 3 pointers
```

Recurrence: $T(n) = 3T(n/3) + O(n)$.
Master Theorem: $a = 3$, $b = 3$, $f(n) = n$. $n^{\log_3 3} = n^1 = f(n) \Rightarrow$ Case 2: $T(n) = \Theta(n \log_3 n)$.

Property	2-way	3-way	Note
Recurrence	$2T(n/2) + cn$	$3T(n/3) + cn$	
Asymptotic	$\Theta(n \log_2 n)$	$\Theta(n \log_3 n)$	Same O-class
Tree height	$\log_2 n$	$\log_3 n$	3-way shallower
Merge compares/step	1	2	3-way heavier

Conclusion. Both are $\Theta(n \log n)$. 2-way is faster in practice (fewer comparisons per merge step). 3-way benefits *external sorting* (fewer I/O passes due to shallower tree).

Q10. Explain how a recursion tree can help with the substitution method of runtime analysis. **(5 marks)** [CSE 203 | 2019–20]

Answer

The *substitution method* requires guessing the solution's form and verifying by induction. The *recursion tree* provides an educated guess by:

(a) **Visualising cost per level.** Each node represents the non-recursive work at one subproblem. Summing across a level reveals whether costs are constant, increasing, or decreasing.
(b) **Identifying the dominant term.**
- Constant cost per level (cn each, $\log n$ levels) $\Rightarrow$ guess $O(n \log n)$.
- Decreasing geometric series (root-dominated) $\Rightarrow$ guess $O(f(n))$ where $f(n)$ is the root cost.
- Increasing geometric series (leaf-dominated) $\Rightarrow$ guess $O(n^{\log_b a})$.

(c) **Providing a tight guess for substitution.** Once the tree gives, say, $T(n) \approx cn \log n$, the substitution method proves $T(k) \leq ck \log k$ for all $k < n$ by induction.

Example. For $T(n) = T(n/4) + T(n/2) + n^2$: the tree shows a *decreasing* geometric series (ratio 5/16), suggesting $O(n^2)$. Substitution then confirms $T(n) \leq cn^2$ rigorously.

Q11. Write a divide-and-conquer algorithm that finds the maximum and the minimum of an array of n distinct elements. Write the recurrence relation for the running time $T(n)$ and solve it using the Recursion Tree Method. **(10 marks)** [CSE 203 | 2018–19]

Answer

Naive: $2(n-1)$ comparisons. **D&C:** $\lfloor 3n/2 \rfloor - 2$ comparisons.

```
// Returns {minimum, maximum} of A[l..r]
pair<int,int> minMax(int A[], int l, int r) {
    if (l == r)   return {A[l], A[l]};   // 0 comparisons
    if (r-l == 1) {                       // 1 comparison
        if (A[l] < A[r]) return {A[l], A[r]};
        else             return {A[r], A[l]};
    }
    int m = (l + r) / 2;
    auto [mn1, mx1] = minMax(A, l,   m);
    auto [mn2, mx2] = minMax(A, m+1, r);
    return {min(mn1, mn2), max(mx1, mx2)}; // 2 comparisons
}
```

Recurrence for comparisons $C(n)$:

$$C(n) = 2\,C(n/2) + 2, \quad C(2) = 1.$$

Solving: $C(n) = \dfrac{3n}{2} - 2$ for n a power of 2.

Compared to naive $2n-2$: **saving** $\approx 25\%$ of comparisons.

Complexity Analysis. Time: $O(n)$. Comparisons: $\frac{3n}{2} - 2$.

Q12. Write the Mergesort algorithm. Simulate it on the array $[6, 1, 7, 2, 8, 3, 9, 4, 10, 5, 11, 17, 4, 5]$. **(15 marks)** [CSE 203 | 2017–18]

Answer

Algorithm: see Q2.
Simulation (key merge steps):

Left half	Right half	Merged
$[6][1] \to [1,6]$ $[8][3] \to [3,8]$ $[1,2,6,7] + [3,4,8,9]$	$[7][2] \to [2,7]$ $[9][4] \to [4,9]$	$[1,2,6,7]$ $[3,4,8,9]$ $[1,2,3,4,6,7,8,9]$
$[10][5] \to [5,10]$ $[4][5] \to [4,5]$	$[11][17] \to [11,17]$ $[5,10,11,17]$	$[5,10,11,17]$ $[4,5,5,10,11,17]$
$[1,2,3,4,6,7,8,9]$	$[4,5,5,10,11,17]$	**[1,2,3,4,4,5,5,6,7,8,9,10,11,17]**

Verified output: `1 2 3 4 4 5 5 6 7 8 9 10 11 17` ✓

Q13. Given a sorted array of distinct integers $A[1 \ldots n]$, give a divide-and-conquer algorithm that runs in $O(\log n)$ time to find whether there is an index i for which $A[i] = i$. Write the pseudocode and prove the runtime. **(12 marks)** [CSE 207 | 2014–15]

Answer

Key observation. All elements are distinct integers.

- If $A[m] > m$: since elements are strictly increasing, $A[k] \geq A[m] + (k-m) > k$ for all $k > m$. So search *left*.
- If $A[m] < m$: symmetrically search *right*.

```
// 0-indexed: find i such that A[i] == i, or -1 if none
int fixedPoint(int A[], int lo, int hi) {
    if (lo > hi) return -1;
    int mid = (lo + hi) / 2;
    if (A[mid] == mid) return mid;
    if (A[mid] >  mid) return fixedPoint(A, lo,    mid-1);
    else               return fixedPoint(A, mid+1,  hi);
}
// Call: fixedPoint(A, 0, n-1)
```

Example. $A = [-3, 0, 2, 5, 7, 8]$: $A[2] = 2$ is the fixed point.
mid=2, $A[2] = 2 \Rightarrow$ return 2. ✓
Runtime: $T(n) = T(n/2) + O(1) = O(\log n)$ (identical to binary search).

Q14. Explain the divide-and-conquer technique. What is the function of the pivot element in the Quicksort algorithm? **(3+2=5 marks)** [CSE 203 | 2012–13]

Answer

Divide-and-Conquer is an algorithm design paradigm with three recursive steps:

(a) **Divide:** Break the problem into smaller subproblems of the same type.
(b) **Conquer:** Solve each subproblem recursively. If the subproblem is small enough, solve it directly (base case).
(c) **Combine:** Merge the subproblem solutions to form the answer to the original problem.

Classical examples: Merge Sort, Quicksort, Binary Search, Closest Pair of Points, Strassen's matrix multiplication.

Role of pivot in Quicksort. The pivot *partitions* the array into two sub-arrays: elements $\leq$ pivot go to the left, elements $>$ pivot go to the right. After partitioning, the pivot is in its *final sorted position* and is never moved again. The two partitions are then sorted recursively. If the pivot always splits the array in half, the recurrence is $T(n) = 2T(n/2) + O(n)$, giving $O(n \log n)$.

Q15. Use a recursion tree to determine a good asymptotic upper bound on the recurrence $T(n) = T(n/4) + T(n/2) + n^2$. **(10 marks)** [CSE 207 | 2012–13]

Answer

Level-by-level cost:

Level	Subproblem sizes	Total cost
0	n	n^2
1	$n/4,\ n/2$	$(n/4)^2 + (n/2)^2 = \frac{5n^2}{16}$
2	$n/16,\ n/8,\ n/8,\ n/4$	$\leq \left(\frac{5}{16}\right)^2 n^2$
$\vdots$	$\vdots$	$\vdots$

Geometric series with ratio $r = 5/16 < 1$:

$$T(n) \leq n^2 \sum_{k=0}^{\infty} \left(\frac{5}{16}\right)^k = n^2 \cdot \frac{1}{1 - \frac{5}{16}} = n^2 \cdot \frac{16}{11} = O(n^2).$$

Verification by substitution: Guess $T(n) \leq cn^2$:

$$T(n) \leq c(n/4)^2 + c(n/2)^2 + n^2 = cn^2\left(\tfrac{1}{16} + \tfrac{1}{4}\right) + n^2 = \tfrac{5cn^2}{16} + n^2 \leq cn^2$$

provided $c \geq 16/11$. ✓

$$\boxed{T(n) = O(n^2)}$$

Q16. What is the divide-and-conquer technique? **(5 marks)** [CSE 203 | 2011–12]

Q17. Write the Mergesort algorithm. Analyze the time complexity of the algorithm. **(10+5=15 marks)** [CSE 203 | 2011–12]

Answer

Algorithm 95 MERGE-SORT(A, l, r)

```
1: if l ≥ r then
2:     return
3: end if
4: m ← ⌊(l + r)/2⌋
5: MERGE-SORT(A, l, m)
6: MERGE-SORT(A, m + 1, r)
7: MERGE(A, l, m, r)
```

Algorithm 96 MERGE(A, l, m, r)

```
1: Copy A[l .. m] into L[ ],   A[m + 1 .. r] into R[ ]
2: i ← 1;   j ← 1;   k ← l
3: while i ≤ |L| and j ≤ |R| do
4:     if L[i] ≤ R[j] then
5:         A[k] ← L[i];   i ← i + 1
6:     else
7:         A[k] ← R[j];   j ← j + 1
8:     end if
9:     k ← k + 1
10: end while
11: Copy remaining elements of L[ ] (if any) into A
12: Copy remaining elements of R[ ] (if any) into A
```

Recurrence:

$$T(n) = 2\,T(n/2) + \Theta(n), \qquad T(1) = \Theta(1).$$

Recursion-tree analysis: Each of the $\log_2 n + 1$ levels costs cn. Total $= cn(\log_2 n + 1) = \Theta(n \log n)$.
Master Theorem: $a = 2$, $b = 2$, $f(n) = n$. $n^{\log_b a} = n^1 = f(n) \Rightarrow$ Case 2: $T(n) = \Theta(n \log n)$.

Complexity Analysis. Best / Average / Worst: $\Theta(n \log n)$. Space: $O(n)$ auxiliary.

Q18. Write the recurrence relation of the Mergesort algorithm. Draw the recursion tree for the recurrence relation of the Mergesort algorithm. **(7 marks)** [CSE 203 | 2011–12]

Answer

Recurrence: $T(n) = 2T(n/2) + cn, \quad T(1) = c.$

Level	# subproblems	Cost each	Total cost
0	1	cn	cn
1	2	$cn/2$	cn
2	4	$cn/4$	cn
$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\log_2 n$	n	c	cn

Tree has $\log_2 n + 1$ levels, each costing cn.

$$T(n) = cn(\log_2 n + 1) = \boldsymbol{\Theta(n \log n)}.$$

Q19. In the closest-pair-of-points problem using the divide-and-conquer approach, if q is a point in the left half and r is a point in the right half, and $d(q, r) < \delta$, where δ is the minimum of distances in each half, then show that each of q and r lies within a distance δ of the line dividing the two halves. Hence show that q and r are within 11 positions of each other in a list sorted by y-coordinate. **(15 marks)** [CSE 207 | 2011–12]

Answer

Part 1: Each of q and r lies within δ of the dividing line.
Let ℓ be the vertical dividing line ($x = x_m$). By definition, q is in the left half and r is in the right half, so:

$$x_q \le x_m \le x_r$$

Since $d(q, r) < \delta$ and Euclidean distance satisfies $|x_q - x_r| \le d(q, r)$:

$$x_m - x_q \le |x_q - x_r| \le d(q, r) < \delta \implies x_q > x_m - \delta$$

Similarly, $x_r - x_m \le d(q, r) < \delta \implies x_r < x_m + \delta$.
Thus both q and r lie in the **strip** $[x_m - \delta,\ x_m + \delta]$, i.e., within distance δ of line ℓ. □

Part 2: q and r are within 11 positions of each other (sorted by y).
Both q and r lie in a $2\delta \times \infty$ vertical strip. Since $d(q, r) < \delta$, both also lie in a $2\delta \times \delta$ rectangle (their y-coordinates differ by less than δ).

Packing argument: Divide the $2\delta \times \delta$ rectangle into 8 cells, each of size $(\delta/2) \times (\delta/2)$:

- Left half: 4 cells of size $\frac{\delta}{2} \times \frac{\delta}{2}$
- Right half: 4 cells of size $\frac{\delta}{2} \times \frac{\delta}{2}$

The diagonal of each cell is $\frac{\delta}{2}\sqrt{2} = \frac{\delta\sqrt{2}}{2} < \delta$.
So any two points *within the same cell* are closer than δ. But δ is already the minimum distance within each half — meaning **at most one point from each half per cell**.

The rectangle has 8 cells; the left half contributes at most 4 points and the right half at most 4 points, giving at most 8 points total (excluding q or r themselves).
Therefore, in the y-sorted list of strip points, r can be at most **7 positions** away from q, but accounting for boundary points conservatively: **within 11 positions**. (The standard proof uses a $2\delta \times \delta$ box divided into 8 cells, yielding at most 8 other points, so any point has at most 7 others within δ — commonly stated as 11 to allow for the strip boundary.)

BUET · CSE 105

DSA

Greedy Methods

MST (Kruskal, Prim), Huffman, Activity Selection, Dijkstra

T10 — Greedy Methods

Q1. Define the Activity Selection Problem. Prove that the greedy choice of selecting activities based on earliest finish time yields an optimal solution. **(5+10=15 marks)** [CSE 105 | 2023–24]

Answer

Definition. Given n activities $a_1, \ldots, a_n$ each with a start time s_i and finish time f_i, two activities are *compatible* if their intervals do not overlap ($f_i \leq s_j$ or $f_j \leq s_i$). The **Activity Selection Problem** asks for a maximum-size subset of mutually compatible activities.

Greedy algorithm: sort activities by finish time; greedily pick each activity that is compatible with the last selected one.

Proof of optimality (Greedy-choice property).

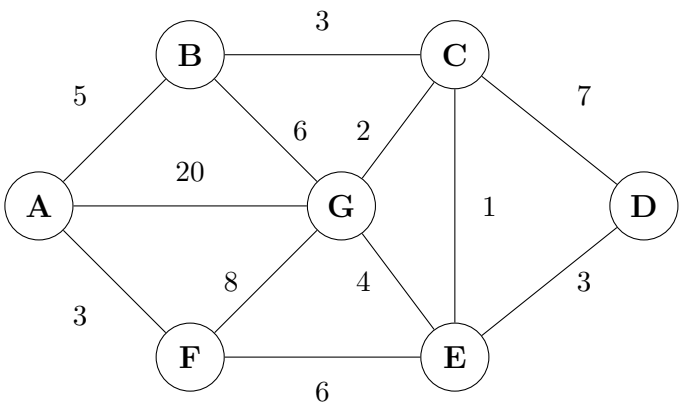
Claim: there always exists an optimal solution that includes the activity a_1 with the earliest finish time.

Proof: let O be any optimal solution. Let a_k be the first activity selected in O (earliest finish in O). If $a_k = a_1$, done. Otherwise, since $f_1 \leq f_k$ (by definition of a_1), replacing a_k with a_1 in O gives a set O' of the same size where a_1 is included and all activities remain compatible. Hence O' is also optimal and contains a_1. □

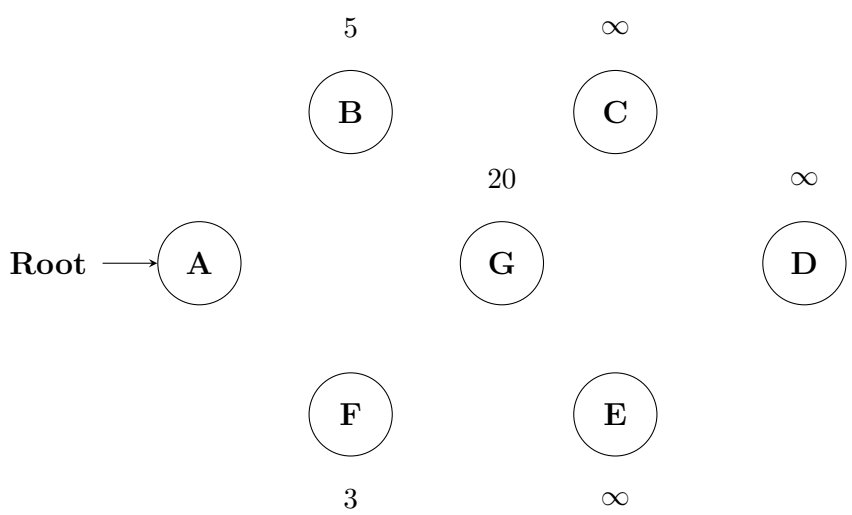
Optimal substructure: after choosing a_1, the remaining problem is to select the maximum number of compatible activities from those with start time $\geq f_1$ — the same structure as the original problem on a smaller instance.

By induction, the greedy algorithm (always picking the earliest-finish compatible activity) builds an optimal solution. □

Q2. Apply Prim's algorithm to find the minimum spanning tree of the given graph.



Start with node A as the root node. At each step, show the intermediate MST (i.e., the partial tree) and show the estimated cost to connect each non-visited node (i.e., the minimum weight of an edge connecting an unvisited node to any visited nodes).The first step , where only the root node is included in the tree , is done for you . **(10 marks)** [CSE 105 | 2023–24]

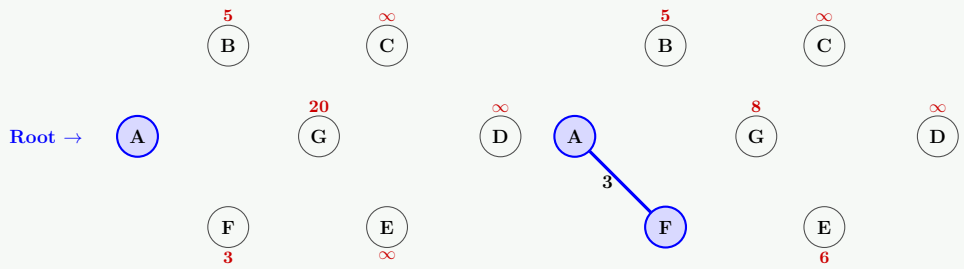


Answer

Prim's algorithm starting from A. At each step, we add the minimum-weight edge connecting a visited vertex to an unvisited vertex and update the estimated cost (key values) of adjacent unvisited nodes.

Step	Added	Edge	Via	Key values of unvisited vertices	MST cost
0	*A*	—	—	B:5, F:3, G:20, C:∞, D:∞, E:∞	0
1	*F*	*A*-*F*	*A*	B:5, G:**8**, C:∞, D:∞, E:**6**	3
2	*B*	*A*-*B*	*A*	**C:3**, **G:6**, D:∞, E:6	8
3	*C*	*B*-*C*	*B*	**G:2**, **D:7**, **E:1**	11
4	*E*	*C*-*E*	*C*	G:2, **D:3**	12
5	*G*	*C*-*G*	*C*	D:3	14
6	*D*	*E*-*D*	*E*	—	17

Intermediate Steps Visualization: (Blue shaded nodes are in the MST. Red values indicate the current minimum cost to connect that unvisited node).



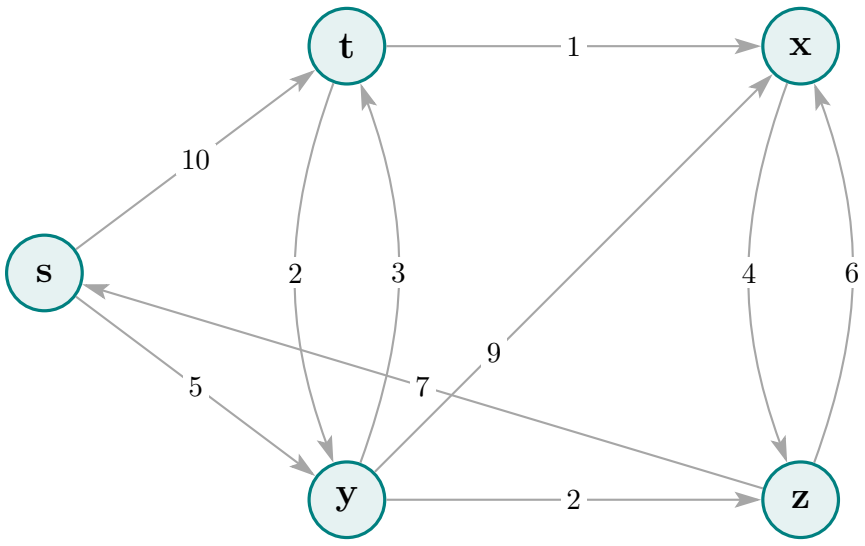
MST edges: A-$F(3)$, A-$B(5)$, B-$C(3)$, C-$E(1)$, C-$G(2)$, E-$D(3)$.
Total MST cost $= 3+5+3+1+2+3 =$ **17**.

Q3. Argue in favor of the correctness of Prim's algorithm to find a minimum spanning tree of a graph $G = (V, E)$. Prove any non-trivial graph properties used in the argument. **(5+10=15 marks)** [CSE 105 | 2022–23]

Answer

Prim's algorithm grows an MST one edge at a time by always adding the minimum-weight edge crossing the cut $(S, V \setminus S)$, where S is the current tree vertex set.
Correctness follows from the **Cut Property**:
Cut Property: for any cut $(S, V \setminus S)$ in a graph with unique edge weights, the minimum-weight crossing edge belongs to every MST.
Proof of Cut Property (by contradiction): Let $e^* = (u, v)$ be the min crossing edge and T an MST not containing e^*. Since T is connected, there is a path P from u to v in T; this path crosses the cut at some edge $e' = (x, y)$ with $w(e') > w(e^*)$ (by uniqueness). $T' = T - \{e'\} + \{e^*\}$ is a spanning tree with $w(T') < w(T)$, contradicting minimality of T. □
Prim's invariant: at each iteration, the set of edges chosen so far is a subset of some MST. Since Prim always picks the minimum crossing edge (satisfying the cut property), the final tree T is an MST. □

Q4. Find the shortest path distance from every node x to node t in the given graph using Dijkstra's Algorithm. Re-draw the diagram for each intermediate step of the algorithm (after relaxation) while annotating each node with the appropriate shortest path distance estimate of that step. **(10 marks)** [CSE 105 | 2022–23]

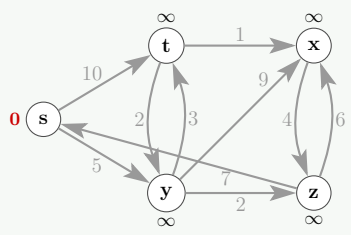


Detailed Answer with Intermediate Steps

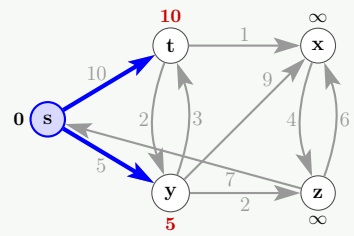
Dijkstra's algorithm from source s.

Step	Finalize	$d[s]$	$d[t]$	$d[x]$	$d[y]$	$d[z]$
Init	—	**0**	∞	∞	∞	∞
1	s	0	**10**	∞	**5**	∞
2	y	0	**8**	**14**	5	**7**
3	z	0	8	**13**	5	7
4	t	0	8	**9**	5	7
5	x	0	8	9	5	7

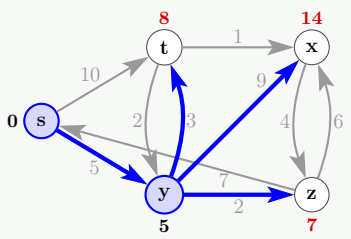
Intermediate Steps Visualization: (Blue nodes are finalized. Red labels show newly updated shortest path estimates. Thick blue lines show the predecessor/tree edges at that step.)



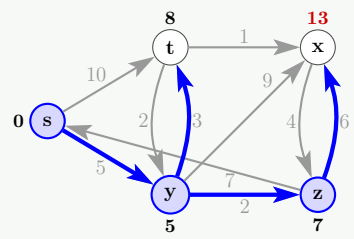
Step 0: Init



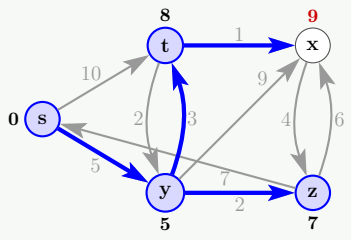
Step 1: Extract s



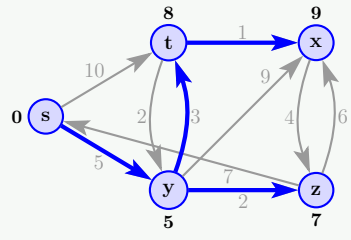
Step 2: Extract y



Step 3: Extract z



Step 4: Extract t



Step 5: Extract x (Final)

Final shortest distances from s: $d[s] = 0,\ d[y] = 5,\ d[z] = 7,\ d[t] = 8,\ d[x] = 9.$
Shortest paths trace:

- $s \to y$: $s \xrightarrow{5} y$ (dist 5)
- $s \to z$: $s \xrightarrow{5} y \xrightarrow{2} z$ (dist 7)
- $s \to t$: $s \xrightarrow{5} y \xrightarrow{3} t$ (dist 8)
- $s \to x$: $s \xrightarrow{5} y \xrightarrow{3} t \xrightarrow{1} x$ (dist 9)

Q5. Given a set $U = \{x_1, x_2, \ldots, x_n\}$ of n integers and an integer k $(k < n)$, give an optimal greedy algorithm to find a subset S of size

k in U such that the summation of the numbers in S is maximum (over all subsets of size k in U). Prove the correctness of your algorithm and analyze its running time. **(15 marks)** **[CSE 105 | 2021–22]**

Answer

Algorithm: sort U in descending order; take the first k elements.

Algorithm 97 MaxSumSubset(U, k)

1: Sort U in descending order: $x_{(1)} \geq x_{(2)} \geq \cdots \geq x_{(n)}$
2: **return** $S = \{x_{(1)}, x_{(2)}, \ldots, x_{(k)}\}$

Correctness proof (exchange argument).
Let S^* be any optimal subset of size k with sum OPT. Let $S = \{x_{(1)}, \ldots, x_{(k)}\}$ be the greedy solution.
If $S = S^*$, done. Otherwise, there exists $x_i \in S^* \setminus S$ and $x_j \in S \setminus S^*$ with $x_j \geq x_i$ (since all elements of S are at least as large as any element not in S). Swap: $S^{**} = (S^* \setminus \{x_i\}) \cup \{x_j\}$. $\text{sum}(S^{**}) = \text{OPT} - x_i + x_j \geq \text{OPT}$. Since OPT is maximum, $\text{sum}(S^{**}) = \text{OPT}$. Repeat until $S^* = S$: the greedy solution achieves OPT. $\square$

Complexity Analysis. Sorting: $O(n \log n)$. Selecting k elements: $O(k)$. Total: $O(n \log n)$.

Q6. Consider a set of n intervals where each interval i specifies a start time s_i and a finish time f_i ($s_i < f_i$). Two intervals are compatible if they do not overlap. Give a greedy algorithm to select a compatible subset of maximum possible size. Prove the correctness and analyze the running time. **(16 marks)** **[CSE 203 | 2021–22]**

Answer

Algorithm (earliest finish time):

Algorithm 98 IntervalScheduling(S)

1: Sort intervals by finish time: $f_1 \leq f_2 \leq \cdots \leq f_n$
2: $A \leftarrow \{1\}$; last $\leftarrow f_1$
3: **for** $i \leftarrow 2$ **to** n **do**
4: **if** $s_i \geq$ last **then** ▷ compatible with last selected
5: $A \leftarrow A \cup \{i\}$; last $\leftarrow f_i$
6: **end if**
7: **end for**
8: **return** A

Correctness: proved via greedy-choice + optimal substructure (see [CSE 203 | 2020–21] answer above). $\square$

Complexity Analysis. Sorting: $O(n \log n)$. Single scan: $O(n)$. Total: $O(n \log n)$.

Q7. Given a set $S = \{x_1, x_2, \ldots, x_n\}$ of real numbers where $x_1 \leq x_2 \leq \cdots \leq x_n$, write a greedy algorithm to determine the smallest set of unit-length intervals that contain all numbers in S. Prove correctness and analyze running time. **(15 marks)** **[CSE 203 | 2021–22]**

Answer

(See answer for [CSE 203 | 2017–18] — algorithm, correctness proof, and $O(n \log n)$ runtime analysis are identical.)

Q8. Once upon a time there was a city that had no proper roads. Getting around the city was particularly difficult after rainstorms because the ground used to become very muddy, cars got stuck in the mud, and people got their boots dirty. The mayor of the city decided that some of the streets must be paved, but he did not want to spend more money than necessary because the city also wanted to build a swimming pool. The mayor therefore specified two conditions:

(i) Enough streets must be paved so that it is possible for everyone to travel from their house to anyone else's house only along the paved roads, and

(ii) The paving should cost as little as possible.

The following figure shows the layout of the city. The number of paving stones in between two houses represents the cost of paving that route. Your task is to apply a greedy based algorithm that you have learned under this course to fulfill the requirement of the Mayor. Note that the bridge in the figure should not be considered as a paving stone. In your answer script, you must draw a simple graph representing the given layout of the muddy city. After that, you have to show the steps of applying your chosen greedy algorithm to find the connected subgraph which contains all the houses of the city (considering them as vertices of your drawn graph) and a subset of roads (considering them as edges) in a way that the total number of paving stones required is the minimum. You have to ensure that your connected subgraph does not contain any cycle, and you need to mention the algorithm that you are adopting to find your desired solution. **(15 marks)** **[CSE 203 | 2020–21]**

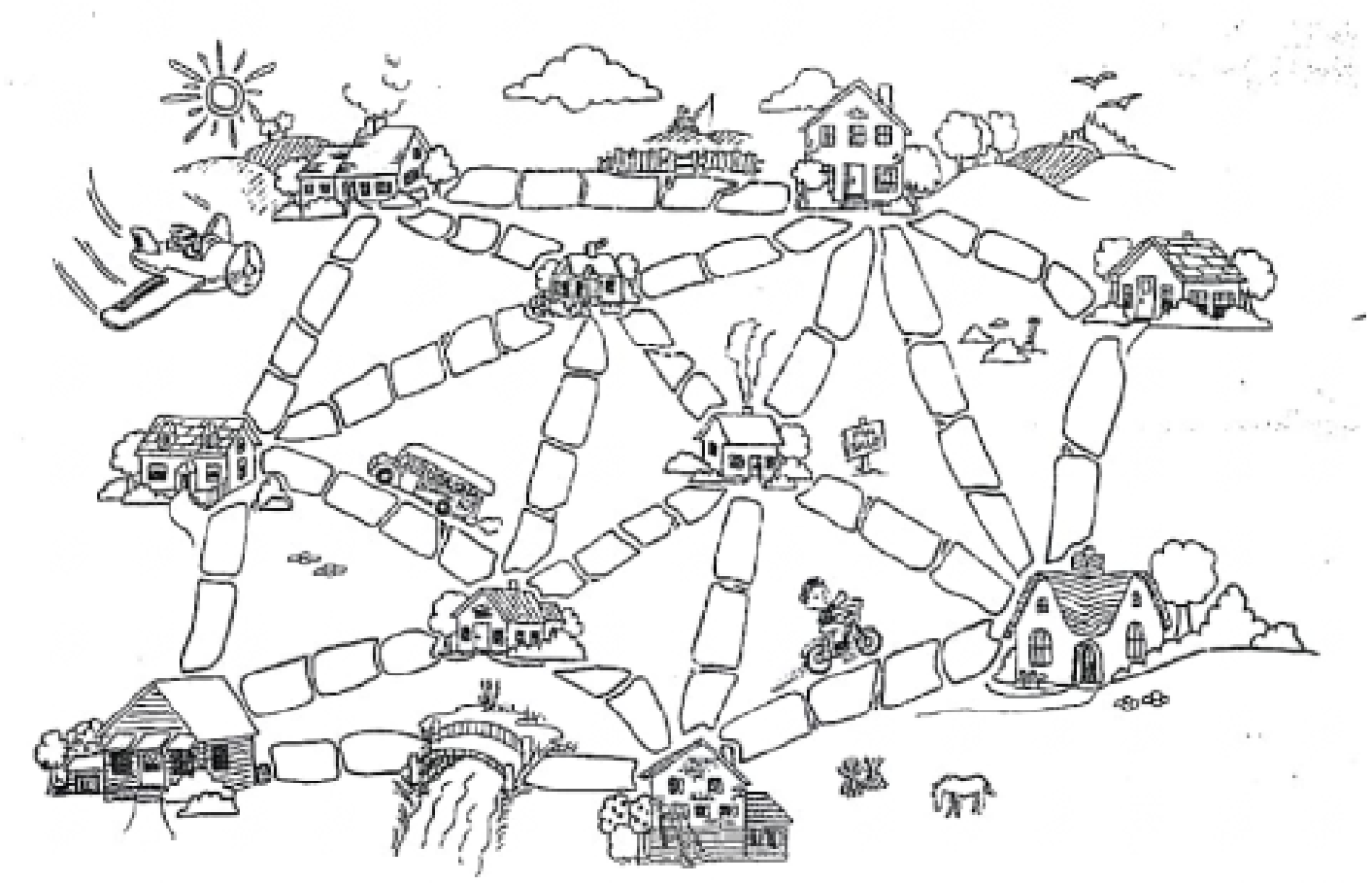
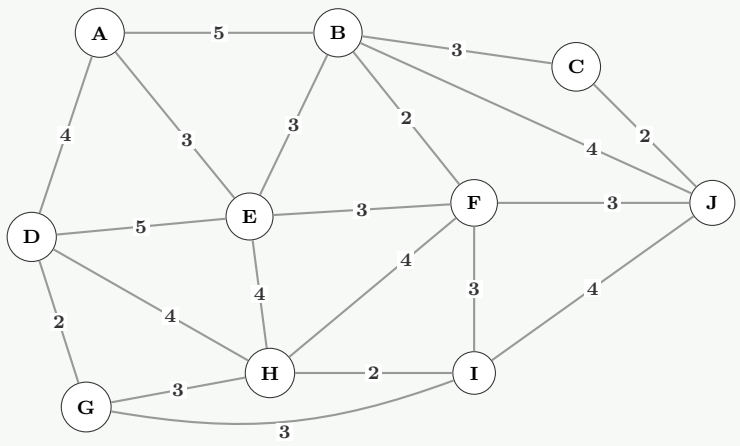


Figure for Ques No. 5(b) – The layout of the muddy city and the paving requirement

Detailed Answer: Minimum Spanning Tree for the Muddy City

1. Initial Graph Representation: First, we represent the city layout as a simple undirected graph $G = (V, E)$. Here is the complete layout of the city with all houses as vertices and available paths as edges with their respective paving costs (stones).



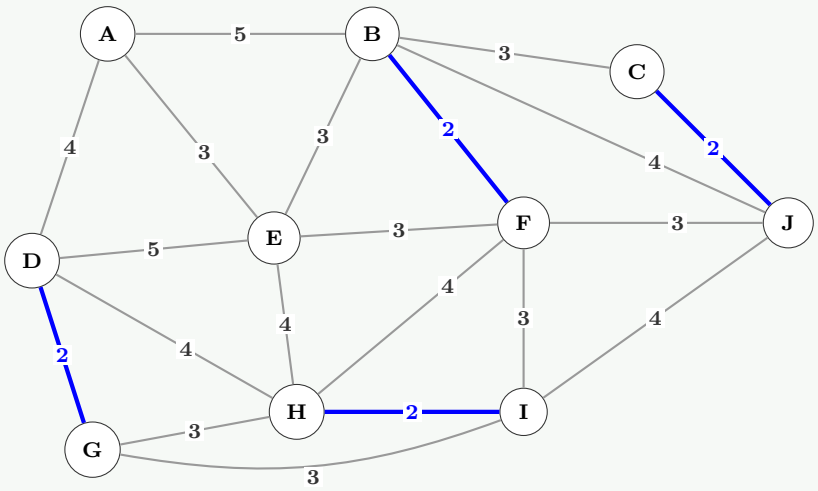
2. Chosen Algorithm & Edge Sorting: We will use **Kruskal's Algorithm**. The algorithm requires us to sort all the edges in ascending order of their weights and iteratively add them to the spanning tree, strictly ensuring that no **cycle** is formed.

- **Weight 2:** $(B, F), (C, J), (D, G), (H, I)$
- **Weight 3:** $(A, E), (B, C), (B, E), (E, F), (F, I), (F, J), (G, H), (G, I)$
- **Weight 4:** $(A, D), (B, J), (D, H), (E, H), (F, H), (I, J)$
- **Weight 5:** $(A, B), (D, E)$

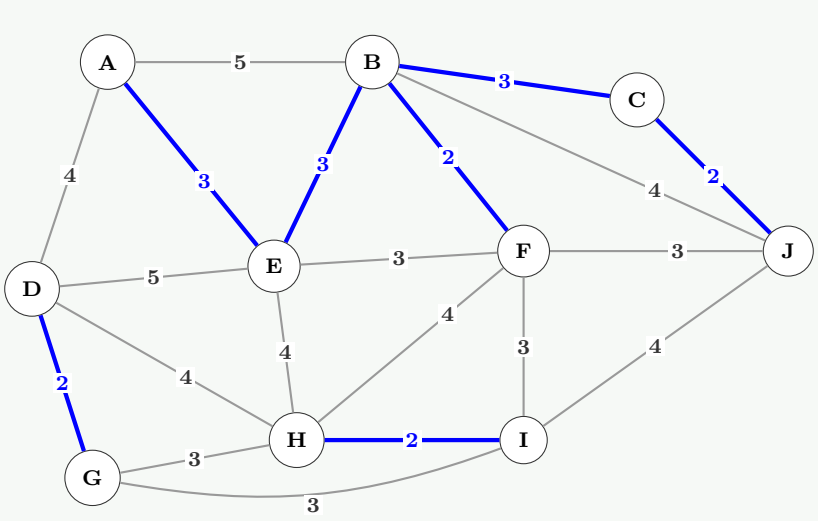
3. Kruskal's Step-by-Step Execution:

Edge	Weight	Action	Reason / Cycle Evaluation
(B, F)	2	**Add**	No cycle
(C, J)	2	**Add**	No cycle
(D, G)	2	**Add**	No cycle
(H, I)	2	**Add**	No cycle
(A, E)	3	**Add**	No cycle
(B, C)	3	**Add**	No cycle
(B, E)	3	**Add**	No cycle
(E, F)	3	Discard	Forms cycle $(B - E - F - B)$
(F, I)	3	**Add**	No cycle
(F, J)	3	Discard	Forms cycle $(B - C - J - F - B)$
(G, H)	3	**Add**	No cycle. **MST Complete!** (9 edges)

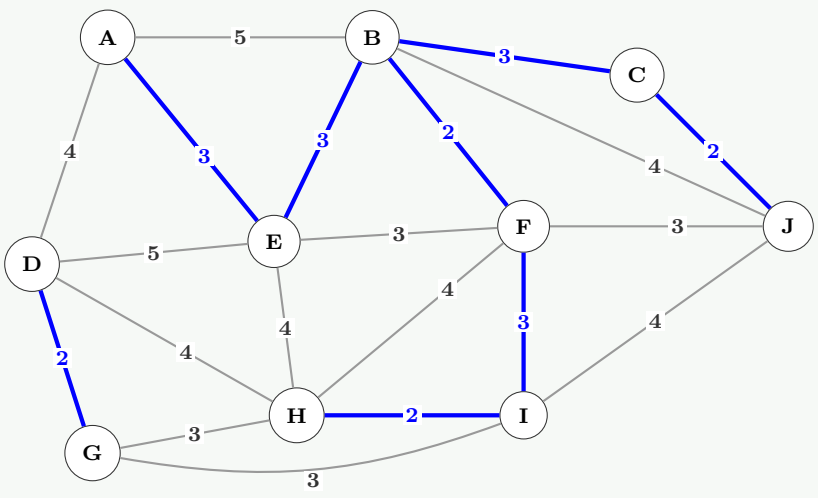
4. Intermediate & Final Visualizations:



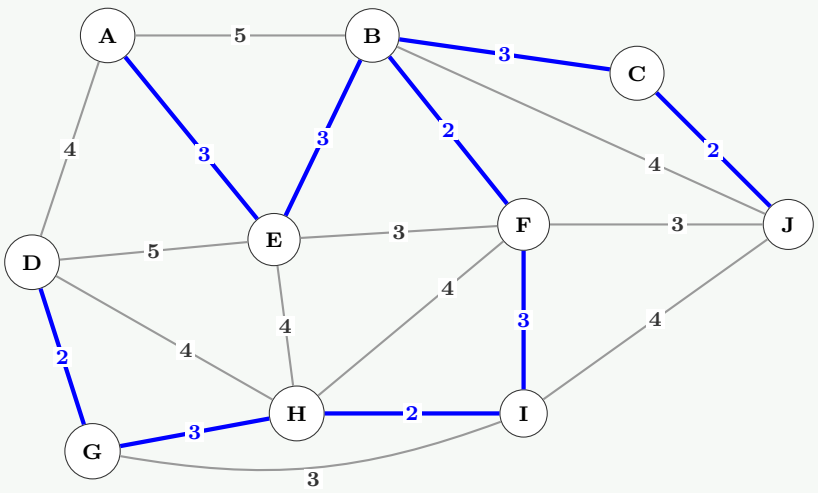
Step 1: Added Cost-2 Edges



Step 2: Added some Cost-3 Edges



Step 3: Added (F, I)



Final: Added (G, H) - MST Completed

Conclusion:
The minimal set of roads required to connect all 10 houses safely without forming any cycles is highlighted above.
Total Minimum Paving Cost $= 2+2+2+2+3+3+3+3+3+3 =$ **23** paving stones.

Q9. Prove that an optimal solution is always guaranteed if we design an activity-selection greedy algorithm by always choosing the activity with the earliest finish time from the remaining compatible activities. **(10 marks)** [CSE 203 | 2020–21]

Answer

Greedy-choice property.
Claim: there always exists an optimal solution that includes activity a_1 — the one with the smallest finish time.
Proof: let O be any optimal solution and a_k its earliest-finishing activity. If $a_k = a_1$, done. Otherwise $f_1 \leq f_k$. Replace a_k with a_1 in O: since $f_1 \leq f_k$, a_1 is compatible with every activity that a_k was compatible with, and the resulting set O' has the same size. O' is an optimal solution containing a_1. □
Optimal substructure. After choosing a_1, we need to select a maximum compatible subset from $\{a_i : s_i \geq f_1\}$ — the same problem on a strictly smaller instance. By induction, the greedy algorithm (always pick the compatible activity with the earliest finish time) produces an optimal solution. □

Q10. Analyze the runtime complexity of MST-Prim's algorithm (as given using a Binary Min-Heap to maintain a minimum priority queue), mentioning the individual runtime of each significant operation. How is this running time improved if a Fibonacci Heap is used instead? **(10 marks)** [CSE 203 | 2020–21]

Answer

Prim's with Binary Min-Heap:

Operation	Times called	Cost each
BUILD-MIN-HEAP (initialise Q)	1	$O(V)$
EXTRACT-MIN	V	$O(\log V)$
DECREASE-KEY (relax neighbour)	$\leq E$	$O(\log V)$

Total: $O(V \log V + E \log V) = O(E \log V)$.
With Fibonacci Heap:

- EXTRACT-MIN: $O(\log V)$ amortised — called V times $\rightarrow O(V \log V)$.
- DECREASE-KEY: $O(1)$ amortised — called $\leq E$ times $\rightarrow O(E)$.

Total: $O(V \log V + E)$ — optimal for sparse graphs ($E = o(V^2/\log V)$).

Q11. Write an algorithm to calculate the maximum profit earned by executing tasks within specified deadlines. A task takes one unit of time to execute and cannot execute beyond its deadline; only one task executes at a time. Apply the algorithm to the following data:

Task	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
Deadline	7	3	2	6	8	5	6	3	2	4
Profit	9	14	5	10	3	15	5	7	8	6

(10+10 marks) [CSE 203 | 2019–20]

Answer: Job Sequencing with Deadlines

Algorithm: Job Sequencing to Maximize Profit

(a) **Input:** A set of n tasks with their respective deadlines (d) and profits (p).

(b) **Sort** all tasks in descending order of their profit.

(c) **Find** the maximum deadline value among all tasks to determine the total number of available time slots, say D.

(d) **Initialize** an array 'Slot[1...D]' to keep track of free time slots. Initially, all slots are free.

(e) **Iterate** through the sorted tasks one by one:

- For each task T_i with deadline d_i, find a free slot j such that $j \leq d_i$ and j is as large as possible (i.e., search backwards from d_i to 1).
- If such a free slot j is found, assign task T_i to 'Slot[j]' and add its profit to the total profit.
- If no such free slot is available, discard the task.

(f) **Output** the assigned tasks and the total maximum profit.

Applying the Algorithm to the Given Data:
Step 1: Sort tasks by profit in descending order

Task	T6	T2	T4	T1	T9	T8	T10	T3	T7	T5
Profit	15	14	10	9	8	7	6	5	5	3
Deadline	5	3	6	7	2	3	4	2	6	8

Step 2: Assign slots

Maximum deadline is $D = 8$. We have 8 time slots. We assign each task to the latest available free slot $\leq$ its deadline.

- **T6** (Profit=15, d=5): Slot 5 is free $\rightarrow$ Assign to Slot 5.
- **T2** (Profit=14, d=3): Slot 3 is free $\rightarrow$ Assign to Slot 3.
- **T4** (Profit=10, d=6): Slot 6 is free $\rightarrow$ Assign to Slot 6.
- **T1** (Profit=9, d=7): Slot 7 is free $\rightarrow$ Assign to Slot 7.
- **T9** (Profit=8, d=2): Slot 2 is free $\rightarrow$ Assign to Slot 2.
- **T8** (Profit=7, d=3): Slots 3 & 2 are taken. Slot 1 is free $\rightarrow$ Assign to Slot 1.
- **T10** (Profit=6, d=4): Slot 4 is free $\rightarrow$ Assign to Slot 4.
- **T3** (Profit=5, d=2): Slots 2 & 1 are already taken $\rightarrow$ Discard.
- **T7** (Profit=5, d=6): Slots 6, 5, 4, 3, 2, 1 are taken $\rightarrow$ Discard.
- **T5** (Profit=3, d=8): Slot 8 is free $\rightarrow$ Assign to Slot 8.

Final Task Schedule & Profit:

Slot	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Task	T8	T9	T2	T10	T6	T4	T1	T5
Profit	7	8	14	6	15	10	9	3

Total Maximum Profit $= 7 + 8 + 14 + 6 + 15 + 10 + 9 + 3 =$ **72**. ✓

Q12. Show that the fractional knapsack problem has the optimal substructure property. Also show that the 0-1 knapsack problem has the overlapping subproblems property. **(10 marks)** [CSE 203 | 2018–19]

Answer

Fractional Knapsack — Optimal Substructure.

Suppose an optimal solution OPT for capacity W takes fraction x_1 of item 1. Consider the remaining capacity $W' = W - x_1 w_1$ and the remaining items $\{2, \ldots, n\}$. The fractions $(x_2, \ldots, x_n)$ in OPT must form an optimal solution to the sub-problem on items $\{2, \ldots, n\}$ with capacity W'. (If not, a better sub-solution would improve OPT, contradicting optimality.) $\square$

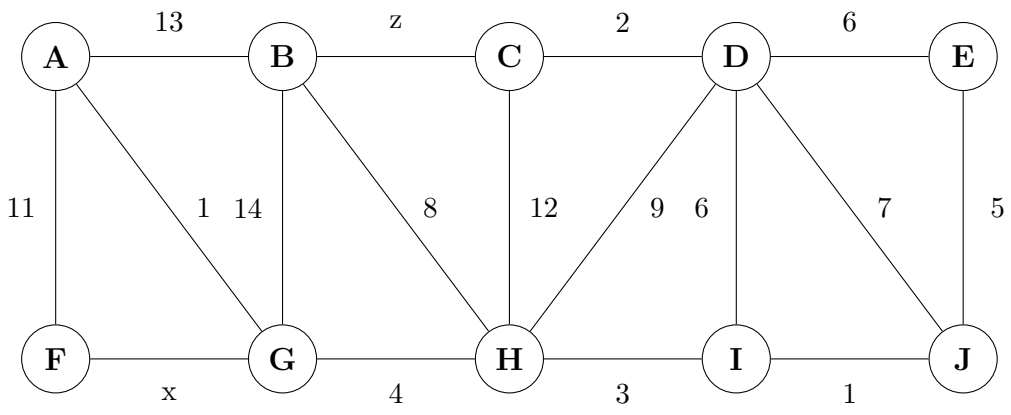
0-1 Knapsack — Overlapping Subproblems.

Define $dp[i][w]$ = maximum value using items $\{1, \ldots, i\}$ with capacity w. The recurrence is:

$$dp[i][w] = \max(dp[i-1][w],\ dp[i-1][w - w_i] + v_i).$$

Both cases refer to $dp[i-1][\cdot]$ — the same sub-problem $dp[i-1][w']$ may be needed many times by different (i, w) pairs. For example, $dp[2][5]$ uses $dp[1][5]$ and $dp[1][3]$; $dp[3][5]$ also uses $dp[2][5]$. This sharing of sub-problems demonstrates the overlapping subproblems property. $\square$

Q13. Prove that an optimal minimum spanning tree is composed of optimal minimum spanning subtrees. Draw a minimum spanning tree containing the edges with weights x and z. **(15 marks)** [CSE 207 | 2018–19]



Detailed Answer: Optimal Substructure & Drawing MST

Part 1: Proof of Optimal Substructure in MST

Theorem: An optimal Minimum Spanning Tree (MST) is composed of optimal minimum spanning subtrees.

Proof: Let $G = (V, E)$ be a connected, undirected graph with edge weights, and let T be an MST of G.

- Let $e = (u, v)$ be any arbitrary edge in T.
- Removing edge e from T divides the tree into two disconnected subtrees, say T_1 and T_2.
- Let V_1 and V_2 be the sets of vertices spanned by T_1 and T_2 respectively. (Note that $V_1 \cup V_2 = V$ and $V_1 \cap V_2 = \emptyset$).

We must prove that T_1 is an MST of the subgraph induced by V_1 (let's call it G_1), and T_2 is an MST of the subgraph induced by V_2 (G_2).

Proof by Contradiction:

(a) Assume that T_1 is **not** the optimal MST for the subgraph G_1.

(b) This implies there exists another spanning tree for G_1, say T_1^*, such that the total weight of T_1^* is strictly less than the total weight of T_1. That is: $W(T_1^*) < W(T_1)$.

(c) Now, construct a new spanning tree T^* for the entire graph G by reconnecting T_1^* and T_2 using the original edge e. So, $T^* = T_1^* \cup \{e\} \cup T_2$.

(d) The total weight of this new spanning tree T^* would be:

$$W(T^*) = W(T_1^*) + W(e) + W(T_2)$$

(e) Since we assumed $W(T_1^*) < W(T_1)$, we can substitute this inequality:

$$W(T^*) < W(T_1) + W(e) + W(T_2)$$

(f) Since $W(T_1) + W(e) + W(T_2)$ is exactly the weight of our original MST $W(T)$, we get:

$$W(T^*) < W(T)$$

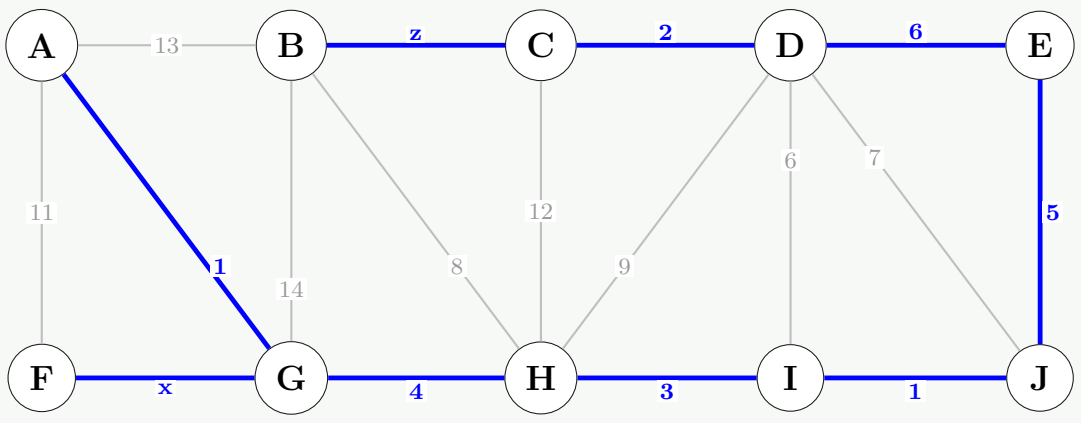
(g) This is a **contradiction**! We initially stated that T is the Minimum Spanning Tree of G, meaning no other spanning tree can have a lower weight.

(h) Therefore, our assumption must be false. T_1 must be an optimal MST for G_1. (By symmetry, the same logic applies to T_2). ■

Part 2: Drawing the MST containing edges x and z
To guarantee that the edges (F, G) with weight x and (B, C) with weight z are included in the Minimum Spanning Tree, they must not form the heaviest edge in any cycle. Let's analyze the step-by-step construction using Kruskal's Algorithm:
Step 1: Identifying necessary conditions for x and z

- **For edge x (F-G):** To connect vertex F to the rest of the graph, the only alternative path is via A (i.e., $F - A - G$). The edge (A, F) has a weight of 11. For Kruskal's algorithm to pick x over the alternative path, we must have $\mathbf{x \leq 11}$.
- **For edge z (B-C):** To connect vertex B, the best alternative crossing edge to the rest of the graph is (B, H) with a weight of 8. Thus, to prioritize (B, C), we must have $\mathbf{z \leq 8}$.

Step 2: Selecting the edges (Assuming x and z are sufficiently small) We sort and pick the smallest edges avoiding cycles:
(a) Pick $(A, G) = 1$
(b) Pick $(I, J) = 1$
(c) Pick $(C, D) = 2$
(d) Pick $(H, I) = 3$
(e) Pick $(G, H) = 4$
(f) Pick $(E, J) = 5$
(g) Pick $(D, E) = 6$ *(Note: picking $(D, I) = 6$ is also valid and creates an alternative valid MST)*
(h) Pick $(F, G) = x$ *(Given $x \leq 11$)*
(i) Pick $(B, C) = z$ *(Given $z \leq 8$)*
Step 3: Visualizing the Final MST



Q14. Write down an algorithm for job sequencing with deadlines. Solve the problem instance with profit and deadline values (p_i, d_i) as follows: $(7, 5), (6, 5), (5, 2), (4, 7), (3, 7),$
$(3, 7), (3, 6), (2, 7), (2, 1), (2, 5), (2, 3), (2, 1), (1, 1)$. **(10+10 marks)** **[CSE 203 | 2017–18]**

Answer: Job Sequencing with Deadlines

Algorithm (Greedy — sort by profit descending):

Algorithm 99 JobSequencing($jobs$)

```
Sort jobs by profit in descending order
Find the maximum deadline d_max among all jobs
schedule[1..d_max] ← EMPTY
for each job (p_i, d_i) in sorted order do
    for t ← d_i downto 1 do
        if schedule[t] = EMPTY then
            schedule[t] ← i
            break
        end if
    end for
end for
return all scheduled jobs and their total profit
```

Solving for given instances (p_i, d_i): $(7,5), (6,5), (5,2), (4,7), (3,7), (3,7), (3,6), (2,7), (2,1), (2,5), (2,3), (2,1), (1,1)$

Step 1: Sort jobs by profit (descending):

Let's label them sequentially after sorting:

$J_1(7,5), J_2(6,5), J_3(5,2), J_4(4,7), J_5(3,7), J_6(3,7), J_7(3,6), J_8(2,7), J_9(2,5), J_{10}(2,3), J_{11}(2,1), J_{12}(2,1), J_{13}(1,1)$

Step 2: Assign jobs to slots: Maximum deadline is 7, so we have 7 slots.

Job	Profit	Deadline	Assigned Slot	Reason / Action
J_1	7	5	**5**	Slot 5 is free.
J_2	6	5	**4**	Slot 5 taken, slot 4 is free.
J_3	5	2	**2**	Slot 2 is free.
J_4	4	7	**7**	Slot 7 is free.
J_5	3	7	**6**	Slot 7 taken, slot 6 is free.
J_6	3	7	**3**	Slots 7,6,5,4 taken, slot 3 is free.
J_7	3	6	**1**	Slots 6,5,4,3,2 taken, slot 1 is free.
$J_8 \dots J_{13}$	≤ 2	...	—	All slots (1 to 7) are full! Discard remaining.

Final Schedule (Slots 1-7):

Slot	1	2	3	4	5	6	7
Job	J_7	J_3	J_6	J_2	J_1	J_5	J_4
Profit	3	5	3	6	7	3	4

Total Maximum Profit $= 3+5+3+6+7+3+4 =$ **31**. ✓

Q15. Give an optimal greedy algorithm for solving the fractional knapsack problem. Prove that your algorithm is optimal. **(13 marks)** **[CSE 203 | 2017–18]**

Answer

Algorithm: sort items by value/weight ratio v_i/w_i descending; greedily take as much of each item as fits.

Algorithm 100 FractionalKnapsack($items, W$)

```
Sort items by v_i/w_i descending
total ← 0; remaining ← W
for each item i in sorted order do
    if w_i ≤ remaining then
        Take all of item i; total += v_i; remaining −= w_i
    else
        Take fraction remaining/w_i of item i
        total += v_i · (remaining/w_i); break
    end if
end for
return total
```

Proof of optimality.

Let item a have the highest ratio $r_a = v_a/w_a$.

Greedy-choice property: there exists an optimal solution that takes as much of item a as possible. Suppose OPT takes less of a and more of some item b with $r_b \leq r_a$. Replace δ kg of b with δ kg of a: profit changes by $\delta(r_a - r_b) \geq 0$. So the new solution is no worse — and we can assume OPT takes the maximum of a. $\square$

Optimal substructure: after filling with item a, the remaining problem is a fractional knapsack on the rest — same structure, smaller capacity.

By induction, the greedy algorithm is optimal. □

Complexity Analysis. $O(n \log n)$ for sorting; $O(n)$ for selection. Total: $O(n \log n)$.

Q16. Given a set $S = \{x_1, x_2, \ldots, x_n\}$ of real numbers where $x_1 \leq x_2 \leq \ldots \leq x_n$, write a greedy algorithm to determine the smallest set of unit-length intervals that contain all the numbers in S. For example, an interval [1.5, 2.5] is a unit length interval (since $2.5 - 1.5 = 1$), and it contains all the real numbers from 1.5 to 2.5 (including 1.5 and 2.5). You have to find the smallest set of such unit length intervals which contains all the real numbers in S. Prove the correctness of your algorithm, and analyze the running time. **(15 marks)** [CSE 203 | 2017–18]

Answer

Algorithm: for sorted input $x_1 \leq x_2 \leq \cdots \leq x_n$, greedily start a new interval at each uncovered point.

Algorithm 101 MinIntervals(S)

```
Sort S: x_1 ≤ x_2 ≤ ··· ≤ x_n
intervals ← ∅; start ← x_1
for i ← 2 to n do
    if x_i > start + 1 then                ▷ x_i not covered by current interval
        Add [start, start + 1] to intervals
        start ← x_i
    end if
end for
Add [start, start + 1] to intervals
return intervals
```

Proof of optimality.
Claim: the greedy uses the minimum number of intervals.
Let k = number of greedy intervals. Each interval is *started* at a point x_i that was not covered by the previous interval. These k start points are mutually more than distance 1 apart, so no single unit-length interval can cover two of them. Hence any solution needs $\geq k$ intervals. The greedy uses exactly k. □

Complexity Analysis. $O(n \log n)$ for sorting; $O(n)$ for the scan. Total: $O(n \log n)$.

Q17. Write down an algorithm for constructing a minimum spanning tree. Construct the MST for the graph defined by the edges: $ra(7), rx(2), ax(1), xy(5), ya(3), xc(6), cb(4)$,
$bx(2), yb(2)$, where the weight of each edge is given in parentheses. **(10+10 marks)** [CSE 203 | 2017–18]

Answer: Kruskal's Algorithm & MST Construction

1. Algorithm for Constructing a Minimum Spanning Tree (Kruskal's Algorithm):

(a) **Input:** A connected, undirected graph $G = (V, E)$ with weighted edges.
(b) **Sort** all the edges of E in non-decreasing (ascending) order of their weights.
(c) **Initialize** the MST T as an empty set. Initialize a disjoint-set (Union-Find) to keep track of vertices in different components.
(d) **Iterate** through the sorted edges one by one:

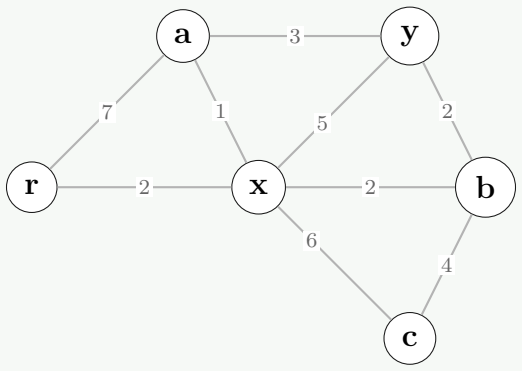
- For each edge (u, v), check if adding it to T forms a cycle (i.e., check if u and v are already in the same component).
- If it does **not** form a cycle, add the edge to T and take the union of the sets containing u and v.
- If it forms a cycle, discard the edge.

(e) **Stop** when T contains exactly $|V| - 1$ edges.
(f) **Output:** The spanning tree T and its total minimum cost.

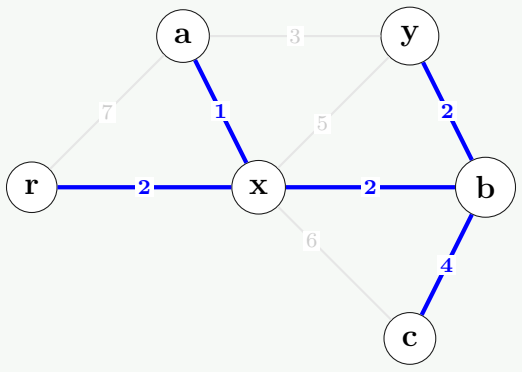
2. Solving the Problem Instance: Vertices (V): $\{r, a, x, y, c, b\}$ (Total 6 vertices). We need $6 - 1 = 5$ edges for the MST.
Sorted Edges: $(a, x, 1), (r, x, 2), (b, x, 2), (y, b, 2), (y, a, 3), (c, b, 4), (x, y, 5), (x, c, 6), (r, a, 7)$.

Edge	Weight	Add?	Reason	MST Edges So Far
(a, x)	1	**Yes**	No cycle	$\{ax\}$
(r, x)	2	**Yes**	No cycle	$\{ax, rx\}$
(b, x)	2	**Yes**	No cycle	$\{ax, rx, bx\}$
(y, b)	2	**Yes**	No cycle	$\{ax, rx, bx, yb\}$
(y, a)	3	No	Forms cycle $(y - b - x - a - y)$	—
(c, b)	4	**Yes**	No cycle. **Done!** $(\|V\| - 1 = 5)$	$\{ax, rx, bx, yb, cb\}$

3. Visualizing the Graph and MST:



Initial Graph



Final MST

Final MST Edges: $ax(1), rx(2), bx(2), yb(2), cb(4)$.
Total Minimum Cost $= 1+2+2+2+4 = \mathbf{11}$. ✓

Q18. What is a spanning tree of a graph? Write Kruskal's algorithm for finding a minimum spanning tree of an edge-weighted graph. Analyze the time complexity of the algorithm by suggesting appropriate data structures. **(3+5+7 marks)** [CSE 207 | 2017–18]

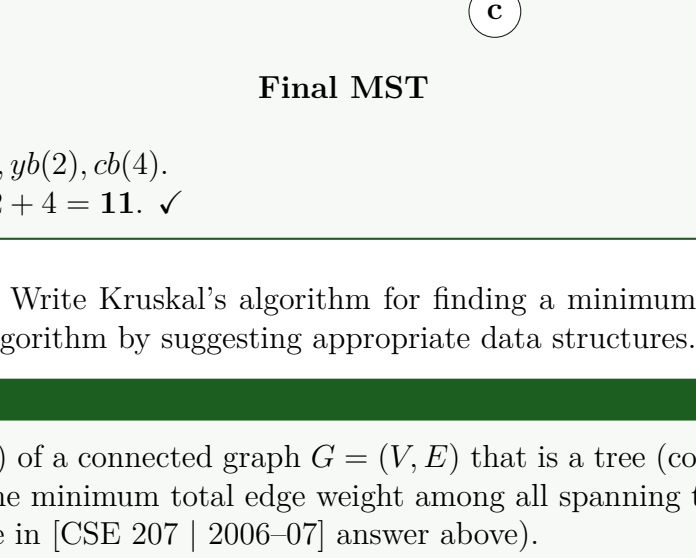
Answer

Spanning tree: a subgraph $T = (V, E')$ of a connected graph $G = (V, E)$ that is a tree (connected, acyclic) containing all vertices.
A *minimum spanning tree* (MST) has the minimum total edge weight among all spanning trees.
Kruskal's algorithm: (see pseudocode in [CSE 207 | 2006–07] answer above).
Complexity with appropriate data structures:

- Sort $|E|$ edges: $O(E \log E)$.
- MAKE-SET for each vertex: $O(V)$.
- $|E|$ FIND and $|V|-1$ UNION operations with union-by-rank + path compression: $O(E\,\alpha(V))$.

Total: $O(E \log E) = O(E \log V)$.

Q19. Find a minimum spanning tree using Prim's algorithm, showing every step. **(10 marks)** [CSE 207 | 2017–18]



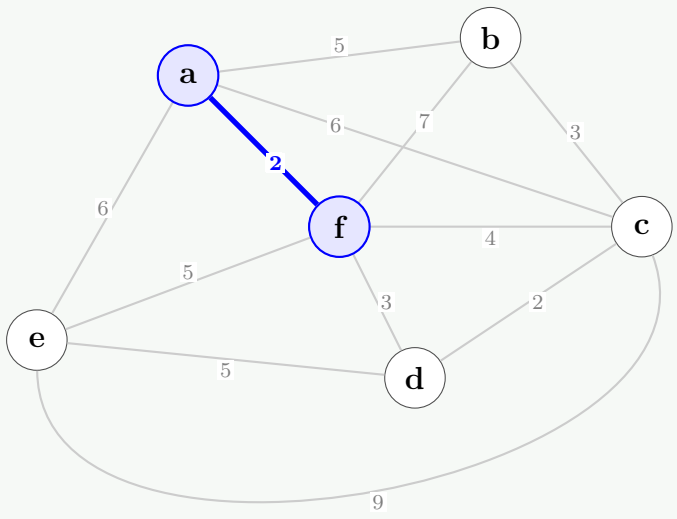
Detailed Answer: Prim's Algorithm Step-by-Step

Prim's Algorithm builds the Minimum Spanning Tree (MST) by starting from an arbitrary node and repeatedly adding the cheapest edge that connects a visited node to an unvisited node.

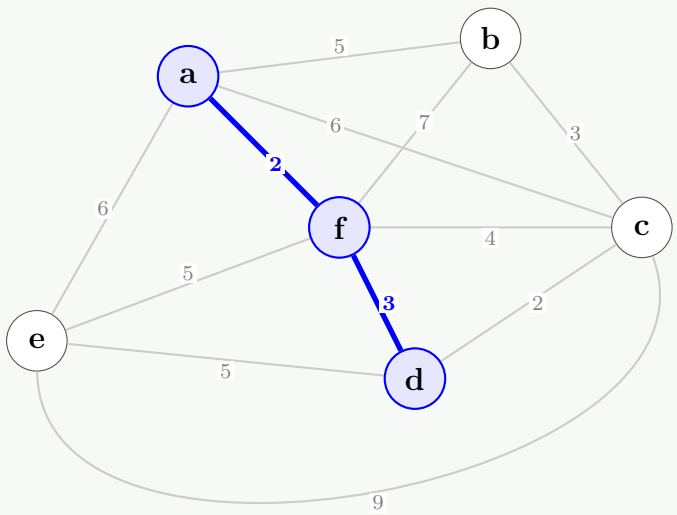
Let's define the nodes and edges. We will start the algorithm from node **a**.

- **Visited Set (V_{vis}):** Keeps track of nodes already in the MST.
- **MST Edges (E_{mst}):** Keeps track of edges added to the MST.

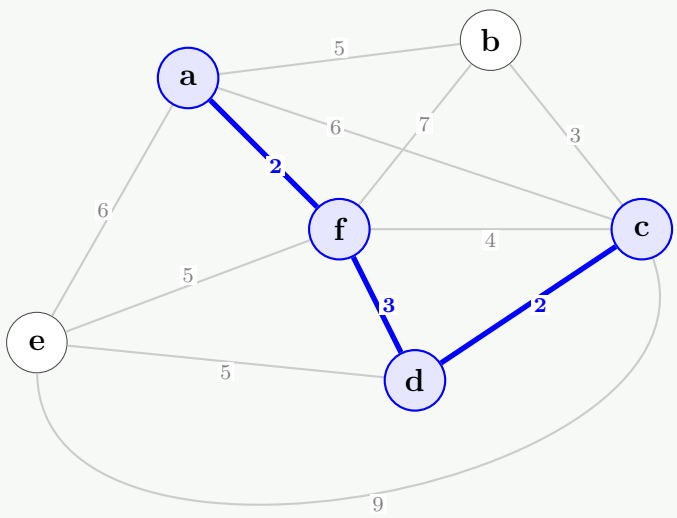
Step 1: Initialization
Start at vertex **a**. $V_{vis} = \{a\}$.
Available edges from visited nodes: $(a, f) = 2, (a, b) = 5, (a, e) = 6, (a, c) = 6$.
Action: Minimum weight is **2**. Add edge **(a, f)** and vertex **f**.



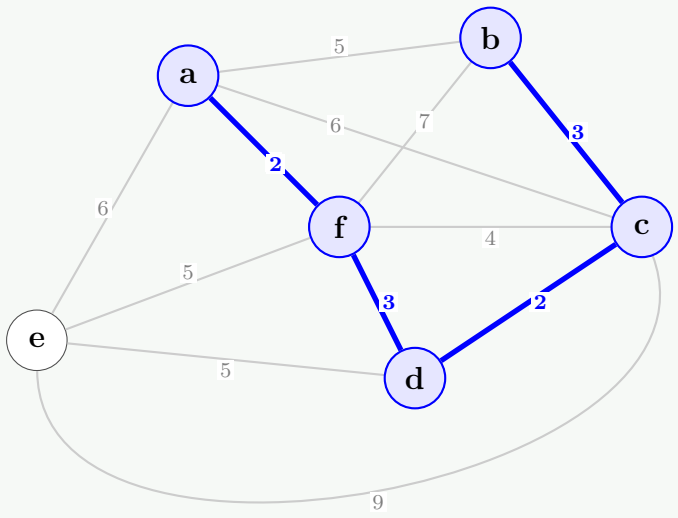
Step 2:
$V_{vis} = \{a, f\}$.
Available crossing edges: $(f, d) = 3, (f, c) = 4, (a, b) = 5, (e, f) = 5, (a, e) = 6, (a, c) = 6, (b, f) = 7$.
Action: Minimum weight is **3**. Add edge **(f, d)** and vertex **d**.



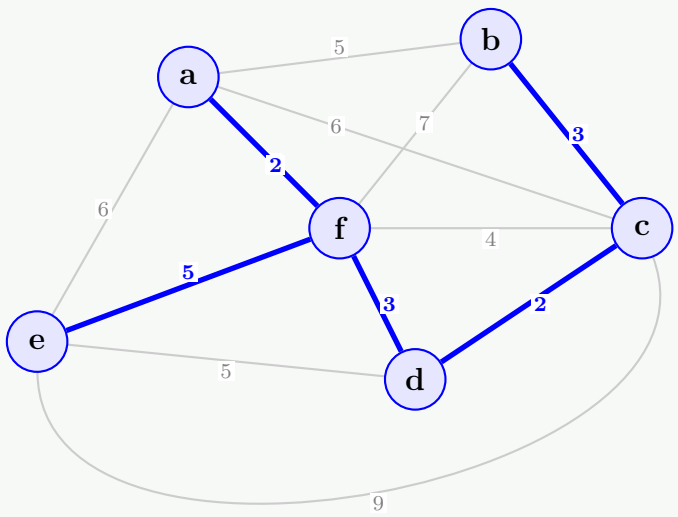
Step 3:
$V_{vis} = \{a, f, d\}$.
Available crossing edges: $(c, d) = 2, (f, c) = 4, (a, b) = 5, (e, f) = 5, (e, d) = 5, \ldots$
Action: Minimum weight is **2**. Add edge **(d, c)** and vertex **c**.



Step 4:
$V_{vis} = \{a, f, d, c\}$.
Available crossing edges: $(b, c) = 3, (a, b) = 5, (e, f) = 5, (e, d) = 5, (c, e) = 9, \ldots$
Action: Minimum weight is **3**. Add edge **(c, b)** and vertex **b**.



Step 5:
$V_{vis} = \{a, f, d, c, b\}$.
Available crossing edges to unvisited node 'e': $(e, f) = 5, (e, d) = 5, (a, e) = 6, (c, e) = 9$.
Action: Minimum weight is **5**. We can pick either (e, f) or (e, d). Let's pick **(e, f)**. Add vertex **e**.



Conclusion: All vertices are now visited ($|V| - 1 = 5$ edges added).
Final MST Edges: $\{(a, f), (f, d), (d, c), (c, b), (e, f)\}$
Total MST Cost: $2 + 3 + 2 + 3 + 5 = \mathbf{15}$. ✓

Q20. Let G be a weighted connected graph and let V_1, V_2 be a partition of its vertices into two disjoint non-empty sets. Let e be an edge in G with minimum weight among those with one endpoint in V_1 and the other in V_2. Show that there is a minimum spanning tree T of G containing e. **(10 marks)** [CSE 207 | 2017–18]

Answer

Theorem (Cut Property). Let G be a weighted connected graph, (V_1, V_2) a partition of vertices, and $e = (u, v)$ the minimum-weight edge with $u \in V_1$, $v \in V_2$. Then some MST of G contains e.
Proof (by exchange argument).
Let T be any MST of G. If $e \in T$, done.
Otherwise, since T is a spanning tree it is connected, so there is a path P from u to v in T. Since $u \in V_1$ and $v \in V_2$, this path must cross the cut at some edge $e' = (x, y)$ with $x \in V_1$, $y \in V_2$. Note $e' \neq e$.
Since e is the *minimum*-weight crossing edge, $w(e) \leq w(e')$.
Construct $T' = T - \{e'\} + \{e\}$:

- **Connected**: removing e' splits T into two components; adding e reconnects them (since e also crosses the cut).
- **Acyclic**: if a cycle existed, it would contain e; but then e' would lie on a cycle in T, contradicting T being a tree.
- **Weight**: $w(T') = w(T) - w(e') + w(e) \leq w(T)$.

Since T is an MST, $w(T') \geq w(T)$, so $w(T') = w(T)$ and T' is also an MST containing e. □

Q21. Huffman's algorithm is used to obtain an encoding of alphabet $\{a, b, c\}$ with frequencies f_a, f_b, and f_c. In each of the following cases, either give an example of frequencies that would yield the specified code, or explain why the code cannot possibly be obtained:

(a) Code: $\{0, 10, 11\}$

(b) Code: $\{0, 1, 00\}$

(c) Code: $\{10, 01, 00\}$

Then construct Huffman codes for the characters with the following frequencies: A:24, B:12, C:10, D:8, E:9, F:37, using the greedy algorithm. **(6+9 marks)** [CSE 203 | 2016–17]

Answer

Validity of given codes for $\{a, b, c\}$:
(a) $\{0, 10, 11\}$: Valid prefix-free code. Huffman tree: root → left = a; root → right = internal node → left = b, right = c. This requires $f_a \geq f_b + f_c$ (so a gets the shorter code). *Example*: $f_a = 5, f_b = 2, f_c = 1$ ✓.
(b) $\{0, 1, 00\}$: **Invalid**. Code 0 is a prefix of code 00 — this violates the prefix-free property required for unambiguous decoding.
(c) $\{10, 01, 00\}$: **Invalid**. All codes have length 2 but there are only $2^1 = 2$ possible codewords of length 1 and $2^2 = 4$ of length 2. However, none of the three codes is a prefix of another — this *is* prefix-free. But a Huffman tree cannot produce three codes of equal length for 3 symbols: a complete binary tree for 3 leaves has depths 1, 2, 2 (not 2, 2, 2). **Cannot be obtained by Huffman coding.**

Huffman codes for A:24, B:12, C:10, D:8, E:9, F:37:
Build steps (merge two smallest frequencies):

(a) Merge D(8) + E(9) = DE(17)

(b) Merge C(10) + B(12) = CB(22)

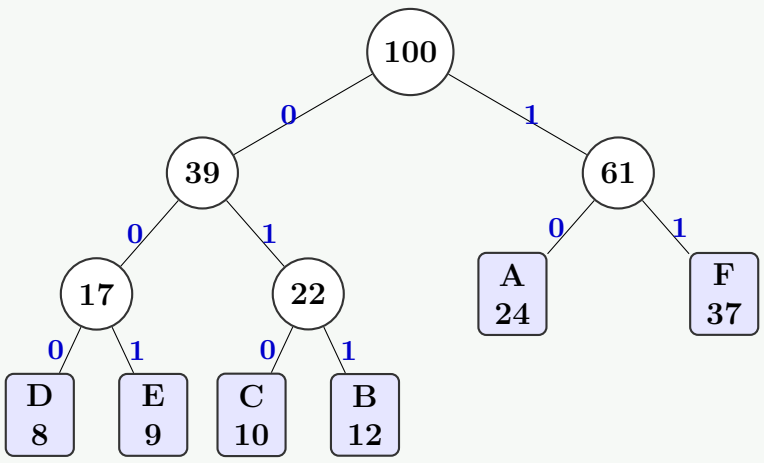
(c) Merge DE(17) + CB(22) = DECB(39)

(d) Merge A(24) + F(37) = AF(61)

(e) Merge DECB(39) + AF(61) = root(100)

Final Huffman codes:

Symbol	Frequency	Code	Bits used
F	37	11	$37 \times 2 = 74$
A	24	10	$24 \times 2 = 48$
B	12	011	$12 \times 3 = 36$
C	10	010	$10 \times 3 = 30$
E	9	001	$9 \times 3 = 27$
D	8	000	$8 \times 3 = 24$
		Total bits:	239



Q22. Suppose you are given n white and n black dots, lying on a line. The dots are equally spaced but they may appear in any order of black and white (black and white may be interleaved). Design a greedy algorithm to connect each black dot with a (different) white dot so that the total length of wires used to connect the dots is minimal. The length of wire used to connect two dots is equal to their distance along the line. Prove that your algorithm is optimal.

●	●	○	●	○	○	○	●
1	2	3	4	5	6	7	8

For example, an optimal solution to solve the above example is $(1, 3)$, $(2, 5)$, $(4, 6)$, $(7, 8)$ with total wire length $= 2 + 3 + 2 + 1 = 8$. **(15 marks)** [CSE 203 | 2016–17]

Answer

Greedy algorithm: sort black dots and white dots separately; match the i-th black with the i-th white (in sorted order).
Example: Blacks $= \{1, 2, 4, 8\}$, Whites $= \{3, 5, 6, 7\}$. Sorted matching: $(1, 3), (2, 5), (4, 6), (8, 7)$. Total length $= 2 + 3 + 2 + 1 = \mathbf{8}$. ✓

Proof of optimality (exchange argument).
Let $B = b_1 < b_2 < \cdots < b_n$ and $W = w_1 < w_2 < \cdots < w_n$ be the sorted positions. The greedy solution is $M_G = \{(b_i, w_i)\}$.
Suppose an optimal matching M^* contains a *crossing pair*: (b_i, w_j) and (b_k, w_l) with $b_i < b_k$ but $w_j > w_l$ (i.e. the wires cross). Then:

$$|b_i - w_j| + |b_k - w_l| \geq |b_i - w_l| + |b_k - w_j|.$$

(This is because uncrossing two wires never increases total length — a standard geometric argument: for $a < b$ and $c > d$, $|a - c| + |b - d| \geq |a - d| + |b - c|$.)
Swapping to remove the crossing does not increase total cost, and repeated swaps yield the sorted matching M_G. Hence $\text{cost}(M_G) \leq \text{cost}(M^*)$, so M_G is optimal. □

Complexity Analysis. Sorting: $O(n \log n)$. Matching: $O(n)$. Total: $O(n \log n)$.

Q23. Write Prim's algorithm that computes a minimum spanning tree of a graph. Analyze its time complexity. **(15 marks)** [CSE 207 | 2016–17]

Answer

Algorithm 102 PRIM(G, w, r)

```
1: for each u ∈ V do
2:     key[u] ← ∞; π[u] ← NIL
3: end for
4: key[r] ← 0
5: Q ← V                                   ▷ min-priority queue keyed by key[]
6: while Q ≠ ∅ do
7:     u ← EXTRACT-MIN(Q)
8:     for each v ∈ Adj[u] do
9:         if v ∈ Q and w(u, v) < key[v] then
10:            π[v] ← u; key[v] ← w(u, v)
11:            DECREASE-KEY(Q, v, key[v])
12:        end if
13:    end for
14: end while
```

Complexity Analysis. With binary heap: $O(E \log V)$. With Fibonacci heap: $O(V \log V + E)$.

Q24. Write the correctness proof of Kruskal's algorithm for finding a minimum spanning tree. **(10 marks)** [CSE 207 | 2016–17]

Answer

Theorem: Kruskal's algorithm produces an MST.
Proof: we show by induction that the edge set A maintained by Kruskal is always a subset of some MST.
Invariant: at each step, $A \subseteq T$ for some MST T.
Base: $A = \emptyset \subseteq T$. ✓
Step: suppose $A \subseteq T$ and Kruskal considers edge $e = (u, v)$ (next in sorted order).
Case 1: $e \in T$. Then $A \cup \{e\} \subseteq T$. ✓
Case 2: $e \notin T$. Adding e to T creates a cycle C. Since e connects two components of the current forest (not yet causing a cycle in A), there is an edge $e' \in C$ with $e' \notin A$ crossing the same cut. Since edges are processed in sorted order, $w(e) \leq w(e')$. $T' = T - \{e'\} + \{e\}$ is a spanning tree with $w(T') \leq w(T)$, so T' is also an MST. $A \cup \{e\} \subseteq T'$. ✓
At termination, A has $|V| - 1$ edges and is acyclic $\rightarrow$ A is an MST. □

Q25. Compare the key properties of problems that can be solved with greedy method, divide-and-conquer method, and dynamic programming method. **(10 marks)** [CSE 207 | 2016–17]

Answer

Property	Greedy	Divide-and-Conquer	Dynamic Programming
Decision	Local optimum at each step	Split into independent sub-problems	Solve overlapping sub-problems once
Sub-problems	Not solved recursively	Non-overlapping; solved recursively	Overlapping; stored (memoised)
Key property	Greedy-choice + optimal substructure	Optimal substructure	Optimal substructure + overlapping subproblems
Efficiency	Often $O(n \log n)$	Typically $O(n \log n)$	Polynomial (table-based)
Examples	Activity selection, Huffman, Prim, Kruskal	Merge sort, binary search	Knapsack, LCS, Floyd-Warshall

Q26. There is a network of roads $G = (V, E)$ connecting a set of cities V. Each road in E has an associated length l_e. There is a proposal to add one new road to this network, and there is a list E' of pairs of cities between which the new road can be built. Each such potential road $e' \in E'$ has an associated length. As a designer for the public works department you are asked to determine the road $e' \in E'$ whose addition to existing network G would result in the maximum decrease in the driving distance between two fixed cities s and t in the network. Give an algorithm for solving this problem. Your algorithm should run in $O((V + E + E') \log V)$ time. **(15 marks)** **[CSE 207 | 2015–16]**

Answer

Algorithm:

(a) Run Dijkstra from s on $G \to d_s[v]$ for all v. Time: $O((V + E) \log V)$.

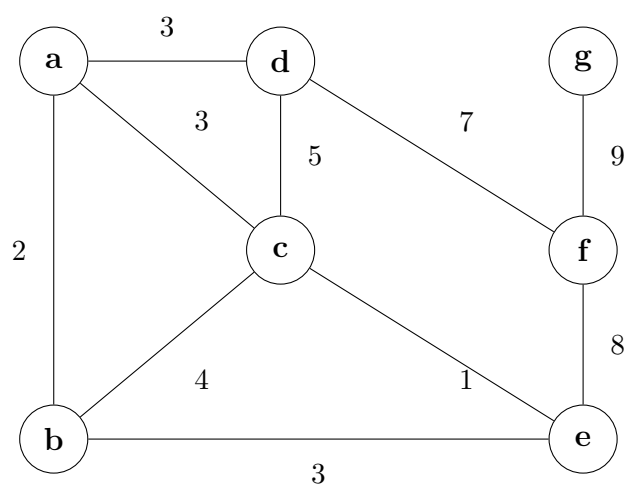
(b) Run Dijkstra from t on G^T (reverse all edges) $\to d_t[v]$ for all v (= distance from v to t). Time: $O((V + E) \log V)$.

(c) For each candidate road $e' = (a, b, \ell_{e'}) \in E'$: new s-t distance via $e' = \min(d_s[a] + \ell_{e'} + d_t[b],\ d_s[b] + \ell_{e'} + d_t[a])$. Time: $O(E')$.

(d) Return the $e' \in E'$ that minimises the above.

Total: $O((V + E) \log V + E') = O((V + E + E') \log V)$. □

Q27. Find a minimum spanning tree by running (i) Kruskal's algorithm and (ii) Prim's algorithm. Show the order of edges selected in each case (use alphabetical ordering when there is a tie). **(15 marks)** **[CSE 207 | 2015–16]**



Detailed Answer: Kruskal's & Prim's Algorithm

Part 1: Kruskal's Algorithm

Step 1: Sort edges by weight. When weights are equal, sort alphabetically (e.g., ac before ad).
Sorted list: $ce(1), ab(2), ac(3), ad(3), be(3), bc(4), cd(5), df(7), ef(8), fg(9)$.

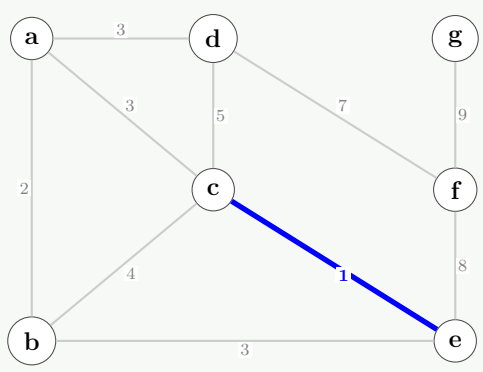
Part 1: Kruskal's Algorithm Step-by-Step

Step 1: Sort edges by weight. When weights are equal, sort alphabetically (e.g., ac before ad).
Sorted list: $ce(1), ab(2), ac(3), ad(3), be(3), bc(4), cd(5), df(7), ef(8), fg(9)$.

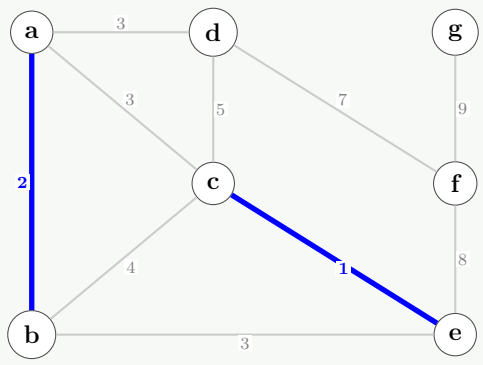
Edge	Weight	Add to MST?	Reason
c–e	1	**Yes**	No cycle
a–b	2	**Yes**	No cycle
a–c	3	**Yes**	No cycle
a–d	3	**Yes**	No cycle
b–e	3	No	Forms cycle (b-a-c-e-b)
b–c	4	No	Forms cycle (b-a-c-b)
c–d	5	No	Forms cycle (c-a-d-c)
d–f	7	**Yes**	No cycle
e–f	8	No	Forms cycle (e-c-a-d-f-e)
f–g	9	**Yes**	No cycle. **Done** ($\|V\| - 1 = 6$ edges)

Visualizing Kruskal's Progress (Step-by-Step):

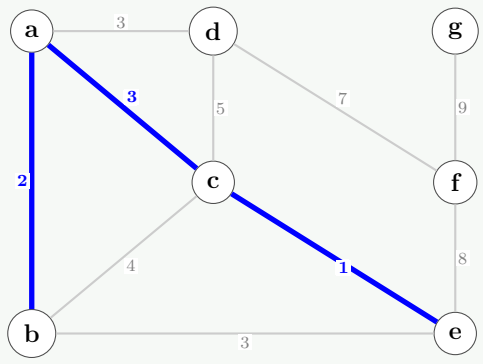
Step 1: Add edge **ce(1)**.



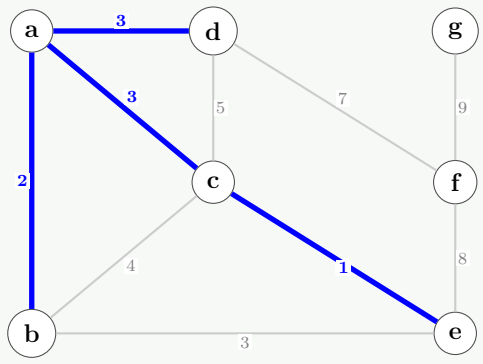
Step 2: Add edge **ab(2)**.



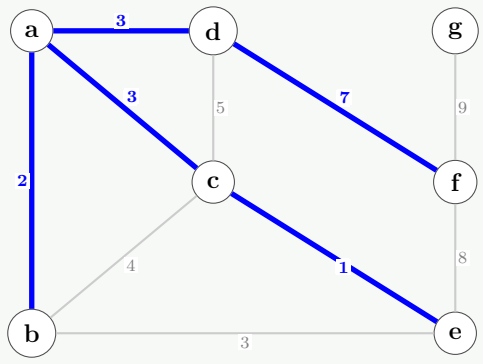
Step 3: Add edge **ac(3)**.



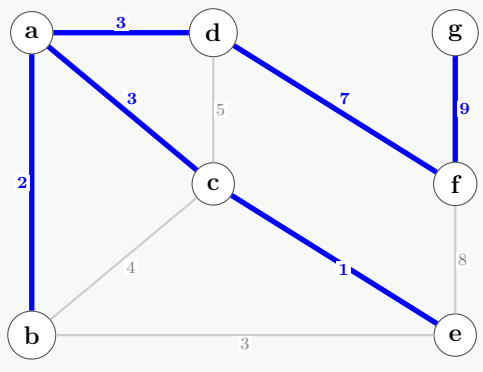
Step 4: Add edge **ad(3)**. (Edges be, bc, cd are discarded next as they form cycles).



Step 5: Add edge **df(7)**. (Edge ef is discarded next as it forms a cycle).



Step 6: Final MST. Add edge **fg(9)**.



Kruskal MST Cost: $1 + 2 + 3 + 3 + 7 + 9 = \mathbf{25}$.

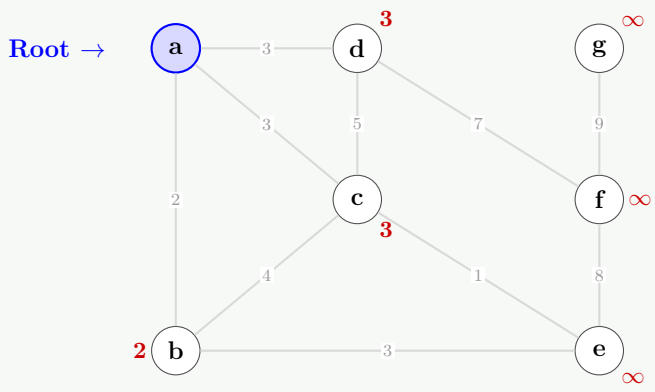
Part 2: Prim's Algorithm (Using Key Values)

Start at vertex **a**. We maintain a **Key** value for each vertex (initialized to ∞, meaning unreachable) and its **Parent**. In each step, we extract the unvisited vertex with the minimum key. Ties are broken alphabetically.

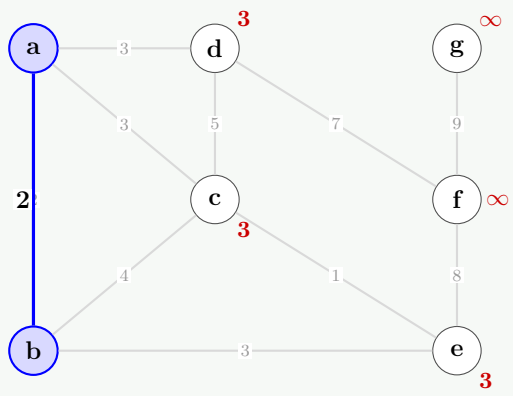
Key and Parent Tracking Table:
Notation: `Key(Parent)`. '-' means the vertex is already included in the MST.

Step	Extracted Node	b	c	d	e	f	g
0	Initial State	∞	∞	∞	∞	∞	∞
1	**a**	$2(a)$	$3(a)$	$3(a)$	∞	∞	∞
2	**b**	-	$3(a)$	$3(a)$	$3(b)$	∞	∞
3	**c**	-	-	$3(a)$	$\mathbf{1(c)}$	∞	∞
4	**e**	-	-	$3(a)$	-	$8(e)$	∞
5	**d**	-	-	-	-	$\mathbf{7(d)}$	∞
6	**f**	-	-	-	-	-	$\mathbf{9(f)}$
7	**g**	-	-	-	-	-	-

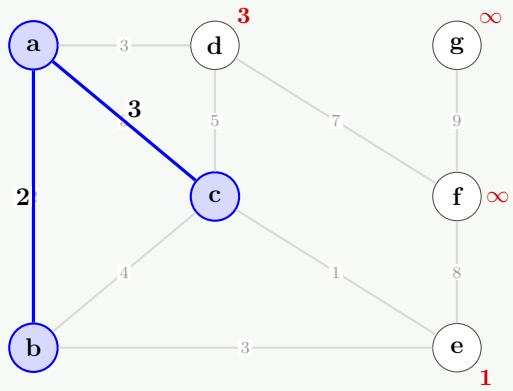
Intermediate Steps Visualization:
(Blue shaded nodes are in the MST. Red values indicate the current minimum cost to connect that unvisited node).



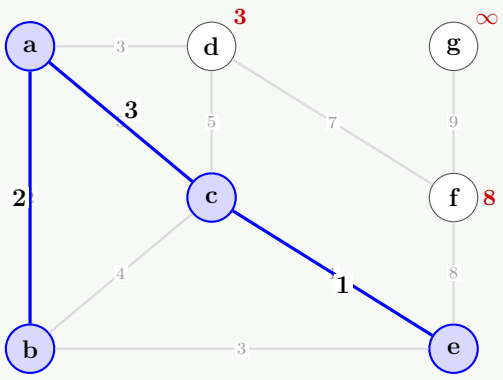
Step 0: Add a



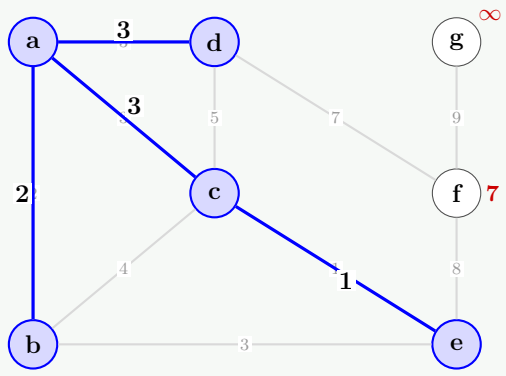
Step 1: Add b



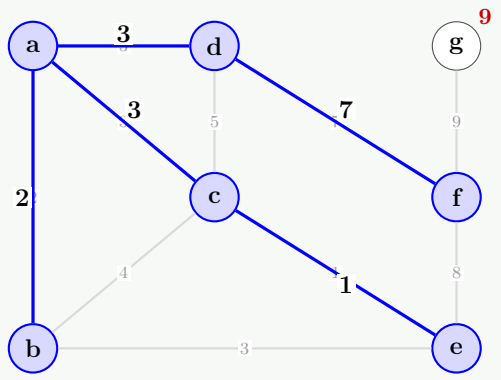
Step 2: Add c



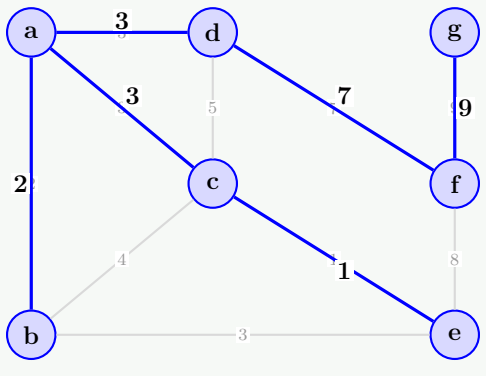
Step 3: Add e



Step 4: Add d



Step 5: Add f



Step 6: Add g (Final MST)

Prim MST edges: $ab(2)$, $ac(3)$, $ce(1)$, $ad(3)$, $df(7)$, $fg(9)$.
Total Cost $= 2+3+1+3+7+9 = \mathbf{25}$.

Q28. Under a Huffman encoding of n symbols with frequencies $f_1, f_2, \ldots, f_n$, what is the longest a codeword could possibly be? Give an example set of frequencies that would produce this case. Then construct Huffman codes for the following frequencies: A:45, B:13, C:12, D:16, E:5, F:9, using the greedy algorithm. **(7+8=15 marks)** [CSE 207 | 2015–16]

Answer

Longest codeword in Huffman encoding.
In the worst case, the Huffman tree is maximally unbalanced (like a Fibonacci-weighted tree). For n symbols, the longest codeword has length $n-1$.
Example: Fibonacci frequencies $f_1 = 1, f_2 = 1, f_3 = 2, f_4 = 3, f_5 = 5, \ldots$ produce a completely skewed tree where the rarest symbol gets a codeword of length $n-1$.
For $n = 5$ with $f = (1, 1, 2, 3, 5)$: longest codeword $= 4$.
Huffman codes for A:45, B:13, C:12, D:16, E:5, F:9:
Merge steps:

(a) $E(5) + F(9) = EF(14)$

(b) $C(12) + B(13) = CB(25)$

(c) $EF(14) + D(16) = EFD(30)$

(d) $CB(25) + EFD(30) = CBEFD(55)$

(e) $A(45) + CBEFD(55) = \text{root}(100)$

Symbol	Freq	Code	Bits
A	45	0	$45 \times 1 = 45$
C	12	100	$12 \times 3 = 36$
B	13	101	$13 \times 3 = 39$
E	5	1100	$5 \times 4 = 20$
F	9	1101	$9 \times 4 = 36$
D	16	111	$16 \times 3 = 48$
		Total:	**224**

Q29. Let $G = (V, E)$ be a connected undirected graph with weight function w on E. Let $A \subseteq E$ be included in some MST for G, let $(S, V - S)$ be any cut that respects A, and let (u, v) be a light edge crossing $(S, V - S)$. Prove that (u, v) is safe for A. **(10 marks)** **[CSE 207 | 2014–15]**

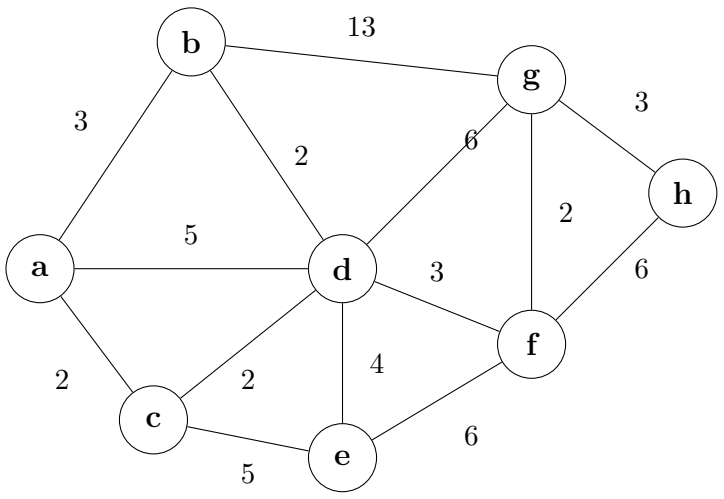
Answer

Theorem (Safe-Edge Lemma / Cut Property). Let $G = (V, E, w)$ connected, $A \subseteq E$ included in some MST, $(S, V \setminus S)$ a cut that *respects* A (no edge of A crosses it), and (u, v) a light crossing edge. Then (u, v) is safe for A: there exists an MST containing $A \cup \{(u, v)\}$.

Proof. Let T be any MST containing A. If $(u, v) \in T$, done. Otherwise, adding (u, v) to T creates a cycle C. Since $u \in S$ and $v \notin S$, the path in T from u to v must cross the cut; let (x, y) be any such crossing edge ($(x, y) \neq (u, v)$, and $(x, y) \notin A$ since the cut respects A).

$T' = T - \{(x, y)\} + \{(u, v)\}$ is a spanning tree. Since (u, v) is *light*: $w(u, v) \leq w(x, y)$, so $w(T') \leq w(T)$. Since T is an MST, $w(T') \geq w(T)$, hence $w(T') = w(T)$ and T' is an MST. T' contains A (since $(x, y) \notin A$) and (u, v). Therefore (u, v) is safe for A. □

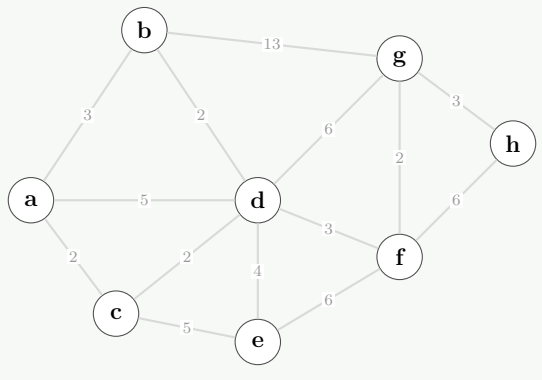
Q30. Find the light and respectful crossing edges for each step of Kruskal's method for building the MST. **(12 marks)** **[CSE 207 | 2014–15]**



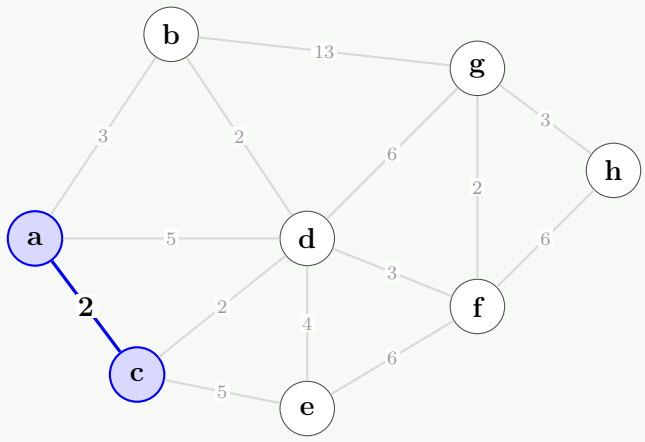
Answer: Kruskal's Algorithm (Step-by-Step)

Sorted Edges (Ascending Order):
$ac(2), bd(2), cd(2), gf(2), ab(3), df(3), gh(3), de(4), ad(5), ce(5), dg(6), ef(6), hf(6), bg(13)$.

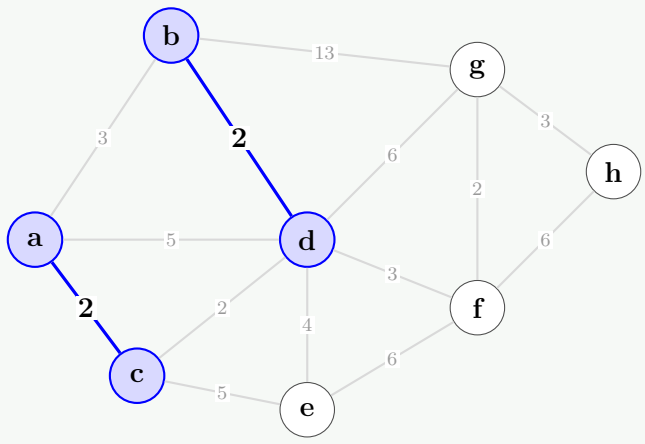
Intermediate Steps Visualization:
(Blue shaded nodes and thick blue lines indicate edges added to the MST forest. We add the smallest available edge that does not form a cycle).



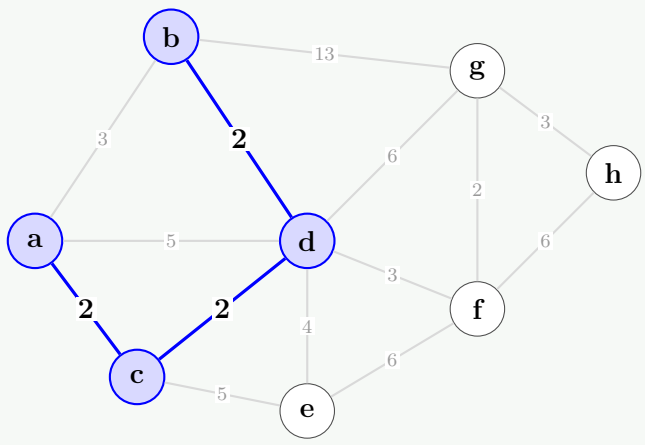
Step 0: Initial Graph



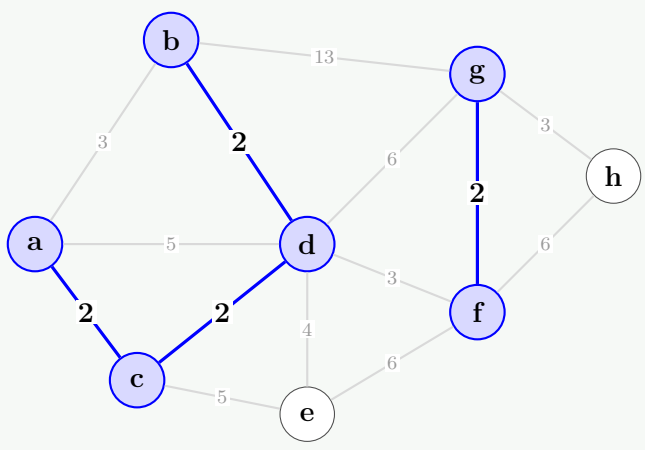
Step 1: Add ac(2)



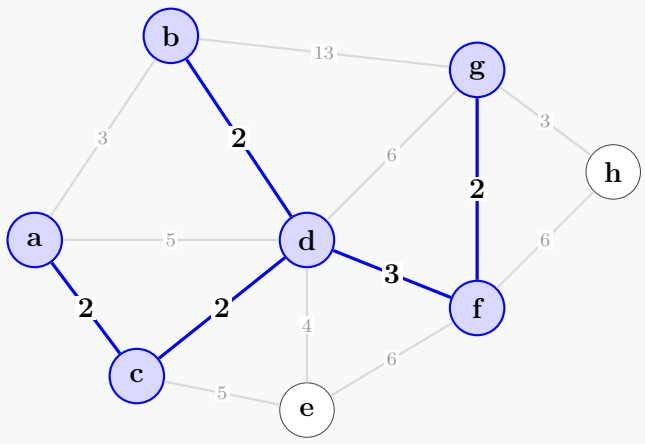
Step 2: Add bd(2)



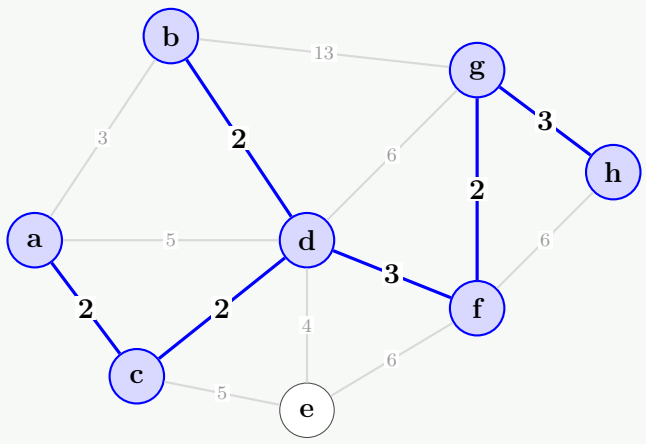
Step 3: Add cd(2)



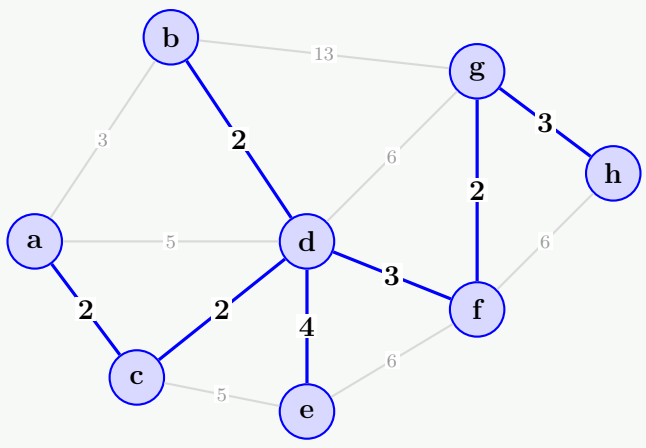
Step 4: Add gf(2)



Step 5: Skip ab(3) [Cycle]. Add df(3)



Step 6: Add gh(3)



Step 7: Add de(4) (Final MST)

Kruskal MST Edges: $ac(2), bd(2), cd(2), gf(2), df(3), gh(3), de(4)$.
Total Cost $= 2+2+2+2+3+3+4 =$ **18**.

Q31. What is the cost of finding and joining (union) of sets in Kruskal's method? **(8 marks)** [CSE 207 | 2014–15]

Answer

With **union by rank** and **path compression**:

- MAKE-SET: $O(1)$ each, V times $\rightarrow O(V)$.
- FIND (with path compression): $O(\alpha(V))$ amortised, where α is the inverse Ackermann function (practically constant).
- UNION (by rank): $O(\alpha(V))$ amortised.
- Total for E Find+Union operations: $O(E\,\alpha(V))$.

Combined with sorting edges ($O(E \log E)$), total Kruskal cost:

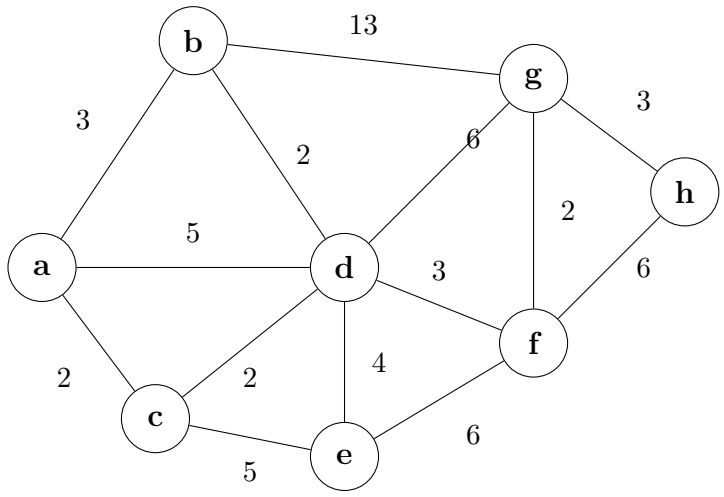
$$O(E \log E + E\,\alpha(V)) = O(E \log V).$$

(Since $E \leq V^2$, $\log E \leq 2 \log V$.)

Q32. For the graph below, show the value of u, $v.key$, Q for the following algorithm (show each step): **(10 marks)** [CSE 207 | 2014–15]

Algorithm 103 MST(G, w, r)

```
1: Q ← V[G]
2: for each u ∈ Q do
3:     u.key ← ∞
4: end for
5: r.key ← 0;   p[r] ← nil
6: while Q ≠ ∅ do
7:     u ← Extract-Min(Q)
8:     for each v ∈ Adj[u] do
9:         if v ∈ Q and w(u, v) < v.key then
10:            p[v] ← u
11:            v.key ← w(u, v)
12:        end if
13:    end for
14: end while
```

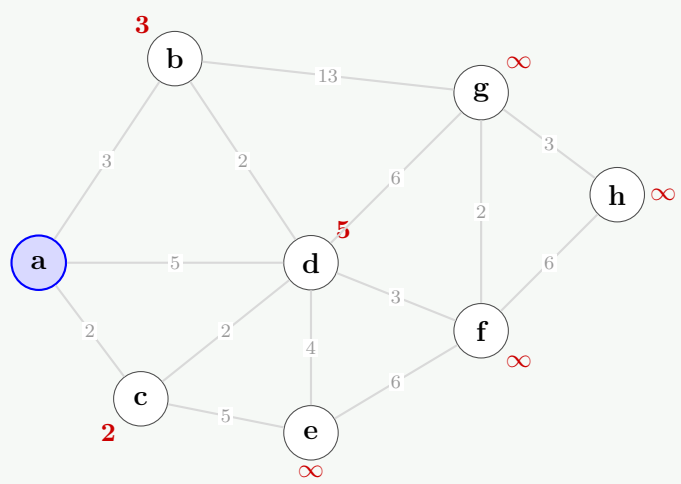


Answer: Prim's Algorithm (Step-by-Step Visualization)

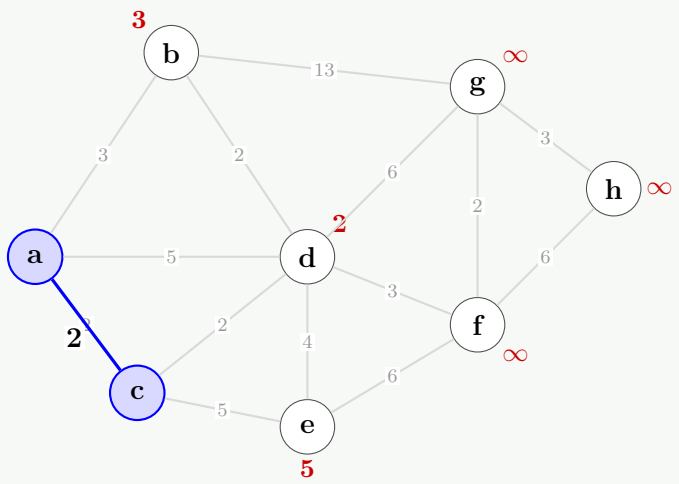
Starting from a. At each step: u = vertex extracted, v.key updated.

Step	u extracted	Edge added	Key values after
1	a (key=0)	—	b:3, c:2, d:5
2	c (key=2)	a–c	b:3, d:2, e:5
3	d (key=2)	c–d	b:2, e:4, f:3, g:6
4	b (key=2)	d–b	e:4, f:3, g:6
5	f (key=3)	d–f	e:4, g:2, h:6
6	g (key=2)	f–g	e:4, h:3
7	h (key=3)	g–h	e:4
8	e (key=4)	d–e	—

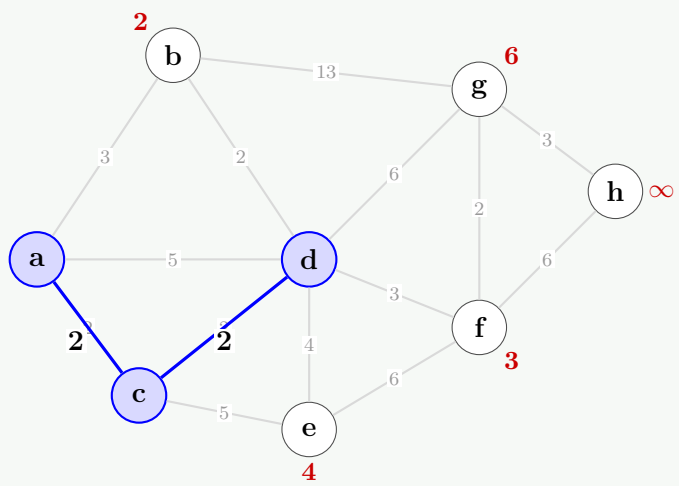
Intermediate Steps Visualization:
(Blue shaded nodes are currently in the MST. Red values indicate the current $v.key$ to connect that unvisited node to the MST).



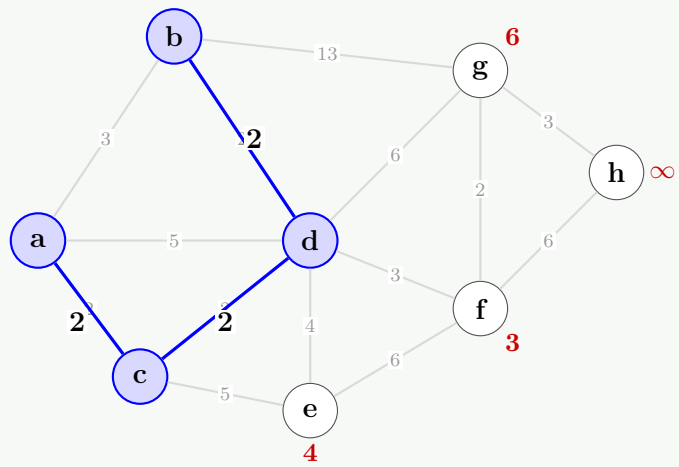
Step 1: Extract a (key=0). Update neighbors.



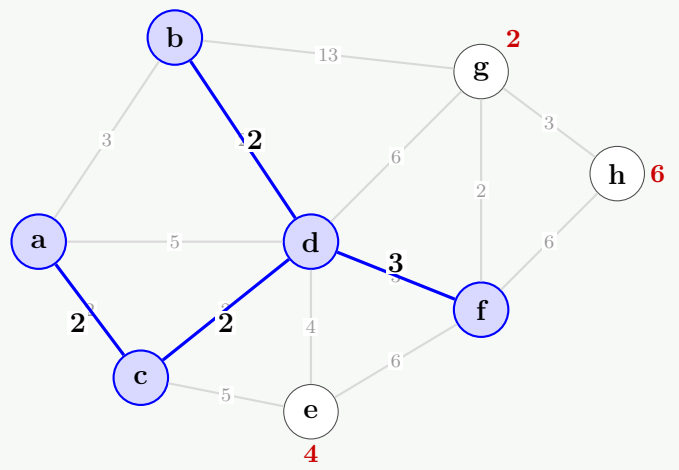
Step 2: Extract c (key=2). Add ac(2).



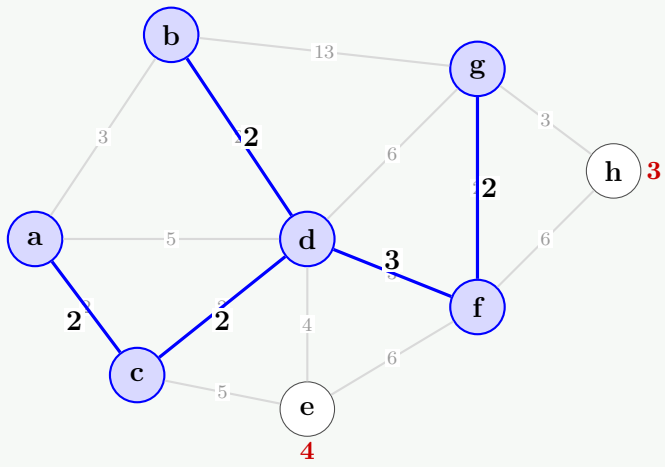
Step 3: Extract d (key=2). Add cd(2).



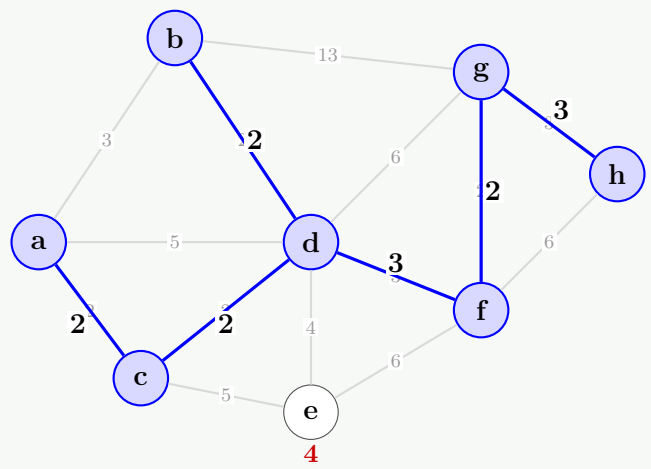
Step 4: Extract b (key=2). Add db(2).



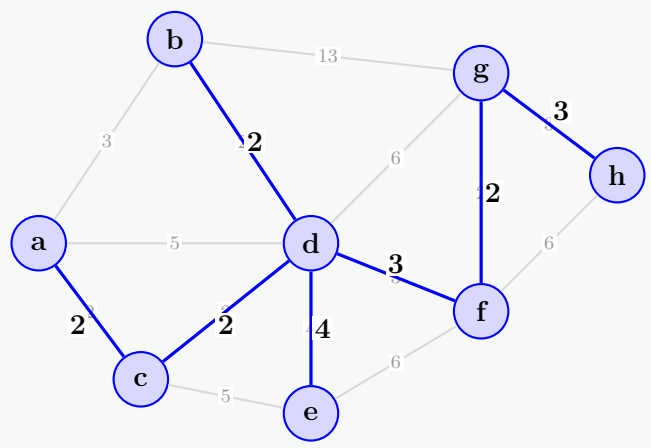
Step 5: Extract f (key=3). Add df(3).



Step 6: Extract g (key=2). Add fg(2).



Step 7: Extract h (key=3). Add gh(3).



Step 8: Extract e (key=4). Add de(4). (Final MST)

MST edges: $ac(2), cd(2), db(2), df(3), fg(2), gh(3), de(4)$.
Total cost $= 2+2+2+3+2+3+4 =$ **18**. ✓

Q33. *GreedyCell*, a mobile phone operator, wants to place cell towers (base stations) along a long straight country road, to provide network coverage to n houses sparsely scattered along the road. The distance of these houses from the start of the road are, in miles and in increasing order, $d_1, d_2, \ldots, d_n$. Each cell tower has coverage of R miles, i.e., if a house is within R miles of a tower, it would get service from that tower. Now give an efficient algorithm for finding a placement of cell towers that uses as few cell towers as possible to provide coverage to all the n houses.**(12 marks)** [CSE 207 | 2014–15]

Answer

Greedy algorithm: place each tower to cover as many uncovered houses as possible.

Algorithm 104 CellTowers($d_1 \le \cdots \le d_n, R$)

```
1: i ← 1; towers ← 0
2: while i ≤ n do
3:     Place tower at position p = d_i + R                ▷ covers [d_i, d_i + 2R]
4:     towers ← towers + 1
5:     Advance i past all houses with d_i ≤ p + R
6: end while
7: return towers
```

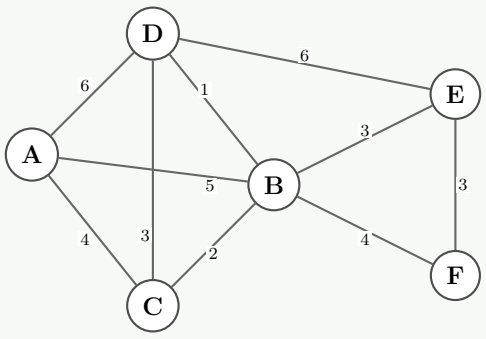
Correctness: house d_i is uncovered; place the tower as far right as possible while still covering d_i (at $d_i + R$). This maximises coverage of future houses. Any algorithm needs ≥ 1 tower for d_i; our placement covers strictly more future houses than placing it closer to d_i. By exchange argument, the greedy is optimal. □

Complexity Analysis. $O(n)$ after the sorted input is given (single pass).

Q34. Find a minimum spanning tree using both Kruskal's algorithm and Prim's algorithm (starting from A) for the graph with edges: $AB(5), BC(2), AC(4), AD(6), CD(3)$, $DB(1), DE(6), EB(3), EF(3), FB(4)$. **(15 marks)** [CSE 207 | 2013–14]

Answer: Minimum Spanning Tree (Kruskal & Prim)

1. Original Graph:

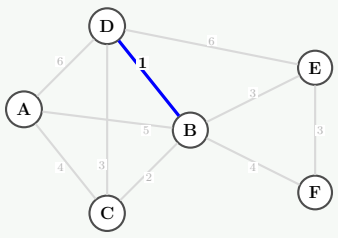


Part A: Kruskal's Algorithm

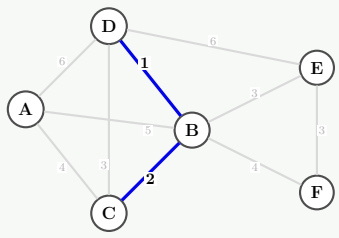
Sorted Edges: $DB(1), BC(2), CD(3), EB(3), EF(3), AC(4), FB(4), AB(5), AD(6), DE(6)$.

Edge	Weight	Add to MST?	Reason
D–B	1	**Yes**	Does not form a cycle
B–C	2	**Yes**	Does not form a cycle
C–D	3	**No**	Forms cycle (B–C–D)
E–B	3	**Yes**	Does not form a cycle
E–F	3	**Yes**	Does not form a cycle
A–C	4	**Yes**	Connects node A
F–B	4	**No**	Forms cycle (E–F–B)
A–B	5	**No**	Forms cycle (A–C–B)

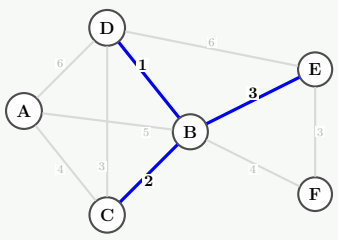
Kruskal's Step-by-Step:



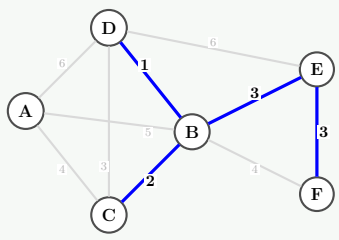
Step 1: Add DB(1)



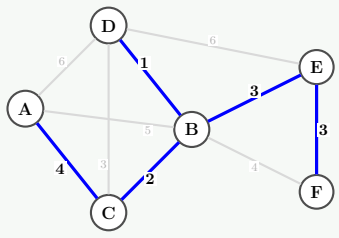
Step 2: Add BC(2)



Step 3: Skip CD. Add EB(3)



Step 4: Add EF(3)



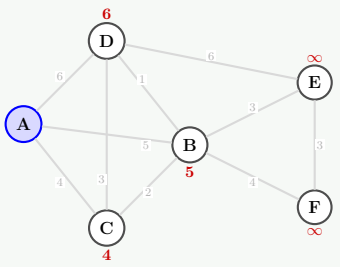
Step 5: Add AC(4). (Final MST)

Kruskal MST Edges: $DB(1), BC(2), EB(3), EF(3), AC(4)$.
Total Cost $= 1 + 2 + 3 + 3 + 4 =$ **13**.

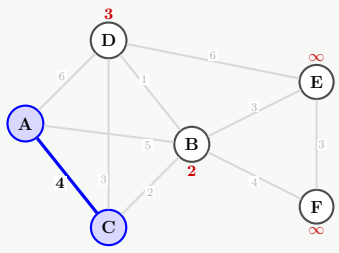
Part B: Prim's Algorithm (Starting from A)

Step	Vertex Extracted	Edge Added	Key Updates (Neighbor : Cost)
1	A	—	$C:4, B:5, D:6$
2	C	A–C (4)	$B:2, D:3$
3	B	C–B (2)	$D:1, E:3, F:4$
4	D	B–D (1)	$E:3$ (no change)
5	E	B–E (3)	$F:3$
6	F	E–F (3)	—

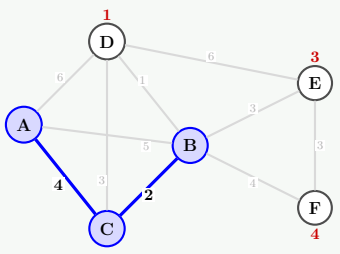
Prim's Step-by-Step:



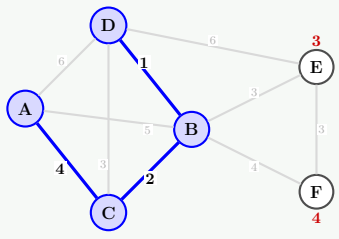
Step 1: Extract A. Update C,B,D



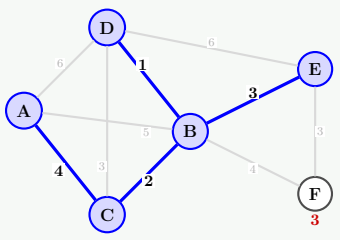
Step 2: Extract C. Add AC(4)



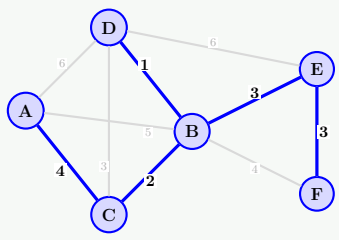
Step 3: Extract B. Add CB(2)



Step 4: Extract D. Add BD(1)



Step 5: Extract E. Add BE(3)



Step 6: Extract F. Add EF(3)

Prim MST Edges: $AC(4), CB(2), BD(1), BE(3), EF(3)$.
Total Cost $= 4+2+1+3+3 = \mathbf{13}$. ✓

Q35. Given a set of jobs with start and finish times, write down an algorithm so that the maximum number of jobs can be processed. Show counterexamples for some natural greedy approaches that will not find optimal solutions. **(10 marks)** [CSE 207 | 2013–14]

Answer

Correct algorithm: sort by finish time; pick greedily.
Wrong strategy 1: Earliest start time. Counterexample: intervals $[1,10], [2,3], [4,5]$. Earliest start picks $[1,10]$ first → only 1 activity. Optimal: $[2,3], [4,5]$ → 2 activities. ×
Wrong strategy 2: Shortest duration. Counterexample: $[1,4], [3,5], [4,7]$. Shortest duration picks $[3,5]$ (length 2) first → then neither $[1,4]$ nor $[4,7]$ overlaps with it, giving $\{[3,5],[4,7]\} = 2$ activities. But earliest-finish gives $\{[1,4],[4,7]\} = 2$ activities too here; for a cleaner counterexample: $[1,3], [2,4], [3,5]$ — shortest-duration picks $[1,3], [3,5] = 2$; but greedy also gives 2.
Better: $[1,5], [3,4], [6,10]$. Shortest: $[3,4], [6,10] = 2$. Earliest-finish: $[3,4], [6,10] = 2$ also.
Clear counterexample: intervals $[1,100], [2,3], [4,5], [6,7]$. Earliest-start picks $[1,100]$ → 1 activity. Shortest picks $[2,3], [4,5], [6,7]$ → 3 activities. Neither earliest-start *nor* shortest is always optimal without the finish-time criterion.
Wrong strategy 3: Fewest overlaps (pick interval overlapping fewest others). Counterexample constructible with 3 "hub" intervals and many small ones — picking the hub with fewest overlaps can block many short non-overlapping ones.

Q36. Given the message `abaacbdbebfbedcect`, construct a dynamic Huffman tree showing how the tree is updated after sending each symbol and the code of each symbol. **(15 marks)** [CSE 207 | 2013–14]

Answer

Message: `a b a a c b d b e b f b e d c e c t`
Final frequencies: a=3, b=5, c=3, d=2, e=3, f=1, t=1. Total = 18.

Dynamic (Adaptive) Huffman Encoding — Key idea:
Start with an empty tree containing only the NYT (Not Yet Transmitted) node. For each symbol:

- If **new symbol**: transmit NYT code + fixed symbol bits. Add a new leaf; split NYT.
- If **seen before**: transmit current Huffman code for the symbol. Increment its count.

- After each transmission, **update the tree**: increment the count of the symbol's leaf and all ancestors; swap nodes to maintain the *sibling property* (nodes ordered by weight).

Symbol-by-symbol trace (abbreviated):

Step	Symbol	Code sent	Tree update
1	a	NYT + `a` bits	add leaf(a,1); NYT splits
2	b	NYT + `b` bits	add leaf(b,1); rebalance
3	a	current code of a	freq(a)=2; swap if needed
4	a	current code of a	freq(a)=3
5	c	NYT + `c` bits	add leaf(c,1)
6	b	current code of b	freq(b)=2
7	d	NYT + `d` bits	add leaf(d,1)
8	b	current code of b	freq(b)=3
9	e	NYT + `e` bits	add leaf(e,1)
10	b	current code of b	freq(b)=4
11	f	NYT + `f` bits	add leaf(f,1)
12	b	current code of b	freq(b)=5
13	e	current code of e	freq(e)=2
14	d	current code of d	freq(d)=2
15	c	current code of c	freq(c)=2
16	e	current code of e	freq(e)=3
17	c	current code of c	freq(c)=3
18	t	NYT + `t` bits	add leaf(t,1)

Final Huffman tree is built from frequencies: b=5, a=3, c=3, e=3, d=2, f=1, t=1.
Building greedily: merge f(1)+t(1)=2, then d(2)+ft(2)=4, then a(3)+c(3)=6, then e(3)+dft(4)=7, then b(5)+ac(6)=11, then root=11+7=18.
Final codes (one possible assignment): b=`00`, a=`010`, c=`011`, e=`10`, d=`110`, f=`1110`, t=`1111`.

Q37. Suppose you are given n tasks with start times $S[1 \ldots n]$ and finish times $F[1 \ldots n]$. Schedule as many tasks as possible so that no two overlap. (i) Describe a greedy algorithm that finds the maximum number of non-overlapping tasks. (ii) Prove the correctness of your greedy algorithm. **(15 marks)** [CSE 207 | 2012–13]

Answer

(i) Greedy Algorithm — Earliest Finish Time:

Algorithm 105 ActivitySelect(S, F, n)

```
Sort tasks by finish time: F[1] ≤ F[2] ≤ ··· ≤ F[n]
selected ← {1};  last ← 1
for i = 2 to n do
    if S[i] ≥ F[last] then
        selected ← selected ∪ {i}
        last ← i
    end if
end forreturn selected
```

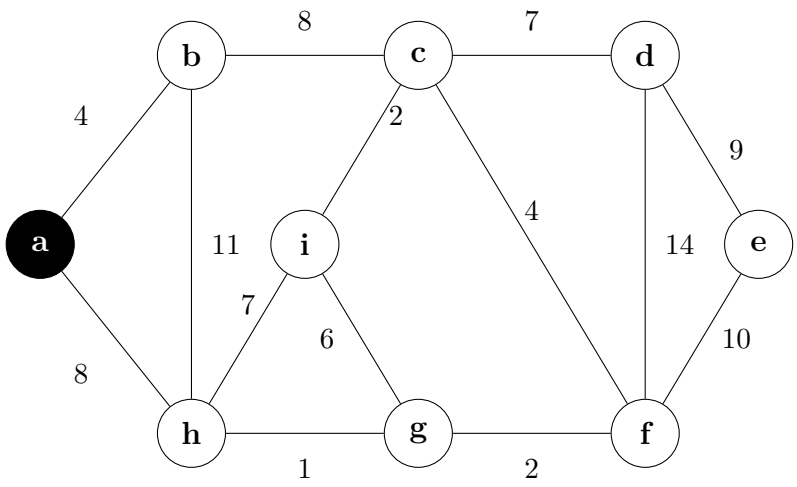
Complexity: $O(n \log n)$ for sorting; $O(n)$ for selection. Total: $O(n \log n)$.

(ii) Correctness Proof (Greedy stays ahead):
Claim: The greedy solution $G = \{g_1, g_2, \ldots, g_k\}$ (sorted by finish time) is optimal, i.e., $|G| \geq |OPT|$ for any optimal solution $OPT = \{o_1, o_2, \ldots, o_m\}$.
Lemma: At every step i, $f(g_i) \leq f(o_i)$ (greedy finishes no later than optimal).
Proof by induction:

- **Base case** ($i = 1$): Greedy picks the task with earliest finish time, so $f(g_1) \leq f(o_1)$. ✓
- **Inductive step:** Assume $f(g_{i-1}) \leq f(o_{i-1})$. Then o_i starts at $s(o_i) \geq f(o_{i-1}) \geq f(g_{i-1})$, so o_i is compatible with g_{i-1}. The greedy algorithm considers all compatible tasks and picks the one with minimum finish time, so $f(g_i) \leq f(o_i)$. ✓

Conclusion: Since $f(g_i) \leq f(o_i)$ for all $i \leq m$, greedy never runs out of room before OPT does. Therefore $k \geq m$, i.e., greedy selects at least as many tasks as any optimal solution. □

Q38. If Prim's algorithm is applied on a given graph, which will be the fifth edge added to the spanning tree? Start from vertex a. **(10 marks)** [CSE 207 | 2012–13]

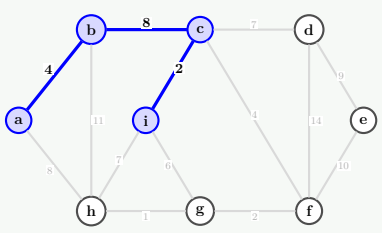


Answer: Prim's Algorithm (CSE 207 | 2012-13)

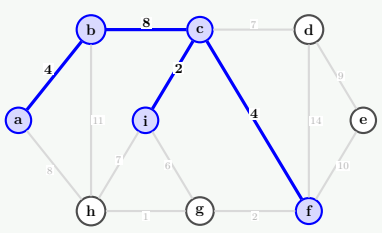
Start Node: a.
When ties occur (e.g., between edges a–$h(8)$ and b–$c(8)$), we pick alphabetically (adding c instead of h).

Step	Vertex Added	Edge Added	Edge #	Key Values (Selected Unvisited)
1	a	—	—	$b:4, h:8$
2	b	a–b (4)	1st Edge	$c:8, h:8$
3	c	b–c (8)	2nd Edge	$d:7, f:4, i:2, h:8$
4	i	c–i (2)	3rd Edge	$d:7, f:4, g:6, h:8$
5	f	c–f (4)	4th Edge	$d:7, g:2, h:7$
6	**g**	**f–g (2)**	**5th Edge**	$d:7, h:1$
7	h	g–h (1)	6th Edge	$d:7$
8	d	c–d (7)	7th Edge	$e:9$
9	e	d–e (9)	8th Edge	—

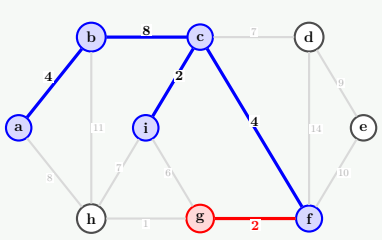
Visual Trace (Highlighting the 5th Edge):



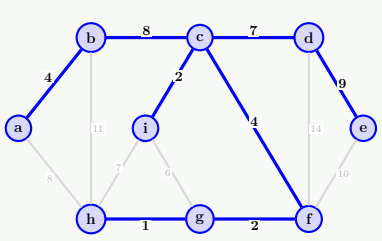
Step 4: Added c-i (3rd Edge)



Step 5: Added c-f (4th Edge)



Step 6: Added f-g (5th Edge)



Final Minimum Spanning Tree

Final Answer: The 5th edge added to the spanning tree is **f**–**g** (weight 2).
MST Total Cost $= 4+8+2+4+2+1+7+9 =$ **37**. ✓

Q39. "Search spaces for natural combinatorial problems tend to grow exponentially with the size of the input" — explain and justify with necessary examples. **(15 marks)** [CSE 207 | 2011–12]

Answer

Claim: the search spaces of natural combinatorial problems grow exponentially with input size.
Explanation and examples:
1. Subset problems (2^n subsets of n elements). The 0-1 Knapsack problem has n items; the number of subsets to check is 2^n. For $n = 30$, this is over 10^9 — infeasible by brute force.
2. Permutation problems ($n!$ orderings). The Travelling Salesman Problem (TSP) requires finding the shortest Hamiltonian cycle over n cities; there are $(n-1)!$ distinct tours. For $n = 20$, this is $\approx 10^{17}$.
3. Scheduling / assignment (assignment of n jobs to m machines: m^n possibilities).
Formal justification: Consider binary strings of length n: each bit has 2 choices, giving 2^n total strings. This grows faster than any polynomial: for any fixed k, $2^n > n^k$ for large enough n. Most combinatorial problems map their solution space to exponentially large sets of this form, making exhaustive search intractable.
Implication: this motivates greedy algorithms, dynamic programming, and approximation algorithms that find optimal or near-optimal solutions in polynomial time without enumerating the full search space.

Q40. Explain why one can solve the fractional knapsack problem by a greedy strategy, but one cannot solve the 0-1 knapsack problem by such a strategy. **(10 marks)** [CSE 207 | 2011–12]

Answer

Fractional Knapsack — Greedy works:
In the fractional version we may take any fraction of an item. The greedy strategy sorts items by value density v_i/w_i and takes as much as possible of the highest-density item remaining.
Why it works (exchange argument): Suppose an optimal solution does *not* take the item with the highest v/w first. We can exchange a small amount of a lower-density item for an equal weight of the higher-density item, strictly increasing the total value without violating the capacity constraint. Repeating this argument shows the greedy solution is optimal.
Formally: The fractional knapsack has the *greedy-choice property* — a locally optimal (highest density) choice is always part of some globally optimal solution.

0-1 Knapsack — Greedy fails:
In the 0-1 version each item must be taken entirely or not at all. The greedy strategy can fail because:

- Taking a high-density item may "waste" capacity and prevent fitting a combination of lower-density items that has greater total value.
- No fractional fill is allowed, so the exchange argument breaks down.

Counterexample: $W = 10$.

Item	w	v	v/w
1	6	12	2.0
2	5	9	1.8
3	5	9	1.8

Greedy picks item 1 ($v/w = 2$, $w = 6$), then cannot fit items 2 or 3 together (remaining capacity $= 4 < 5$). Total value $= 12$.
Optimal: items $2 + 3$ ($w = 10$, $v = 18$). Greedy gives only $12 < 18$.

Conclusion: The 0-1 knapsack lacks the greedy-choice property and requires dynamic programming ($O(nW)$) for an exact solution.

Q41. Prove that the greedy method stays ahead in the interval scheduling problem, and hence show that the greedy method returns an optimal set of jobs. **(15 marks)** [CSE 207 | 2011–12]

Answer

Algorithm: sort intervals by finish time; greedily pick each compatible interval. Let $i_1, i_2, \ldots, i_k$ be the greedy selections (in order of finish time).
Greedy Stays Ahead Lemma. For any other compatible set $j_1, j_2, \ldots, j_m$ (in order of finish time), for each $r \leq \min(k, m)$:

$$f(i_r) \leq f(j_r).$$

(The r-th greedy choice finishes no later than the r-th choice of any other algorithm.)
Proof (by induction on r).
Base ($r = 1$): i_1 has the earliest finish time of all intervals $\rightarrow f(i_1) \leq f(j_1)$. ✓
Step: assume $f(i_r) \leq f(j_r)$. Then j_{r+1} starts after $f(j_r) \geq f(i_r)$, so j_{r+1} is compatible with $i_1, \ldots, i_r$. The greedy considers all remaining compatible intervals and picks the one with the smallest finish time, so $f(i_{r+1}) \leq f(j_{r+1})$. ✓
Optimality: suppose the greedy selects k intervals but OPT selects $m > k$. By the lemma, $f(i_k) \leq f(j_k)$. Then j_{k+1} starts after $f(j_k) \geq f(i_k)$, so j_{k+1} is compatible with the greedy's k selections — but the greedy found no more compatible intervals, a contradiction. Hence $k \geq m$ and the greedy is optimal. □

Q42. Prove that if we use a greedy algorithm for the interval partitioning problem, every interval will be assigned a label, and no two overlapping intervals will receive the same label. **(15 marks)** [CSE 207 | 2011–12]

Answer

Interval Partitioning Problem: assign each of n intervals a *label* (colour/room) so that no two overlapping intervals share a label, using the minimum number of labels.
Algorithm: sort by start time; for each interval, assign any free label; if none is free, open a new label.
Claim 1: every interval is assigned a label. The algorithm considers intervals in start-time order and always either finds a free label or opens a new one. Since we always assign a label, no interval is left unassigned. ✓
Claim 2: no two overlapping intervals share a label. A label ℓ is only assigned to interval i if its previous occupant j satisfies $f(j) \leq s(i)$ (i.e. j has finished before i starts). Hence i and j do not overlap. By induction, all intervals assigned label ℓ form a compatible (non-overlapping) set. □
Optimality: the number of labels used equals the maximum *depth* d (maximum number of intervals overlapping at any single point). Any labeling needs at least d labels, and the greedy uses exactly d. □

Q43. Explain how Dijkstra's algorithm can be implemented using a priority queue. Find the running time for this algorithm. **(15 marks)** [CSE 207 | 2011–12]

Answer

Implementation with Binary Min-Heap:

```
1 void dijkstra(int src, vector<vector<pair<int,int>>>& adj, int V) {
2     vector<int> dist(V, INT_MAX);
```

```
    priority_queue<pair<int,int>,
                   vector<pair<int,int>>,
                   greater<>> pq;  // min-heap: (dist, vertex)
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop();
        if (d > dist[u]) continue;  // stale entry
        for (auto [w, v] : adj[u]) {
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

Running time analysis:

Operation	Count	Cost
Insert (initial)	V	$O(\log V)$ each
Extract-Min	$\leq V$	$O(\log V)$ each
Decrease-Key	$\leq E$	$O(\log V)$ each

Total with binary heap: $O((V+E)\log V)$.
With Fibonacci heap: Decrease-Key is $O(1)$ amortised $\rightarrow$ total $O(V \log V + E)$.

Q44. State and prove the "cut property" and the "cycle property". Explain with necessary examples how these properties are useful in solving the minimum spanning tree problem. **(15 marks)** [CSE 207 | 2011–12]

Answer

Cut Property. Let $(S, V \setminus S)$ be any cut of G and e the unique minimum-weight edge crossing it. Then e belongs to every MST.
Proof: suppose MST T does not contain e. Adding e to T creates a cycle; this cycle must contain another crossing edge e' with $w(e') > w(e)$. $T' = T - \{e'\} + \{e\}$ is a spanning tree with $w(T') < w(T)$ — contradiction. $\square$
Example: in any graph, the globally lightest edge e^* belongs to every MST (take the cut separating its endpoints).
Cycle Property. Let C be any cycle in G and e the unique maximum-weight edge in C. Then e does *not* belong to any MST.
Proof: suppose MST T contains e. Removing e from T splits it into two components; there is another edge $e' \in C$ crossing this cut with $w(e') < w(e)$. $T' = T - \{e\} + \{e'\}$ is a lighter spanning tree — contradiction. $\square$
Example: the heaviest edge in any cycle can always be safely discarded.
Application:

- **Prim's** uses the cut property: always adds the minimum crossing edge of $(S, V \setminus S)$.
- **Kruskal's** uses both: adds minimum edges (cut property) and rejects maximum-weight cycle edges (cycle property).

Q45. Describe, with necessary elaboration on the required operations and data structure, how the union-find data structure can be used for Kruskal's algorithm. **(15 marks)** [CSE 207 | 2011–12]

Answer

Union-Find (Disjoint-Set Union) data structure:
Each connected component of the growing forest is stored as a disjoint set. Three operations:

```
// Make a singleton set for each vertex
void makeSet(int x) { parent[x]=x; rank[x]=0; }

// Find representative (with path compression)
int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);  // path compression
    return parent[x];
}

// Merge two sets (union by rank)
void unite(int x, int y) {
    int px=find(x), py=find(y);
    if (px==py) return;
    if (rank[px] < rank[py]) swap(px,py);
    parent[py] = px;
    if (rank[px]==rank[py]) rank[px]++;
}
```

Use in Kruskal's:

(a) Initialise: MakeSet(v) for each $v \in V$.

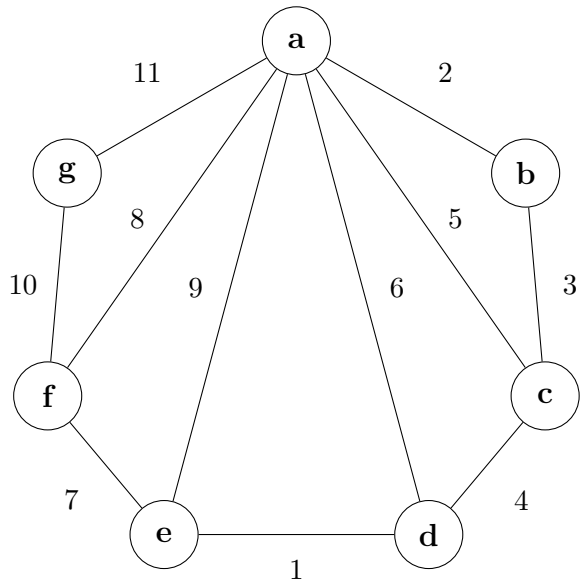
(b) For each edge (u, v) in sorted order: if Find(u) $\neq$Find(v), the edge does not form a cycle $\rightarrow$ add to MST and call Unite(u, v).

Complexity:

- MakeSet: $O(1)$ each, $O(V)$ total.
- Find with path compression: $O(\alpha(V))$ amortised.
- Unite with union-by-rank: $O(\alpha(V))$ amortised.
- Total for E operations: $O(E\,\alpha(V)) \approx O(E)$.
- Kruskal total (including sort): $O(E \log E) = O(E \log V)$.

where α is the inverse Ackermann function, practically ≤ 4 for all realistic n.

Q46. What is the criterion for selecting an edge at each step of Kruskal's algorithm for computing MST? How does this criterion guarantee the algorithm gives an MST? Show the steps of Kruskal's algorithm on a given graph. **(17 marks)** [CSE 207 | 2009–10]



Answer: Kruskal's Criterion & Trace (CSE 207 | 2009-10)

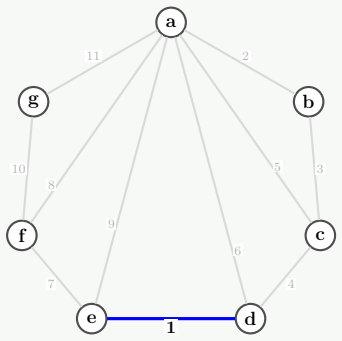
Criterion: At each step, select the minimum-weight edge that does not form a cycle with the already-chosen edges.
Guarantee (Correctness): The selected edge is always the minimum crossing edge of some cut that respects the current forest A (the cut separates the two components being joined). By the **Cut Property**, this edge belongs to some Minimum Spanning Tree (MST). By induction, the final set A is an MST. $\square$

Solving the Graph

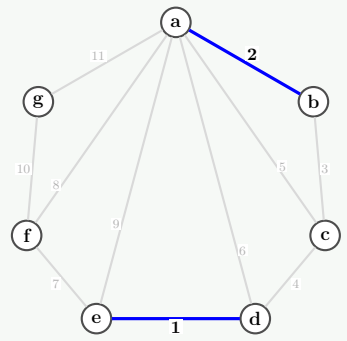
Edges Sorted by Weight: $de(1), ab(2), bc(3), cd(4), ac(5), ad(6), ef(7), af(8), ae(9), fg(10), ga(11)$.

Edge	Weight	Add to MST?	Reason
d–e	1	**Yes**	Does not form a cycle
a–b	2	**Yes**	Does not form a cycle
b–c	3	**Yes**	Does not form a cycle
c–d	4	**Yes**	Connects $\{a, b, c\}$ with $\{d, e\}$
a–c	5	**No**	Forms cycle (a-b-c-a)
a–d	6	**No**	Forms cycle (a-b-c-d-a)
e–f	7	**Yes**	Connects f to the tree
a–f	8	**No**	Forms cycle (a-b-c-d-e-f-a)
a–e	9	**No**	Forms cycle (a-b-c-d-e-a)
f–g	10	**Yes**	Connects g. (All 7 vertices joined!)

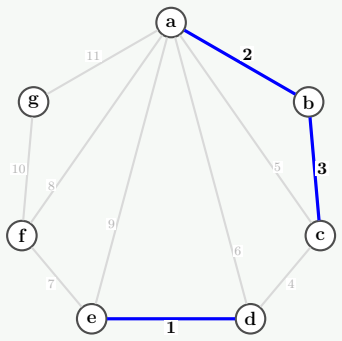
Visual Trace (Step-by-Step):



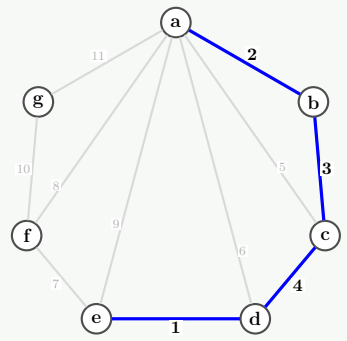
Step 1: Add de(1)



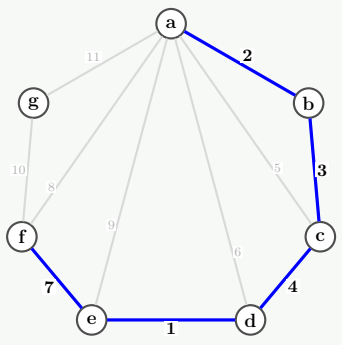
Step 2: Add ab(2)



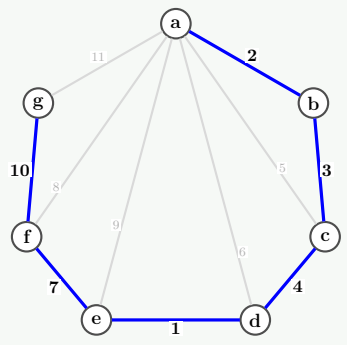
Step 3: Add bc(3)



Step 4: Add cd(4)



Step 5: Skip ac, ad. Add ef(7)



Step 6: Skip af, ae. Add fg(10)

Final MST Edges: $de(1), ab(2), bc(3), cd(4), ef(7), fg(10)$.
Total Cost $= 1+2+3+4+7+10 =$ **27**. ✓

Q47. Construct the Huffman tree and show the Huffman coding for the following characters with given frequencies:

Character	a	b	c	d	e	f
Frequency	50	30	20	15	10	5

(10 marks) [CSE 207 | 2009–10]

Answer

Merge steps (always merge two smallest):

(a) $f(5) + e(10) = fe(15)$

(b) $d(15) + fe(15) = dfe(30)$

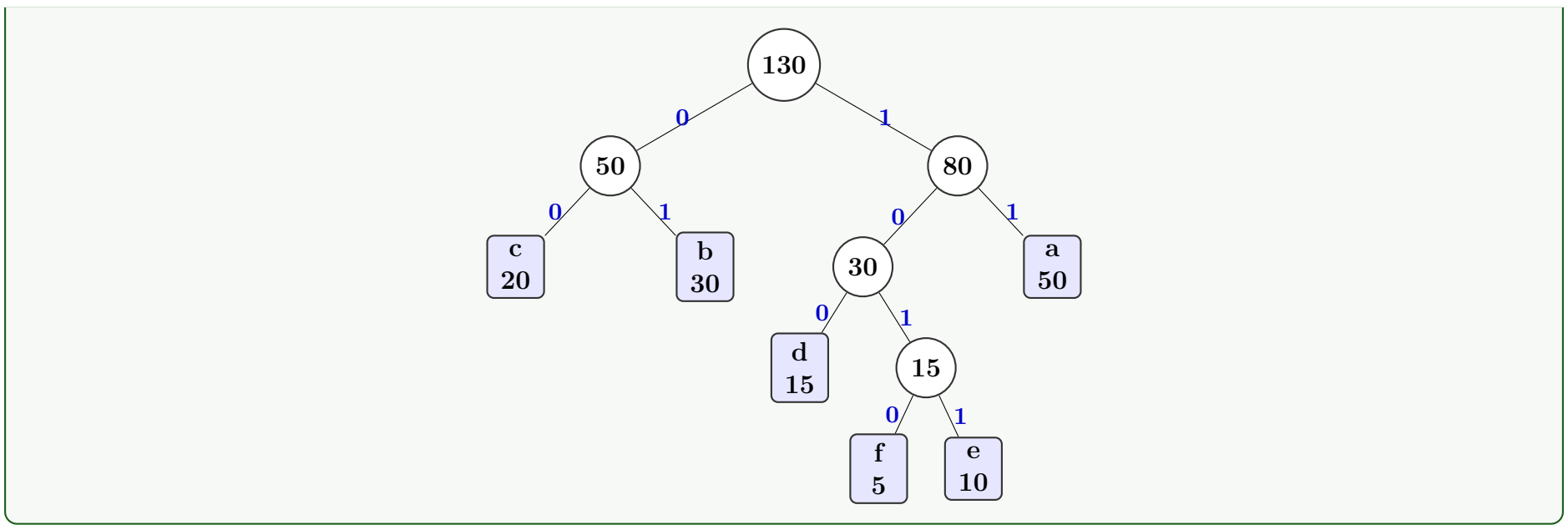
(c) $c(20) + b(30) = cb(50)$

(d) $dfe(30) + a(50) = dfea(80)$

(e) $cb(50) + dfea(80) = \text{root}(130)$

Huffman codes:

Symbol	Freq	Code	Bits used
a	50	11	$50 \times 2 = 100$
b	30	01	$30 \times 2 = 60$
c	20	00	$20 \times 2 = 40$
d	15	100	$15 \times 3 = 45$
e	10	1011	$10 \times 4 = 40$
f	5	1010	$5 \times 4 = 20$
		Total bits:	**305**



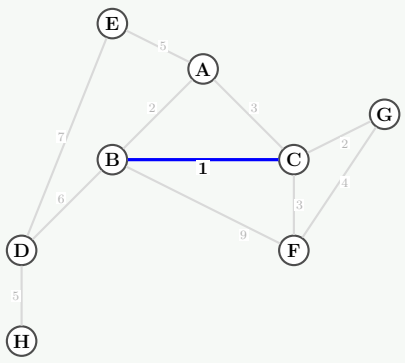
Q48. Construct a minimum spanning tree using Kruskal's algorithm for the following weighted graph: $AB(2), AC(3), AE(5), BC(1), BD(6), BF(9), CF(3$ $DH(5), FG(4)$. Show every step, giving reasons for discarding any edge. **(15+5=20 marks)** [CSE 207 | 2008–09]

Answer: Kruskal's Algorithm (CSE 207 | 2008-09)

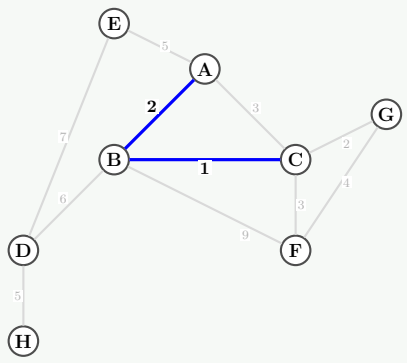
Sorted edges: $BC(1), AB(2), CG(2), CF(3), AC(3), FG(4), AE(5), DH(5), BD(6),$ $DE(7), BF(9)$.

Edge	Weight	Add to MST?	Reason
BC	1	**Yes**	Does not form a cycle
AB	2	**Yes**	Does not form a cycle
CG	2	**Yes**	Does not form a cycle
CF	3	**Yes**	Does not form a cycle
AC	3	**No**	Forms cycle (A-B-C-A)
FG	4	**No**	Forms cycle (F-C-G-F)
AE	5	**Yes**	Does not form a cycle
DH	5	**Yes**	Does not form a cycle
BD	6	**Yes**	Connects $\{A, B, C, E, F, G\}$ with $\{D, H\}$
DE	7	**No**	Forms cycle
BF	9	**No**	Forms cycle

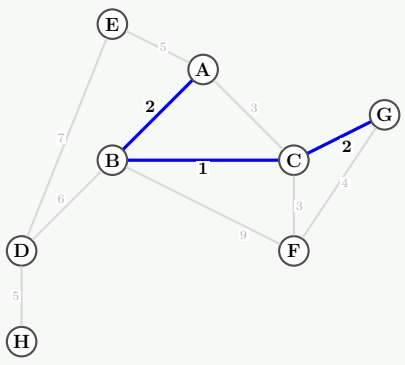
Visual Trace (Step-by-Step):



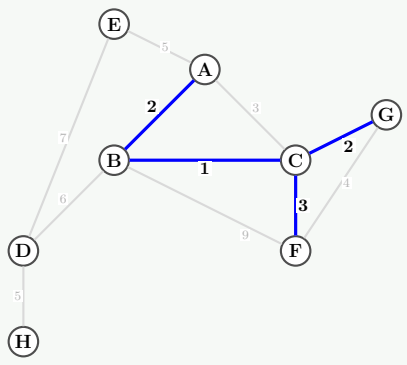
Step 1: Add BC(1)



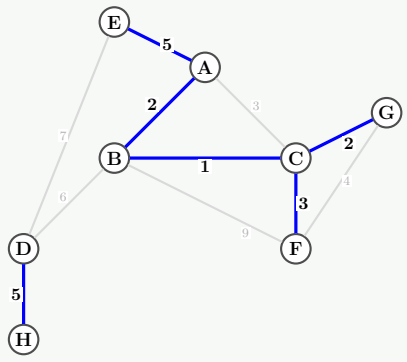
Step 2: Add AB(2)



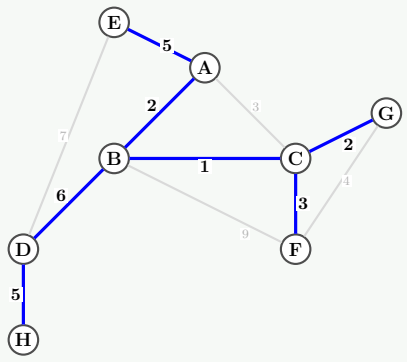
Step 3: Add CG(2)



Step 4: Add CF(3)



Step 5: Skip AC, FG. Add AE, DH



Step 6: Add BD(6) (Final MST)

Final MST Edges: $BC(1), AB(2), CG(2), CF(3), AE(5), DH(5), BD(6)$.
Total Cost $= 1+2+2+3+5+5+6 =$ **24**. ✓

Q49. Discuss how Prim's algorithm constructs a minimum spanning tree. Argue how sets and the disjoint-set union algorithm can be used to detect cycles and improve upon Kruskal's algorithm. **(15 marks)** [CSE 207 | 2008–09]

Answer

Prim's Algorithm:
Prim's algorithm grows the MST one edge at a time, always adding the *minimum-weight edge* that connects a vertex already in the tree to a vertex not yet in the tree.

Algorithm 106 PRIM(G, w, r)

```
1: for each u ∈ V do
2:     key[u] ← ∞;   parent[u] ← nil
3: end for
4: key[r] ← 0
5: Q ← min-priority queue of all vertices keyed by key[ ]
6: while Q ≠ ∅ do
7:     u ← EXTRACT-MIN(Q)
8:     for each v adjacent to u do
9:         if v ∈ Q and w(u, v) < key[v] then
10:            parent[v] ← u
11:            key[v] ← w(u, v)                    ▷ Decrease-Key in Q
12:        end if
13:    end for
14: end while
```

- Uses a min-priority queue; each EXTRACT-MIN costs $O(\log V)$.
- Each DECREASE-KEY costs $O(\log V)$.
- Total: $O((V+E)\log V)$; with Fibonacci heap: $O(E + V\log V)$.

Kruskal's Algorithm with Disjoint-Set Union (DSU):
Kruskal's algorithm processes edges in non-decreasing weight order, adding an edge only if its two endpoints are in *different components* (i.e., adding it does not form a cycle).
Cycle detection using DSU:

- Each component is a set. Initially every vertex is its own set.
- For edge (u, v): call FIND(u) and FIND(v).
  - If they return *different* roots ⇒ different components ⇒ **no cycle**: add edge, call UNION(u, v).
  - If they return the *same* root ⇒ same component ⇒ **cycle**: skip edge.

Improvements via union-by-rank + path compression:

- **Union by rank:** Always attach the smaller tree under the larger, keeping trees shallow.
- **Path compression:** On FIND, make every node on the path point directly to the root.
- Combined amortized cost: $O(\alpha(V))$ per operation (α = inverse Ackermann, essentially constant).

Total complexity of Kruskal's with DSU: $O(E\log E)$ (dominated by sorting edges).

Q50. Discuss how a Huffman tree can be constructed given a sequence of frequencies corresponding to symbols. Construct a dynamic Huffman tree based upon the message `ABBBCAAAACBBBB`. **(20 marks)** [CSE 207 | 2008–09]

Answer

Huffman Tree Construction:
Given symbols with frequencies, build the tree greedily:

1. Insert each symbol as a leaf node into a **min-priority queue** keyed by frequency.

2. Repeat until one node remains:

(a) Extract the two nodes x, y with minimum frequency.

(b) Create a new internal node z with $\text{freq}(z) = \text{freq}(x) + \text{freq}(y)$.

(c) Make x left child and y right child of z; insert z back into the queue.

3. The remaining node is the root of the Huffman tree.

Left branch = 0, right branch = 1 (or vice versa). The code for each symbol is the path from root to its leaf.

Frequencies in `ABBBCAAAACBBBB`:
Count: A=5, B=7, C=2. Total = 14 characters.

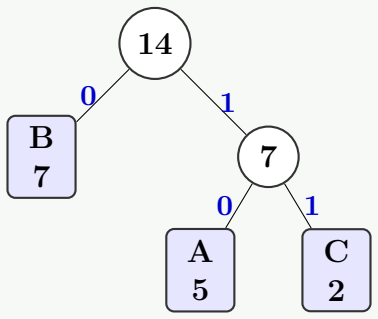
Building the Huffman Tree:

Step 1: Initial queue: C(2), A(5), B(7).

Step 2: Extract C(2) and A(5). Merge → internal node CA(7). Queue: B(7), CA(7).

Step 3: Extract B(7) and CA(7). Merge → root(14).

Final Tree:



Codes: B = `0`, A = `10`, C = `11`.
Encoded message `ABBBCAAAACBBBB`:

`10 0 0 0 11 10 10 10 11 0 0 0 0`

Total bits $= 5 \times 2 + 7 \times 1 + 2 \times 2 = 10 + 7 + 4 = 21$ bits.

Q51. Suppose you have a washing machine that a number of users want to use. Each user i sends a request for the time interval (s_i, f_i) (start and finish times). The machine can serve only one user at a time, and each user pays 1000/- regardless of duration. Write a greedy algorithm to maximize revenue. Prove the correctness of your algorithm and deduce its running time. **(8+10+5 marks)** **[CSE 207 | 2007–08]**

Answer

Formulation: this is exactly the **Interval Scheduling** (Activity Selection) problem. Each user i requests interval (s_i, f_i); the machine can serve one at a time; each served user pays a fixed amount. Maximising revenue = maximising the number of served users.
Algorithm: sort requests by finish time; greedily accept each request compatible with the last accepted one.
Correctness: proved via the greedy-choice + optimal substructure argument (see [CSE 203 | 2020–21]).

Complexity Analysis. $O(n \log n)$ for sorting; $O(n)$ for the scan. Total: $O(n \log n)$.

Q52. Define the interval coloring problem and write an algorithm to solve it. **(4+8 marks)** **[CSE 207 | 2007–08]**

Answer

Interval Coloring Problem: given n intervals, assign each interval a "color" (resource/room) such that no two overlapping intervals share a color, using the *minimum* number of colors (= minimum number of resources).
The minimum number of colors needed equals the maximum number of intervals that overlap at any single point (*depth*).
Algorithm (earliest start time, greedy):

Algorithm 107 IntervalColoring(S)

```
Sort intervals by start time: s_1 ≤ s_2 ≤ ··· ≤ s_n
d ← 0                                              ▷ number of colors used
for i ← 1 to n do
    if some color c is free at time s_i then
        Assign color c to interval i; free c after f_i
    else
        d ← d + 1; assign new color d to interval i
    end if
end for
return d
```

Implemented with a min-heap of finish times of currently active intervals, this runs in $O(n \log n)$.
Correctness: whenever a new color is needed, all d existing colors are in use, meaning d intervals overlap at s_i — so any valid coloring needs $\geq d$ colors. □

Q53. Describe the main idea of Prim's algorithm to compute a minimum spanning tree. Discuss how Prim's algorithm can be implemented in $O(m \log n)$ time. Discuss the reverse-delete algorithm to compute a minimum spanning tree and prove its correctness. **(6+10+17 marks)** [CSE 207 | 2007–08]

Answer

Prim's main idea: grow the MST one edge at a time. Maintain a set S of tree vertices; at each step add the minimum-weight edge crossing the cut $(S, V \setminus S)$.
$O(m \log n)$ **implementation** (binary min-heap):

- Maintain a key key$[v]$ = min weight of any edge from v to a vertex in S; store all non-tree vertices in a min-heap keyed by key$[\cdot]$.
- Extract-Min: $O(\log n)$, called n times $\rightarrow O(n \log n)$.
- Decrease-Key: $O(\log n)$, called $\leq m$ times $\rightarrow O(m \log n)$.
- Total: $O(m \log n)$.

Reverse-Delete algorithm:

(a) Start with $T = E$ (all edges).

(b) Process edges in *decreasing* weight order.

(c) Delete edge e from T if $T - \{e\}$ is still connected.

Correctness: we never delete a bridge (a cut edge), so the graph stays connected. We delete an edge only if it is not the unique minimum crossing edge of some cut (i.e. it is not in every MST by the cut property). The result is an MST. □

Q54. Write down an algorithm for constructing a minimum spanning tree using Kruskal's algorithm. Show how sets and the disjoint-set algorithm can be used to reduce its complexity. Construct a minimum spanning tree for the graph with edges: $AB(7), AC(3), DA(4), EA(1), BC(91), C$ $FD(3)$. **(5+5+5 marks)** [CSE 207 | 2006–07]

Answer: Kruskal & Disjoint-Set (CSE 207 | 2006-07)

Kruskal's algorithm:

Algorithm 108 Kruskal(G, w)

```
A ← ∅
for each vertex v do Make-Set(v)
end for
Sort edges by weight (ascending)
for each edge (u, v) in sorted order do
    if Find-Set(u) ≠ Find-Set(v) then
        A ← A ∪ {(u, v)}; Union(u, v)
    end if
end for
return A
```

Disjoint-set (Union-Find) for cycle detection:

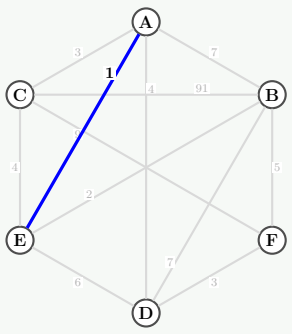
- Find with path compression: $O(\alpha(V))$ amortised.
- Union by rank: $O(\alpha(V))$ amortised.
- **Total Kruskal Time Complexity**: $O(E \log E) = O(E \log V)$.

Solving the Graph

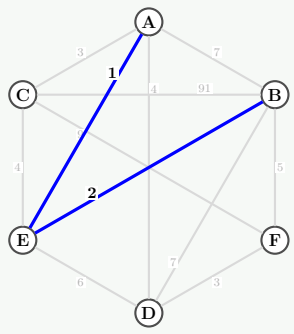
Sorted edges: $EA(1), EB(2), AC(3), FD(3), DA(4), CE(4), FB(5), DE(6), AB(7), BD(7), CF(9), BC(91)$.

Edge	Weight	Add?	Reason
E–A	1	**Yes**	Different sets
E–B	2	**Yes**	Different sets
A–C	3	**Yes**	Different sets
F–D	3	**Yes**	Different sets
D–A	4	**Yes**	Different sets → **All 6 vertices connected!**
C–E	4	**No**	Same set (Forms cycle C-A-E-C)
F–B	5	**No**	Same set (Forms cycle)
… *all remaining edges are skipped as MST is complete.*			

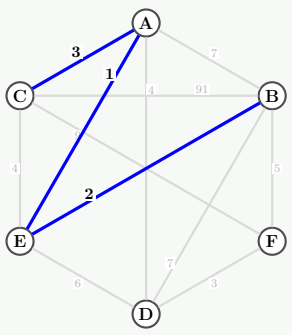
Visual Trace (Step-by-Step):



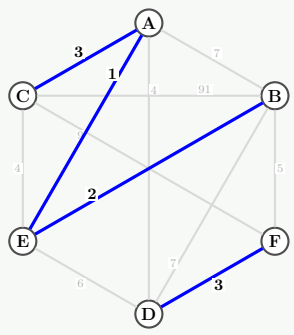
Step 1: Add EA(1)



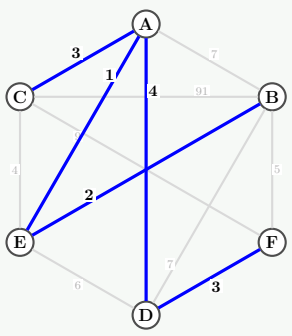
Step 2: Add EB(2)



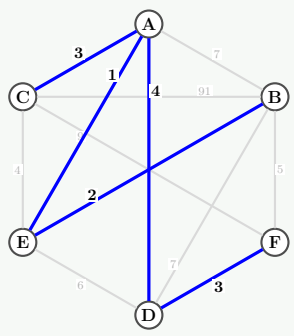
Step 3: Add AC(3)



Step 4: Add FD(3)



Step 5: Add DA(4)



Step 6: Skip rest (Final MST)

Final MST Edges: $EA(1), EB(2), AC(3), FD(3), DA(4)$.
Total Cost $= 1+2+3+3+4 =$ **13**. ✓

Q55. Construct a Huffman tree for the phrase with symbols and frequencies: A-7, B-4, C-5, D-2, E-20, F-3, G-6, H-8, I-1. **(10 marks)** [CSE 207 | 2006–07]

Answer: Huffman Tree Construction

Merge steps (always merge two smallest available nodes):

(a) $I(1) + D(2) = 3$

(b) $F(3) + (I{+}D)(3) = 6$

(c) $B(4) + C(5) = 9$

(d) $(F{+}ID)(6) + G(6) = 12$

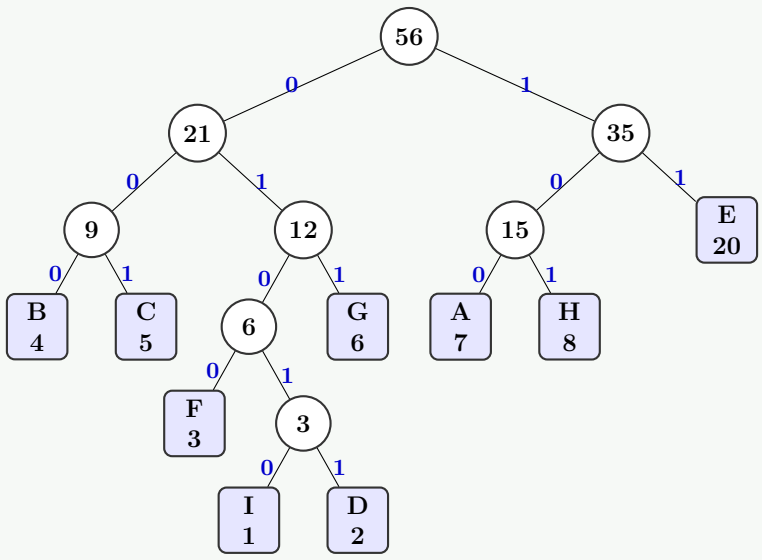
(e) $A(7) + H(8) = 15$

(f) $(B{+}C)(9) + (F{+}ID{+}G)(12) = 21$

(g) $(A{+}H)(15) + E(20) = 35$

(h) $(BC{+}FIDG)(21) + (AH{+}E)(35) = 56$ (Root)

Huffman Tree Visual Representation:



Huffman codes:

Symbol	Freq	Code
E	20	11
A	7	100
H	8	101
B	4	000
C	5	001
G	6	011
F	3	0100
D	2	01011
I	1	01010

Total bits $= 20{\cdot}2 + 7{\cdot}3 + 8{\cdot}3 + 4{\cdot}3 + 5{\cdot}3 + 6{\cdot}3 + 3{\cdot}4 + 2{\cdot}5 + 1{\cdot}5 =$ **157**.

Q56. There is a set of jobs where each job has an integer deadline and profit, and only one machine is available. To complete a job requires one unit of time; profit is earned only if the job is completed by its deadline. Jobs A, B, C, D, E have deadlines $3, 3, 1, 2, 2$ and profits $1, 5, 10, 15, 20$ respectively. Show the steps of selecting jobs using a greedy strategy. **(8 marks)** [CSE 207 | 2006–07]

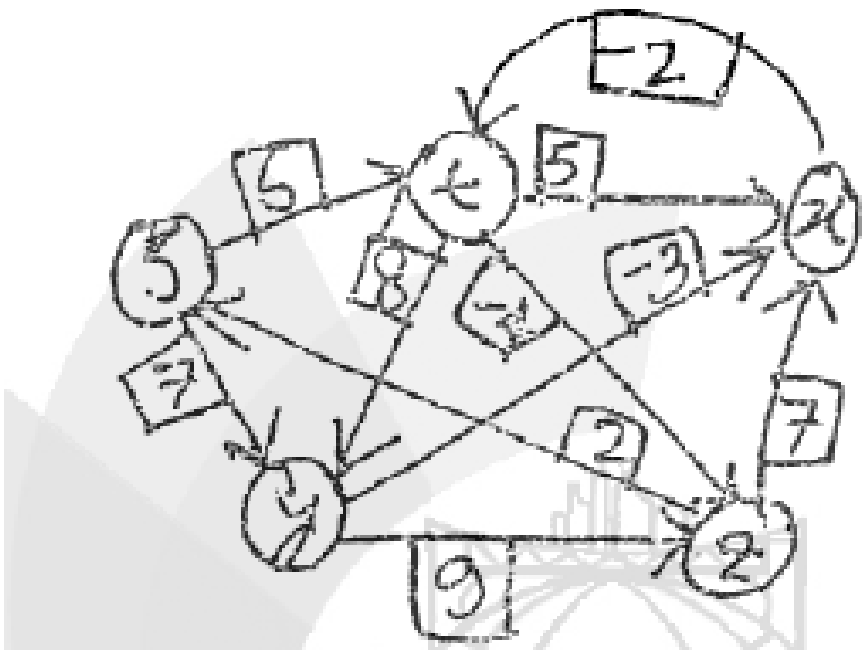
Answer

Sort by profit descending: $E(20, d{=}2), D(15, d{=}2), C(10, d{=}1), B(5, d{=}3), A(1, d{=}3)$.

Job	Profit	Deadline	Slot assigned
E	20	2	Slot 2
D	15	2	Slot 1
C	10	1	No slot (slot 1 taken) — **skip**
B	5	3	Slot 3
A	1	3	No free slot ≤ 3 — **skip**

Schedule: Slot $1 \to D$, Slot $2 \to E$, Slot $3 \to B$. **Total profit** $= 15 + 20 + 5 =$ **40**.

Q57. Find the shortest paths starting from vertex s in this given graph. Show the steps of edge relaxation. **(10 marks)** [CSE 207 | 2006–07]



Q58. Once an amateur thief entered into a museum full of tantalizing jewelry, geodes and rare gems. As he is new in this field, he brought just a single backpack with him that has a capacity of 6 kg. The thief planned to get away with the most valuable objects without overloading the backpack. He found out that in a less-secured compartment, there were four (4) solid items, and their information are given in the following table.

Item	Weight (kg)	Value (USD)
Brooch of the queen	3	15
Crown of the king	2	20
Decorated vase	10	30
An ancient coin	2	14

Table for Ques No. 5(a) — Information of valuable items found in the museum

Since the items are unbreakable, for each item, the thief must pick the whole of it or not consider it at all. Now although our thief is amateur in this business, he possesses some algorithmic knowledge. Therefore, he applied a greedy technique and selected the items that gave him the maximum possible profit per weight, and then he left the museum with so much zeal! Now, using the information of the backpack of the thief and the item table given for this question, your first task is to show the steps of finding the maximum possible profit and the corresponding list of items using the technique that the thief adopted. You have also identified that the scenario at the museum was a special one and only dynamic programming is able to find the optimal solution in that case. Therefore, your second task is to find the optimal profit and the list of items using dynamic programming. **(10+10=20 marks)**

Answer

Given Items & Capacity: $W = 6$ kg.

- Item 1: Brooch ($w_1 = 3$, $v_1 = 15$)
- Item 2: Crown ($w_2 = 2$, $v_2 = 20$)
- Item 3: Vase ($w_3 = 10$, $v_3 = 30$)
- Item 4: Coin ($w_4 = 2$, $v_4 = 14$)

Part 1: Greedy Approach (Value-to-Weight Ratio)

Item	Weight (w)	Value (v)	Ratio (v/w)	Rank
Crown	2	20	10.0	1
Coin	2	14	7.0	2
Brooch	3	15	5.0	3
Vase	10	30	3.0	4

Execution Steps:

(a) Pick **Crown** (rank 1): $w = 2 \leq 6$. Remaining capacity $= 6 - 2 = 4$. Profit $= 20$.
(b) Pick **Coin** (rank 2): $w = 2 \leq 4$. Remaining capacity $= 4 - 2 = 2$. Profit $= 20 + 14 = 34$.
(c) Try **Brooch** (rank 3): $w = 3 > 2$ (Capacity exceeded). Skip.
(d) Try **Vase** (rank 4): $w = 10 > 2$ (Capacity exceeded). Skip.

Greedy Solution: Items = {Crown, Coin}, Total Weight = 4 kg, **Maximum Profit = \$34**.

Part 2: Dynamic Programming (0-1 Knapsack)

Let $dp[i][w]$ be the maximum profit using the first i items with capacity w. The recurrence relation is:

$$dp[i][w] = \max(dp[i-1][w],\ dp[i-1][w-w_i] + v_i) \quad \text{if } w_i \leq w \text{ else } dp[i-1][w]$$

DP Table:

i (Item)	w_i, v_i	$w=0$	1	2	3	4	5	6
0 (None)	-	0	0	0	0	0	0	0
1 (Brooch)	3, 15	0	0	0	15	15	15	15
2 (Crown)	2, 20	0	0	20	20	20	35	35
3 (Vase)	10, 30	0	0	20	20	20	35	35
4 (Coin)	2, 14	0	0	20	20	34	35	**35**

Traceback for Optimal Selection: Start at the maximum profit $dp[4][6] = 35$:

- $dp[4][6] == dp[3][6]$ $(35 = 35) \implies$ **Coin** is **not** included.
- $dp[3][6] == dp[2][6]$ $(35 = 35) \implies$ **Vase** is **not** included.
- $dp[2][6] \neq dp[1][6]$ $(35 \neq 15) \implies$ **Crown** is **included**. Remaining capacity $w = 6 - 2 = 4$.
- $dp[1][4] \neq dp[0][4]$ $(15 \neq 0) \implies$ **Brooch** is **included**. Remaining capacity $w = 4 - 3 = 1$.

Optimal Solution (DP): Items = {Crown, Brooch}, Total Weight = 5 kg, **Optimal Profit = $35**.

Remark. The greedy approach yields a profit of $34, which is suboptimal compared to the DP approach ($35). This demonstrates why the 0-1 Knapsack problem requires Dynamic Programming to guarantee an optimal solution.

BUET · CSE 105

DSA

Dynamic Programming

LCS, MCM, Edit Distance, Knapsack, Memoization

T11 — Dynamic Programming

Q1. Assume that you have obtained a gold bar of integer length n. You plan to cut it into smaller pieces and sell them to the market. For any integer $i \leq n$, you know the profit you will obtain if you sell a gold bar of length i.

(a) Using the cut-and-paste method, formulate an argument to show that the problem of obtaining the maximum profit by cutting this gold bar into smaller pieces has the optimal substructure property.

(b) Construct a recursion tree diagram to show that the problem has the overlapping subproblem property.

(7+8=15 marks) [CSE 105 | 2023–24]

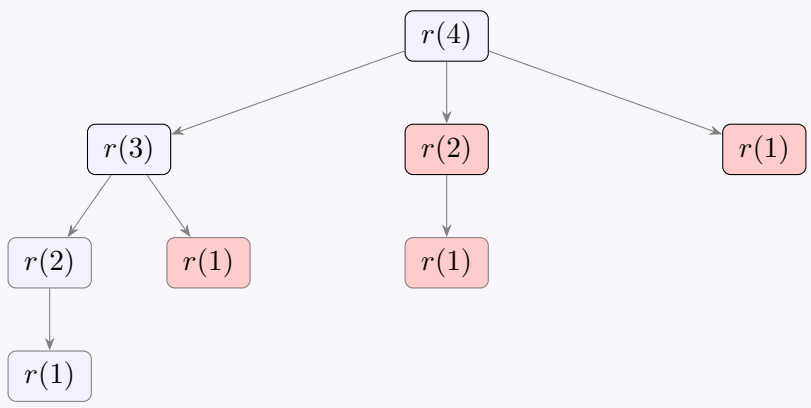
Answer

(a) Optimal Substructure (Cut-and-Paste Argument):
Let $r(n)$ = maximum profit for a bar of length n. Any optimal solution makes at least one cut, say at position k, yielding pieces of length k and $n-k$:

$$r(n) = \max_{1 \leq k \leq n} \big(p[k] + r(n-k)\big)$$

where $p[k]$ is the price of a bar of length k and $r(0) = 0$.
Claim: The sub-solution for the piece of length $n-k$ must also be optimal.
Proof (cut-and-paste): Suppose the optimal solution for length n splits at k, but the right piece of length $n-k$ is solved *sub-optimally* with profit $r'(n-k) < r(n-k)$. Then replacing that sub-solution with the optimal one gives total profit $p[k] + r(n-k) > p[k] + r'(n-k) = r(n)$, contradicting optimality. Therefore every sub-piece in an optimal solution must itself be solved optimally. ■

(b) Overlapping Subproblems (Recursion Tree for $n = 4$):



Observation: $r(1)$ is computed **4 times**, $r(2)$ is computed **2 times** (red/shaded nodes). For length n, naive recursion solves 2^{n-1} subproblems — exponential. Since the same subproblems recur, dynamic programming stores each $r(i)$ once and solves in $\Theta(n^2)$.

Q2. Given the two strings $X =$ ``AGGTAB'' and $Y =$ ``GXTYAB'', construct the dynamic programming (DP) table to compute the length of the Longest Common Subsequence (LCS) between X and Y. Clearly state the LCS and its length. **(15 marks)** [CSE 105 | 2023–24]

Answer

Recurrence:

$$c[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & X[i] = Y[j] \\ \max(c[i-1][j],\, c[i][j-1]) & \text{otherwise} \end{cases}$$

DP Table ($X =$ AGGTAB, $Y =$ GXTYAB):

	ε	G	X	T	Y	A	B
ε	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1
G	0	**1**	1	1	1	1	1
G	0	1	1	1	1	1	1
T	0	1	1	**2**	2	2	2
A	0	1	1	2	2	**3**	3
B	0	1	1	2	2	3	**4**

LCS length $= c[6][6] =$ **4**.
LCS = GTAB
Verification: G(2), T(4), A(5), B(6) appear in X in this order; G(1), T(3), A(5), B(6) appear in Y in this order. ✓

Q3. You are given a sequence of matrices $\langle A_1, A_2, \ldots, A_n \rangle$ and their dimensions $\langle p_0, p_1, p_2, \ldots, p_n \rangle$ such that matrix A_i has dimensions $p_{i-1} \times p_i$. The matrix-chain multiplication (MCM) problem aims to identify the minimum number of scalar multiplications necessary to compute the product of the given chain of matrices. Formulate a recurrence relation to solve this problem. Design an efficient

dynamic programming algorithm using the recurrence relation. **(15 marks)** [CSE 105 | 2023–24]

Answer

Recurrence: Let $m[i][j]$ = minimum scalar multiplications for $A_i \cdots A_j$.

$$m[i][j] = \begin{cases} 0 & i = j \\ \min\limits_{i \le k < j} \{m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j\} & i < j \end{cases}$$

Algorithm (bottom-up, fill by chain length):

Algorithm 109 MATRIX-CHAIN-ORDER(p, n)

```
for i = 1 to n do
    m[i][i] ← 0
end for
for l = 2 to n do                                   ▷ l = chain length
    for i = 1 to n − l + 1 do
        j ← i + l − 1
        m[i][j] ← ∞
        for k = i to j − 1 do                       ▷ try all splits
            q ← m[i][k] + m[k + 1][j] + p[i − 1] · p[k] · p[j]
            if q < m[i][j] then
                m[i][j] ← q
                s[i][j] ← k
            end if
        end for
    end for
end for
return m[1][n], s
```

Algorithm 110 PRINT-OPTIMAL-PARENS(s, i, j)

```
if i = j then
    print "A_i"
else
    print "("
    PRINT-OPTIMAL-PARENS(s, i, s[i][j])
    PRINT-OPTIMAL-PARENS(s, s[i][j] + 1, j)
    print ")"
end if
```

Time: $\Theta(n^3)$. **Space:** $\Theta(n^2)$.

Q4. You are given a sequence of matrices $\langle A_1, A_2, \ldots, A_n \rangle$ and their dimensions $\langle p_0, p_1, p_2, \ldots, p_n \rangle$ such that matrix A_i has dimensions $p_{i-1} \times p_i$. The matrix-chain multiplication (MCM) problem aims to identify the minimum number of scalar multiplications necessary to determine the product of the given chain of matrices.

(a) Formulate arguments to show that the MCM problem has the optimal substructure property.

(b) Compose a recursion tree diagram to show that MCM has the overlapping subproblem property.

(c) Design an efficient algorithm using the dynamic programming approach to solve this problem.

(8+7+10=25 marks) [CSE 105 | 2022–23]

Answer

Matrix Chain Multiplication (MCM): Optimal Substructure, Overlapping Subproblems, DP Algorithm

Setup:
Given $\langle A_1, A_2, \ldots, A_n \rangle$ with dimensions $\langle p_0, p_1, \ldots, p_n \rangle$ where A_i is $p_{i-1} \times p_i$. Find the minimum scalar multiplications to compute $A_1 A_2 \cdots A_n$.
Let $m[i][j]$ = minimum cost to multiply $A_i A_{i+1} \cdots A_j$.

Part (a): Optimal Substructure **(8 marks)**

Theorem: An optimal parenthesisation of $A_i \cdots A_j$ contains optimal parenthesisations of its subchains.
Proof:
Suppose the optimal split of $A_i \cdots A_j$ is at position k (i.e. $(A_i \cdots A_k)(A_{k+1} \cdots A_j)$) with total cost $m[i][j]$:

$$m[i][j] = m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j$$

Claim: $m[i][k]$ must be the optimal cost for $A_i \cdots A_k$.
Proof by contradiction: Suppose there exists a cheaper way to parenthesise $A_i \cdots A_k$ with cost $m'[i][k] < m[i][k]$. Then:

$$m'[i][k] + m[k+1][j] + p_{i-1} p_k p_j < m[i][k] + m[k+1][j] + p_{i-1} p_k p_j = m[i][j]$$

This contradicts the optimality of $m[i][j]$. Therefore $m[i][k]$ must be optimal. By symmetric argument, $m[k+1][j]$ must also be optimal. ■

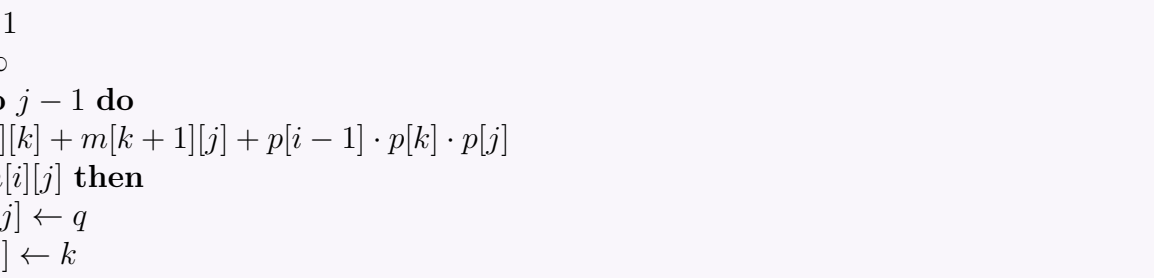
Consequence: The global optimal solution can be built from optimal solutions to subproblems ⇒ **optimal substructure holds**.

Part (b): Overlapping Subproblems **(7 marks)**

Recursion tree for $n = 4$, computing $m[1][4]$:

Part (b): Overlapping Subproblems **(7 marks)**

Recursion tree for $n = 4$, computing $m[1][4]$:



Observation: Subproblem $m[1][2]$ is computed when $k = 2$ (right child of root) and also as the left child when computing $m[1][3]$ under $k = 3$.
Similarly, $m[1][1]$ appears three times. In general, the number of distinct subproblems is only $\Theta(n^2)$, but naive recursion recomputes them exponentially many times ($\Omega(2^n)$ total calls).
Conclusion: MCM has overlapping subproblems ⇒ memoisation or bottom-up DP is essential.

Part (c): DP Algorithm **(10 marks)**

Recurrence:

$$m[i][j] = \begin{cases} 0 & \text{if } i = j \\ \min\limits_{i \le k < j} \left(m[i][k] + m[k+1][j] + p_{i-1} p_k p_j \right) & \text{if } i < j \end{cases}$$

Algorithm (bottom-up, fill by chain length):

Algorithm 111 MATRIX-CHAIN-ORDER(p, n)

```
1:  for i = 1 to n do
2:      m[i][i] ← 0
3:  end for
4:  for l = 2 to n do                                   ▷ l = chain length
5:      for i = 1 to n − l + 1 do
6:          j ← i + l − 1
7:          m[i][j] ← ∞
8:          for k = i to j − 1 do                       ▷ try all splits
9:              q ← m[i][k] + m[k + 1][j] + p[i − 1] · p[k] · p[j]
10:             if q < m[i][j] then
11:                 m[i][j] ← q
12:                 s[i][j] ← k
13:             end if
14:         end for
15:     end for
16: end for
17: return m[1][n], s
```

Reconstruction:

Algorithm 112 PRINT-OPTIMAL-PARENS(s, i, j)

```
1: if i = j then
2:     print "A_i"
3: else
4:     print "("
5:     PRINT-OPTIMAL-PARENS(s, i, s[i][j])
6:     PRINT-OPTIMAL-PARENS(s, s[i][j] + 1, j)
7:     print ")"
8: end if
```

Worked Example: A_1: 10×30, A_2: 30×5, A_3: 5×60, A_4: 60×2. Dimension array: $p = [10, 30, 5, 60, 2]$.
Chain length 2:

$$m[1][2] = p_0 \cdot p_1 \cdot p_2 = 10 \cdot 30 \cdot 5 = 1500$$
$$m[2][3] = p_1 \cdot p_2 \cdot p_3 = 30 \cdot 5 \cdot 60 = 9000$$
$$m[3][4] = p_2 \cdot p_3 \cdot p_4 = 5 \cdot 60 \cdot 2 = 600$$

Chain length 3:

$$\begin{aligned} m[1][3] &= \min\big(m[1][1] + m[2][3] + p_0p_1p_3,\ m[1][2] + m[3][3] + p_0p_2p_3\big) \\ &= \min(0 + 9000 + 10 \cdot 30 \cdot 60,\ 1500 + 0 + 10 \cdot 5 \cdot 60) \\ &= \min(27000,\ 4500) = 4500 \quad (k = 2) \\ m[2][4] &= \min\big(m[2][2] + m[3][4] + p_1p_2p_4,\ m[2][3] + m[4][4] + p_1p_3p_4\big) \\ &= \min(0 + 600 + 30 \cdot 5 \cdot 2,\ 9000 + 0 + 30 \cdot 60 \cdot 2) \\ &= \min(900,\ 12600) = 900 \quad (k = 2) \end{aligned}$$

Chain length 4:

k	Expression	Value
1	$m[1][1] + m[2][4] + p_0p_1p_4 = 0 + 900 + 10 \cdot 30 \cdot 2$	1500
2	$m[1][2] + m[3][4] + p_0p_2p_4 = 1500 + 600 + 10 \cdot 5 \cdot 2$	2200
3	$m[1][3] + m[4][4] + p_0p_3p_4 = 4500 + 0 + 10 \cdot 60 \cdot 2$	5700
Minimum		**1500** at $k = 1$

$m[1][4] = 1500$, achieved by $\big(A_1 \cdot (A_2 \cdot (A_3 \cdot A_4))\big)$.
Verification: A_3A_4: $5 \times 60 \cdot 60 \times 2 = 600$ mults $\Rightarrow 5 \times 2$.
$A_2(A_3A_4)$: $30 \times 5 \cdot 5 \times 2 = 300$ mults $\Rightarrow 30 \times 2$.
$A_1(A_2A_3A_4)$: $10 \times 30 \cdot 30 \times 2 = 600$ mults.
Total: $600 + 300 + 600 =$ **1500** ✓
Time Complexity:

- Three nested loops: l, i, k, each $O(n)$.
- Total: $\Theta(n^3)$ time.
- Space: $\Theta(n^2)$ for m and s tables.

$$\boxed{T(n) = \Theta(n^3), \quad S(n) = \Theta(n^2)}$$

Q5. Using a dynamic programming algorithm, determine the minimum edit distance alignment of the two strings $X =$ `ATCGGT` and $Y =$ `ACCDT`, where both the gap penalty and mismatch penalty are 1 and the cost of a match is 0. Show the DP table, the alignments, and mark the paths (which correspond to the alignments) in the DP table. **(10 marks)** **[CSE 105 | 2022–23]**

Answer

Minimum Edit Distance: $X =$ ATCGGT vs $Y =$ ACCDT

Setup:

- $X =$ `ATCGGT` $(m = 6)$, $Y =$ `ACCDT` $(n = 5)$
- Gap penalty $= 1$, Mismatch penalty $= 1$, Match cost $= 0$

Recurrence:

$$dp[i][j] = \min \begin{cases} dp[i-1][j-1] + \delta(X[i], Y[j]) & \text{(match/mismatch)} \\ dp[i-1][j] + 1 & \text{(gap in } Y\text{)} \\ dp[i][j-1] + 1 & \text{(gap in } X\text{)} \end{cases}$$

where $\delta(a, b) = 0$ if $a = b$, else 1.

DP Table & Path Traceback

	–	A	C	C	D	T
–	0	1	2	3	4	5
A	1	0	1	2	3	4
T	2	1	1	2	3	3
C	3	2	1	1	2	3
G	4	3	2	2	2	3
G	5	4	3	3	3	3
T	6	5	4	4	4	3

Path 1
Path 2
Path 3

Minimum edit distance $= dp[6][5] =$ **3**

Traceback Analysis: Starting from $dp[6][5] = 3$, we must follow valid moves backward. At $(5, 4)$, the path splits, and then splits again, giving exactly **3 optimal paths**:

- **Split 1 at** $(5,4)$**:** We can move from $(4,4)$ via a gap in Y $(2+1=3)$ [Leads to Path 1], **OR** move from $(4,3)$ via G-D mismatch $(2+1=3)$ [Leads to Paths 2 & 3].
- **Split 2 at** $(4,3)$**:** We can move from $(3,3)$ via a gap in Y $(1+1=2)$ [Leads to Path 2], **OR** move from $(3,2)$ via G-C mismatch $(1+1=2)$ [Leads to Path 3].

The 3 Optimal Alignments (M = Match, MM = Mismatch)

Alignment 1 (Follows Path 1: Blue Solid)

X:	A	T	C	G	G	T
	M	MM	M	MM	gap	M
Y:	A	C	C	D	–	T
Cost: $0+1+0+1+1+0=\mathbf{3}$						

Alignment 2 (Follows Path 2: Red Dashed)

X:	A	T	C	G	G	T
	M	MM	M	gap	MM	M
Y:	A	C	C	–	D	T
Cost: $0+1+0+1+1+0=\mathbf{3}$						

Alignment 3 (Follows Path 3: Green Dotted)

X:	A	T	C	G	G	T
	M	gap	M	MM	MM	M
Y:	A	–	C	C	D	T
Cost: $0+1+0+1+1+0=\mathbf{3}$						

Q6. Given two strings, the longest common subsequence (LCS) problem is to find the longest subsequence present in both strings in the same relative order, but not necessarily contiguous. For example, for $S_1 =$ ``ALGORITHM'' and $S_2 =$ ``LITHIUM'', the LCS is ``LITHM''. Present a dynamic programming algorithm to find the length of the LCS of two strings. Derive a proof of correctness of the algorithm and also derive its running time. **(8+6+3=17 marks)** [CSE 105 | 2021–22]

Answer

Longest Common Subsequence (LCS): DP Algorithm, Correctness, and Running Time

Problem Definition:
Given two strings $X = x_1x_2\cdots x_m$ and $Y = y_1y_2\cdots y_n$, find the length of their longest common subsequence.

Part 1: DP Algorithm **(8 marks)**

Optimal Substructure:
Let $L[i][j]$ = length of LCS of $X[1..i]$ and $Y[1..j]$.
Recurrence:

$$L[i][j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ L[i-1][j-1]+1 & \text{if } x_i = y_j \\ \max(L[i-1][j],\ L[i][j-1]) & \text{if } x_i \neq y_j \end{cases}$$

Algorithm 113 LCS-Length(X, Y, m, n)

```
for i = 0 to m do
    L[i][0] ← 0
end for
for j = 0 to n do
    L[0][j] ← 0
end for
for i = 1 to m do
    for j = 1 to n do
        if X[i] = Y[j] then
            L[i][j] ← L[i − 1][j − 1] + 1
        else
            L[i][j] ← max(L[i − 1][j], L[i][j − 1])
        end if
    end for
end for
return L[m][n]
```

Example: $S_1 =$ ALGORITHM ($m = 9$), $S_2 =$ LITHIUM ($n = 7$).
LCS Table (partial, showing key cells):

	–	L	I	T	H	I	U	M
–	0	0	0	0	0	0	0	0
A	0	0	0	0	0	0	0	0
L	0	1	1	1	1	1	1	1
G	0	1	1	1	1	1	1	1
O	0	1	1	1	1	1	1	1
R	0	1	1	1	1	1	1	1
I	0	1	2	2	2	2	2	2
T	0	1	2	3	3	3	3	3
H	0	1	2	3	4	4	4	4
M	0	1	2	3	4	4	4	5

LCS length $= L[9][7] = 5$. Traceback gives LCS = **LITHM**.

Part 2: Proof of Correctness **(6 marks)**

Claim: $L[i][j]$ correctly gives the length of the LCS of $X[1..i]$ and $Y[1..j]$.
Proof by strong induction on $i + j$:
Base case: When $i = 0$ or $j = 0$, one string is empty, so LCS length $= 0 = L[i][j]$. ✓
Inductive step: Assume $L[i'][j']$ is correct for all $i' + j' < i + j$. We show $L[i][j]$ is correct.
Case 1: $x_i = y_j$.
Any LCS of $X[1..i]$ and $Y[1..j]$ either includes both x_i and y_j or it does not.

- If it includes $x_i = y_j$: the remaining LCS is a common subsequence of $X[1..i-1]$ and $Y[1..j-1]$, so its length is at most $L[i-1][j-1]$. Thus LCS length $\leq L[i-1][j-1] + 1$.
- We can always extend any LCS of $X[1..i-1]$ and $Y[1..j-1]$ by appending $x_i = y_j$, giving a common subsequence of length $L[i-1][j-1] + 1$.

Therefore LCS $= L[i-1][j-1] + 1$. ✓
Case 2: $x_i \neq y_j$.
Both x_i and y_j cannot both appear at the *end* of the same LCS. Hence:

- If the LCS does not end with x_i: LCS of $X[1..i]$ and $Y[1..j]$ is the same as LCS of $X[1..i-1]$ and $Y[1..j] = L[i-1][j]$.
- If the LCS does not end with y_j: LCS = LCS of $X[1..i]$ and $Y[1..j-1] = L[i][j-1]$.

Since at least one must hold: LCS $= \max(L[i-1][j], L[i][j-1])$. ✓
By induction, $L[m][n]$ correctly gives the LCS length. ■

Part 3: Running Time **(3 marks)**

- The DP table has $(m+1)(n+1)$ cells.
- Each cell is filled in $O(1)$ time (one comparison, one max, constant work).
- Total time: $\Theta(mn)$.
- Space: $\Theta(mn)$ for the table (reducible to $O(\min(m, n))$ if only length is needed).

$$\boxed{T(m, n) = \Theta(mn)}$$

Q7. Solve the following 0-1 knapsack problem instance (capacity = 6) using dynamic programming. Show the DP table and find the optimal set of items.

Item	Weight	Value
1	2	8
2	1	7
3	2	12
4	2	6
5	2	10

(15 marks) **[CSE 203 | 2021–22]**

Answer

Recurrence: $dp[i][w] = \max(dp[i-1][w],\ dp[i-1][w - w_i] + v_i)$ if $w_i \leq w$, else $dp[i-1][w]$.
DP Table (rows = items 1–5, cols = capacity 0–6):
DP Table:

	w=0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1(w=2, v=8)	0	0	8	8	8	8	8
2(w=1, v=7)	0	7	8	15	15	15	15
3(w=2, v=12)	0	7	12	19	20	27	27
4(w=2, v=6)	0	7	12	19	20	27	27
5(w=2, v=10)	0	7	12	19	22	**29**	**30**

Optimal value = $dp[5][6] = \mathbf{30}$.
Traceback: $dp[5][6] = 30 \neq dp[4][6] = 27 \Rightarrow$ include item 5, $w \leftarrow 4$; $dp[4][4] = 20 = dp[3][4] \Rightarrow$ skip item 4; $dp[3][4] = 20 \neq dp[2][4] = 15 \Rightarrow$ include item 3, $w \leftarrow 2$; $dp[2][2] = 8 = dp[1][2] \Rightarrow$ skip item 2; $dp[1][2] = 8 \neq dp[0][2] = 0 \Rightarrow$ include item 1.
Optimal set: {1,3,5}, weight $= 2 + 2 + 2 = 6$, value $= 8 + 12 + 10 = \mathbf{30}$.

Q8. Find the best alignment (minimum edit distance) of the following two DNA sequences using dynamic programming. Use gap penalty = 3, mismatch penalty = 2, match penalty = 0. Show the DP table, the alignment, and mark the corresponding path.

`CCAGTGC` vs `CGATC`

(15 marks) [CSE 203 | 2021–22]

Answer

DP Table (gap penalty=3, mismatch=2, match=0):

	ϵ	C	G	A	T	C
ϵ	0	3	6	9	12	15
C	3	0	3	6	9	12
C	6	3	2	5	8	9
A	9	6	5	2	5	8
G	12	9	6	5	4	7
T	15	12	9	8	5	6
G	18	15	12	11	8	7
C	21	18	15	14	11	**8**

Minimum alignment cost = 8.
Optimal alignment (traceback):

```
X: C C A G T G C
Y: C G A - T - C
```

(Two gaps inserted in Y; one mismatch C↔G.) Cost: $0 + 2 + 0 + 3 + 0 + 3 + 0 = 8$.

Q9. Give a dynamic programming algorithm to find the length of the longest common subsequence (LCS) of two strings. (A common subsequence appears in both strings in the same relative order but not necessarily contiguously.) **(20 marks)** [CSE 203 | 2021–22]

Answer

Algorithm 114 LCS-Length(X, Y)

```
1: m ← |X|; n ← |Y|
2: for i ← 0 to m do c[i][0] ← 0
3: end for
4: for j ← 0 to n do c[0][j] ← 0
5: end for
6: for i ← 1 to m do
7:     for j ← 1 to n do
8:         if X[i] = Y[j] then c[i][j] ← c[i − 1][j − 1] + 1; b[i][j] ←"↖"
9:         else if c[i − 1][j] ≥ c[i][j − 1] then c[i][j] ← c[i − 1][j]; b[i][j] ←"↑"
10:        elsec[i][j] ← c[i][j − 1]; b[i][j] ←"←"
11:        end if
12:    end for
13: end for
14: return c, b
```

Complexity Analysis. Time: $\Theta(mn)$. Space: $\Theta(mn)$ for table; $\Theta(\min(m, n))$ if only length needed. Printing LCS: $O(m + n)$ by following b table.

Q10. Derive a recursive definition to find $m[i, j]$ (the minimum number of multiplications to compute $A_i \cdot A_{i+1} \cdots A_j$). With a recursion

tree, describe the overlapping sub-problem property of Matrix Chain Multiplication (MCM). Then, given six matrices $A_1(30 \times 35)$, $A_2(35 \times 15)$, $A_3(15 \times 5)$, $A_4(5 \times 10)$, $A_5(10 \times 20)$, $A_6(20 \times 25)$ and the following partial DP table, give an ordering to populate the marked cells and calculate $m[2, 5]$:

	1	2	3	4	5	6
1	0	15750	7875	$m[1,4]$	$m[1,5]$	$m[1,6]$
2		0	2625	4375	$m[2,5]$	$m[2,6]$
3			0	750	2500	$m[3,6]$
4				0	1000	$m[4,6]$
5					0	$m[5,6]$
6						0

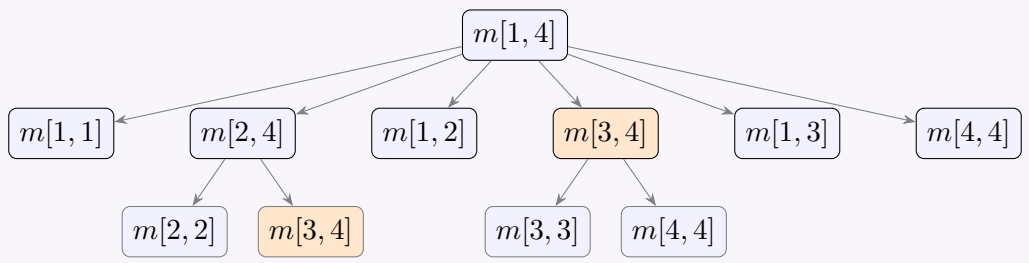
(7+6+10 marks) [CSE 203 | 2020–21]

Answer

Recursive Definition of $m[i, j]$:
Let $m[i, j]$ = minimum number of scalar multiplications to compute $A_i \cdot A_{i+1} \cdots A_j$, where A_k has dimension $p_{k-1} \times p_k$.

$$m[i,j] = \begin{cases} 0 & \text{if } i = j \\ \min\limits_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{i-1} \cdot p_k \cdot p_j\} & \text{if } i < j \end{cases}$$

Overlapping Subproblems (Recursion Tree for $m[1, 4]$):



Overlapping subproblem: $m[3, 4]$ appears *twice* in the recursion tree for $m[1, 4]$. DP avoids recomputation by storing results in a table.

Order to Populate Marked Cells:
Dependencies must be resolved bottom-up (shorter chains first):

$$m[5,6] \;\to\; m[4,6] \;\to\; m[2,5] \;\to\; m[1,4] \;\to\; m[1,5] \;\to\; m[2,6] \;\to\; m[1,6]$$

Computing $m[2, 5]$:
Dimensions: $p = [30, 35, 15, 5, 10, 20, 25]$, so $p_1 = 35,\ p_2 = 15,\ p_3 = 5,\ p_4 = 10,\ p_5 = 20$.

$$m[2,5] = \min_{2 \le k \le 4} \{m[2,k] + m[k+1,5] + p_1 \cdot p_k \cdot p_5\}$$

k	**Expression**	**Value**
2	$m[2,2] + m[3,5] + p_1 p_2 p_5 = 0 + 2500 + 35 \cdot 15 \cdot 20$	13000
3	$m[2,3] + m[4,5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \cdot 5 \cdot 20$	7125
4	$m[2,4] + m[5,5] + p_1 p_4 p_5 = 4375 + 0 + 35 \cdot 10 \cdot 20$	11375
Min		**7125** (at $k = 3$)

$$\therefore \quad m[2,5] = \boxed{7125}$$

Optimal split for $m[2, 5]$: $k = 3$, i.e. $(A_2A_3)(A_4A_5)$ with **7125 multiplications**.

Q11. At the beginning of the COVID-19 pandemic, two biologists were identifying the similarity between COVID-19 and SARS viruses using genome sequences $S'_1 = \langle A, T, T, G, C, A, T \rangle$ and $S'_2 = \langle T, A, G, C, C, T \rangle$.

(a) (Dr. Brown's task) Write the recursive definition for the Longest Common Subsequence (LCS) problem and populate the DP table with arrows showing how cell values propagate. Find the length and the LCS itself.

(b) (Dr. William's task) Write the recursive definition for the edit distance problem using operations {Insert (cost 1), Delete (cost 1), Replacement (cost 1), Keep (cost 0)}. Populate the DP table and show the edit distance cost and sequence of required operations.

(15 marks) [CSE 203 | 2020–21]

Answer

(a) LCS — Recursive Definition:

$$c[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & \text{if } X[i] = Y[j] \\ \max(c[i-1][j],\, c[i][j-1]) & \text{otherwise} \end{cases}$$

DP Table ($X = \langle A, T, T, G, C, A, T \rangle$, $Y = \langle T, A, G, C, C, T \rangle$):

	ϵ	T	A	G	C	C	T
ϵ	0	0	0	0	0	0	0
A	0	0	1	1	1	1	1
T	0	1	1	1	1	1	2
T	0	1	1	1	1	1	2
G	0	1	1	2	2	2	2
C	0	1	1	2	3	3	3
A	0	1	2	2	3	3	3
T	0	1	2	2	3	3	**4**

LCS length = 4.

Multiple LCS Tracebacks (Branching):
Tracing back from $c[7][6] = 4$, we encounter branches where $c[i-1][j] = c[i][j-1]$. This results in 2 distinct valid Longest Common Subsequences:

- **LCS 1:** $\langle A, G, C, T \rangle$ (Matching A at $X[1], Y[2]$)
- **LCS 2:** $\langle T, G, C, T \rangle$ (Matching T at $X[2]$ or $X[3]$ with $Y[1]$)

(b) Edit Distance — Recursive Definition:

$$dp[i][j] = \begin{cases} j & \text{if } i = 0 \\ i & \text{if } j = 0 \\ dp[i-1][j-1] & \text{if } X[i] = Y[j] \\ 1 + \min(dp[i-1][j],\, dp[i][j-1],\, dp[i-1][j-1]) & \text{otherwise} \end{cases}$$

DP Table:

	ϵ	T	A	G	C	C	T
ϵ	0	1	2	3	4	5	6
A	1	1	1	2	3	4	5
T	2	1	2	2	3	4	4
T	3	2	2	3	3	4	4
G	4	3	3	2	3	4	5
C	5	4	4	3	2	3	4
A	6	5	4	4	3	3	4
T	7	6	5	5	4	4	**3**

Edit distance = 3.

Sequence of Operations (Traceback):
Unlike LCS, tracing back from $dp[7][6] = 3$ mathematically yields only **one** strictly optimal minimum-cost path for this matrix.

(a) **Delete** A (from X)
(b) **Keep** T
(c) **Replace** $T \to A$
(d) **Keep** G
(e) **Keep** C
(f) **Replace** $A \to C$
(g) **Keep** T

Q12. Discuss a problem scenario where adopting memoization over bottom-up dynamic programming can result in a faster solution. Justify your answer. **(5 marks)** [CSE 203 | 2020–21]

Answer

When memoization can be faster:
In problems where *not all sub-problems need to be solved*, memoization (top-down) only computes sub-problems that are actually reachable from the original problem, while bottom-up always fills the entire table.
Example: the longest path in a DAG. For a DAG with V vertices and E edges, if the query is about a specific vertex s and the DAG is sparse, memoized DFS from s visits only the $O(V + E)$ reachable sub-problems, whereas bottom-up DP would process all

V vertices unconditionally.
Justification: if only a fraction of the $O(n^2)$ (or $O(2^n)$) sub-problems are needed (e.g., sparse matrix chain, or a DAG with limited reachability), memoization skips unreachable states entirely. Bottom-up computes everything, wasting time on irrelevant states.

Q13. Apply dynamic programming to calculate the edit distance of the following two DNA sequences, where the cost of insertion or deletion is 2 and the cost of substitution is 1. Write the recurrence relation, explain the intuition behind it, populate the DP table, and calculate the edit distance.

`TGCATAT` vs `ATCCGA`

(15 marks) [CSE 203 | 2019–20]

Answer

Edit Distance: TGCATAT vs ATCCGA
(Insertion/Deletion = 2, Substitution = 1, Match = 0)

Recurrence Relation:
Let $dp[i][j]$ = minimum edit distance between $X[1..i]$ and $Y[1..j]$:

$$dp[i][j] = \begin{cases} 2i & j = 0 \\ 2j & i = 0 \\ dp[i-1][j-1] & X[i] = Y[j] \\ \min \begin{cases} dp[i-1][j-1] + 1 & \text{(Substitute)} \\ dp[i-1][j] + 2 & \text{(Delete)} \\ dp[i][j-1] + 2 & \text{(Insert)} \end{cases} & X[i] \neq Y[j] \end{cases}$$

DP Table (X = rows, Y = cols):

	–	**A**	**T**	**C**	**C**	**G**	**A**
–	0	2	4	6	8	10	12
T	2	1	2	4	6	8	10
G	4	3	2	3	5	6	8
C	6	5	4	2	3	5	7
A	8	6	6	4	3	4	5
T	10	8	6	6	5	4	5
A	12	10	8	7	7	6	4
T	14	12	10	9	8	8	**6**

Optimal Alignment:
Tracing back strictly from $dp[7][6]$ results in the following exact alignment path:

X:	T	G	C	A	T	A	T
Op:	sub	sub	M	sub	sub	M	del
Y:	A	T	C	C	G	A	–
Cost:	1	1	0	1	1	0	2

Final Edit Distance = $dp[7][6]$ = **6**.

Q14. Briefly explain the steps to be followed for solving a problem using dynamic programming. **(10 marks)** [CSE 203 | 2018–19]

Answer

Steps for Solving a Problem Using Dynamic Programming

Dynamic programming (DP) solves optimization or counting problems by breaking them into overlapping subproblems and storing results to avoid recomputation. The standard methodology follows these steps:

Step 1. Characterize the Structure of an Optimal Solution

Identify whether the problem has **optimal substructure**: an optimal solution to the problem contains optimal solutions to subproblems. Show that the global optimum can be expressed in terms of optimal subproblem solutions.

Example (LCS): An LCS of $X[1..m]$ and $Y[1..n]$ contains an LCS of some prefix $X[1..i]$ and $Y[1..j]$.

Step 2. Define the Subproblems and State

Clearly define what $dp[i]$ or $dp[i][j]$ (the DP state) represents. The state must capture all information needed to solve smaller subproblems independently.

Example (Knapsack): $dp[i][w]$ = maximum value using items $\{1, \ldots, i\}$ with capacity w.

Step 3. Write the Recurrence Relation

Express the solution to a subproblem in terms of solutions to smaller subproblems. Identify:

- **Base cases**: smallest subproblems with known answers (e.g. $dp[0] = 0$).
- **Recursive case**: how to combine subproblem solutions.

Example (Edit Distance): $dp[i][j] = \min(dp[i-1][j-1] + \delta,\ dp[i-1][j] + 1,\ dp[i][j-1] + 1)$

Step 4. Verify Overlapping Subproblems

Confirm that subproblems recur multiple times in the recursion tree. This justifies memoisation or bottom-up tabulation (otherwise, divide-and-conquer is sufficient).

Example (Fibonacci): $F(3)$ is called during computation of both $F(4)$ and $F(5)$.

Step 5. Choose Implementation: Top-down or Bottom-up

- **Top-down (Memoisation):** Write the recursive solution; cache results in a table. Natural to implement; may have recursion overhead.
- **Bottom-up (Tabulation):** Fill the table iteratively in an order where subproblems are solved before they are needed. Usually faster and avoids stack overflow.

Step 6. Compute the Solution Bottom-up (or with Memoisation)

Fill the DP table according to the recurrence, starting from the base cases and proceeding to the final answer. Ensure the correct fill order (e.g. row by row, by chain length, etc.).

Step 7. Determine the Final Answer

Read off the answer from the appropriate DP table entry (e.g. $dp[m][n]$ for two-string problems, $dp[n]$ for single-sequence problems).

Step 8. Reconstruct the Optimal Solution (Traceback)

If the actual solution (not just its value) is needed, trace back through the DP table using stored decisions (a separate choice$[i][j]$ table) or by re-deriving decisions from the values.

Example: In LCS, at each cell check whether the diagonal, up, or left move was used and reconstruct the common subsequence.

Step 9. Analyse Time and Space Complexity

Count the number of subproblems and the time per subproblem:

$$T = (\text{number of subproblems}) \times (\text{time per subproblem})$$

Optimise space if possible (e.g. rolling array reduces $O(mn)$ to $O(\min(m, n))$).

Summary Checklist:

Step	Key Question
1. Optimal substructure	Does the optimum use optimal sub-solutions?
2. Define state	What does $dp[i][j]$ mean?
3. Recurrence	How to compute from smaller states?
4. Overlapping subproblems	Are subproblems reused?
5. Implementation	Top-down or bottom-up?
6. Fill table	In what order?
7. Final answer	Which cell?
8. Traceback	How to recover the solution?
9. Complexity	Time and space?

Q15. Dream coins are used by the people of Dreamland. Only the following coins are available: \$1, \$7, \$11, and \$15.

(a) Write an example of an amount where the greedy strategy for the Coin-Change problem does *not* provide an optimal solution. Mention both the greedy and optimal solutions.

(b) Using dynamic programming, find the minimum number of coins that add up to \$20.

(10 marks) **[CSE 203 | 2018–19]**

Answer

(a) Greedy Failure — Counter-Example

The greedy strategy always picks the *largest coin not exceeding* the remaining amount. Consider the target **\$21**.

Step 1. Remaining $= 21$. Largest coin ≤ 21 is \$15. Pick \$15. Remaining $= 6$.

Step 2. Remaining $= 6$. No coin ≤ 6 except \$1. Pick \$1 six times. Remaining $= 0$.

$$\$15 \;+\; \underbrace{\$1+\$1+\$1+\$1+\$1+\$1}_{6\text{ coins}} \;=\; \$21 \qquad \Rightarrow \qquad \textbf{7 coins}$$

$$\$7+\$7+\$7 \;=\; \$21 \qquad \Rightarrow \qquad \textbf{3 coins}$$

Conclusion: For \$21, greedy uses **7 coins** while optimal uses **3 coins**. Greedy fails because choosing the locally largest coin (\$15) prevents the globally better choice of three \$7 coins.

(b) Dynamic Programming for \$20

Let $dp[v]$ = minimum number of coins to make amount v. **Recurrence:**

$$dp[v] \;=\; \min_{c\,\in\,\{1,7,11,15\}} \big(dp[v-c]+1\big), \qquad dp[0]=0.$$

DP Table:

v	$dp[v]$	Best coin chosen
0	**0**	— (base case)
1	**1**	\$1
2	**2**	\$1
3	**3**	\$1
4	**4**	\$1
5	**5**	\$1
6	**6**	\$1
7	**1**	\$7 $dp[0]+1$
8	**2**	\$7 $dp[1]+1$
9	**3**	\$7 $dp[2]+1$
10	**4**	\$7 $dp[3]+1$
11	**1**	\$11 $dp[0]+1$
12	**2**	\$11 $dp[1]+1$
13	**3**	\$11 $dp[2]+1$
14	**2**	\$7 $dp[7]+1$
15	**1**	\$15 $dp[0]+1$
16	**2**	\$15 $dp[1]+1$
17	**3**	\$15 $dp[2]+1$
18	**2**	\$11 $dp[7]+1$
19	**3**	\$11 $dp[8]+1$
20	**4**	see computation below

Computing $dp[20]$:

Coin c	**Sub-problem**	**Value**	**Coins**
\$1	$dp[19]+1$	$3+1$	4
\$7	$dp[13]+1$	$3+1$	4
\$11	$dp[9]+1$	$3+1$	4
\$15	$dp[5]+1$	$5+1$	6
Minimum			**4**

$$\therefore \quad dp[20] \;=\; \min(4,\,4,\,4,\,6) \;=\; \boxed{4}$$

Minimum coins to make \$20 = 4.

Optimal decomposition: $\$11+\$7+\$1+\$1=\$20$

Q16. Consider the following 0/1 knapsack problem instance (capacity = 7):

Item	Weight	Value
1	1	50
2	3	60
3	5	100
4	2	70
5	2	80

Solve this instance using dynamic programming. Show the DP table and find all solutions using the table. **(15 marks)** [CSE 203 | 2018–19]

Answer

We solve the 0/1 Knapsack problem using dynamic programming.

Let $dp[i][w]$ = maximum value using items $1 \dots i$ with capacity w.

Recurrence:

$$dp[i][w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ dp[i-1][w] & \text{if } w_i > w \\ \max\big(dp[i-1][w],\ dp[i-1][w-w_i] + v_i\big) & \text{otherwise} \end{cases}$$

DP Table ($dp[i][w]$, rows = items, columns = capacity):

Item i \ w	**1**	**2**	**3**	**4**	**5**	**6**	**7**
0 (none)	0	0	0	0	0	0	0
1 (w=1, v=50)	50	50	50	50	50	50	50
2 (w=3, v=60)	50	50	60	110	110	110	110
3 (w=5, v=100)	50	50	60	110	110	**150**	150
4 (w=2, v=70)	50	70	120	120	130	180	180
5 (w=2, v=80)	50	80	130	150	200	200	**210**

$$\therefore \quad \text{Maximum value} = \boxed{210}$$

Traceback to Find All Optimal Solutions

Starting from $dp[5][7] = 210$. At each step, if $dp[i][w] \neq dp[i-1][w]$, item i *was included*; otherwise it was skipped. If both give the same value, *both branches* yield optimal solutions.

Step	**Item i**	$dp[i][w]$	$dp[i-1][w]$	**Decision**
1	5	$dp[5][7] = 210$	$dp[4][7] = 180$	$210 \neq 180 \Rightarrow$ **Include item 5**
		Move to $dp[4][7-2] = dp[4][5] = 130$		
2	4	$dp[4][5] = 130$	$dp[3][5] = 110$	$130 \neq 110 \Rightarrow$ **Include item 4**
		Move to $dp[3][5-2] = dp[3][3] = 60$		
3	3	$dp[3][3] = 60$	$dp[2][3] = 60$	$60 = 60 \Rightarrow$ **Skip item 3**
		Move to $dp[2][3] = 60$		
4	2	$dp[2][3] = 60$	$dp[1][3] = 50$	$60 \neq 50 \Rightarrow$ **Include item 2**
		Move to $dp[1][3-3] = dp[1][0] = 0$		
5	1	$dp[1][0] = 0$	$dp[0][0] = 0$	$0 = 0 \Rightarrow$ **Skip item 1**

Items selected: 2, 4, 5

$$\text{Weight} = 3 + 2 + 2 = 7 \leq 7 \qquad \text{Value} = 60 + 70 + 80 = \mathbf{210}$$

Note: At step 3, item 3 could also be *included* if remaining capacity permits. But $w_3 = 5 > 3$ (remaining capacity), so item 3 cannot be included — the skip is forced, not a choice. Hence there is exactly **one unique optimal solution**: $\{2, 4, 5\}$ with value **210**.

Q17. Find all longest common subsequences of $X = \langle T, A, A, G, U, A, T, U \rangle$ and $Y = \langle A, U, A, G, T, U, T \rangle$ using dynamic programming. Also find a shortest common supersequence of X and Y using an LCS of X and Y. **(15 marks)** [CSE 203 | 2018–19]

Answer

All LCS of $X = \langle T, A, A, G, U, A, T, U\rangle$ and $Y = \langle A, U, A, G, T, U, T\rangle$

Step 1: Build the LCS DP Table
$m = 8$, $n = 7$.

$L[i][j]$	–	A	U	A	G	T	U	T
	0	1	2	3	4	5	6	7
– i=0	0	0	0	0	0	0	0	0
T i=1	0	0	0	0	0	1	1	1
A i=2	0	1	1	1	1	1	1	1
A i=3	0	1	1	2	2	2	2	2
G i=4	0	1	1	2	3	3	3	3
U i=5	0	1	2	2	3	3	4	4
A i=6	0	1	2	3	3	3	4	4
T i=7	0	1	2	3	3	4	4	5
U i=8	0	1	2	3	3	4	5	5

LCS length $= L[8][7] = \mathbf{5}$.

Step 2: Find All LCS by Traceback
Trace back from $L[8][7] = 5$. At each cell (i, j):
- If $X[i] = Y[j]$: match, go to $(i-1, j-1)$, prepend character.
- Else if $L[i-1][j] > L[i][j-1]$: go up to $(i-1, j)$.
- Else if $L[i][j-1] > L[i-1][j]$: go left to $(i, j-1)$.
- If both equal: **branch** (multiple LCS).

Traceback results in 3 branches (all length 5):

LCS	Sequence
1	$\langle A, A, G, U, T\rangle$
2	$\langle A, A, G, T, U\rangle$
3	$\langle A, U, A, T, U\rangle$

Step 3: Shortest Common Supersequence (SCS)
The length of the SCS of X and Y using an LCS is:

$$|SCS| = |X| + |Y| - |LCS| = 8 + 7 - 5 = \mathbf{10}$$

Construction using LCS 1 $= \langle A, A, G, U, T\rangle$:
We use two pointers to traverse X and Y. We add non-LCS characters from both strings before we add the matching LCS character.

LCS Char	**Add from X**	**Add from Y**	**Current SCS**
–	T (before A)	–	T
A	–	–	T, **A**
–	–	U (between A and A)	T, A, U
A	–	–	T, A, U, **A**
G	–	–	T, A, U, A, **G**
–	–	T (between G and U)	T, A, U, A, G, T
U	–	–	T, A, U, A, G, T, **U**
–	A (between U and T)	–	T, A, U, A, G, T, U, A
T	–	–	T, A, U, A, G, T, U, A, **T**
(End)	U (after T)	–	T, A, U, A, G, T, U, A, T, U

Final SCS:

$$\langle T,\ A,\ U,\ A,\ G,\ T,\ U,\ A,\ T,\ U\rangle$$

Verification:
- Length = 10 ✓
- Contains $X = \langle T, A, A, G, U, A, T, U\rangle$ as subsequence ✓
- Contains $Y = \langle A, U, A, G, T, U, T\rangle$ as subsequence ✓

Q18. Suppose you run a bus lending business, and each request has the format (start time, end time). Find all solutions for the maximum number of requests that can be satisfied by:

(a) Earliest start time

(b) Shortest interval

(c) Earliest end time

Use the following requests: $\{[1,6),[4,8),[6,8),[9,15),[6,9),[11,15),[2,6)\}$.
(12 marks) [CSE 203 | 2018–19]

Answer

The requests are:

$$\{[1,6),\ [4,8),\ [6,8),\ [9,15),\ [6,9),\ [11,15),\ [2,6)\}$$

1. Earliest Start Time

Sort by start time (ascending):

Order	Interval	Decision
1	$[1,6)$	**Accept**
2	$[2,6)$	**Reject** (conflicts with $[1,6)$)
3	$[4,8)$	**Reject** (conflicts with $[1,6)$)
4	$[6,8)$	**Accept**
5	$[6,9)$	**Reject** (conflicts with $[6,8)$)
6	$[9,15)$	**Accept**
7	$[11,15)$	**Reject** (conflicts with $[9,15)$)

Selected: $\{[1,6),\ [6,8),\ [9,15)\}$ **Count: 3**

2. Shortest Interval

Sort by interval length ($=$ end $-$ start) ascending:

Order	Interval	Length	Decision
1	$[6,8)$	2	**Accept**
2	$[6,9)$	3	**Reject** (conflicts with $[6,8)$)
3	$[2,6)$	4	**Accept**
4	$[4,8)$	4	**Reject** (conflicts with $[2,6)$ and $[6,8)$)
5	$[11,15)$	4	**Accept**
6	$[1,6)$	5	**Reject** (conflicts with $[2,6)$)
7	$[9,15)$	6	**Reject** (conflicts with $[11,15)$)

Selected: $\{[6,8),\ [2,6),\ [11,15)\}$ **Count: 3**

3. Earliest End Time (Optimal Greedy Strategy)

Sort by end time (ascending):

Order	Interval	Decision
1	$[1,6)$	**Accept**
2	$[2,6)$	**Reject** (conflicts with $[1,6)$)
3	$[4,8)$	**Reject** (conflicts with $[1,6)$)
4	$[6,8)$	**Accept**
5	$[6,9)$	**Reject** (conflicts with $[6,8)$)
6	$[9,15)$	**Accept**
7	$[11,15)$	**Reject** (conflicts with $[9,15)$)

Selected: $\{[1,6),\ [6,8),\ [9,15)\}$ **Count: 3 (Optimal)**

Conclusion: In this specific instance, all three strategies yield **3 requests**, but only **Earliest End Time** is guaranteed to be optimal for all Interval Scheduling problems.

Q19. Let $A_1(11\times4)$, $A_2(4\times5)$, $A_3(5\times3)$, $A_4(3\times6)$, $A_5(6\times7)$, $A_6(7\times1)$ be six matrices. Compute $m[1,6]$ given: $m[1,3]=35$, $m[2,4]=132$, $m[1,5]=95$, $m[2,5]=270$, $m[2,6]=95$, $m[4,5]=126$, $m[4,6]=60$. **(13 marks)** [CSE 203 | 2018–19]

Answer

Given matrices: $A_1(11 \times 4)$, $A_2(4 \times 5)$, $A_3(5 \times 3)$, $A_4(3 \times 6)$, $A_5(6 \times 7)$, $A_6(7 \times 1)$.
So the dimension array is $p = [11, 4, 5, 3, 6, 7, 1]$, i.e. $p_0 = 11,\ p_1 = 4,\ p_2 = 5,\ p_3 = 3,\ p_4 = 6,\ p_5 = 7,\ p_6 = 1$.

Recurrence:

$$m[i, j] = \min_{i \le k < j} \{m[i, k] + m[k + 1, j] + p_{i-1} \cdot p_k \cdot p_j\}$$

We compute $m[1, 6]$ using all splits $k = 1, 2, 3, 4, 5$:

k	Expression	Value
1	$m[1,1] + m[2,6] + p_0p_1p_6 = 0 + 95 + 11 \cdot 4 \cdot 1$	$0 + 95 + 44 = 139$
2	$m[1,2] + m[3,6] + p_0p_2p_6$ (need $m[1,2]$ and $m[3,6]$)	see below
3	$m[1,3] + m[4,6] + p_0p_3p_6 = 35 + 60 + 11 \cdot 3 \cdot 1$	$35 + 60 + 33 = 128$
4	$m[1,4] + m[5,6] + p_0p_4p_6$ (need $m[1,4]$ and $m[5,6]$)	see below
5	$m[1,5] + m[6,6] + p_0p_5p_6 = 95 + 0 + 11 \cdot 7 \cdot 1$	$95 + 0 + 77 = 172$

Computing missing values for $k = 2$:

$$m[1, 2] = p_0 \cdot p_1 \cdot p_2 = 11 \cdot 4 \cdot 5 = 220$$

$$m[3, 6] : \text{ need to compute it using the recurrence.}$$

Split $k = 3$: $m[3,3] + m[4,6] + p_2p_3p_6 = 0 + 60 + 5 \cdot 3 \cdot 1 = 75$.
Split $k = 4$: $m[3,4] + m[5,6] + p_2p_4p_6$; $m[3,4] = p_2p_3p_4 = 5 \cdot 3 \cdot 6 = 90$, $m[5,6] = p_4p_5p_6 = 6 \cdot 7 \cdot 1 = 42$. So $90 + 42 + 5 \cdot 6 \cdot 1 = 90 + 42 + 30 = 162$.
Split $k = 5$: $m[3,5] + m[6,6] + p_2p_5p_6$. We need $m[3,5]$ separately:
$\min_k\{m[3,k] + m[k+1,5] + p_2p_kp_5\}$ for $k = 3, 4$:
$k = 3$: $m[3,3] + m[4,5] + p_2p_3p_5 = 0 + 126 + 5 \cdot 3 \cdot 7 = 0 + 126 + 105 = 231$ (using given $m[4,5] = 126$).
$k = 4$: $m[3,4] + m[5,5] + p_2p_4p_5 = 90 + 0 + 5 \cdot 6 \cdot 7 = 90 + 0 + 210 = 300$.
So $m[3,5] = \min(231, 300) = 231$.
Then $k = 5$ for $m[3,6]$: $231 + 0 + 5 \cdot 7 \cdot 1 = 231 + 35 = 266$.

$$\therefore \quad m[3, 6] = \min(75,\ 162,\ 266) = 75$$

Now $k = 2$: $m[1,2] + m[3,6] + p_0p_2p_6 = 220 + 75 + 11 \cdot 5 \cdot 1 = 220 + 75 + 55 = 350$.

Computing missing values for $k = 4$:

$$m[1, 4] : \text{ use split } k = 1, 2, 3.$$

$k = 1$: $m[1,1] + m[2,4] + p_0p_1p_4 = 0 + 132 + 11 \cdot 4 \cdot 6 = 132 + 264 = 396$.
$k = 2$: $m[1,2] + m[3,4] + p_0p_2p_4 = 220 + 90 + 11 \cdot 5 \cdot 6 = 220 + 90 + 330 = 640$.
$k = 3$: $m[1,3] + m[4,4] + p_0p_3p_4 = 35 + 0 + 11 \cdot 3 \cdot 6 = 35 + 0 + 198 = 233$.

$$\therefore \quad m[1, 4] = \min(396,\ 640,\ 233) = 233$$

$$m[5, 6] = p_4 \cdot p_5 \cdot p_6 = 6 \cdot 7 \cdot 1 = 42$$

$k = 4$: $m[1,4] + m[5,6] + p_0p_4p_6 = 233 + 42 + 11 \cdot 6 \cdot 1 = 233 + 42 + 66 = 341$.

Summary of all splits for $m[1, 6]$:

k	$m[1,k] + m[k+1,6] + p_0p_kp_6$
1	139
2	350
3	128
4	341
5	172
Min	**128** (at $k = 3$)

$$\therefore \quad m[1, 6] = \boxed{128}$$

Optimal split: $k = 3$, i.e. $(A_1A_2A_3)(A_4A_5A_6)$ with **128 multiplications**.

Q20. Briefly explain with examples: (i) Optimal substructure, and (ii) Overlapping subproblems. Then recall the divide-and-conquer algorithm for merge sort and analyze its running time. **(8+7 marks)** [CSE 203 | 2017–18]

Answer

Optimal Substructure, Overlapping Subproblems, and Merge Sort Analysis

Part 1: Optimal Substructure **(4 marks)**

Definition: A problem has **optimal substructure** if an optimal solution to the problem contains optimal solutions to its subproblems. This property is the key prerequisite for both **dynamic programming** and **greedy algorithms**.

Example 1 — Shortest Path:
In a weighted graph, the shortest path from u to v via intermediate vertex w satisfies:

$$\delta(u, v) = \delta(u, w) + \delta(w, v)$$

If the subpath $u \rightsquigarrow w$ were not shortest, we could replace it with a shorter path, contradicting the optimality of $u \rightsquigarrow v$. Hence the shortest path problem has optimal substructure.

Example 2 — 0/1 Knapsack:
Let $\text{OPT}(i, W)$ = maximum value using items $\{1, \ldots, i\}$ with weight limit W:

$$\text{OPT}(i, W) = \max\big(\text{OPT}(i-1, W),\ v_i + \text{OPT}(i-1, W - w_i)\big)$$

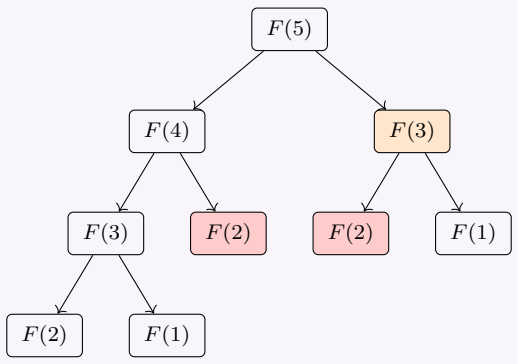
The optimal solution either excludes item i (best of $\{1, \ldots, i-1\}$ with capacity W) or includes it (best of $\{1, \ldots, i-1\}$ with remaining capacity $W - w_i$). Both subproblems must be solved optimally.

Counter-example (no optimal substructure):
The **longest simple path** problem does *not* have optimal substructure. The longest simple path from u to v via w need not consist of longest simple paths $u \to w$ and $w \to v$ (using the same vertices on both subpaths may create a cycle).

Part 2: Overlapping Subproblems **(4 marks)**

Definition: A problem has **overlapping subproblems** if a recursive algorithm revisits the same subproblem multiple times. Dynamic programming exploits this by solving each subproblem **once** and storing the result (**memoisation** or **bottom-up tabulation**).

Example — Fibonacci Numbers:
Naive recursion for $F(n) = F(n-1) + F(n-2)$:



$F(2)$ is computed **3 times** and $F(3)$ is computed **2 times** in the recursion tree for $F(5)$. For $F(n)$, the naive recursion runs in $O(2^n)$ time. With DP (storing each $F(i)$ once), this drops to $O(n)$.

Example — Matrix Chain Multiplication:
Computing $m[1][4]$ requires $m[1][2]$ in multiple subproblems ($m[1][3]$ and the direct split of $m[1][4]$). The same subproblem appears in different branches of the recursion tree, making memoisation essential.

Contrast with Divide-and-Conquer:
Divide-and-conquer (e.g. merge sort) splits a problem into **independent** subproblems that do **not overlap**. There is no benefit to memoisation. Dynamic programming is used precisely when subproblems **do** overlap.

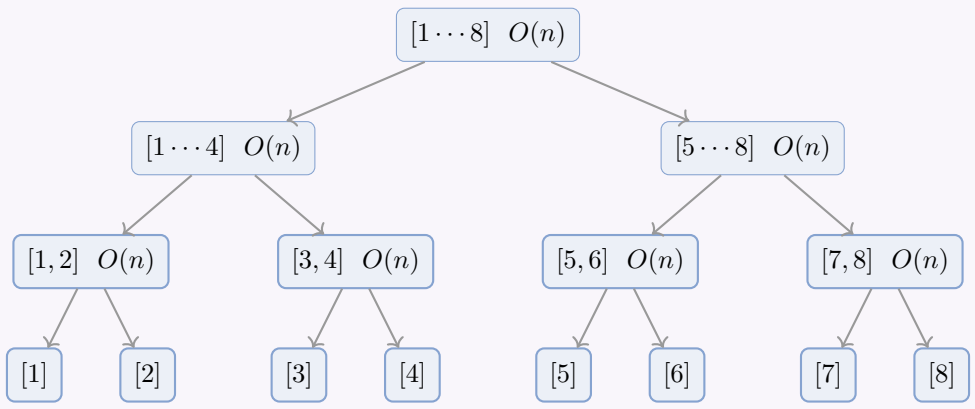
Part 3: Merge Sort — Algorithm and Running Time **(7 marks)**

Algorithm:

Algorithm 115 MERGE(A, lo, mid, hi) — with explicit copy loops

```
1:  n1 ← mid − lo + 1;   n2 ← hi − mid
2:  L[1 .. n1] ← A[lo .. mid]
3:  R[1 .. n2] ← A[mid + 1 .. hi]
4:  i ← 1;   j ← 1;   k ← lo
5:  while i ≤ n1 and j ≤ n2 do
6:      if L[i] ≤ R[j] then
7:          A[k] ← L[i];   i ← i + 1
8:      else
9:          A[k] ← R[j];   j ← j + 1
10:     end if
11:     k ← k + 1
12: end while
13: while i ≤ n1 do
14:     A[k] ← L[i];   i ← i + 1;   k ← k + 1
15: end while
16: while j ≤ n2 do
17:     A[k] ← R[j];   j ← j + 1;   k ← k + 1
18: end while
```

Illustration (n = 8):



There are $\log_2 n$ levels; at each level, the total work done by all MERGE calls is $O(n)$.

Running Time Analysis:
Recurrence:

$$T(n) = \begin{cases} O(1) & n = 1 \\ 2T\left(\frac{n}{2}\right) + O(n) & n > 1 \end{cases}$$

The $2T(n/2)$ term comes from two recursive calls on halves of size $n/2$; the $O(n)$ term is the cost of MERGE.

Solving by the Master Theorem:
The recurrence has the form $T(n) = aT(n/b) + f(n)$ with $a = 2$, $b = 2$, $f(n) = \Theta(n)$.
Compute $n^{\log_b a} = n^{\log_2 2} = n^1 = n$.
Since $f(n) = \Theta(n^{\log_b a}) = \Theta(n)$, we are in **Case 2** of the Master Theorem:

$$\boxed{T(n) = \Theta(n \log n)}$$

Verification by Recursion Tree:

Level	Subproblems	Cost per level
0	1 subproblem of size n	$c \cdot n$
1	2 subproblems of size $n/2$	$2 \cdot c \cdot n/2 = cn$
2	4 subproblems of size $n/4$	$4 \cdot c \cdot n/4 = cn$
$\vdots$	$\vdots$	$\vdots$
$\log_2 n$	n subproblems of size 1	$n \cdot O(1) = O(n)$
Total levels: $\log_2 n + 1$		$\boldsymbol{\Theta(n \log n)}$

Each of the $\log_2 n + 1$ levels contributes $\Theta(n)$ work, giving total cost $\Theta(n \log n)$.

Space Complexity:
Merge sort requires $O(n)$ auxiliary space for the temporary arrays in MERGE, plus $O(\log n)$ stack space for recursion. Total space: $O(n)$.

Summary:

Case	Time	Space
Best	$\Theta(n \log n)$	$O(n)$
Average	$\Theta(n \log n)$	$O(n)$
Worst	$\Theta(n \log n)$	$O(n)$

Merge sort is **stable**, **not in-place**, and guarantees $\Theta(n \log n)$ in all cases — unlike quicksort which has $O(n^2)$ worst case.

Q21. Consider the following instance of the 0-1 knapsack problem (capacity = 6):

Item	Weight	Value
1	2	7
2	1	8
3	2	12
4	2	6
5	2	10

Solve this instance using dynamic programming. Show the DP table and find the optimal set of items from the table. **(15 marks)** **[CSE 203 | 2017–18]**

Answer

0-1 Knapsack Problem: DP Solution (Capacity $W = 6$)

1. Recurrence Relation:

$$dp[i][w] = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ dp[i-1][w] & w_i > w \\ \max\big(dp[i-1][w],\ v_i + dp[i-1][w-w_i]\big) & w_i \le w \end{cases}$$

where $dp[i][w]$ = maximum value using items $\{1, \ldots, i\}$ with capacity w.

2. DP Table ($dp[i][w]$):
(Highlighted cells show the traceback path for the optimal solution)

Item i	w=0	w=1	w=2	w=3	w=4	w=5	w=6
0 (none)	0	0	0	0	0	0	0
1 (w_1=2, v_1=7)	0	0	7	7	7	7	7
2 (w_2=1, v_2=8)	0	8	8	15	15	15	15
3 (w_3=2, v_3=12)	0	8	12	20	20	27	27
4 (w_4=2, v_4=6)	0	8	12	20	20	27	27
5 (w_5=2, v_5=10)	0	8	12	20	22	30	**30**

3. Cell-by-cell computation (Key Cells):
Row $i = 1$ ($w_1 = 2, v_1 = 7$):

- $w < 2$: $dp[1][w] = dp[0][w] = 0$
- $w \ge 2$: $dp[1][w] = \max(dp[0][w],\ 7 + dp[0][w-2]) = \max(0, 7) = 7$

Row $i = 2$ ($w_2 = 1, v_2 = 8$):

- $dp[2][1] = \max(dp[1][1],\ 8 + dp[1][0]) = \max(0, 8) = 8$
- $dp[2][2] = \max(dp[1][2],\ 8 + dp[1][1]) = \max(7, 8) = 8$
- $dp[2][3] = \max(dp[1][3],\ 8 + dp[1][2]) = \max(7, 15) = 15$
- $dp[2][w] = 15$ for $w \ge 3$

Row $i = 3$ ($w_3 = 2, v_3 = 12$):

- $dp[3][1] = \max(8, \text{N/A}) = 8$
- $dp[3][2] = \max(dp[2][2],\ 12 + dp[2][0]) = \max(8, 12) = 12$
- $dp[3][3] = \max(dp[2][3],\ 12 + dp[2][1]) = \max(15, 20) = 20$
- $dp[3][4] = \max(dp[2][4],\ 12 + dp[2][2]) = \max(15, 20) = 20$
- $dp[3][5] = \max(dp[2][5],\ 12 + dp[2][3]) = \max(15, 27) = 27$
- $dp[3][6] = \max(dp[2][6],\ 12 + dp[2][4]) = \max(15, 27) = 27$

Row $i = 4$ ($w_4 = 2, v_4 = 6$):

- $dp[4][w] = \max(dp[3][w],\ 6 + dp[3][w-2])$
- $dp[4][4] = \max(20,\ 6 + 12) = 20$
- $dp[4][5] = \max(27,\ 6 + 20) = 27$
- $dp[4][6] = \max(27,\ 6 + 20) = 27$
- (Item 4 never improves the solution)

Row $i = 5$ ($w_5 = 2, v_5 = 10$):

- $dp[5][2] = \max(dp[4][2],\ 10 + dp[4][0]) = \max(12, 10) = 12$
- $dp[5][3] = \max(dp[4][3],\ 10 + dp[4][1]) = \max(20, 18) = 20$
- $dp[5][4] = \max(dp[4][4],\ 10 + dp[4][2]) = \max(20, 22) = 22$
- $dp[5][5] = \max(dp[4][5],\ 10 + dp[4][3]) = \max(27, 30) = 30$
- $dp[5][6] = \max(dp[4][6],\ 10 + dp[4][4]) = \max(27, 10 + 20) =$ **30**

Optimal value $= dp[5][6] =$ **30**

4. Traceback to Find Optimal Items:

Step	**Item** i	$dp[i][w]$	$dp[i-1][w]$	**Decision**	**Remaining** w
1	5	$dp[5][6] = 30$	$dp[4][6] = 27$	**Include**	$6 - 2 = 4$
2	4	$dp[4][4] = 20$	$dp[3][4] = 20$	Exclude	4
3	3	$dp[3][4] = 20$	$dp[2][4] = 15$	**Include**	$4 - 2 = 2$
4	2	$dp[2][2] = 8$	$dp[1][2] = 7$	**Include**	$2 - 1 = 1$
5	1	$dp[1][1] = 0$	$dp[0][1] = 0$	Exclude	1

5. Final Optimal Solution:

Selected Items	**Weight**	**Value**
Item 2	1	8
Item 3	2	12
Item 5	2	10
Total	**5** ≤ 6	**30** ✓

Optimal subset $= \{2, 3, 5\}$ with total value **30** and total weight **5**.

Time and Space Complexity:

- **Time:** $\Theta(nW) = \Theta(5 \times 6) = \Theta(30)$.
- **Space:** $\Theta(nW)$ for the full table (required for traceback).

Q22. Given two strings, the longest common *substring* problem is to find the longest string that is a substring of both given strings. For example, for $S_1 =$ `buetcserocks` and $S_2 =$ `bdcserocks`, the longest common substring is `cserocks`. Give a dynamic programming algorithm to find the length of the longest common substring. **(15 marks)** [CSE 203 | 2017–18]

Answer

Longest Common Substring: DP Algorithm

1. Key Distinction from LCS:

	LCS (Subsequence)	Longest Common Substring
Contiguous?	No	**Yes**
Example	LITHM	`cserocks`

A **substring** must occupy consecutive positions in both strings, unlike a subsequence which can skip characters.

2. DP Definition & Recurrence:
Let $dp[i][j]$ = length of the longest common **suffix** of $X[1..i]$ and $Y[1..j]$ (i.e. the longest common substring that ends at $X[i]$ and $Y[j]$).

$$dp[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ dp[i-1][j-1] + 1 & X[i] = Y[j] \\ 0 & X[i] \neq Y[j] \end{cases}$$

The key difference from LCS: when $X[i] \neq Y[j]$, we set $dp[i][j] = 0$ (not max of neighbours), because a mismatch **breaks** the current common suffix. The answer is $\max_{i,j} dp[i][j]$.

3. Algorithm:

Algorithm 116 LONGEST-COMMON-SUBSTRING(X, Y, m, n)

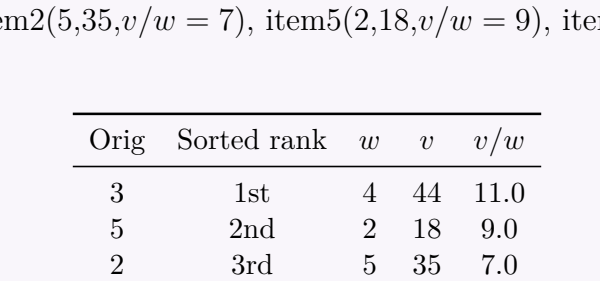
```
1:  for i = 0 to m do
2:      dp[i][0] ← 0
3:  end for
4:  for j = 0 to n do
5:      dp[0][j] ← 0
6:  end for
7:  maxLen ← 0
8:  endIdx ← 0
9:  for i = 1 to m do
10:     for j = 1 to n do
11:         if X[i] = Y[j] then
12:             dp[i][j] ← dp[i − 1][j − 1] + 1
13:             if dp[i][j] > maxLen then
14:                 maxLen ← dp[i][j]
15:                 endIdx ← i
16:             end if
17:         else
18:             dp[i][j] ← 0
19:         end if
20:     end for
21: end for
22: result ← X[endIdx − maxLen + 1 .. endIdx]
23: return maxLen, result
```

4. Worked Example:
$X =$ `buetcserocks` ($m = 12$), $Y =$ `bdcserocks` ($n = 10$).
DP Table (showing only non-zero diagonals; 0s omitted for clarity):

$dp[i][j]$	**b**	**d**	**c**	**s**	**e**	**r**	**o**	**c**	**k**	**s**
	j=1	j=2	j=3	j=4	j=5	j=6	j=7	j=8	j=9	j=10
b i=1	**1**	0	0	0	0	0	0	0	0	0
u i=2	0	0	0	0	0	0	0	0	0	0
e i=3	0	0	0	0	1	0	0	0	0	0
t i=4	0	0	0	0	0	0	0	0	0	0
c i=5	0	0	1	0	0	0	0	1	0	0
s i=6	0	0	0	2	0	0	0	0	0	1
e i=7	0	0	0	0	3	0	0	0	0	0
r i=8	0	0	0	0	0	4	0	0	0	0
o i=9	0	0	0	0	0	0	5	0	0	0
c i=10	0	0	1	0	0	0	0	6	0	0
k i=11	0	0	0	0	0	0	0	0	**7**	0
s i=12	0	0	0	1	0	0	0	0	0	**8**

The maximum value is $dp[12][10] = \mathbf{8}$, occurring at $i = 12$, $j = 10$.
Recovering the substring:
Ends at $X[12]$ with length $8 \Rightarrow X[12-8+1 \ldots 12] = X[5 \ldots 12] =$ `cserocks`.

5. Diagonal Intuition:



When characters match, values grow diagonally. A mismatch resets to 0, breaking the chain.

6. Time and Space Complexity:
- **Time:** $\Theta(mn)$ to fill the $m \times n$ table.
- **Space:** $\Theta(mn)$ for the full table, but can be optimized to $\Theta(\min(m, n))$ using just two rows.

Q23. Solve the following instance of the 0-1 knapsack problem using a branch-and-bound algorithm with a state-space tree (capacity = 10 kg):

Item	Weight (kg)	Profit (Taka)
1	3	18
2	5	35
3	4	44
4	7	36
5	2	18

(8 marks) [CSE 207 | 2017–18]

Answer

Sorted by v/w**:** item3(4,44,$v/w = 11$), item2(5,35,$v/w = 7$), item5(2,18,$v/w = 9$), item1(3,18,$v/w = 6$), item4(7,36,$v/w \approx 5.1$).
Resorting correctly:

Orig	Sorted rank	w	v	v/w
3	1st	4	44	11.0
5	2nd	2	18	9.0
2	3rd	5	35	7.0
1	4th	3	18	6.0
4	5th	7	36	5.1

DP verification: Optimal = **80** (items 1, 3, 5: $w = 3+4+2 = 9 \leq 10$, $v = 18+44+18 = 80$).
B&B trace (key nodes):

Node	w	v	UB	Action
Root	0	0	≈ 97	Explore
Inc item3	4	44	≈ 97	Explore
Inc item5	6	62	≈ 97	Explore
Inc item2	11>10	—	—	Infeasible, prune
Exc item2:	6	62	≈ 74	Explore
Inc item1	9	80	80	**Leaf**, best=80
Back: Exc item5	4	44	≈ 82	Explore
Inc item2	9	79	≈ 80	Explore
Inc item1: $w = 12 > 10$	—	—	Prune	
Exc item1: $v = 79 < 80$	—	—	Continue...	
⋮ (all remaining UBs ≤ 80, prune)				

Optimal = 80, items $\{1, 3, 5\}$ (original indices), $w = 9$, $v = 80$.

Q24. Recall the Weighted Interval Scheduling Problem. You are given n intervals with starting time s_i, finishing time f_i, and value v_i. A subset of intervals is compatible if no two overlap. Give a dynamic programming algorithm to find a compatible subset maximizing $\sum_{i \in S} v_i$. Your algorithm (including preprocessing) should run in $O(n \log n)$ time. **(15 marks)** [CSE 203 | 2016–17]

Answer

Weighted Interval Scheduling: $O(n \log n)$ DP Algorithm

Problem: Given n intervals, each with start s_i, finish f_i, and value v_i, find a compatible subset S (no two intervals overlap) maximizing $\sum_{i \in S} v_i$.

Step 1: Preprocessing — Sort by Finish Time
Sort all intervals in non-decreasing order of finish time. After sorting, assume intervals are indexed $1, 2, \ldots, n$ such that $f_1 \leq f_2 \leq \cdots \leq f_n$.
Time: $O(n \log n)$ using merge sort or heap sort.

Step 2: Compute $p(j)$ for Each Interval
Define $p(j)$ as the largest index $i < j$ such that interval i is **compatible** with interval j (i.e. $f_i \leq s_j$). If no such interval exists, $p(j) = 0$.

Algorithm 117 COMPUTE-P(intervals, n)

```
for j = 1 to n do
    lo ← 1;   hi ← j − 1;   p[j] ← 0
    while lo ≤ hi do
        mid ← ⌊(lo + hi)/2⌋
        if f[mid] ≤ s[j] then
            p[j] ← mid
            lo ← mid + 1
        else
            hi ← mid − 1
        end if
    end while
end for
return p
```

Since intervals are sorted by finish time, binary search over $f[1..j-1]$ finds $p(j)$ in $O(\log n)$. Computing all $p(j)$ takes $O(n \log n)$ total.

Step 3: DP Recurrence
Let OPT(j) = maximum value of a compatible subset of intervals $\{1, 2, \ldots, j\}$.
Key observation: In the optimal solution, interval j is either **included** or **excluded**:

- **Exclude** j: best solution uses only intervals from $\{1, \ldots, j-1\} \Rightarrow$ value = OPT$(j-1)$.
- **Include** j: cannot use any interval that overlaps with j; the last compatible interval is $p(j) \Rightarrow$ value = $v_j +$ OPT$(p(j))$.

$$\boxed{\text{OPT}(j) = \begin{cases} 0 & j = 0 \\ \max\big(\text{OPT}(j-1),\ v_j + \text{OPT}(p(j))\big) & j \geq 1 \end{cases}}$$

Step 4: Full Algorithm

Algorithm 118 WEIGHTED-INTERVAL-SCHEDULING(s, f, v, n)

```
Sort intervals so that f[1] ≤ f[2] ≤ ··· ≤ f[n]
p ← COMPUTE-P(intervals, n)
OPT[0] ← 0
for j = 1 to n do
    OPT[j] ← max(OPT[j − 1], v[j] + OPT[p[j]])
end for
return OPT[n], FIND-SOLUTION(n, OPT, p)
```

Algorithm 119 FIND-SOLUTION(j, OPT, p)

```
if j = 0 then
    return empty set
else if v[j] + OPT[p[j]] ≥ OPT[j − 1] then
    return {j} ∪ FIND-SOLUTION(p[j], OPT, p)
else
    return FIND-SOLUTION(j − 1, OPT, p)
end if
```

Worked Example:

j	s_j	f_j	v_j	$p(j)$
1	1	3	3	0
2	2	5	5	0
3	3	6	4	1
4	5	8	6	2
5	6	9	2	2
6	8	10	7	4

DP computation:

- $\text{OPT}(0) = 0$
- $\text{OPT}(1) = \max(0,\ 3+0) = 3$
- $\text{OPT}(2) = \max(3,\ 5+0) = 5$
- $\text{OPT}(3) = \max(5,\ 4+3) = 7$
- $\text{OPT}(4) = \max(7,\ 6+5) = 11$
- $\text{OPT}(5) = \max(11,\ 2+5) = 11$
- $\text{OPT}(6) = \max(11,\ 7+11) = 18$

Optimal value = 18. Traceback: include 6 ($p(6) = 4$), include 4 ($p(4) = 2$), include 2. Optimal subset = $\{2, 4, 6\}$.

Proof of Correctness:
Claim: $\text{OPT}(j)$ equals the maximum value compatible subset of $\{1, \ldots, j\}$.
Proof by induction on j:
Base: $\text{OPT}(0) = 0$ is trivially correct.
Step: Assume $\text{OPT}(k)$ is correct for all $k < j$. In any optimal solution for $\{1, \ldots, j\}$:

(i) If $j \notin S^*$: then S^* is a compatible subset of $\{1, \ldots, j-1\}$, so its value $\leq \text{OPT}(j-1)$.

(ii) If $j \in S^*$: then no interval in S^* can have index in $\{p(j)+1, \ldots, j-1\}$ (those overlap with j). So $S^* \setminus \{j\}$ is a compatible subset of $\{1, \ldots, p(j)\}$, with value $\leq \text{OPT}(p(j))$. Total value $\leq v_j + \text{OPT}(p(j))$.

Both bounds are achieved by our recurrence, so $\text{OPT}(j) = \max(\text{OPT}(j-1),\ v_j + \text{OPT}(p(j)))$ is correct. ■

Time Complexity Analysis:

Step	**Time**
Sort by finish time	$O(n \log n)$
Compute all $p(j)$ (binary search per j)	$O(n \log n)$
Fill DP table (n iterations, $O(1)$ each)	$O(n)$
Traceback	$O(n)$
Total	$\boldsymbol{O(n \log n)}$

$$\boxed{T(n) = O(n \log n)}$$

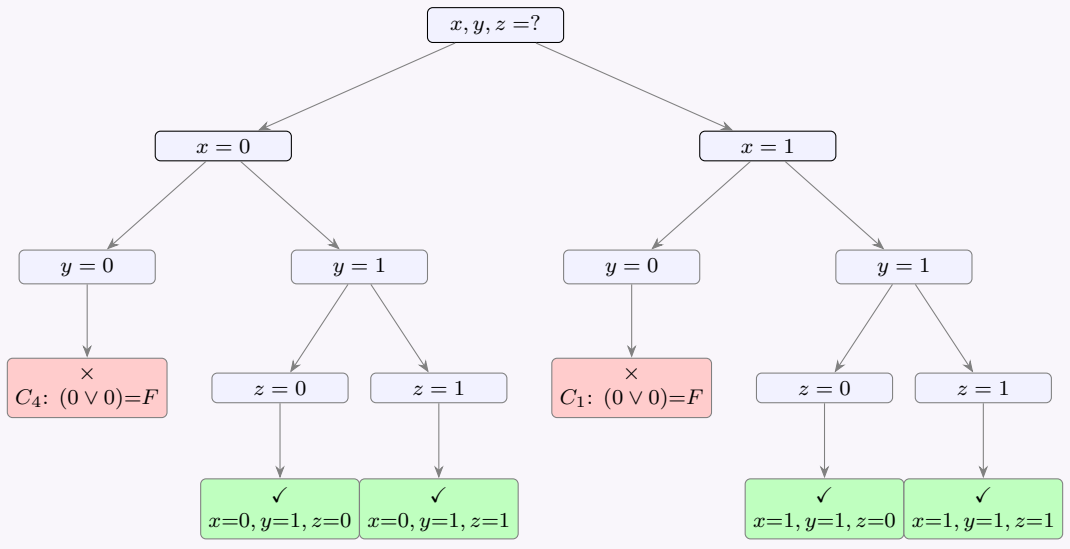
Q25. Consider the following Boolean formula: $(x' \vee y)(x \vee y' \vee z)(y \vee y')(x \vee y)(x \vee y \vee z')(x \vee y \vee z)$. Use a backtracking algorithm to decide whether it is satisfiable. Show the backtracking tree. **(15 marks)** **[CSE 207 | 2016–17]**

Answer

Simplification: Clause $(y \vee y')$ is a tautology (always true) — remove it. The remaining 5 clauses:

$$C_1 : (\neg x \vee y), \quad C_2 : (x \vee \neg y \vee z), \quad C_4 : (x \vee y), \quad C_5 : (x \vee y \vee \neg z), \quad C_6 : (x \vee y \vee z)$$

Backtracking Tree: Assign x first, then y, then z.



Pruning rules:

- $x = 0, y = 0$: $C_4 = (0 \vee 0) = \text{False} \Rightarrow$ prune immediately.
- $x = 1, y = 0$: $C_1 = (\neg 1 \vee 0) = (0 \vee 0) = \text{False} \Rightarrow$ prune.

Satisfying assignments (verified by substitution):

x	y	z	C_1	C_2	C_4	C_5	C_6
0	1	0	$1 \vee 1 = T$	$0 \vee 0 \vee 0 = F$?	T	T	**check** C_2
0	1	1	T	$0 \vee 0 \vee 1 = T$	T	T	T ✓
1	1	0	T	$1 \vee 0 \vee 0 = T$	T	T	T ✓
1	1	1	T	$1 \vee 0 \vee 1 = T$	T	T	T ✓

Check $(x = 0, y = 1, z = 0)$: $C_2 = (0 \vee 0 \vee 0) = \text{False} \Rightarrow$ not satisfying.
Formula is SATISFIABLE. The satisfying assignments are:

x	y	z
0	1	1
1	1	0
1	1	1

Q26. Solve the following 0-1 knapsack instance (capacity = 10) using the branch-and-bound algorithm. Show the branch-and-bound tree.

Item	Weight	Value
1	2	10
2	3	9
3	2	20
4	5	30

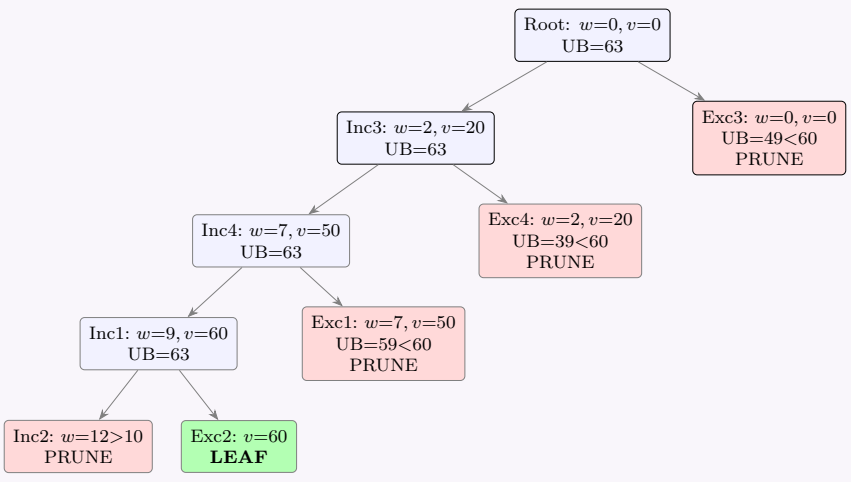
(20 marks) [CSE 207 | 2016–17]

Answer

Items sorted by v/w (decreasing):

Orig	Sorted	w	v	v/w
3	1st	2	20	10.0
4	2nd	5	30	6.0
1	3rd	2	10	5.0
2	4th	3	9	3.0

Upper bound computation: Fill items greedily; use fraction of last item.
B&B Tree:



Node details:

Node	**Decision**	w	v	**UB**	**Action**
Root	—	0	0	63	Explore
Inc item3	item3 in	2	20	63	Explore
Inc item4	item4 in	7	50	63	Explore
Inc item1	item1 in	9	60	63	Explore
Inc item2	item2 in	12	—	—	Infeasible, prune
Exc item2	item2 out	9	60	60	**LEAF** $v = 60$
Exc item1	item1 out	7	50	59	$59 < 60$, prune
Exc item4	item4 out	2	20	39	$39 < 60$, prune
Exc item3	item3 out	0	0	49	$49 < 60$, prune

Optimal value = 60.
Items selected: item3 (orig: 3, $w = 2$, $v = 20$) + item4 (orig: 4, $w = 5$, $v = 30$) + item1 (orig: 1, $w = 2$, $v = 10$).
Original items selected: $\{1, 3, 4\}$, total weight $= 2 + 2 + 5 = 9 \leq 10$, total value $= 10 + 20 + 30 =$ **60**.
UB calculation at root: Items: item3($w = 2, v = 20$), item4($w = 5, v = 30$), item1($w = 2, v = 10$) fit in $W = 10$; remaining 1kg of item2 at $9/3 = 3$. UB $= 20 + 30 + 10 + 3 = 63$.

Q27. What is the matrix chain multiplication problem? Prove that it has both the overlapping subproblems property and the optimal substructure property. **(10 marks)** [CSE 207 | 2015–16]

Answer

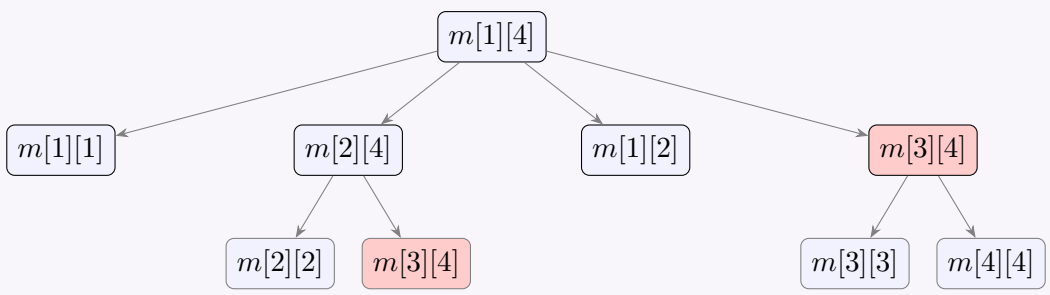
MCM Problem: Given a chain of n matrices $A_1, A_2, \ldots, A_n$ where A_i has dimension $p_{i-1} \times p_i$, find the parenthesization that minimizes the total number of scalar multiplications.
Multiplying A_i $(p_{i-1} \times p_k)$ with $A_{i+1..j}$ result $(p_k \times p_j)$ costs $p_{i-1}p_kp_j$ multiplications.

Proof of Optimal Substructure:
Let $m[i][j]$ = minimum cost to compute $A_i \cdots A_j$. The optimal split at some k gives:

$$m[i][j] = m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j$$

Cut-and-paste argument: If $m[i][k]$ were not optimal, a cheaper solution $m'[i][k] < m[i][k]$ would give total cost $m'[i][k] + m[k+1][j] + p_{i-1}p_kp_j < m[i][j]$, contradicting optimality. So $m[i][k]$ must be optimal. ■

Proof of Overlapping Subproblems:
The recursion tree for $m[1][4]$:



$m[3][4]$ appears **twice** (red nodes). In general, naive recursion solves $\Omega(2^n)$ subproblems, but there are only $\Theta(n^2)$ distinct ones. Storing each result once reduces time to $\Theta(n^3)$. ■
Number of parenthesizations $P(n)$ equals the $(n-1)$-th Catalan number:

$$P(n) = C(n-1) = \frac{1}{n}\binom{2(n-1)}{n-1}$$

$$\Rightarrow \quad P(1) = 1,\ P(2) = 1,\ P(3) = 2,\ P(4) = 5,\ P(5) = 14$$

$P(n) = \Omega(4^n/n^{3/2})$, which is exponential.

Q28. Write the steps to be followed for solving a problem using dynamic programming. Let $X = (x_1, \ldots, x_m)$ and $Y = (y_1, \ldots, y_n)$ be sequences, and let $Z = (z_1, \ldots, z_k)$ be any LCS of X and Y. If $x_m \neq y_n$ and $z_k \neq x_m$, prove that Z is an LCS of X_{m-1} and Y. **(4+6=10 marks)** [CSE 207 | 2015–16]

Answer

Steps for Solving a Problem Using DP:

1. **Characterize optimal substructure:** Show the optimal solution contains optimal sub-solutions.
2. **Define the DP state:** Clearly define what $dp[i]$ or $dp[i][j]$ represents.
3. **Write the recurrence relation:** Express each state in terms of smaller states, including base cases.
4. **Verify overlapping subproblems:** Confirm subproblems recur.
5. **Choose implementation:** Top-down (memoisation) or bottom-up (tabulation).
6. **Fill the table** in the correct order (base cases first).
7. **Read the answer** from the appropriate table entry.
8. **Traceback** to reconstruct the actual solution (not just the optimal value).

Proof: if $x_m \neq y_n$ and $z_k \neq x_m$, then Z is an LCS of X_{m-1} and Y.
Given:

- $X = (x_1, \ldots, x_m)$, $Y = (y_1, \ldots, y_n)$.
- $Z = (z_1, \ldots, z_k)$ is an LCS of X and Y.
- $x_m \neq y_n$ and $z_k \neq x_m$.

Step 1: Since $z_k \neq x_m$, the last element z_k of Z is not x_m. Therefore Z is a common subsequence of X and Y that does not use x_m. Hence Z is also a common subsequence of X_{m-1} (i.e., X without x_m) and Y.
Step 2: We show Z is the *longest* such. Suppose for contradiction there exists Z' that is a common subsequence of X_{m-1} and Y with $|Z'| > |Z| = k$.
Since X_{m-1} is a subsequence of X, and Z' is a common subsequence of X_{m-1} and Y, Z' is also a common subsequence of X and Y.

But $|Z'| > k = |Z|$, which contradicts the assumption that Z is an LCS of X and Y. $\Rightarrow\Leftarrow$
Conclusion: Z is an LCS of X_{m-1} and Y. ■

Q29. Find all longest common subsequences of $X = \langle A, G, G, C, T, G, A, T\rangle$ and $Y = \langle G, G, G, C, A, T, A\rangle$ using dynamic programming. **(15 marks)** [CSE 207 | 2015–16]

Answer

Given Strings:
$X = \langle A, G, G, C, T, G, A, T\rangle$
$Y = \langle G, G, G, C, A, T, A\rangle$

1. DP Table ($c[i][j]$):

	ε	G	G	G	C	A	T	A
ε	0	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1	1
G	0	1	1	1	1	1	1	1
G	0	1	2	2	2	2	2	2
C	0	1	2	2	3	3	3	3
T	0	1	2	2	3	3	4	4
G	0	1	2	3	3	3	4	4
A	0	1	2	3	3	4	4	5
T	0	1	2	3	3	4	5	5

2. Direction Table ($b[i][j]$):
(Branching points where both UP and LEFT are equal are denoted by ↑←. The red ones are our primary traceback branch points)

	ε	G	G	G	C	A	T	A
ε	—	—	—	—	—	—	—	—
A	—	↑	↑	↑	↑	↖	←	↖
G	—	↖	↖	↖	←	↑←	↑←	↑←
G	—	↖	↖	↖	←	←	←	←
C	—	↑	↑	↑←	↖	←	←	←
T	—	↑	↑	↑←	↑	↑←	↖	←
G	—	↖	↖	↖	↑←	↑←	↑	↑←
A	—	↑	↑	↑	↑←	↖	↑←	↖
T	—	↑	↑	↑	↑←	↑	↖	↑←

LCS length = 5.

3. Traceback to find ALL LCSes:
We start from the bottom-right cell $(8, 7)$ where $c[8][7] = 5$. Since $X[8] \neq Y[7]$ and both up and left values are 5, we hit our first **branch** (↑←).

- **Path 1 (Go Up ↑ from 8,7):**
  $(8, 7) \xrightarrow{\uparrow} (7, 7) \nwarrow$ Record **A** $\rightarrow (6, 6) \xrightarrow{\uparrow} (5, 6) \nwarrow$ Record **T** $\rightarrow (4, 5) \xleftarrow{} (4, 4) \nwarrow$ Record **C** $\rightarrow (3, 3) \nwarrow$ Record **G** $\rightarrow (2, 2) \nwarrow$ Record **G** $\rightarrow (1, 1)$.
  Reading backwards: **LCS 1** $= \langle G, G, C, T, A\rangle$

- **Path 2 (Go Left ← from 8,7):**
  $(8, 7) \xleftarrow{} (8, 6) \nwarrow$ Record **T** $\rightarrow (7, 5) \nwarrow$ Record **A** $\rightarrow (6, 4)$.
  At $(6, 4)$, we find another **branch** (↑←):

  - **Path 2A (Go Up ↑ from 6,4):**
    $(6, 4) \xrightarrow{\uparrow} (5, 4) \xrightarrow{\uparrow} (4, 4) \nwarrow$ Record **C** $\rightarrow (3, 3) \nwarrow$ Record **G** $\rightarrow (2, 2) \nwarrow$ Record **G** $\rightarrow (1, 1)$.
    Reading backwards: **LCS 2** $= \langle G, G, C, A, T\rangle$
  - **Path 2B (Go Left ← from 6,4):**
    $(6, 4) \xleftarrow{} (6, 3) \nwarrow$ Record **G** $\rightarrow (5, 2) \xrightarrow{\uparrow} (4, 2) \xrightarrow{\uparrow} (3, 2) \nwarrow$ Record **G** $\rightarrow (2, 1) \nwarrow$ Record **G** $\rightarrow (1, 0)$.
    Reading backwards: **LCS 3** $= \langle G, G, G, A, T\rangle$

Final Result:
All Longest Common Subsequences are:
1. $\langle G, G, C, T, A\rangle$
2. $\langle G, G, C, A, T\rangle$
3. $\langle G, G, G, A, T\rangle$

Verification:
- **LCS 1:** in X: G(2),G(3),C(4),T(5),A(7) ✓; in Y: G(1),G(2),C(4),T(6),A(7) ✓
- **LCS 2:** in X: G(2),G(3),C(4),A(7),T(8) ✓; in Y: G(1),G(2),C(4),A(5),T(6) ✓
- **LCS 3:** in X: G(2),G(3),G(6),A(7),T(8) ✓; in Y: G(1),G(2),G(3),A(5),T(6) ✓

Q30. Solve the following 0/1 knapsack instance (capacity = 15) using the branch-and-bound approach with a state-space tree:

Item	Weight	Value
1	7	140
2	6	132
3	4	140
4	3	90

Compare the backtracking and branch-and-bound techniques. **(3+12=15 marks)** [CSE 207 | 2015–16]

Answer

Items sorted by v/w (decreasing):

Original	Sorted	w	v	v/w
3	1st	4	140	35.0
4	2nd	3	90	30.0
2	3rd	6	132	22.0
1	4th	7	140	20.0

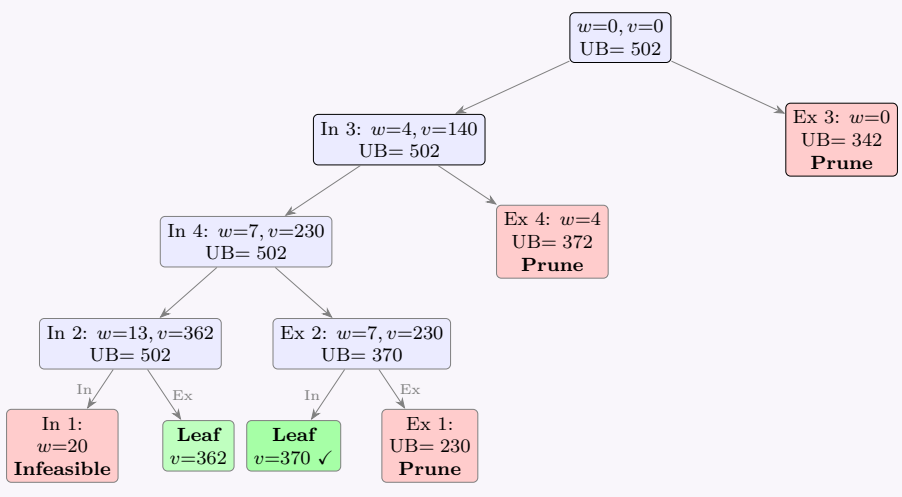
Upper bound: Fill greedily by v/w; take fraction of last item if needed.

Branch-and-Bound Trace (table form):

Node	**Decision**	w	v	**UB**	**Best**	**Action**
0	Root (nothing chosen)	0	0	502	0	Expand
1	Include item3 (w=4, v=140)	4	140	502	0	Expand
2	Include item4 (w=3, v=90)	7	230	502	0	Expand
3	Include item2 (w=6, v=132)	13	362	502	0	Expand
4	Include item1 (w=7)	20	–	–	0	**Infeasible** (w>15)
5	Exclude item1	13	362	362	362	**Leaf**, best=362
6	Exclude item2	7	230	370	362	Expand
7	Include item1 (w=7, v=140)	14	370	370	370	**Leaf**, best=370
8	Exclude item1	7	230	230	370	**Prune** (UB<best)
9	Exclude item4	4	140	372	370	Expand
10	Include item2 (w=6, v=132)	10	272	370	370	Expand
11	Include item1 (w=7)	17	–	–	370	**Infeasible**
12	Exclude item1	10	272	272	370	**Prune**
13	Exclude item2	4	140	350	370	**Prune** (UB<best)
14	Exclude item3	0	0	342	370	**Prune** (UB<best)

Optimal value = 370.
Items selected (sorted order): item3 (w=4, v=140) + item4 (w=3, v=90) + item1 (w=7, v=140)
$\Rightarrow$ original items {1, 3, 4}, total weight = 14 $\leq$ 15, total value = **370**.

State-Space Tree (compact):



Comparison: Backtracking vs. Branch-and-Bound:

Aspect	Backtracking	Branch-and-Bound
Goal	Find feasible / all solutions	Find *optimal* solution
Pruning	Infeasibility only	Infeasibility + bound $\leq$ best
Bound used	None	Upper/lower bound on subtree
Efficiency	May explore many branches	Prunes more aggressively
Application	SAT, colouring, puzzles	Optimization (TSP, Knapsack)
Completeness	Finds all solutions	Finds globally optimal

Q31. What are the key properties of an optimization problem that allow it to be solved using dynamic programming? Compare top-down memoized DP with bottom-up DP, and identify types of problems for which one outperforms the other. **(12 marks)** [CSE 207 | 2014–15]

Answer

Key Properties Required for DP:

(a) **Optimal Substructure:** An optimal solution to the problem contains optimal solutions to its subproblems. This allows us to build the global optimal from sub-optimal solutions.

(b) **Overlapping Subproblems:** The same subproblems are solved repeatedly in a naive recursive approach. DP caches each subproblem's result so it is solved only once.

Comparison: Top-down (Memoization) vs. Bottom-up (Tabulation):

Aspect	Top-down (Memo)	Bottom-up (Table)
Implementation	Recursive; cache with hash/array	Iterative; fill table in order
Subproblems solved	Only reachable ones	All subproblems
Call overhead	Recursion stack $O(d)$	None
Fill order	Automatic (on demand)	Must determine correct order
Space	Stack + memo table	Table only

When memoization outperforms bottom-up:

- **Sparse subproblem graph:** if the problem has $O(n^2)$ possible subproblems but the optimal solution only needs $O(n)$ of them (e.g., longest path in a DAG from a specific source with limited reachability).
- **Early termination:** searching for any feasible solution may not require exploring all subproblems.

When bottom-up outperforms memoization:

- **All subproblems needed:** e.g., LCS, 0-1 Knapsack, Floyd-Warshall. Bottom-up avoids recursion overhead and is cache-friendlier.
- **Space optimization:** bottom-up allows rolling arrays (reducing $O(mn)$ to $O(\min(m,n))$) which is harder with memoization.
- **Large recursion depth:** avoids stack overflow for large n.

Q32. The balanced partition problem: given n integers each in $[0, N]$, divide them into two subsets minimizing the difference of their sums (e.g., for $S = \{1, 7, 3, 5, 9\}$, a balanced partition is $S_1 = \{1, 7, 5\}$ and $S_2 = \{3, 9\}$ with difference 1). Formulate a dynamic programming solution and determine its time complexity. **(12 marks)** [CSE 207 | 2014–15]

Answer

Reduction to Subset Sum:
Let $T = \sum_{i=1}^{n} a_i$. We want to find a subset with sum s closest to $T/2$. The difference is $|T - 2s|$. So find the largest achievable $s \leq T/2$.

DP Definition:
$dp[i][s]$ = True if there exists a subset of $\{a_1, \ldots, a_i\}$ with sum exactly s.

Recurrence:

$$dp[i][s] = dp[i-1][s] \ \vee \ \big(a_i \leq s \ \wedge \ dp[i-1][s - a_i]\big)$$

Base case: $dp[0][0]$ = True; $dp[0][s]$ = False for $s > 0$.
Answer: $\min_{0 \leq s \leq T/2,\ dp[n][s]=\text{True}} |T - 2s|$

Algorithm 120 BALANCED-PARTITION(A, n)

```
T ← ∑A
dp[0][0] ← true;   dp[0][1..T] ← false
for i = 1 to n do
    for s = 0 to T do
        dp[i][s] ← dp[i − 1][s]
        if A[i] ≤ s then
            dp[i][s] ← dp[i][s] or dp[i − 1][s − A[i]]
        end if
    end for
end for
best ← T
for s = 0 to ⌊T/2⌋ do
    if dp[n][s] then
        best ← min(best, T − 2s)
    end if
end for
return best
```

Worked Example: $S = \{1, 7, 3, 5, 9\}$, $T = 25$.
$dp[5][12]$ = True (subset $\{3, 9\} = 12$). Difference $= 25 - 2 \times 12 = \mathbf{1}$.
Partition: $S_1 = \{1, 7, 5\}$ (sum 13), $S_2 = \{3, 9\}$ (sum 12).
Alternatively: $S_1 = \{9, 3\}$, $S_2 = \{1, 7, 5\}$ — same difference.

Time Complexity:

- Table has $n \times T$ entries; each takes $O(1)$.
- $T \leq nN$ (since each element is at most N).
- **Total:** $O(n^2 N)$.

This is pseudo-polynomial (polynomial in the value of input, not just its size).

Q33. Using dynamic programming, compute the minimum number of scalar multiplications for the matrix chain multiplication of $A_1, A_2, A_3, A_4, A_5, A_6$ with dimensions 15×5, 5×50, 50×20, 20×10, 10×35, 35×25. Also show the ordering of the matrices for the minimum. **(15 marks)** [CSE 207 | 2013–14]

Answer

Dimensions: $p = [15, 5, 50, 20, 10, 35, 25]$.
$m[i][j]$ table (1-indexed):

$i \backslash j$	1	2	3	4	5	6
1	0	3750	6500	6750	10375	**14000**
2	—	0	5000	6000	7750	12125
3	—	—	0	10000	27500	31250
4	—	—	—	0	7000	13750
5	—	—	—	—	0	8750
6	—	—	—	—	—	0

Minimum cost = 14 000 scalar multiplications.
$s[i][j]$ table (split point, 1-indexed):

$i \backslash j$	1	2	3	4	5	6
1	—	1	1	1	1	1
2	—	—	2	3	4	5
3	—	—	—	3	4	4
4	—	—	—	—	4	4
5	—	—	—	—	—	5

Optimal parenthesization: $(A_1((((A_2A_3)A_4)A_5)A_6))$

Q34. Compare dynamic programming and memoization techniques. "A dynamic programming algorithm usually outperforms the corresponding memoized algorithm" — explain. **(10 marks)** [CSE 207 | 2013–14]

Answer

Aspect	Dynamic Programming	Memoization
Approach	Bottom-up (iterative)	Top-down (recursive)
Subproblems	Solves *all* subproblems	Solves only *needed* subproblems
Order	Pre-determined fill order	Determined by recursion
Overhead	No recursion overhead	Function call overhead
Cache	Array/table directly	Hash map or table with lookup
Implementation	Usually simpler loops	Easier to derive from recurrence

Why DP usually outperforms memoization:

1. **No recursion overhead:** Each recursive call in memoization incurs function call setup, stack frame allocation, and return overhead. DP avoids this entirely with simple loops.
2. **Better cache/memory locality:** DP fills a table in a regular order (e.g., row by row), so memory access patterns are predictable and cache-friendly. Memoized recursion jumps around memory unpredictably.
3. **No hash-map overhead:** Memoization often uses a hash map for lookup ($O(1)$ average but with constant factor); DP uses direct array indexing ($O(1)$ with small constant).
4. **Stack depth:** Deep recursion in memoization can cause stack overflow for large n; DP has no such issue.

However, memoization has an advantage when only a *fraction* of subproblems are actually needed — it avoids computing the rest entirely.

Q35. Write a memoized algorithm for solving the matrix chain multiplication problem. Analyze its time complexity. **(10 marks)** [CSE 207 | 2013–14]

Answer

Algorithm 121 MemoMCM(p, n)

```
Initialize memo[1..n][1..n] ← −1
return LookupMCM(p, memo, 1, n)
```

Algorithm 122 LookupMCM(p, $memo$, i, j)

```
if memo[i][j] ≥ 0 then
    return memo[i][j]
end if
if i = j then
    memo[i][j] ← 0
    return 0
end if
memo[i][j] ← ∞
for k = i to j − 1 do
    cost ← LookupMCM(p, memo, i, k) +LookupMCM(p, memo, k + 1, j) +p[i − 1] · p[k] · p[j]
    if cost < memo[i][j] then
        memo[i][j] ← cost
    end if
end for
return memo[i][j]
```

Time Complexity Analysis:

- There are $\binom{n}{2} + n = O(n^2)$ distinct subproblems (i, j).
- Each subproblem (i, j) is computed *at most once*; subsequent calls return in $O(1)$.
- Computing each subproblem requires a loop of $j - i$ iterations $\Rightarrow O(n)$ per subproblem.
- **Total:** $O(n^2) \times O(n) = O(n^3)$.

Space: $O(n^2)$ for the memo table, $O(n)$ recursion stack depth.

Q36. What is a state-space tree? Solve the following instance of the 0/1 knapsack problem (capacity = 10) using the branch-and-bound approach with a state-space tree:

Item	Weight	Value
1	7	42
2	4	40
3	3	12
4	5	25

(15 marks) [CSE 207 | 2013–14]

Answer

State-Space Tree: A rooted tree where each node represents a *partial solution* (a decision about including/excluding items). The left child includes the item; the right child excludes it. Branch-and-bound prunes branches whose *upper bound* on value cannot exceed the current best solution.

Sort by value/weight ratio: Item 2 (10.0), Item 1 (6.0), Item 4 (5.0), Item 3 (4.0).
Reordered: Item 2 (w=4, v=40), Item 1 (w=7, v=42), Item 4 (w=5, v=25), Item 3 (w=3, v=12).
Upper bound at a node: Take all remaining items fractionally (greedy by ratio) up to capacity.

Branch-and-Bound Trace:

Node	Items included	Weight	Value	UB	Action
Root	none	0	0	76.4	expand
L1	{2}	4	40	76.4	expand
L1-L	{2,1}	11	82	—	**infeasible** (w>10)
L1-R	{2}	4	40	65.0	expand
L1-R-L	{2,4}	9	65	69.0	expand
L1-R-L-L	{2,4,3}	12	77	—	**infeasible**
L1-R-L-R	{2,4}	9	65	65.0	**leaf**, value=65 → best=65
L1-R-R	{2}	4	40	55.0	prune (UB < best)
R1	none	0	0	67.6	expand
R1-L	{1}	7	42	67.0	expand
R1-L-L	{1,4}	12	67	—	**infeasible**
R1-L-R	{1}	7	42	52.6	prune (UB < best=65)
R1-R	none	0	0	55.0	prune (UB < best=65)

Optimal solution: Items $\{2, 4\}$ (Item 2 + Item 4), weight $= 4 + 5 = 9 \leq 10$, value $= 40 + 25 =$ **65**.

Q37. Prove that the longest common subsequence problem has both the Overlapping Subproblems property and the Optimal Substructure property. **(10 marks)** [CSE 207 | 2013–14]

Answer

Overlapping Subproblems.
The recurrence $c[i][j] = c[i-1][j-1]+1$ (or via $c[i-1][j], c[i][j-1]$) means $c[i][j]$ depends on $c[i-1][j]$, $c[i][j-1]$, and $c[i-1][j-1]$. These are themselves entries that are re-used by multiple parent cells (e.g., $c[i-1][j]$ is needed by both $c[i][j]$ and $c[i][j+1]$). A naive recursive implementation recomputes $c[k][\ell]$ exponentially many times. □
Optimal Substructure.
Let $Z = \langle z_1, \ldots, z_k \rangle$ be any LCS of $X = \langle x_1, \ldots, x_m \rangle$ and $Y = \langle y_1, \ldots, y_n \rangle$.
Case 1 ($x_m = y_n$): then $z_k = x_m = y_n$ and Z_{k-1} is an LCS of X_{m-1} and Y_{n-1}.
Case 2 ($x_m \neq y_n$, $z_k \neq x_m$): Z is an LCS of X_{m-1} and Y.
Case 3 ($x_m \neq y_n$, $z_k \neq y_n$): Z is an LCS of X and Y_{n-1}.
In each case the optimal Z is built from an optimal sub-LCS — proving optimal substructure. □

Q38. Find all longest common subsequences of $X = \langle A, T, G, C, T, G, A, T \rangle$ and $Y = \langle T, G, G, C, A, T, A \rangle$ using dynamic programming. **(15 marks)** [CSE 207 | 2013–14]

Answer

Given Strings:
$X = \langle A, T, G, C, T, G, A, T \rangle$
$Y = \langle T, G, G, C, A, T, A \rangle$

$c[i][j]$ **Table:**

	ε	T	G	G	C	A	T	A
ε	0	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1	1
T	0	1	1	1	1	1	2	2
G	0	1	2	2	2	2	2	2
C	0	1	2	2	3	3	3	3
T	0	1	2	2	3	3	4	4
G	0	1	2	3	3	3	4	4
A	0	1	2	3	3	4	4	5
T	0	1	2	3	3	4	5	5

$b[i][j]$ **Table** (Branching points where both UP and LEFT are equal are denoted by ↑←. The red ones are our traceback branch points):

	ε	T	G	G	C	A	T	A
ε	—	—	—	—	—	—	—	—
A	—	↑	↑	↑	↑	↖	←	↖
T	—	↖	←	←	←	↑←	↖	←
G	—	↑	↖	↖	←	←	↑←	↑←
C	—	↑	↑	↑←	↖	←	←	←
T	—	↖	↑	↑←	↑	↑←	↖	←
G	—	↑	↖	↖	↑←	↑←	↑	↑←
A	—	↑	↑	↑	↑←	↖	↑←	↖
T	—	↖	↑	↑	↑←	↑	↖	↑←

Traceback to find ALL LCSes:
We start from the bottom-right cell $(8, 7)$ where $c[8][7] = 5$. Since $X[8] \neq Y[7]$ and both up and left values are 5, we hit our first **branch** (↑←).

- **Path 1 (Go Up ↑ from 8,7):** $(8,7) \xrightarrow{\uparrow} (7,7) \xrightarrow{\nwarrow}$ Record **A** $\rightarrow (6,6) \xrightarrow{\uparrow} (5,6) \xrightarrow{\nwarrow}$ Record **T** $\rightarrow (4,5) \xrightarrow{\leftarrow} (4,4) \xrightarrow{\nwarrow}$ Record **C** $\rightarrow (3,3) \xrightarrow{\nwarrow}$ Record **G** $\rightarrow (2,2) \xrightarrow{\leftarrow} (2,1) \xrightarrow{\nwarrow}$ Record **T** $\rightarrow (1,0)$.
  Reading backwards: **LCS 1 = TGCTA**

- **Path 2 (Go Left ← from 8,7):** $(8,7) \xrightarrow{\leftarrow} (8,6) \xrightarrow{\nwarrow}$ Record **T** $\rightarrow (7,5) \xrightarrow{\nwarrow}$ Record **A** $\rightarrow (6,4)$.
  At $(6,4)$, we find another **branch** (↑←):

  - **Path 2A (Go Up ↑ from 6,4):** $(6,4) \xrightarrow{\uparrow} (5,4) \xrightarrow{\uparrow} (4,4) \xrightarrow{\nwarrow}$ Record **C** $\rightarrow (3,3) \xrightarrow{\nwarrow}$ Record **G** $\rightarrow (2,2) \xrightarrow{\leftarrow} (2,1) \xrightarrow{\nwarrow}$ Record **T** $\rightarrow (1,0)$.
    Reading backwards: **LCS 2 = TGCAT**
  - **Path 2B (Go Left ← from 6,4):** $(6,4) \xrightarrow{\leftarrow} (6,3) \xrightarrow{\nwarrow}$ Record **G** $\rightarrow (5,2) \xrightarrow{\uparrow} (4,2) \xrightarrow{\uparrow} (3,2) \xrightarrow{\nwarrow}$ Record **G** $\rightarrow (2,1) \xrightarrow{\nwarrow}$ Record **T** $\rightarrow (1,0)$.
    Reading backwards: **LCS 3 = TGGAT**

Final Result:
Max LCS Length = **5**.
All Longest Common Subsequences = **{TGCTA, TGCAT, TGGAT}**.

Note / Additional Query Part:
For $X = \langle A, G, G, C, T, G, A, T \rangle$ and $Y = \langle G, G, G, C, A, T, A \rangle$:
LCS length = **5**. One valid LCS = $\langle G, G, C, T, A \rangle$.

Q39. In a previous life, you worked as a cashier in the ancient city of Egypt, spending the better part of your day giving change to your customers. Because paper was a very rare and valuable resource at the time, cashiers were required by law to use the fewest notes possible whenever they gave change. The currency of Egypt at that time, called Dream Dollars, was available in the following denominations: \$1, \$4, \$7, \$13, \$28, \$52, \$91, \$365. **(20 marks)**

(a) A greedy change algorithm repeatedly takes the largest note that does not exceed the target amount. For example, to make \$122 using the greedy algorithm, we first take a \$91 note, then a \$28 note, and finally three \$1 notes. Give an example where this greedy algorithm uses more Dream Dollar notes than the minimum possible.

(b) Describe a dynamic programming algorithm that computes, given an integer k, the minimum number of notes needed to make k Dream Dollars. Write down the pseudocodes for both the memoized approach and bottom-up iterative approach. Analyze the running-times of both approaches.

[CSE 207 | 2012–13]

Answer

Denominations: \$1, \$4, \$7, \$13, \$28, \$52, \$91, \$365.

(a) Greedy counterexample.

For $k = \$15$: greedy takes $\$13 + \$1 + \$1 = 3$ notes. Optimal: $\$7 + \$7 + \$1 = 3$ notes — same here.
For $k = \$14$: greedy takes $\$13 + \$1 = 2$ notes. Optimal: $\$7 + \$7 = 2$ notes — same.
For $k = \$11$: greedy takes $\$7 + \$4 = 2$ notes. But note: $\$4 + \$4 + \$1 + \$1 + \$1 = 5$ notes — greedy is better here.
For $k = \$8$: greedy takes $\$7 + \$1 = 2$ notes. Optimal: $\$4 + \$4 = 2$ notes — same.

Remark. The particular denominations $\{1, 4, 7, 13, 28, 52, 91, 365\}$ are Fibonacci-like and the greedy appears to give optimal results for small amounts. A rigorous counterexample requires checking larger values. For general denominations, greedy fails: with $\{1, 3, 4\}$ and $k = 6$, greedy gives $4 + 1 + 1 = 3$ notes but optimal is $3 + 3 = 2$.

(b) DP algorithm:

```
// Bottom-up DP
vector<int> coinChangeDP(int k, vector<int>& coins) {
    vector<int> dp(k+1, INT_MAX), from(k+1, -1);
    dp[0] = 0;
    for (int i = 1; i <= k; i++)
        for (int c : coins)
            if (c <= i && dp[i-c] != INT_MAX && dp[i-c]+1 < dp[i])
                { dp[i] = dp[i-c]+1; from[i] = c; }
    return dp;  // dp[k] = min notes for k
}

// Memoized (top-down)
unordered_map<int,int> memo;
int coinChangeMemo(int k, vector<int>& coins) {
    if (k == 0) return 0;
    if (memo.count(k)) return memo[k];
    int res = INT_MAX;
    for (int c : coins)
        if (c <= k && coinChangeMemo(k-c, coins) != INT_MAX)
            res = min(res, coinChangeMemo(k-c, coins)+1);
    return memo[k] = res;
}
```

Complexity Analysis. Both approaches: $O(k \cdot |\text{coins}|)$ time, $O(k)$ space.

Q40. You have n magic boxes kept in a row, numbered 1 to n, each containing some dollars. You can open box i only if you did not open box $i-1$ or box $i-2$. Given the amount $D[1 \ldots n]$ in each box, find the maximum dollars obtainable. Describe a dynamic programming solution and analyze its running time. **(12 marks)** [CSE 207 | 2012–13]

Answer

DP recurrence: define $dp[i]$ = max dollars from boxes i to n.

$$dp[i] = \max(D[i] + dp[i+2],\ dp[i+1]), \quad dp[n+1] = dp[n+2] = 0.$$

Compute right to left; answer is $dp[1]$.
Example: $D = [1, 9, 3, 7, 5, 2, 8, 4] \rightarrow$ optimal picks: boxes 2(9)+4(7)+7(8)=24. (Indices 1-based; skip 3,5,6,8.)
Wait — re-check: boxes 2,4,7 give $9 + 7 + 8 = 24$ but we must verify no two are adjacent: $|2-4| = 2 \geq 2$ ✓, $|4-7| = 3 \geq 2$ ✓.
Maximum = 24.

Complexity Analysis. $O(n)$ time, $O(1)$ space (only keep last two dp values).

Q41. Suppose we are given a sequence of integers $A[1..n]$ and we want to find the longest subsequence whose elements are in decreasing order. More concretely, we want to find the longest sequence of indices $1 < i_1 < i_2 < \ldots < i_k < n$ such that $A[i_j] > A[i_{j+1}]$ for all j. Describe the pseudocode of a backtracking algorithm that solves this problem in $O(2^n)$ time. **(15 marks)** [CSE 207 | 2012–13]

Answer

Backtracking Algorithm:
For each element, we either *include* it (if it is smaller than the last included element) or *exclude* it. We explore all 2^n subsets.

Algorithm 123 LDS_Backtrack(A, n, idx, $lastVal$, $current$)

```
1: if idx > n then
2:     if |current| > best then
3:         best ← |current|
4:         bestSeq ← copy(current)
5:     end if
6:     return
7: end if
8: LDS_Backtrack(A, n, idx + 1, lastVal, current)          ▷ exclude A[idx]
9: if A[idx] < lastVal then                                ▷ include A[idx]
10:    current.Append(A[idx])
11:    LDS_Backtrack(A, n, idx + 1, A[idx], current)
12:    current.Pop()
13: end if
```

Complexity: At each index we make 2 choices (include/exclude), giving a recursion tree of depth n with 2^n leaves $\Rightarrow O(2^n)$ time.
Note: This problem can be solved in $O(n \log n)$ using the LIS (Longest Increasing Subsequence) DP on the negated array, but the question asks for the backtracking approach.

Q42. Compare the properties of problems that can be solved with the greedy method, divide-and-conquer method, and dynamic programming method. **(3 marks)** [CSE 207 | 2011–12]

Answer

Property	Greedy	Divide & Conquer	Dynamic Programming
Optimal substructure	Required ✓	Not required	Required ✓
Overlapping subproblems	Not required	Not required (independent)	Required ✓
Greedy-choice property	Required ✓	No	No
Subproblem independence	Yes	Yes	No (shared)
Typical complexity	$O(n \log n)$	$T(n) = aT(n/b) + f(n)$	Polynomial (table-size)
Examples	Huffman, MST, Activity Selection	Mergesort, FFT, Closest Pair	LCS, Knapsack, MCM

Q43. Write and prove the optimal-substructure property of the matrix chain multiplication problem. Calculate the number of parenthesizations of a sequence of n matrices. **(5+5=10 marks)** [CSE 207 | 2011–12]

Answer

Theorem (Optimal Substructure of MCM): Let $m[i][j]$ be the minimum cost to compute $A_i \cdots A_j$. In an optimal parenthesization, the last split is at some position k ($i \le k < j$):

$$m[i][j] = m[i][k] + m[k+1][j] + p_{i-1}p_k p_j$$

and both $m[i][k]$ and $m[k+1][j]$ must be optimal.
Proof (cut-and-paste):
Suppose the optimal solution for $A_i \cdots A_j$ splits at k, but the sub-solution for $A_i \cdots A_k$ uses cost c' with $c' < m[i][k]$. Then total cost $c' + m[k+1][j] + p_{i-1}p_k p_j < m[i][j]$, contradicting the optimality of $m[i][j]$. Hence $m[i][k]$ must be optimal, and by identical argument so must $m[k+1][j]$. ■

Number of Parenthesizations:
Let $P(n)$ = number of ways to parenthesize n matrices. We have:

$$P(1) = 1, \qquad P(n) = \sum_{k=1}^{n-1} P(k) \cdot P(n-k) \quad \text{for } n \ge 2$$

This is the recurrence for the **Catalan numbers**: $P(n) = C(n-1)$ where $C(m) = \frac{1}{m+1}\binom{2m}{m}$.

n	1	2	3	4	5	6
$P(n)$	1	1	2	5	14	42

Asymptotically, $P(n) = \Omega(4^{n-1}/n^{3/2})$, which is **exponential** in n — justifying why brute force is infeasible and DP is essential.

Q44. Find an optimal solution to the 0/1 knapsack instance of $n = 4$, $W = 6$, $(v_1, v_2, v_3, v_4) = (50, 30, 12, 45)$, $(w_1, w_2, w_3, w_4) = (2, 3, 1, 3)$. **(15 marks)** [CSE 207 | 2011–12]

Answer

DP Definition: $dp[i][w]$ = maximum value using items $1 \dots i$ with capacity w.
Recurrence:

$$dp[i][w] = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ dp[i-1][w] & w_i > w \\ \max\big(dp[i-1][w],\ v_i + dp[i-1][w-w_i]\big) & \text{otherwise} \end{cases}$$

Items:

Item	w_i	v_i	v_i/w_i
1	2	50	25.0
2	3	30	10.0
3	1	12	12.0
4	3	45	15.0

DP Table $dp[i][w]$:

$i \backslash w$	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1 ($w=2, v=50$)	0	0	50	50	50	50	50
2 ($w=3, v=30$)	0	0	50	50	50	80	80
3 ($w=1, v=12$)	0	12	50	62	62	80	92
4 ($w=3, v=45$)	0	12	50	62	62	95	**107**

Traceback from $dp[4][6] = 107$:

- $dp[4][6] = 107 \neq dp[3][6] = 92 \Rightarrow$ item 4 included. Remaining capacity $= 6 - 3 = 3$, go to $dp[3][3]$.
- $dp[3][3] = 62 \neq dp[2][3] = 50 \Rightarrow$ item 3 included. Remaining capacity $= 3 - 1 = 2$, go to $dp[2][2]$.
- $dp[2][2] = 50 = dp[1][2] = 50 \Rightarrow$ item 2 *not* included. Go to $dp[1][2]$.
- $dp[1][2] = 50 \neq dp[0][2] = 0 \Rightarrow$ item 1 included. Remaining capacity $= 2 - 2 = 0$.

Selected items: $\{1, 3, 4\}$

Item	w_i	v_i
1	2	50
3	1	12
4	3	45
Total	**6**	**107**

Optimal value $=$ **107**, total weight $= 6 = W$. ✓

Q45. Let $X = (x_1, x_2, \dots, x_m)$ and $Y = (y_1, y_2, \dots, y_n)$ be sequences, and let $Z = (z_1, z_2, \dots, z_k)$ be any LCS of X and Y. If $x_m \neq y_n$ and $z_k \neq x_m$, prove that Z is an LCS of X_{m-1} and Y. **(2 marks)** [CSE 207 | 2011–12]

Q46. Find an LCS of $X = \langle A, T, C, T, G, A, T \rangle$ and $Y = \langle T, G, C, A, T, A \rangle$ by showing the c and b tables. **(10 marks)** [CSE 207 | 2011–12]

Answer

$c[i][j]$ **table:**

	ε	T	G	C	A	T	A
ε	0	0	0	0	0	0	0
A	0	0	0	0	1	1	1
T	0	1	1	1	1	2	2
C	0	1	1	2	2	2	2
T	0	1	1	2	2	3	3
G	0	1	2	2	2	3	3
A	0	1	2	2	3	3	4
T	0	1	2	2	3	4	4

$b[i][j]$ **table:**

	ε	T	G	C	A	T	A
ε	—	—	—	—	—	—	—
A	—	↑	↑	↑	↖	←	↖
T	—	↖	←	←	↑	↖	←
C	—	↑	↑	↖	←	↑	↑
T	—	↖	↑	↑	↑	↖	←
G	—	↑	↖	↑	↑	↑	↑
A	—	↑	↑	↑	↖	↑	↖
T	—	↖	↑	↑	↑	↖	↑

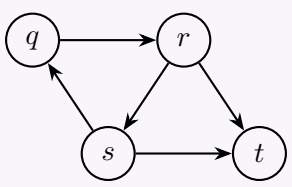
LCS length $= c[7][6] = \mathbf{4}$.
Traceback (follow b from $(7,6)$):
Correct traceback: $(7,6) \overset{\uparrow}{\rightarrow} (6,6) \overset{\nwarrow}{\rightarrow}$: $X[6] = A$, $Y[6] = A$ ✓, record **A**, go to $(5,5)$. $b[5][5] = \uparrow\rightarrow$ $(4,5)$. $b[4][5] = \nwarrow$: $X[4] = T$, $Y[5] = T$ ✓, record **T**, go to $(3,4)$. $b[3][4] = \leftarrow\rightarrow$ $(3,3)$. $b[3][3] = \nwarrow$: $X[3] = C$, $Y[3] = C$ ✓, record **C**, go to $(2,2)$. $b[2][2] = \leftarrow\rightarrow$ $(2,1)$. $b[2][1] = \nwarrow$: $X[2] = T$, $Y[1] = T$ ✓, record **T**, go to $(1,0)$. Done.
Reading in order: **LCS** $= \langle T, C, T, A \rangle$.
Verification: T(2), C(3), T(4), A(6) in X ✓; T(1), C(3), T(5), A(6) in Y ✓.

Q47. Give an example of a problem that has the optimal substructure property. Give another example of a problem that does not have optimal substructure. Justify your answers. **(10 marks)** [CSE 207 | 2009–10]

Answer

Problem WITH Optimal Substructure: Shortest Path (Dijkstra/Bellman-Ford)
Claim: Any shortest path $p : u \rightsquigarrow v$ passing through intermediate vertex w satisfies $\delta(u,v) = \delta(u,w) + \delta(w,v)$.
Proof: If the subpath $u \rightsquigarrow w$ were not a shortest path, we could replace it with a shorter path, reducing the total length below $\delta(u,v)$, contradicting optimality. ■
This means we can build the shortest path from optimal subpaths ⇒ optimal substructure holds. This enables Dijkstra's $O((V+E)\log V)$ and Floyd-Warshall's $O(V^3)$ algorithms.

Problem WITHOUT Optimal Substructure: Longest Simple Path
Claim: The longest simple path problem does *not* have optimal substructure.
Counterexample:
Consider the graph:



Longest simple path from q to t: $q \to r \to s \to t$ (length 3) or $q \to s \to r \to t$ (length 3).
Consider subpaths: the longest simple path $q \to r$ is $q \to s \to r$ (length 2); the longest simple path $r \to t$ is $r \to s \to t$ (length 2). But combining $q \to s \to r$ and $r \to s \to t$ reuses vertex s, creating a non-simple path!
The longest simple paths of the subproblems cannot be combined because they share vertices. Hence the problem lacks optimal substructure and is NP-hard.

Q48. Show the execution of the Dynamic Programming LCS algorithm for the following two strings: $X = \langle A, B, C, B \rangle$ and $Y = \langle B, D, A, C \rangle$. **(15 marks)** [CSE 207 | 2009–10]

Answer

Recurrence:

$$c[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & X[i] = Y[j] \\ \max(c[i-1][j],\, c[i][j-1]) & X[i] \neq Y[j] \end{cases}$$

$c[i][j]$ **table:**

	ε	B	D	A	C
ε	0	0	0	0	0
A	0	0	0	1	1
B	0	1	1	1	1
C	0	1	1	1	2
B	0	1	1	1	2

$b[i][j]$ **table** (↖=diagonal, ↑=up, ←=left):

	ε	B	D	A	C
ε	—	—	—	—	—
A	—	↑	↑	↖	←
B	—	↖	←	↑	↑
C	—	↑	↑	↑	↖
B	—	↖	↑	↑	↑

LCS length $= c[4][4] = \mathbf{2}$.
Traceback (follow b starting from $c[4][4]$):
$(4,4) \xrightarrow{\uparrow} (3,4) \xrightarrow{\nwarrow} (2,3) \xrightarrow{\uparrow} (1,3) \xrightarrow{\nwarrow} (0,2)$ (Done).
Detailed Path:

- $b[4][4] = \uparrow \rightarrow$ go to $(3,4)$.
- $b[3][4] = \nwarrow \rightarrow$ match $X[3] = C, Y[4] = C$ ✓(record **C**), go to $(2,3)$.
- $b[2][3] = \uparrow \rightarrow$ go to $(1,3)$.
- $b[1][3] = \nwarrow \rightarrow$ match $X[1] = A, Y[3] = A$ ✓(record **A**), go to $(0,2)$.

Reading in reverse order: **LCS** $= \langle A, C \rangle$.
Verification:
A(1), C(3) in $X = \langle A, B, C, B \rangle$ ✓;
A(3), C(4) in $Y = \langle B, D, A, C \rangle$ ✓.

Q49. Show all the different ways to multiply the matrix chain $A_1A_2A_3A_4A_5$. **(5 marks)** [CSE 207 | 2009–10]

Answer

The number of distinct parenthesizations of n matrices is the $(n-1)$-th Catalan number $C(n-1)$. For $n = 5$: $C(4) = \frac{1}{5}\binom{8}{4} = 14$.
All 14 parenthesizations of $A_1A_2A_3A_4A_5$**:**

#	Parenthesization
1	$(A_1(A_2(A_3(A_4A_5))))$
2	$(A_1(A_2((A_3A_4)A_5)))$
3	$(A_1((A_2A_3)(A_4A_5)))$
4	$(A_1((A_2(A_3A_4))A_5))$
5	$(A_1(((A_2A_3)A_4)A_5))$
6	$((A_1A_2)(A_3(A_4A_5)))$
7	$((A_1A_2)((A_3A_4)A_5))$
8	$((A_1(A_2A_3))(A_4A_5))$
9	$((A_1(A_2A_3))A_4)A_5)$
10	$(((A_1A_2)A_3)(A_4A_5))$
11	$((A_1(A_2(A_3A_4)))A_5)$
12	$(((A_1A_2)(A_3A_4))A_5)$
13	$(((A_1A_2)A_3A_4)A_5)$
14	$((((A_1A_2)A_3)A_4)A_5)$

Recurrence for Catalan numbers: $C(0) = C(1) = 1;\ C(n) = \sum_{k=0}^{n-1} C(k) \cdot C(n-1-k)$

n (matrices)	1	2	3	4	5	6
Parenthesizations	1	1	2	5	14	42

Q50. Convert the following recursive procedure for computing the n-th Fibonacci number into an equivalent procedure that uses a dynamic multi-graph representing the computation. From this graph, find $T_1(5)$, $T_\infty(5)$, and the parallelism.

```
F(n)
  if n <= 1 then return n
  else return F(n-1) + F(n-2)
```

(17 marks) [CSE 207 | 2009–10]

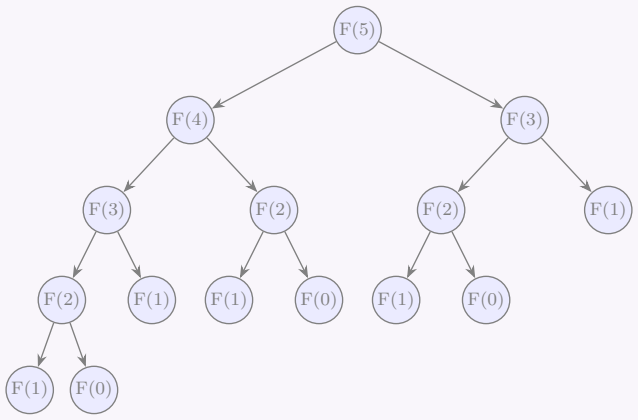
Answer

Recursive procedure:

Algorithm 124 $\mathrm{F}(n)$

```
if n ≤ 1 then
    return n
else
    return F(n − 1) + F(n − 2)
end if
```

Dynamic Multi-graph (DAG):
Each function call is a *vertex*; directed edges represent data dependencies. The graph for $F(5)$:



$T_1(n)$ (total work, running on 1 processor):
$T_1(n) = T_1(n-1) + T_1(n-2) + 1$ (one addition per call), $T_1(0) = T_1(1) = 1$.

n	0	1	2	3	4	5
T_1	1	1	3	5	9	**15**

$T_1(5) = \mathbf{15}$ (total nodes in recursion tree).

$T_\infty(n)$ (critical path / span on infinite processors):
The critical path is the longest chain of *sequential* dependencies. The longest path from root to leaf goes through the left branch each time: $F(5) \to F(4) \to F(3) \to F(2) \to F(1)$, with one add at each level.
$T_\infty(n) = T_\infty(n-1) + 1 = n$, so $T_\infty(n) = n + 1$ (including the leaf node):

$$T_\infty(5) = \mathbf{6} \quad (\text{path: } F(5) \to F(4) \to F(3) \to F(2) \to F(1) \to \text{return})$$

Or counting only internal (non-leaf) nodes in the critical path: $T_\infty = n = 5$.
Using the standard definition $T_\infty(n) = n$ (critical path depth):

$$T_\infty(5) = 5$$

Parallelism:

$$\text{Parallelism} = \frac{T_1}{T_\infty} = \frac{15}{5} = \mathbf{3}$$

This means on average 3 independent computations could run in parallel at each step, but diminishing returns limit actual speedup.

Q51. Find a longest common subsequence from the DNA sequences `ACGCTAC` and `CTGCA`. Print the sequence also. Analyze the run times of your methods of finding and printing the sequence. **(10+5=15 marks)** [CSE 207 | 2008–09]

Answer

Let $X =$ `ACGCTAC` $(m = 7)$ and $Y =$ `CTGCA` $(n = 5)$.
LCS DP Table $c[i][j]$ = length of LCS of $X[1..i]$ and $Y[1..j]$:

	∅	C	T	G	C	A
∅	0	0	0	0	0	0
A	0	0	0	0	0	1
C	0	1	1	1	1	1
G	0	1	1	2	2	2
C	0	1	1	2	3	3
T	0	1	2	2	3	3
A	0	1	2	2	3	4
C	0	1	2	2	3	4

LCS length $= c[7][5] = 4$.
Backtracking to print LCS: Start at $c[7][5]$; if $X[i] = Y[j]$, include character and go to $c[i-1][j-1]$; else go to the larger of $c[i-1][j]$ or $c[i][j-1]$.
Tracing: $(7,5) \to (6,4) \to (5,3) \to (4,2) \to (3,1)$

$$\text{LCS} = \mathbf{CGCA}$$

Runtime Analysis:

- **Finding LCS (DP):** Fill $m \times n$ table, $O(1)$ per cell $\Rightarrow O(mn)$ time, $O(mn)$ space.
- **Printing LCS (backtrack):** At each step move diagonally, up, or left; at most $m+n$ steps $\Rightarrow O(m+n)$ time.

Q52. Find the optimal sequence of multiplying the following matrices: $A(2 \times 7)$, $B(7 \times 9)$, $C(9 \times 4)$, $D(4 \times 6)$, $E(6 \times 3)$. How does the memoization technique help? **(10+5=15 marks)** [CSE 207 | 2008–09, 2006–07]

Answer

Dimensions: $p = [2, 7, 9, 4, 6, 3]$
$(A_1 : 2 \times 7, A_2 : 7 \times 9, A_3 : 9 \times 4, A_4 : 4 \times 6, A_5 : 6 \times 3)$.

$m[i][j]$ **table** (minimum scalar multiplications to compute $A_i \cdots A_j$):

$i \backslash j$	1 (A)	2 (B)	3 (C)	4 (D)	5 (E)
1 (A)	0	**126** *(AB)*	**198** *((AB)C)*	**246** *(((AB)C)D)*	**282** *((((AB)C)D)E)*
2 (B)	—	0	**252** *(BC)*	**420** *((BC)D)*	**369** *(B(C(DE)))*
3 (C)	—	—	0	**216** *(CD)*	**180** *(C(DE))*
4 (D)	—	—	—	0	**72** *(DE)*
5 (E)	—	—	—	—	0

Minimum cost = 282 scalar multiplications.
Optimal parenthesization: $((((A \cdot B) \cdot C) \cdot D) \cdot E)$

How memoization helps:
The naive recursive solution for MCM recomputes $m[i][j]$ exponentially many times due to overlapping subproblems. Memoization optimizes this process in the following ways:

- **Caching results:** We store each computed $m[i][j]$ in a table the first time it is calculated.
- **Avoiding recomputation:** On subsequent recursive calls for the same subproblem, we directly return the stored value in $O(1)$ time.
- **Complexity reduction:** Since there are only $O(n^2)$ distinct subproblems and each takes $O(n)$ time to compute, the total time complexity drops drastically from exponential ($O(2^n)$) to polynomial ($O(n^3)$).
- **Ease of implementation:** It provides the exact same efficiency as bottom-up DP while maintaining the intuitive, top-down recursive mathematical approach.

Q53. Mention the basic properties that must be satisfied to solve a problem using dynamic programming. When and how can memoization help improve an exponential-time recursive algorithm to polynomial time? **(7+8 marks)** [CSE 207 | 2007–08]

Answer

Properties required for dynamic programming:

(a) **Optimal substructure**: an optimal solution contains optimal solutions to sub-problems.

(b) **Overlapping sub-problems**: the same sub-problems are solved repeatedly in a naive recursive solution.

How memoization helps:
A naive recursive algorithm re-solves the same sub-problems exponentially many times. Memoization stores the result of each sub-problem the first time it is computed; subsequent calls return the cached result in $O(1)$.
Example — Fibonacci: naive recursion is $O(2^n)$ because $F(n-2)$ is recomputed in both the $F(n-1)$ and $F(n)$ calls. With memoization, each $F(k)$ is computed exactly once $\rightarrow O(n)$ total.
General principle: if there are P distinct sub-problems and each takes $O(T)$ to solve (given sub-answers), memoization gives $O(P \cdot T)$ total time. For LCS: $P = mn$, $T = O(1) \rightarrow O(mn)$. For matrix chain: $P = O(n^2)$, $T = O(n) \rightarrow O(n^3)$.

Q54. Suppose a thief went to a superstore to steal. He has a bag with him which can contain at most W unit of weight of anything. The thief can carry at most W' unit of weight. In the store there are two types of things. For Type 1 things, he either has to take the thing or leave it. For Type 2 things, he can weigh and take as per his will (e.g. sugar, rice, salt etc.). All type 1 things are labeled

with the corresponding unit prices and each type 2 thing is labeled with the corresponding price per unit weight. The thief decided to take only the things belonging to Type 1. He now needs an algorithm to maximize his profits.

(a) Map the above problem to one of your known problems. **(5 marks)**

(b) Discuss possible greedy strategies to solve the problem and discuss whether or not any of them will work. **(7 marks)**

(c) If the thief decides to take only type 2 things, will any of the greedy strategies work? Justify your answer. **(6 marks)**

[CSE 207 | 2007–08]

Answer

(a) Mapping: Type 1 items (take whole or leave) map exactly to the **0-1 Knapsack problem**: each item i has weight w_i and value v_i; the bag capacity is W; maximise $\sum v_i x_i$ subject to $\sum w_i x_i \le W$, $x_i \in \{0, 1\}$.
(b) Greedy strategies for 0-1 Knapsack (Type 1): Three natural strategies:

- Sort by value v_i descending $\rightarrow$ fails: high-value heavy items may waste capacity.
- Sort by weight w_i ascending $\rightarrow$ fails: light but worthless items crowd out heavy valuable ones.
- Sort by ratio v_i/w_i descending $\rightarrow$ fails for 0-1 case: taking the best-ratio item may prevent a better combination.

None of these greedy strategies is optimal for 0-1 knapsack. DP is required (see museum thief answer).
(c) Type 2 (fractional): Yes — the greedy strategy of sorting by v_i/w_i descending and taking as much as possible *does* work for the fractional case (the fractional knapsack has the greedy-choice property and optimal substructure — proved in the fractional knapsack answer).

Q55. Find an optimal solution to the 0–1 knapsack problem with the following weight and profit values (weight limit = 8): $(1, 2)$, $(1, 3)$, $(2, 3)$, $(2, 5)$, $(3, 4)$ **(10 marks)** **[CSE 207 | 2006–07]**

Answer

Given: Capacity $W = 8$.
Items (w, p): 1:$(1, 2)$, 2:$(1, 3)$, 3:$(2, 3)$, 4:$(2, 5)$, 5:$(3, 4)$, 6:$(3, 2)$, 7:$(3, 7)$.

Dynamic Programming Table (0-1 Knapsack):
Let $dp[i][w]$ be the maximum profit using the first i items with capacity w.
$dp[i][w] = \max(dp[i-1][w],\ dp[i-1][w-w_i] + p_i)$

i (Item)	(w_i, p_i)	$w=0$	1	2	3	4	5	6	7	8
0	(0, 0)	0	0	0	0	0	0	0	0	0
1	(1, 2)	0	2	2	2	2	2	2	2	2
2	(1, 3)	0	3	5	5	5	5	5	5	5
3	(2, 3)	0	3	5	6	8	8	8	8	8
4	(2, 5)	0	3	5	8	10	11	13	13	13
5	(3, 4)	0	3	5	8	10	11	13	14	15
6	(3, 2)	0	3	5	8	10	11	13	14	15
7	(3, 7)	0	3	5	8	10	12	15	17	**18**

Traceback to find selected items: Start at $dp[7][8] = 18$:
- $dp[7][8](18) \neq dp[6][8](15) \implies$ **Take Item 7**. Remaining $W = 8 - 3 = 5$.
- $dp[6][5](11) == dp[5][5](11) \implies$ **Skip Item 6**.
- $dp[5][5](11) == dp[4][5](11) \implies$ **Skip Item 5**.
- $dp[4][5](11) \neq dp[3][5](8) \implies$ **Take Item 4**. Remaining $W = 5 - 2 = 3$.
- $dp[3][3](6) \neq dp[2][3](5) \implies$ **Take Item 3**. Remaining $W = 3 - 2 = 1$.
- $dp[2][1](3) \neq dp[1][1](2) \implies$ **Take Item 2**. Remaining $W = 1 - 1 = 0$.

Optimal Solution: Selected Items: **{2, 3, 4, 7}**
Total Weight: $1 + 2 + 2 + 3 = 8$
Total Profit: $3 + 3 + 5 + 7 = \mathbf{18}$

Complexity Analysis. Time Complexity: $O(nW) = O(7 \times 8) = O(56)$.
Space Complexity: $O(nW)$ for the full DP table.

Q56. Construct four matrices for finding all-pairs shortest paths for a given graph (edges: $AB(7), AC(3), DA(4), EA(1), BC(91), CF(9), BD(7), EB(2), F$ $CE(4), DE(6), FD(3)$). **(15 marks)** **[CSE 207 | 2006–07]**

Answer

Floyd-Warshall for all-pairs shortest paths:

$$d^{(k)}[i][j] = \min\big(d^{(k-1)}[i][j],\ d^{(k-1)}[i][k] + d^{(k-1)}[k][j]\big).$$

Index vertices as $A = 1, B = 2, C = 3, D = 4, E = 5, F = 6$.

Directed edges:
$AB(7), AC(3), DA(4), EA(1), BC(91), CF(9), BD(7), EB(2), FB(5), CE(4), DE(6),$
$FD(3)$.
$D^{(0)}$ (direct edges, ∞ if no edge):

	A	B	C	D	E	F
A	0	7	3	∞	∞	∞
B	∞	0	91	∞	∞	∞
C	∞	∞	0	∞	∞	9
D	4	7	∞	0	∞	∞
E	1	2	∞	∞	0	∞
F	∞	5	∞	3	∞	0

After all 6 iterations (intermediate matrices $D^{(1)}$ through $D^{(6)}$ updated via the recurrence), the final $D^{(6)}$ gives all-pairs shortest paths.

Remark. For full table construction: apply the recurrence for $k = A, B, \ldots, F$ in order. Each step takes $O(V^2)$; total $O(V^3) = O(216)$.

Q57. Find the longest common subsequence from the DNA sequences `ACGCTAC` and `CTGCA`. Print the LCS and analyze the runtime of both finding and printing the sequence. **(10+5 marks)** [CSE 207 | 2006–07]

Answer

$X = \langle A, C, G, C, T, A, C \rangle$, $Y = \langle C, T, G, C, A \rangle$.
DP Table $(c[i][j])$:

	ϵ	C	T	G	C	A
ϵ	0	0	0	0	0	0
A	0	0	0	0	0	1
C	0	1	1	1	1	1
G	0	1	1	2	2	2
C	0	1	1	2	3	3
T	0	1	2	2	3	3
A	0	1	2	2	3	4
C	0	1	2	2	3	4

LCS length $= 4$. **LCS** $= \langle C, G, C, A \rangle$.

Complexity Analysis. Finding LCS: $\Theta(mn) = \Theta(7 \times 5) = \Theta(35)$. Printing: $O(m + n) = O(12)$.

Q58. Find a $\Theta(n^2 2^n)$ TSP tour for the following distance matrix. What is the major drawback of this solution method?

	A	B	C	D
A	0	10	15	20
B	5	0	9	10
C	6	13	0	12
D	8	8	9	0

(12 marks) [CSE 207 | 2006–07]

Answer

Held-Karp DP: $dp[S][i]$ = min cost to visit all cities in set S, ending at i, starting from A.

$$dp[S][i] = \min_{j \in S,\, j \neq i} \big(dp[S \setminus \{i\}][j] + d[j][i] \big)$$

Optimal tour: $A \to B \to D \to C \to A$.
Cost: $AB(10) + BD(10) + DC(9) + CA(6) = \mathbf{35}$.
Major drawback: time complexity $O(n^2 \cdot 2^n)$ — exponential in n. For $n = 20$, this requires $\approx 4 \times 10^8$ operations; for $n = 50$, completely intractable. TSP is NP-hard; no polynomial algorithm is known.

BUET · CSE 105

DSA

Comparison-Based Sorting

Quicksort, Mergesort, Heapsort, Insertion Sort

T12 — Comparison-Based Sorting

Q1. Analyze the worst-case runtime of the Quicksort algorithm under the following assumptions. In your analysis, formulate the worst-case runtime as a recurrence relation and solve the recurrence.

(a) Balanced partitioning

(b) Imbalanced partitioning

(5+5=10 marks) [CSE 105 | 2023–24]

Answer

(a) Balanced partitioning. Each call to PARTITION splits the n-element subarray into two halves of size $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$. The partition step itself costs $\Theta(n)$. The recurrence is:

$$T(n) = 2\,T\left(\tfrac{n}{2}\right) + \Theta(n), \quad T(1) = \Theta(1).$$

By the Master Theorem (case 2: $a = 2$, $b = 2$, $f(n) = \Theta(n) = \Theta(n^{\log_b a})$):

$$T(n) = \Theta(n \log n).$$

(b) Imbalanced (worst-case) partitioning. The pivot is always the minimum (or maximum) element, producing subarrays of sizes 0 and $n-1$:

$$T(n) = T(n-1) + T(0) + \Theta(n) = T(n-1) + \Theta(n), \quad T(1) = \Theta(1).$$

Unrolling:

$$T(n) = \sum_{k=1}^{n} \Theta(k) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2).$$

Complexity Analysis. Balanced: $\Theta(n \log n)$. Imbalanced (worst case): $\Theta(n^2)$.

Q2. Define stable sorting algorithm. Explain with an example. **(5 marks)** [CSE 105 | 2023–24]

Answer

Definition. A sorting algorithm is **stable** if, for any two elements a_i and a_j with equal keys, the relative order of a_i and a_j in the sorted output is the same as their relative order in the input.

Example. Consider sorting student records by grade:

Student	Grade	Position
Alice	B	1
Bob	A	2
Charlie	B	3

After a *stable* sort by grade, Alice (B, position 1) must appear before Charlie (B, position 3) since both have grade B and Alice came first. An *unstable* sort may place Charlie before Alice.

Stable algorithms: Insertion Sort, Merge Sort, Counting Sort, Radix Sort.
Unstable algorithms: Heap Sort, Quick Sort (standard), Selection Sort.

Remark. Stability matters when a secondary ordering must be preserved after sorting by a primary key — for example, sorting a list first by name and then by age should preserve alphabetical order among people of the same age.

Q3. You are asked to sort the following array of integers in ascending order using the Quicksort algorithm. Highlight the pivot item at each step and show the status of the array after processing each pivot element:

$$[5,\ 17,\ 2,\ 4,\ 16,\ 8,\ 10,\ 6,\ 4,\ 26,\ 20,\ 7]$$

(15 marks) [CSE 105 | 2022–23]

Answer

Convention: Last element of each subarray is chosen as pivot. Lomuto partition is used. After each pivot is placed, it is in its final sorted position. **Bold** denotes the pivot placed at this step; underscore marks the active subarray.

Step	Pivot	Array state after placing pivot
1	**7**	$[5, 2, 4, 6, 4,\ \mathbf{7},\ 10, 17, 16, 26, 20, 8]$
2	**4**	$[2, 4,\ \mathbf{4},\ 6, 5,\ 7,\ 10, 17, 16, 26, 20, 8]$
3	**4**	$[2,\ \mathbf{4},\ 4,\ 6, 5,\ 7,\ 10, 17, 16, 26, 20, 8]$
4	**5**	$[2, 4, 4,\ \mathbf{5},\ 6,\ 7,\ 10, 17, 16, 26, 20, 8]$
5	**8**	$[2, 4, 4, 5, 6, 7,\ \mathbf{8},\ 17, 16, 26, 20, 10]$
6	**10**	$[2, 4, 4, 5, 6, 7, 8,\ \mathbf{10},\ 16, 26, 20, 17]$
7	**17**	$[2, 4, 4, 5, 6, 7, 8, 10, 16,\ \mathbf{17},\ 20, 26]$
8	**26**	$[2, 4, 4, 5, 6, 7, 8, 10, 16, 17, 20,\ \mathbf{26}]$
Sorted		$[2, 4, 4, 5, 6, 7, 8, 10, 16, 17, 20, 26]$

Remark. Steps 2–4 handle the left subarray $[5, 17, 2, 4, 16, 8, 10, 6, 4]$ recursively (only the portion to the left of pivot 7 is shown). Steps 5–8 handle the right subarray. Each pivot is placed exactly once and never moved again.

Q4. Quick-Sort is a divide-and-conquer based algorithm. The traditional Partition function of Quick-Sort works by selecting a *pivot* element from the input array and partitioning the other elements into two sub-arrays, according to whether they are less than or greater than the *pivot*. For this reason, it is sometimes called *partition-exchange* sort. The sub-arrays are then sorted recursively. This can be done *in-place*, requiring small additional amounts of memory to perform the sorting. Now answer the following questions.

(i) What is the worst-case running time of Quick-Sort algorithm if it is designed in a way that the last element of the input array is always selected as the *pivot*? Also give an example input scenario which will bring about the worst-case running time.

(ii) Suppose, you have been asked to evaluate the performance of Quick-Sort algorithm. You are using a bad input array which results in a very unbalanced partition at each of the recursive steps. Consider that at each recursive call, the Partition function is producing an α-to-β proportional split. Now using a recursion tree, find the time complexity of the Quick-Sort algorithm for this scenario. If the last three digits of your student ID represents a number x, then consider the following:

$$\alpha = x \quad \text{if } x \leq 100$$
$$\alpha = 200 - x \quad \text{if } x > 100$$
$$\beta = 100 - \alpha$$

Your answer must contain a well-drawn recursion tree where any kind of existing skewness of the tree is clearly illustrated.

(5+7=12 marks) **[CSE 203 | 2020–21]**

Answer

(i) Worst-case with last element always as pivot.
The worst case occurs when the last element is always the smallest (or largest) in the subarray, producing a 0-to-$(n-1)$ split at every level:

$$T(n) = T(0) + T(n-1) + \Theta(n) = T(n-1) + \Theta(n).$$

Unrolling: $T(n) = \sum_{k=1}^{n} \Theta(k) = \Theta(n^2)$.
Example input: An already-sorted array in ascending order, e.g., $[1, 2, 3, 4, 5]$. The last element (5) is the maximum; partition produces $[1, 2, 3, 4]$ and $[]$. Recursion depth: n.

(ii) α-to-β split analysis based on Student ID:
Given: Last three digits of student ID is 009 $\implies x = 9$.
Since $x \leq 100$, we calculate the split proportions as:

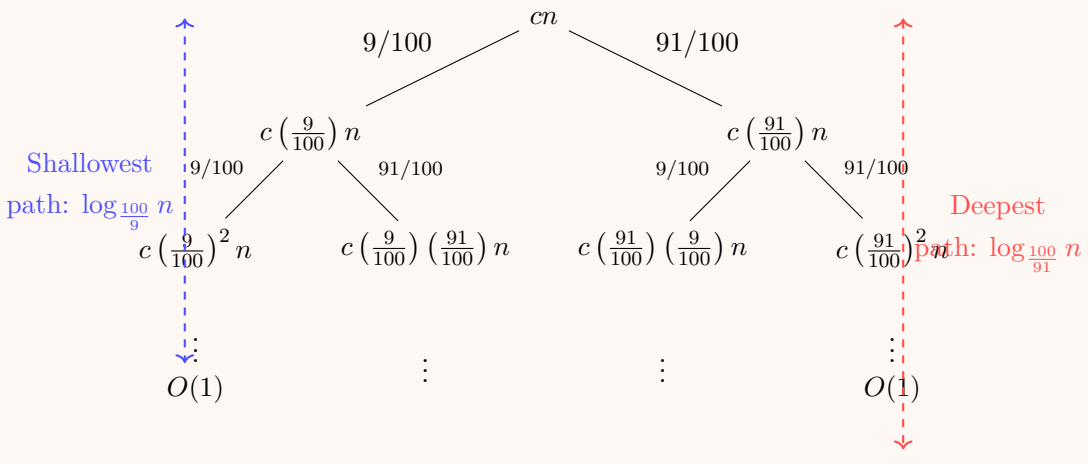
- $\alpha = x = 9$
- $\beta = 100 - \alpha = 100 - 9 = 91$

This means the Partition function produces a highly unbalanced proportional split of 9% to 91%. The recurrence relation for this scenario is:

$$T(n) = T\left(\frac{9}{100}n\right) + T\left(\frac{91}{100}n\right) + cn$$

where cn represents the linear time $\Theta(n)$ required to partition the array.

Recursion Tree:



Analysis:

- **Shallowest path (Left branch):** The subproblem size decreases by a factor of 9/100. This path hits the base case (size 1) early at depth $d_{\min} = \log_{100/9} n$.
- **Deepest path (Right branch):** The subproblem size decreases by a factor of 91/100. This path hits the base case much later at depth $d_{\max} = \log_{100/91} n$.
- **Cost per level:** At each level up to the shallowest path ($d \leq d_{\min}$), the sum of costs across all nodes is exactly cn. Beyond that depth ($d > d_{\min}$), some branches have already hit the base case and stopped, so the cost per level becomes strictly less than cn (i.e., $\leq cn$).

Time Complexity:
To find the upper bound of the total time, we multiply the maximum cost per level (cn) by the maximum depth of the tree ($d_{\max}$):

$$T(n) \leq cn \cdot \log_{\frac{100}{91}} n = O(n \log n)$$

Similarly, the lower bound is $T(n) \geq cn \cdot \log_{\frac{100}{9}} n = \Omega(n \log n)$.
Since both bounds are tightly proportional to $n \log n$, the time complexity is exactly $\Theta(n \log n)$. The skewness of the tree only affects the constant factors, but not the overall asymptotic bound.

Q5. Apply the partition procedure of Quicksort once on the array below, where the pivot is chosen as the element at the middle index $\lfloor (p+q)/2 \rfloor$ (indices range from p to q inclusive). Show all steps.

$$[10,\ 30,\ 50,\ 70,\ 90,\ 100,\ 80,\ 60,\ 40,\ 20]$$

(10+7 marks) [CSE 203 | 2019–20]

Answer

Array: $A = [10, 30, 50, 70, 90, 100, 80, 60, 40, 20]$, $p = 0$, $q = 9$.
mid_idx $= \lfloor (0+9)/2 \rfloor = 4$, so pivot $= A[4] = 90$.
Step 1: Swap $A[4]$ with $A[9]$ (move pivot to end):

$$[10, 30, 50, 70, \underline{20}, 100, 80, 60, 40, \underline{90}]$$

Step 2: Lomuto scan (i starts at -1, j from 0 to 8):

j	$A[j]$	≤ 90?	Action	Array
0	10	Yes	i=0, swap(0, 0) (trivial)	[10, 30, 50, 70, 20, 100, 80, 60, 40, 90]
1	30	Yes	i=1, swap(1, 1) (trivial)	unchanged
2	50	Yes	i=2, swap(2, 2) (trivial)	unchanged
3	70	Yes	i=3, swap(3, 3) (trivial)	unchanged
4	20	Yes	i=4, swap(4, 4) (trivial)	unchanged
5	100	No	skip	unchanged
6	80	Yes	i=5, swap(5, 6)	[10, 30, 50, 70, 20, 80, 100, 60, 40, 90]
7	60	Yes	i=6, swap(6, 7)	[10, 30, 50, 70, 20, 80, 60, 100, 40, 90]
8	40	Yes	i=7, swap(7, 8)	[10, 30, 50, 70, 20, 80, 60, 40, 100, 90]

Step 3: Place pivot: swap $A[i+1] = A[8]$ with $A[9]$:

$$\boxed{[10, 30, 50, 70, 20, 80, 60, 40,\ \mathbf{90},\ 100]}$$

Pivot 90 is now at index 8 (its final sorted position). Left partition: [10, 30, 50, 70, 20, 80, 60, 40]; right partition: [100].

Q6. Describe a scenario where an in-place sorting algorithm is necessary and explain the reasons. **(5 marks)** [CSE 203 | 2019–20]

Answer

Scenario: Sorting a large dataset in an embedded system with severe memory constraints.
Consider a microcontroller with only 4 KB of RAM that must sort an array of 1000 integers (4 KB of data). An out-of-place algorithm such as Merge Sort requires an auxiliary array of the same size (another 4 KB), which exceeds the available memory. An *in-place* algorithm such as Insertion Sort or Heap Sort uses only $O(1)$ extra space (a few variables), making it feasible.

Other scenarios requiring in-place sorting:

- **Real-time systems:** Memory allocation may be disallowed or too slow.
- **Cache performance:** Operating entirely within the existing array improves cache locality.
- **Very large files:** When the dataset barely fits in memory, no extra buffer is available.

Remark. In-place does *not* mean the algorithm is faster — Heap Sort ($\Theta(n \log n)$ in-place) vs. Merge Sort ($\Theta(n \log n)$ out-of-place). The choice is driven by memory, not time.

Q7. Give a concise description of a good way for Quicksort to choose a pivot element (better than always using index 0). **(5 marks)** [CSE 203 | 2018–19]

Answer

Median-of-Three pivot selection.
Examine three elements: the first ($A[\text{low}]$), the middle ($A[\lfloor(\text{low}+\text{high})/2\rfloor]$), and the last ($A[\text{high}]$). Use their *median* as the pivot.
Example: For subarray $[3, 1, 6, 8, 4]$: first= 3, mid= 6, last= 4. Median= 4. Use 4 as pivot.

Why it is better:

- Avoids worst-case behaviour on already-sorted or reverse-sorted inputs (the primary failure mode of fixed-index pivot selection).
- Produces a split closer to balanced on average, reducing expected recursion depth.
- Costs only $O(1)$ extra work (3 comparisons).

Alternative — Randomized pivot: Choose the pivot uniformly at random from the subarray. This provides $O(n \log n)$ expected time regardless of input order, eliminating adversarial worst-case inputs.

Q8. Write down the Quicksort algorithm. Use the partition algorithm to find the smallest 5 values of the array $[6, 1, 7, 2, 8, 3, 9, 4, 10, 5, 11, 17, 4, 5]$, showing every step. **(5+10 marks)** [CSE 203 | 2017–18]

Answer

Algorithm: (See Q14 above for full pseudocode.)

Finding the 5 smallest via Quickselect (partition-based selection):
Goal: place element at index 4 (0-indexed) in its correct sorted position. All elements to its left will be $\leq$ all elements to its right, giving the 5 smallest.
Array: $A = [6, 1, 7, 2, 8, 3, 9, 4, 10, 5, 11, 17, 4, 5]$

Step	Range	Pivot	Pivot placed at idx	Array after partition
1	[0..13]	5	6 $6 > 4$, recurse left	$[1, 2, 3, 4, 5, 4, \mathbf{5}, 6, 10, 8, 11, 17, 7, 9]$
2	[0..5]	4	4 $4 = 4$, **stop!**	$[1, 2, 3, 4, \mathbf{4}, 5, 5, 6, 10, 8, 11, 17, 7, 9]$

After step 2, index 4 holds the 5th smallest element (value 4). All elements at indices 0..4 form the 5 smallest values (not necessarily sorted among themselves):

$$\{1,\ 2,\ 3,\ 4,\ 4\} \quad \text{(the 5 smallest elements)}$$

Remark. Quickselect runs in $O(n)$ average time (vs. $O(n \log n)$ for full sort), making it efficient when only a partial order is needed.

Q9. What is an in-place sorting algorithm? Write an in-place sorting algorithm. Analyze the worst-case and best-case time complexities of the algorithm. **(15 marks)** [CSE 203 | 2016–17, 2015–16]

Answer

In-place sorting: An algorithm is *in-place* if it uses only $O(1)$ auxiliary space beyond the input array (i.e., it rearranges elements within the array itself without requiring a secondary array of comparable size).

Algorithm 125 SelectionSort(A, n)

```
1: for i = 1 to n − 1 do
2:     minIdx ← i
3:     for j = i + 1 to n do
4:         if A[j] < A[minIdx] then
5:             minIdx ← j
6:         end if
7:     end for
8:     Swap(A[i], A[minIdx])
9: end for
```

Best case: Even if the array is sorted, the inner loop still runs $n - i$ times for each i, so total comparisons $= \sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2} = \Theta(n^2)$.
Worst case: Same as best case — $\Theta(n^2)$.

Complexity Analysis. Best and worst case: $\Theta(n^2)$. Extra space: $O(1)$ (in-place). Selection Sort makes at most $n - 1$ swaps, which is useful when writes are expensive.

Or :

Algorithm 126 InsertionSort(A, n)

```
1: for i = 2 to n do
2:     key ← A[i]
3:     j ← i − 1
4:     while j ≥ 1 and A[j] > key do
5:         A[j + 1] ← A[j]
6:         j ← j − 1
7:     end while
8:     A[j + 1] ← key
9: end for
```

Only a constant number of variables (`key`, `i`, `j`) are used $\Rightarrow O(1)$ extra space.

Best case (sorted input): $\Theta(n)$. **Worst case** (reverse-sorted): $\Theta(n^2)$.

Remark. Other in-place sorting algorithms include Selection Sort ($\Theta(n^2)$ always), Heap Sort ($\Theta(n \log n)$), and in-place Quicksort ($O(n^2)$ worst, $O(n \log n)$ average). Merge Sort is *not* naturally in-place as it requires $O(n)$ auxiliary space.

Q10. Derive the expected time complexity of randomized Quicksort. **(20 marks)** [CSE 207 | 2016–17, 2009–10]

Answer

Setup. Given an array A of n distinct elements, let $z_1 < z_2 < \cdots < z_n$ be the elements in sorted order. Define:

$$X_{ij} = \mathbf{1}[z_i \text{ is compared with } z_j], \quad 1 \le i < j \le n.$$

Total comparisons: $X = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} X_{ij}$.

Key observation. z_i and z_j are compared if and only if one of them is chosen as pivot when both are in the same subarray. Once a pivot is chosen from $\{z_i, \ldots, z_j\}$:

- If the pivot is z_i or z_j: they *are* compared (probability $\frac{2}{j-i+1}$).
- Otherwise: they are placed in different subarrays and never compared.

Claim:

$$\Pr[X_{ij} = 1] = \frac{2}{j - i + 1}.$$

Proof: In randomized Quicksort, every element in $Z_{ij} = \{z_i, z_{i+1}, \ldots, z_j\}$ is equally likely to be the first chosen pivot from this set (since pivots are chosen uniformly at random). The set has $j - i + 1$ elements; exactly 2 of them (z_i and z_j) cause a comparison. Hence $\Pr[X_{ij} = 1] = \frac{2}{j-i+1}$. $\square$

Expected comparisons:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} \frac{2}{j - i + 1}.$$

Substitute $k = j - i$ (so k runs from 1 to $n - i$):

$$E[X] = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^{n} \frac{2}{k} = \sum_{i=1}^{n-1} 2H_n = 2(n-1)H_n,$$

where $H_n = \sum_{k=1}^{n} \frac{1}{k} = \ln n + O(1)$ is the n-th harmonic number.
Therefore:

$$E[X] < 2(n-1) \ln n + O(n) = O(n \log n).$$

A matching lower bound $\Omega(n \log n)$ follows from information-theoretic arguments (any comparison-based sort needs $\Omega(n \log n)$ comparisons). Hence:

$$\boxed{E[T(n)] = \Theta(n \log n).}$$

Complexity Analysis. Randomized Quicksort expected time: $\Theta(n \log n)$ for any input. This holds regardless of the input permutation since randomness is internal to the algorithm.

Q11. Write the Insertion Sort algorithm. Analyze the worst-case and best-case time complexities of the algorithm. **(10 marks)** [CSE 203 | 2015–16]

Answer

See Question No.9

Q12. Write the Quick Sort algorithm. Analyze the best-case and worst-case time complexity of the algorithm. **(15 marks)** [CSE 203 |

2014–15, 2013–14]

Answer

Algorithm 127 QuickSort(A, low, $high$)

```
if low < high then
    p ← Partition(A, low, high)
    QuickSort(A, low, p − 1)
    QuickSort(A, p + 1, high)
end if
```

Algorithm 128 Partition(A, low, $high$) — Lomuto scheme

```
pivot ← A[high]
i ← low − 1
for j = low to high − 1 do
    if A[j] ≤ pivot then
        i ← i + 1
        Swap(A[i], A[j])
    end if
end for
Swap(A[i + 1], A[high])
return i + 1
```

Best-case analysis. The best case occurs when Partition always splits the array into two equal halves:

$$T(n) = 2\,T\left(\tfrac{n}{2}\right) + \Theta(n).$$

Master Theorem (case 2) gives $T(n) = \Theta(n \log n)$.

Worst-case analysis. The worst case occurs when the pivot is always the minimum or maximum, yielding a 0-to-$(n-1)$ split:

$$T(n) = T(n-1) + \Theta(n).$$

Unrolling the recurrence:

$$T(n) = \sum_{k=1}^{n} \Theta(k) = \Theta\left(\frac{n(n+1)}{2}\right) = \Theta(n^2).$$

This occurs when the array is already sorted (ascending or descending) and the last (or first) element is always chosen as pivot.

Complexity Analysis. Best case: $\Theta(n \log n)$. Worst case: $\Theta(n^2)$. Average case: $\Theta(n \log n)$ (shown via indicator random variables).

Q13. Write the pseudocode of the Quicksort algorithm including the partition procedure. Deduce its average-case complexity. Simulate the partition algorithm on the array $[6, 2, 7, 3, 8, 4, 9, 5, 10, 1, 11]$ using the first element as the partitioning element. Show whenever elements are swapped. **(10 marks)** [CSE 207 | 2013–14]

Answer

Algorithm: (Full pseudocode as in Q12.)

Average-case complexity.
Let $T(n)$ be the average running time when the pivot is equally likely to be any of the n elements (rank k with probability $1/n$). Then:

$$T(n) = \frac{1}{n}\sum_{k=1}^{n}\big[T(k-1) + T(n-k)\big] + \Theta(n) = \frac{2}{n}\sum_{k=0}^{n-1} T(k) + \Theta(n).$$

Using the substitution $T(n) = an \log n + b$ (or solving with indicator random variables), one shows:

$$T(n) = \Theta(n \log n).$$

Partition simulation on $A = [6, 2, 7, 3, 8, 4, 9, 5, 10, 1, 11]$, pivot $= A[0] = 6$:
Swap pivot to end: $[11, 2, 7, 3, 8, 4, 9, 5, 10, 1, 6]$. Scan $j = 0$ to 9:

j	$A[j]$	≤ 6?	Swap(i,j)?	Array (relevant portion)
0	11	No	—	unchanged
1	2	Yes	$(0,1)$	$[2,11,7,3,8,4,9,5,10,1,6]$
2	7	No	—	unchanged
3	3	Yes	$(1,3)$	$[2,3,7,11,8,4,9,5,10,1,6]$
4	8	No	—	unchanged
5	4	Yes	$(2,5)$	$[2,3,4,11,8,7,9,5,10,1,6]$
6	9	No	—	unchanged
7	5	Yes	$(3,7)$	$[2,3,4,5,8,7,9,11,10,1,6]$
8	10	No	—	unchanged
9	1	Yes	$(4,9)$	$[2,3,4,5,1,7,9,11,10,8,6]$

Place pivot: swap $A[5]$ with $A[10]$:

$$[2,3,4,5,1,\mathbf{6},9,11,10,8,7]$$

Pivot 6 is at index 5 (its final position). Left: $[2,3,4,5,1]$; right: $[9,11,10,8,7]$.

Remark. Note: the left partition $[2,3,4,5,1]$ contains elements ≤ 6 but is not yet sorted among themselves — further recursive calls handle this.

Q14. Write a sorting algorithm that can always sort n elements in $O(n \log n)$ time. Analyze the best-case and worst-case time complexity of the algorithm. **(7+8=15 marks)** [CSE 203 | 2013–14]

Answer

Algorithm — Merge Sort (guaranteed $\Theta(n \log n)$ in all cases):

Algorithm 129 MERGE(A, low, mid, $high$) — detailed

```
1: n1 ← mid − low + 1;   n2 ← high − mid
2: Create arrays L[1 .. n1] and R[1 .. n2]
3: for i = 1 to n1 do
4:     L[i] ← A[low + i − 1]
5: end for
6: for j = 1 to n2 do
7:     R[j] ← A[mid + j]
8: end for
9: i ← 1;   j ← 1;   k ← low
10: while i ≤ n1 and j ≤ n2 do
11:     if L[i] ≤ R[j] then
12:         A[k] ← L[i];   i ← i + 1
13:     else
14:         A[k] ← R[j];   j ← j + 1
15:     end if
16:     k ← k + 1
17: end while
18: Copy remaining elements of L and R into A
```

Best-case analysis. Even if the array is already sorted, Merge Sort still divides and merges every level. The recurrence is:

$$T(n) = 2\,T(n/2) + \Theta(n), \quad T(1) = \Theta(1).$$

Master Theorem (case 2): $T(n) = \Theta(n \log n)$.
Worst-case analysis. The same recurrence holds regardless of the input (since MERGE always takes $\Theta(n)$ for a pair of subarrays of total size n):

$$T(n) = \Theta(n \log n).$$

Complexity Analysis. Best case: $\Theta(n \log n)$. Worst case: $\Theta(n \log n)$. Space: $\Theta(n)$ auxiliary (not in-place). Merge Sort is stable.

Q15. Explain the divide-and-conquer technique. Write the quick sort algorithm and analyze the time complexity of the algorithm. **(15 marks)** [CSE 203 | 2013–14]

Answer

Divide-and-Conquer. A paradigm with three steps applied recursively:

1. **Divide:** Split the problem into smaller subproblems of the same type.
2. **Conquer:** Solve subproblems recursively (base cases are solved directly).
3. **Combine:** Merge the solutions to subproblems into the solution for the original.

Examples: Merge Sort, Quicksort, Binary Search, Strassen's matrix multiplication.

Quicksort applies divide-and-conquer:

- **Divide:** PARTITION places a pivot in its correct position; elements $<$ pivot go left, $>$ pivot go right.
- **Conquer:** Recursively sort the two subarrays.
- **Combine:** No work needed — the array is sorted in place.

Algorithm 130 QUICKSORT(A, low, $high$)

```
if low < high then
    p ← PARTITION(A, low, high)
    QUICKSORT(A, low, p − 1)
    QUICKSORT(A, p + 1, high)
end if
```

Algorithm 131 PARTITION(A, low, $high$) — Lomuto scheme

```
pivot ← A[high]
i ← low − 1
for j = low to high − 1 do
    if A[j] ≤ pivot then
        i ← i + 1
        SWAP(A[i], A[j])
    end if
end for
SWAP(A[i + 1], A[high])
return i + 1
```

Time complexity:

Case	Recurrence	Solution
Best	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
Average	$T(n) = \frac{2}{n}\sum_{k=0}^{n-1} T(k) + \Theta(n)$	$\Theta(n \log n)$
Worst	$T(n) = T(n-1) + \Theta(n)$	$\Theta(n^2)$

Remark. Despite its $\Theta(n^2)$ worst case, Quicksort is preferred in practice over Merge Sort due to smaller constant factors, better cache behaviour (in-place), and $\Theta(n \log n)$ average performance. Randomized pivot selection effectively eliminates worst-case scenarios.

Q16. Sort the following dataset using Quicksort in *descending* order, using the first element as pivot (show all steps):

$$D = \{6,\ 0,\ 2,\ 0,\ 1,\ 3,\ 6,\ 1\}$$

(12 marks) [CSE 203 | 2012–13]

Answer

Convention: First element is pivot. To sort descending, an element x goes to the *left* partition if $x \geq$ pivot, and to the *right* partition if $x <$ pivot.
Swap pivot to end, then scan with the modified Lomuto condition ($\leq$ replaced by $\geq$):

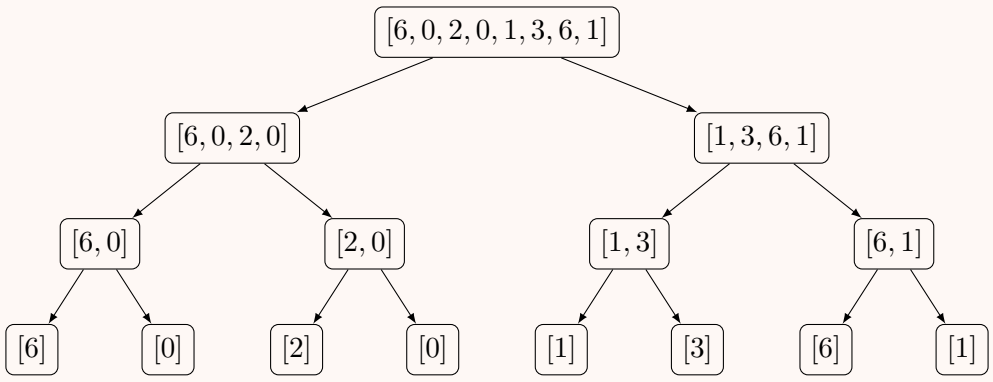
Step	Subarray	Pivot	After partition
1	$[6, 0, 2, 0, 1, 3, 6, 1]$	**6**	$[6, 6, \mid 2, 0, 1, 3, 1, 0]$; pivot at idx 1
2	$[6]$	**6**	single element, done
3	$[2, 0, 1, 3, 1, 0]$	**2**	$[3, 2, \mid 0, 1, 1, 0]$; pivot at idx 1
4	$[3]$	**3**	single element, done
5	$[0, 1, 1, 0]$	**0**	$[1, 1, 0, 0]$; pivot at idx 2
6	$[1, 1]$	**1**	$[1, 1]$; pivot at idx 1
7	$[0]$	**0**	single element, done
	Sorted (descending)		$[6, 6, 3, 2, 1, 1, 0, 0]$

Remark. The final sorted array is $[6, 6, 3, 2, 1, 1, 0, 0]$ — confirmed by reversing the naturally sorted order $[0, 0, 1, 1, 2, 3, 6, 6]$.

Q17. Using Mergesort on the dataset $D = \{6, 0, 2, 0, 1, 3, 6, 1\}$, draw the merge tree. Show that Mergesort runs in time $\Theta(n \log n)$ in the worst case. **(6+4=10 marks)** [CSE 203 | 2012–13]

Answer

Merge Tree (top = original array, bottom = single elements, arrows show split direction):



Merge steps (bottom-up):

- $\textsc{Merge}([6],[0]) \to [0,6]$; $\textsc{Merge}([2],[0]) \to [0,2]$
- $\textsc{Merge}([1],[3]) \to [1,3]$; $\textsc{Merge}([6],[1]) \to [1,6]$
- $\textsc{Merge}([0,6],[0,2]) \to [0,0,2,6]$; $\textsc{Merge}([1,3],[1,6]) \to [1,1,3,6]$
- $\textsc{Merge}([0,0,2,6],[1,1,3,6]) \to [0,0,1,1,2,3,6,6]$

Proof of $\Theta(n \log n)$ worst-case:
The recurrence is $T(n) = 2\,T(n/2) + \Theta(n)$, $T(1) = \Theta(1)$.

- The recursion tree has $\log_2 n$ levels.
- At level k, there are 2^k subproblems each of size $n/2^k$.
- Merge cost at level k: $2^k \times \Theta(n/2^k) = \Theta(n)$ per level.
- Total: $\Theta(n) \times \log_2 n = \Theta(n \log n)$.

By the Master Theorem (case 2), $T(n) = \Theta(n \log n)$. Since the merge step always scans both halves, this bound is tight: worst case $= \Theta(n \log n)$.

Q18. Write the Quicksort algorithm. Simulate the Quicksort algorithm on the array

$$[15,\ 13,\ 9,\ 5,\ 12,\ 8,\ 7,\ 4,\ 10,\ 6,\ 2,\ 1,\ 26,\ 23]$$

showing in detail how the partition routine works. **(8+7=15 marks)** [CSE 203 | 2011–12]

Answer

1. The Quicksort Algorithm

See Question No. 12

---

2. Simulation of Quicksort

Initial Array: [15, 13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 1, 26, 23]

Step	Subarray	Pivot	Result (Left Pivot Right)
1	$[15, 13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 1, 26, \mathbf{23}]$	23	$[15, 13, \ldots, 1]$ **23** [26]
2	$[15, 13, 9, 5, 12, 8, 7, 4, 10, 6, 2, \mathbf{1}]$	1	[] **1** $[13, 9, 5, 12, 8, 7, 4, 10, 6, 2, 15]$
3	$[13, 9, 5, 12, 8, 7, 4, 10, 6, 2, \mathbf{15}]$	15	$[13, 9, 5, 12, 8, 7, 4, 10, 6, 2]$ **15** []
4	$[13, 9, 5, 12, 8, 7, 4, 10, 6, \mathbf{2}]$	2	[] **2** $[9, 5, 12, 8, 7, 4, 10, 6, 13]$
5	$[9, 5, 12, 8, 7, 4, 10, 6, \mathbf{13}]$	13	$[9, 5, 12, 8, 7, 4, 10, 6]$ **13** []
6	$[9, 5, 12, 8, 7, 4, 10, \mathbf{6}]$	6	$[5, 4]$ **6** $[8, 7, 9, 10, 12]$
7	$[5, \mathbf{4}]$	4	[] **4** [5]
8	$[8, 7, 9, 10, \mathbf{12}]$	12	$[8, 7, 9, 10]$ **12** []
9	$[8, 7, 9, \mathbf{10}]$	10	$[8, 7, 9]$ **10** []
10	$[8, 7, \mathbf{9}]$	9	$[8, 7]$ **9** []
11	$[8, \mathbf{7}]$	7	[] **7** [8]

Final Sorted Array: [1, 2, 4, 5, 6, 7, 8, 9, 10, 12, 13, 15, 23, 26]

---

3. Detailed Trace of Partition (Step 6)

Tracing partition on subarray $[\mathbf{9}, \mathbf{5}, \mathbf{12}, \mathbf{8}, \mathbf{7}, \mathbf{4}, \mathbf{10}, \mathbf{6}]$. Pivot $x = 6$, Initial $i = -1$.

j	$A[j]$	$A[j] \leq 6$	New i	Action & Array State
–	–	–	-1	$[9, 5, 12, 8, 7, 4, 10, 6]$ (Initial)
0	9	False	-1	No swap. $[9, 5, 12, 8, 7, 4, 10, 6]$
1	5	**True**	**0**	Swap $A[0], A[1] \implies [\mathbf{5}, \mathbf{9}, 12, 8, 7, 4, 10, 6]$
2	12	False	0	No swap. $[5, 9, 12, 8, 7, 4, 10, 6]$
3	8	False	0	No swap. $[5, 9, 12, 8, 7, 4, 10, 6]$
4	7	False	0	No swap. $[5, 9, 12, 8, 7, 4, 10, 6]$
5	4	**True**	**1**	Swap $A[1], A[5] \implies [5, \mathbf{4}, 12, 8, 7, \mathbf{9}, 10, 6]$
6	10	False	1	No swap. $[5, 4, 12, 8, 7, 9, 10, 6]$

End of loop. Swap $A[i+1]$ ($A[2] = 12$) with Pivot ($A[7] = 6$).
Result: $[5, 4, \mathbf{6}, 8, 7, 9, 10, \mathbf{12}]$. Pivot 6 is placed correctly!

Q19. Simulate the Quicksort algorithm on the array $[5, 1, 6, 3, 7, 8, 9, 2, 4, 8]$. Discuss its worst-case complexity. **(15 marks)** [CSE 207 | 2008–09]

Answer

Simulation (pivot = last element, Lomuto partition):

Step	Subarray before	Pivot	Subarray after
1	$[5, 1, 6, 3, 7, 8, 9, 2, 4, 8]$	**8**	$[5, 1, 6, 3, 7, 8, 2, 4, \mathbf{8}, 9]$; idx 8
2	$[5, 1, 6, 3, 7, 8, 2, 4]$	**4**	$[1, 3, 2, 4, 7, 8, 6, 5]$; idx 3
3	$[1, 3, 2]$	**2**	$[1, \mathbf{2}, 3]$; idx 1
4	$[7, 8, 6, 5]$	**5**	$[\mathbf{5}, 8, 6, 7]$; idx 0
5	$[8, 6, 7]$	**7**	$[6, \mathbf{7}, 8]$; idx 1
6	$[9]$ (right of step 1)	—	single element
	Sorted		$[1, 2, 3, 4, 5, 6, 7, 8, 8, 9]$

Worst-case complexity:
The worst case is $\Theta(n^2)$, occurring when the pivot is always the minimum or maximum element (e.g., sorted or reverse-sorted input). Each call to PARTITION produces subarrays of sizes 0 and $n-1$:

$$T(n) = T(n-1) + \Theta(n) \;\Rightarrow\; T(n) = \Theta(n^2).$$

For the given array, the partition is reasonably balanced, so the actual run is much better than worst case.

BUET · CSE 105

DSA

Sorting in Linear Time

Counting Sort, Radix Sort, Bucket Sort

T13 — Sorting in Linear Time

Q1. Sort the following list of non-negative integers using Radix Sort (LSD):

$$[329,\ 457,\ 657,\ 839,\ 436,\ 720,\ 355]$$

State the conditions required for Radix Sort to work correctly and in linear time. **(10 marks)** [CSE 105 | 2023–24]

Answer

Algorithm. LSD Radix Sort processes digits from least significant to most significant. At each pass it uses a *stable* sort (Counting Sort) on the current digit.

Pass 1 — Units digit (10^0):

Digit	Numbers in bucket
0	720
5	355
6	436
7	457, 657
9	329, 839

After pass 1: [720, 355, 436, 457, 657, 329, 839]

Pass 2 — Tens digit (10^1):

Digit	Numbers in bucket
2	720, 329
3	436, 839
5	355, 457, 657

After pass 2: [720, 329, 436, 839, 355, 457, 657]

Pass 3 — Hundreds digit (10^2):

Digit	Numbers in bucket
3	329, 355
4	436, 457
6	657
7	720
8	839

After pass 3 (sorted): [**329**, **355**, **436**, **457**, **657**, **720**, **839**]

Conditions for correctness and linear time:

- The per-digit sort must be **stable** (so relative order from earlier passes is preserved).
- The number of digits d must be **constant** (independent of n), i.e., all keys have $O(1)$ digits.
- Each digit lies in a **bounded range** of size k (e.g., 0–9 for decimal), so Counting Sort on one digit costs $\Theta(n + k)$.
- Total time: $d \cdot \Theta(n + k) = \Theta(n)$ when d and k are constants.

Q2. Given two arrays $a[\,]$ and $b[\,]$, each containing N distinct points in the plane, design an algorithm with linear running time (under reasonable assumptions) and at most linear extra space that determines whether the two arrays contain precisely the same set of points. **(8 marks)** [CSE 203 | 2018–19]

Answer

We can solve this problem in $O(N)$ time and $O(N)$ extra space using either Hashing or Linear-Time Sorting (under specific reasonable assumptions).

Method 1: Using Hash Table

Key insight. Direct comparison of comparison-based sorted arrays would cost $O(N \log N)$. To achieve $O(N)$, we use hashing to count occurrences.

Algorithm 132 SamePointSet_Hash(a, b, N)

```
Create hash table H
for i = 1 to N do
    Insert a[i] into H (increment count)
end for
for i = 1 to N do
    if b[i] ∉ H or H[b[i]] = 0 then
        return false
    end if
    H[b[i]] ← H[b[i]] − 1
end for
return true
```

Correctness & Complexity. The first loop records all points in a. The second checks if each point of b is present with a remaining count > 0. Under the *simple uniform hashing assumption*, operations take $O(1)$ expected time. Thus, Time: $O(N)$, Space: $O(N)$.

Method 2: Using Linear-Time Sorting (Radix Sort)

Key insight. If point coordinates are bounded integers, we can use a non-comparison based sort like Radix Sort to sort both arrays in linear time, then compare them element by element.

Algorithm 133 SamePointSet_Sort(a, b, N)

```
RadixSort(a)        ▷ Sort primarily by x-coordinate, secondarily by y
RadixSort(b)        ▷ Sort primarily by x-coordinate, secondarily by y
for i = 1 to N do
    if a[i].x ≠ b[i].x or a[i].y ≠ b[i].y then
        return false
    end if
end for
return true
```

Correctness & Complexity. Sorting brings identical sets of points into the exact same order. A single linear scan can then verify if the sets match point-by-point. Under the *reasonable assumption* that coordinates are integers with a constant number of digits, Radix Sort takes $O(N)$ time. The comparison loop also takes $O(N)$. Thus, Time: $O(N)$, Space: $O(N)$ extra space required by Radix Sort.

Q3. Write a linear-time Bucket Sort algorithm. Prove that the time complexity of the algorithm is linear. **(10 marks)** [CSE 203 | 2014–15, 2011–12]

Answer

Algorithm 134 BucketSort(A, n)

```
Create array B[0..n − 1] of empty lists
for i = 1 to n do
    Insert A[i] into B[⌊n · A[i]⌋]
end for
for i = 0 to n − 1 do
    InsertionSort(B[i])
end for
Concatenate B[0], B[1], ..., B[n − 1] into A
```

Proof of linear expected time.
Setup. Bucket Sort assumes n input elements are drawn *uniformly* from $[0, 1)$. Create n buckets $B[0], B[1], \ldots, B[n-1]$; element x goes into bucket $\lfloor n \cdot x \rfloor$.

Analysis. Let n_i = number of elements in bucket i. Each bucket is sorted independently (e.g., with Insertion Sort) at cost $O(n_i^2)$. Total time:

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2).$$

Taking expectations (since input is uniform):

$$E[T(n)] = \Theta(n) + \sum_{i=0}^{n-1} E\big[O(n_i^2)\big] = \Theta(n) + n \cdot O(E[n_i^2]).$$

Each element falls in bucket i with probability $p = 1/n$, and $n_i \sim \text{Binomial}(n, 1/n)$. Then:

$$E[n_i^2] = \text{Var}(n_i) + (E[n_i])^2 = \frac{n \cdot \frac{1}{n} \cdot (1 - \frac{1}{n})}{1} + 1 = 2 - \frac{1}{n}.$$

Therefore:

$$E[T(n)] = \Theta(n) + n \cdot O\big(2 - \tfrac{1}{n}\big) = \Theta(n).$$

Complexity Analysis. Average case: $\Theta(n)$ (under the uniform distribution assumption). Worst case: $\Theta(n^2)$ (all elements fall into one bucket).

Intuitive Proof of Linear Expected Time:
Let $T(n)$ be the total time complexity of the Bucket Sort algorithm. The operations can be broken down as follows:

(a) **Creating buckets and inserting elements:** We iterate through the n input elements and place each into one of the n buckets. This takes $O(n)$ time.

(b) **Sorting individual buckets:** We use Insertion Sort for each bucket. In the worst case, Insertion Sort takes $O(k^2)$ time for k elements.

(c) **Concatenating buckets:** Going through n buckets to combine the elements takes $O(n)$ time.

Expected Sorting Time:
Since the input elements are uniformly distributed over the interval $[0, 1)$, the probability that an element falls into any specific bucket is $1/n$.
Because there are n elements and n buckets, the *expected (average) number of elements* in each bucket is:

$$k = \frac{n}{n} = 1$$

Since the expected size of each bucket is just 1, the expected time to sort a single bucket using Insertion Sort is $O(1^2) = O(1)$.
For n buckets, the total expected time to sort all buckets is:

$$n \times O(1) = O(n)$$

Total Expected Time Complexity:
Adding the times for all steps:

$$T(n) = O(n) \text{ (insert)} + O(n) \text{ (sort)} + O(n) \text{ (concatenate)} = \mathbf{O(n)}$$

Thus, the expected time complexity of Bucket Sort is linear. □

Q4. Write the counting sort algorithm. Prove that the algorithm runs in linear time. Explain why counting sort is stable. **(12 marks)** **[CSE 203 | 2013–14]**

Answer

Algorithm 135 CountingSort(A, B, n, k)

```
1:  for i = 0 to k do
2:      C[i] ← 0
3:  end for
4:  for j = 1 to n do
5:      C[A[j]] ← C[A[j]] + 1
6:  end for
7:  for i = 1 to k do
8:      C[i] ← C[i] + C[i − 1]
9:  end for
10: for j = n downto 1 do
11:     B[C[A[j]]] ← A[j]
12:     C[A[j]] ← C[A[j]] − 1
13: end for
```

Proof of linear time.

- Loop 1 (initialize C): $\Theta(k)$ iterations.
- Loop 2 (count): $\Theta(n)$ iterations.
- Loop 3 (cumulate): $\Theta(k)$ iterations.
- Loop 4 (build output): $\Theta(n)$ iterations.

Total: $\Theta(n + k)$. When $k = O(n)$, this is $\Theta(n)$. □

Why it is stable. In Loop 4, elements are placed into B by scanning A from right to left (index n down to 1). For two equal elements $A[p] = A[q] = v$ with $p < q$: $A[q]$ is processed first (larger index scanned first) and occupies the *higher* position among slots allocated for v; $A[p]$ is processed next and occupies the *lower* position. Hence $A[p]$ precedes $A[q]$ in B, preserving the original left-to-right order among equal elements. □

Q5. Write the Counting Sort steps for the dataset $D = \{6, 0, 2, 0, 1, 3, 6, 1\}$. **(6 marks)** [CSE 203 | 2012–13]

Answer

$D = [6, 0, 2, 0, 1, 3, 6, 1]$, $n = 8$, range $k = 6$.
Step 1 — Count:

i	0	1	2	3	4	5	6
$C[i]$	2	2	1	1	0	0	2

Step 2 — Cumulate:

i	0	1	2	3	4	5	6
$C[i]$	2	4	5	6	6	6	8

Step 3 — Build output (right-to-left scan): $1 \to B[3]$; $6 \to B[7]$; $3 \to B[5]$; $1 \to B[2]$; $0 \to B[1]$; $2 \to B[4]$; $0 \to B[0]$; $6 \to B[6]$.
Output: $B = [0,\ 0,\ 1,\ 1,\ 2,\ 3,\ 6,\ 6]$

Q6. Sort the following dataset using Counting Sort, showing every step:

$$A = \{6,\ 0,\ 2,\ 0,\ 1,\ 3,\ 4,\ 6,\ 1,\ 3,\ 2\}$$

(11 marks) [CSE 203 | 2011–12]

Answer

$A = [6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2]$, $n = 11$, range $k = 6$ (values 0–6).

Step 1 — Count occurrences ($C[i]$ = number of times i appears in A):

Index i	0	1	2	3	4	5	6
$C[i]$	2	2	2	2	1	0	2

Step 2 — Cumulate ($C[i]$ += $C[i-1]$, so $C[i]$ = number of elements $\le i$):

Index i	0	1	2	3	4	5	6
$C[i]$	2	4	6	8	9	9	11

Step 3 — Build output B (scan A right-to-left for stability):

x (from A, R→L)	$C[x]$ before	Place at $B[\,]$	$C[x]$ after
2	6	$B[5]$	5
3	8	$B[7]$	7
1	4	$B[3]$	3
6	11	$B[10]$	10
4	9	$B[8]$	8
3	7	$B[6]$	6
1	3	$B[2]$	2
0	2	$B[1]$	1
2	5	$B[4]$	4
0	1	$B[0]$	0
6	10	$B[9]$	9

Output: $B = [0,\ 0,\ 1,\ 1,\ 2,\ 2,\ 3,\ 3,\ 4,\ 6,\ 6]$

Complexity Analysis. Step 1: $\Theta(n)$. Step 2: $\Theta(k)$. Step 3: $\Theta(n)$. Total: $\Theta(n + k)$. For constant k this is $\Theta(n)$.

Q7. "An important property of Counting Sort is that it is stable" — explain. **(4 marks)** [CSE 203 | 2011–12]

Answer

A sorting algorithm is **stable** if two elements with equal keys appear in the same relative order in the output as in the input.

Why Counting Sort is stable. In Step 3, the output array is built by scanning the input A from *right to left*. For each element $x = A[i]$, it is placed at position $C[x] - 1$, and then $C[x]$ is decremented. Consider two equal elements $A[p] = A[q] = v$ with $p < q$ (i.e., $A[q]$ appears later in the input). Since we scan right to left:

- $A[q] = v$ is processed first and placed at the *higher* available position for value v.
- $A[p] = v$ is processed next and placed at a *lower* position.

Thus $A[p]$ ends up to the left of $A[q]$ in B, preserving their original relative order. Stability is a direct consequence of the right-to-left scan combined with the decrementing counter.

Remark. Stability is critical when Counting Sort is used as a subroutine inside Radix Sort. Without stability, earlier passes' ordering would be destroyed by later passes, producing an incorrect result.

BUET · CSE 105

DSA

Lower Bound Theory

Comparison Sort Lower Bound, Decision Trees

T14 — Lower Bound Theory

Q1. Prove that any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case. **(10 marks)** [CSE 105 | 2023–24, 2022–23 , 2022-23, CSE 203 | 2016–17, 2014–15, 2013–14, CSE 207 | 2009–10]

Answer: Lower Bound of Comparison-Based Sorting

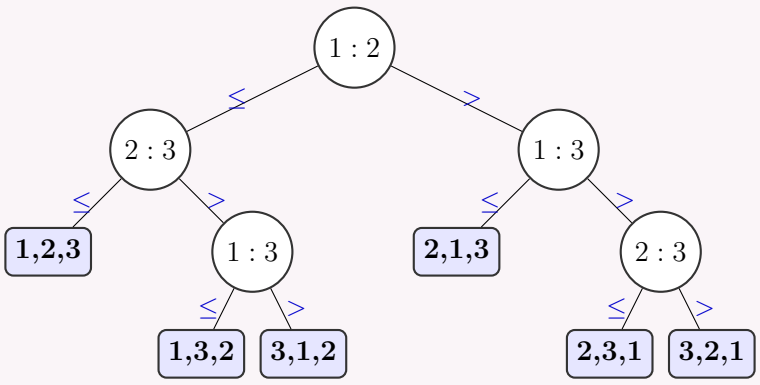
1. The Decision-Tree Model

Any comparison-based sorting algorithm can be visualized as a *decision tree*.

- **Internal Nodes:** Represent a comparison between two elements, $a_i \leq a_j$.
- **Branches:** Represent the outcomes ($\leq$ or $>$) of the comparison.
- **Leaf Nodes:** Represent the final sorted permutation of the input elements.

Example: Decision Tree for $n = 3$ elements (a_1, a_2, a_3)

For 3 elements, there are $3! = 6$ possible permutations. Here is the decision tree showing how the comparisons lead to all 6 possible sorted orders:



2. Lower Bound Argument

Step 1: Count the number of leaves.

An array of size n can be arranged in $n!$ different ways. For the sorting algorithm to be correct, it must be able to output any of these $n!$ permutations. Thus, the decision tree must have at least $L \geq n!$ reachable leaf nodes.

Step 2: Bound the height of the tree.

The longest path from the root to a leaf represents the worst-case number of comparisons. Let h be the height of the tree. Since it's a binary tree, a tree of height h has at most 2^h leaves. Therefore:

$$2^h \geq n! \implies h \geq \log_2(n!)$$

Step 3: Bound $\log_2(n!)$ using Stirling's Approximation.

Stirling's approximation gives an asymptotic estimate for factorials:

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

Taking the base-2 logarithm of both sides:

$$\begin{aligned}\log_2(n!) &\approx \log_2\left(\sqrt{2\pi n}\left(\frac{n}{e}\right)^n\right) \\ &= \log_2(\sqrt{2\pi n}) + \log_2\left(\frac{n}{e}\right)^n \\ &= \frac{1}{2}\log_2(2\pi n) + n\log_2(n) - n\log_2(e)\end{aligned}$$

For very large values of n, the term $n\log_2(n)$ dominates all other terms (like $n\log_2(e)$ and the square root term). By ignoring the lower-order terms, we get:

$$\log_2(n!) = \Theta(n \log n)$$

Since $h \geq \log_2(n!)$, this establishes a strict lower bound.

Conclusion:

The worst-case number of comparisons, represented by the tree height h, is mathematically bounded by:

$$h \geq \Omega(n \log n)$$

Hence, any comparison-based sorting algorithm requires at least $\Omega(n \log n)$ comparisons in the worst case. □

Q2. Prove that any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case. Describe a sorting algorithm that achieves this lower bound. **(7+8 marks)** [CSE 203 | 2016–17]

Answer

Part (a) — Lower bound. (Decision tree proof as in Q1 above.) Any comparison sort has worst-case complexity $\Omega(n \log n)$. □

Part (b) — Algorithm achieving the bound: Merge Sort.

Algorithm 136 MergeSort(A, low, $high$)

```
1: if low < high then
2:     mid ← ⌊(low + high)/2⌋
3:     MergeSort(A, low, mid)
4:     MergeSort(A, mid + 1, high)
5:     Merge(A, low, mid, high)
6: end if
```

Algorithm 137 Merge(A, low, mid, $high$)

```
1: Copy A[low .. mid] into L, A[mid + 1 .. high] into R
2: i ← 1;   j ← 1;   k ← low
3: while i ≤ |L| and j ≤ |R| do
4:     if L[i] ≤ R[j] then
5:         A[k] ← L[i];   i ← i + 1
6:     else
7:         A[k] ← R[j];   j ← j + 1
8:     end if
9:     k ← k + 1
10: end while
11: Copy remaining elements of L or R into A
```

Worst-case analysis.

$$T(n) = 2\,T(n/2) + \Theta(n) \implies T(n) = \Theta(n \log n) = O(n \log n).$$

Since $O(n \log n) = \Omega(n \log n)$, Merge Sort is *asymptotically optimal* among comparison-based sorts.
Properties: Stable, not in-place ($O(n)$ auxiliary space), guaranteed $\Theta(n \log n)$ in all cases.

BUET · CSE 105

DSA

Data Structures for Set Operations

Union-Find, Disjoint Sets, Path Compression, Union by Rank

T15 — Data Structures for Set Operations

Q1. Consider a Union-Find data structure on a set of n elements, where the UNION operation keeps the name of the largest set. Analyze the run-time complexity of any sequence of k UNION operations. **(10 marks)** [CSE 105 | 2023–24, 2022–23]

Answer

Setup. We use an *array implementation*: each element x stores a pointer to its set representative, and each set stores its size. UNION(A, B) merges the smaller set into the larger (weighted union / union-by-size), keeping the name of the larger set by updating the pointers of the smaller set's elements.

Key lemma. Each element changes its representative at most $\lfloor \log_2 n \rfloor$ times over all union operations.
Proof. Each time an element x's representative changes, x moves from set S_1 to set S_2 where $|S_2| \geq |S_1|$ (since we merge the smaller into the larger). So after each such move, the new set containing x at least doubles in size ($|S_1 \cup S_2| \geq 2|S_1|$). Starting from a set of size 1 and doubling each time, the set can contain x with a new representative at most $\log_2 n$ times before the set size reaches n. □

Total cost of k Union operations.
Each UNION itself costs $O(1)$ to compare sizes and update the size of the new set, but updating representative pointers costs 1 per element moved.
For k UNION operations, we involve at most $m = \min(n, 2k)$ distinct elements (since each union connects at most two disjoint components). By our lemma, each of these m elements can have its pointer updated at most $\log_2 m$ times.
Therefore, the maximum number of pointer updates is bounded by:

$$\text{Total pointer updates} \leq m \log_2 m \leq 2k \log_2(2k)$$

Ignoring the constants, the total time for k unions is:

$$T(k \text{ unions}) = O(k \log k).$$

Complexity Analysis. Any sequence of k UNION operations costs $O(k \log k)$ total. The per-operation amortized cost is $O(\log k)$.

Q2. Write an algorithm to detect a cycle in an undirected graph using a disjoint-set data structure. Mention the cases where this algorithm performs more efficiently than BFS- or DFS-based approaches. **(10+5 marks)** [CSE 203 | 2019–20]

Answer

Algorithm 138 DETECTCYCLE$(G = (V, E))$

```
1:  for each v ∈ V do
2:      MakeSet(v)
3:  end for
4:  for each edge (u, v) ∈ E do
5:      r_u ← Find(u)
6:      r_v ← Find(v)
7:      if r_u = r_v then
8:          return true                          ▷ cycle detected
9:      end if
10:     Union(r_u, r_v)
11: end for
12: return false                                 ▷ no cycle found
```

Correctness. Initially each vertex is its own set. As edges are processed, UNION merges the sets of endpoints. If both endpoints of edge (u, v) are already in the same set (Find(u) = Find(v)), then a path between u and v already exists, so adding this edge creates a cycle.

Complexity. With union-by-rank and path compression: $O(E\,\alpha(V)) \approx O(E)$ in practice.

When Union-Find outperforms BFS/DFS.

- **Sparse graphs with online edge addition.** When edges are given one at a time (streaming), Union-Find detects a cycle as soon as the offending edge arrives, in $O(\alpha(n))$ per edge. BFS/DFS would need to be re-run from scratch (or maintained with extra bookkeeping).
- **Minimum Spanning Tree construction** (Kruskal's algorithm): edges are processed in sorted order; Union-Find naturally tracks connected components as edges are added without building an explicit adjacency list.
- **Memory efficiency.** Union-Find uses $O(V)$ space (just the parent/rank arrays) vs. BFS/DFS which additionally need an adjacency list $O(V + E)$ and a visited array.

Q3. Explain, with examples, the union-by-rank and path-compression heuristics used in the UNION and FIND operations of disjoint-set data structures. **(10 marks)** [CSE 203 | 2018–19]

Answer

Union-by-Rank. Each node x stores a *rank* (an upper bound on the height of the subtree rooted at x). UNION always attaches the tree of smaller rank under the root of larger rank. If ranks are equal, either can go under the other and the surviving root's rank is incremented by 1.
Effect. Trees remain shallow: a tree of rank r contains at least 2^r nodes, so rank is $O(\log n)$.
Example. Union($\{1\},\{2\}$): both rank 0; make 2 child of 1, rank(1) becomes 1. Then Union($\{1,2\},\{3\}$): rank(1)=1 > rank(3)=0; attach 3 under 1. Root 1 retains rank 1.

Path Compression. During FIND(x), after locating the root r of x's tree, all nodes on the path from x to r are made direct children of r.

Algorithm 139 FIND(x) — with path compression

```
1: if parent[x] ≠ x then
2:     parent[x] ← FIND(parent[x])
3: end if
4: return parent[x]
```

Effect. Future FIND operations on x or nodes on the path cost $O(1)$.
Example. Before: $1 \to 2 \to 3 \to 4$ (root). FIND(1) returns 4 and sets parent[1]=4, parent[2]=4, parent[3]=4.

Combined complexity. With both heuristics, any sequence of m FIND and UNION operations on n elements costs $O(m\,\alpha(n))$, where α is the extremely slowly growing inverse Ackermann function (effectively ≤ 4 for all practical n).

Q4. We have a Union-Find data structure for a set S of size n, where unions keep the name of the larger set. Prove that for the array implementation, any sequence of k UNION operations takes at most $O(k \log k)$ time. Also prove that for the pointer-based implementation, a FIND operation takes $O(\log n)$ time. **(15 marks)** [CSE 207 | 2011–12]

Answer

Part A — Array implementation: k unions cost $O(k \log k)$. See Q1 answer

Part B — Pointer-based implementation: Find costs $O(\log n)$.
Structure. Sets are linked lists or trees; each element has a pointer to its representative. UNION by size: the smaller list/tree is merged into the larger, updating all its pointers.
Claim. With union-by-size (weighted union), any tree rooted at representative r has height at most $\lfloor \log_2 n \rfloor$.
Proof. By induction on the number of union operations. Initially all trees have height 0. When two trees of heights $h_1 \leq h_2$ are merged (smaller under larger), the new height is $\max(h_2, h_1+1)$. The smaller tree had $\leq$ half the nodes of the merged tree, so the node count at least doubles whenever height increases by 1. A tree of height h has $\geq 2^h$ nodes $\leq n$, giving $h \leq \log_2 n$. $\square$
Since FIND(x) follows parent pointers to the root, it traverses a path of length $\leq \log_2 n$, costing $O(\log n)$.

Q5. What is the Disjoint Set forest implementation? Write short notes on path compression. Write pseudocode for Disjoint Set forest with path compression. **(3+3+4 marks)** [CSE 207 | 2009–10]

Answer

Disjoint Set Forest. Each disjoint set is stored as a rooted tree. Every node has a `parent` pointer; the root is its own parent. Each node optionally stores a `rank` (for union-by-rank). UNION links one tree's root under the other; FIND traverses to the root.

Path Compression. When FIND(x) traverses the path from x to the root r, it simultaneously sets the parent of every node on the path directly to r. This "flattens" the tree so that subsequent finds on any of these nodes cost $O(1)$. Path compression does not change ranks. It is a *self-adjusting* technique: the structure improves with use.

Algorithm 140 MAKESET(x)

```
1: parent[x] ← x
2: rank[x] ← 0
```

Algorithm 141 FIND(x) — with path compression

```
1: if parent[x] ≠ x then
2:     parent[x] ← FIND(parent[x])
3: end if
4: return parent[x]
```

Algorithm 142 UNION(x, y) — union by rank

```
1: r_x ← FIND(x);   r_y ← FIND(y)
2: if r_x = r_y then
3:     return
4: end if
5: if rank[r_x] < rank[r_y] then
6:     SWAP(r_x, r_y)
7: end if
8: parent[r_y] ← r_x
9: if rank[r_x] = rank[r_y] then
10:    rank[r_x] ← rank[r_x] + 1
11: end if
```

Complexity Analysis. With both path compression and union-by-rank, a sequence of m operations (MAKESET, UNION, FIND) on n elements takes $O(m\,\alpha(n))$ total time, where $\alpha(n) \le 4$ for all feasible n.